

Manual Maestro de Linux y Shell Scripting

Edición 2026 · v3 · 35 módulos

Índice

[Manual Maestro de Linux y Shell Scripting — Edición 2026](#)

[Módulo 1 — GNU, Linux, kernel y distribuciones](#)

[Módulo 2 — Terminal, CLI, shell, Bash, prompt y ayuda](#)

[Módulo 3 — Navegación inicial: pwd, ls y cd](#)

[Módulo 4 — Rutas absolutas y relativas; ~, ., .. y nombres con espacios](#)

[Módulo 5 — Árbol de archivos, FHS, /etc, /usr, /var, /tmp, /proc, /sys y enlaces](#)

[Módulo 6 — Crear, copiar, mover, renombrar y borrar con seguridad](#)

[Módulo 7 — Leer archivos con cat, less, head y tail](#)

[Módulo 8 — stdin, stdout, stderr y redirecciones](#)

[Módulo 9 — Tuberías \(pipes\) y composición de comandos](#)

[Módulo 10 — grep, find y locate: buscar texto y archivos](#)

[Módulo 11 — Usuarios y grupos: whoami, id, UID, GID e identidad](#)

[Módulo 12 — Permisos, propietarios, chmod, chown, umask, sudo y ACL básica](#)

[Módulo 13 — Procesos: ps, top y htop opcional](#)

[Módulo 14 — jobs, fg, bg, señales y kill](#)

[Módulo 15 — Paquetes, repositorios y actualizaciones](#)

[Módulo 16 — Diferencias entre Debian, Ubuntu, Fedora y RHEL](#)

[Módulo 17 — systemd, unidades y servicios; otros sistemas init en contexto](#)

[Módulo 18 — Logs y diagnóstico inicial con journalctl](#)

[Módulo 19 — Redes básicas: IP, DNS, rutas, ip y ss](#)

[Módulo 20 — SSH en sistemas propios o expresamente autorizados](#)

[Módulo 21 — Firewall: nftables, UFW y firewalld según el entorno](#)

[Módulo 22 — vi/Vim: edición segura de archivos de texto](#)

[Módulo 23 — Primer script Bash y shebang](#)

[Módulo 24 — Variables, entrada, argumentos, expansiones y quoting en Bash](#)

[Módulo 25 — Códigos de salida y composición de órdenes en Bash](#)

[Módulo 26 — Decisiones con if, test, \[\], \[\[\]\], case y aritmética](#)

[Módulo 27 — Bucles, lectura de líneas y nombres de archivo seguros](#)

[Módulo 28 — Funciones, parámetros y ámbito en Bash](#)

[Módulo 29 — Manejo de errores, trap, mktemp, límites de set -e/set -u y ShellCheck](#)

[Módulo 30 — Automatización con cron y temporizadores de systemd](#)

[Módulo 31 — Archivos y compresión con tar, gzip y xz](#)

[Módulo 32 — Copias, sincronización y restauración con rsync](#)

[Módulo 33 — Git y GitHub para scripts: puente al itinerario específico de Git](#)

[Módulo 34 — Almacenamiento: lsblk, df, du, montaje y fstab](#)

[Módulo 35 — Defensa, actualizaciones, mínimo privilegio, auditoría y AIDE](#)

v3 auditada · Primera entrega: arquitectura y Módulo 1

Fecha editorial: 6 de octubre de 2026. Estado: trabajo en progreso. La revisión de esta entrega no equivale a validar los 35 módulos ni a certificar un sistema.

Empieza aquí: [Módulo 1 — GNU, Linux, kernel y distribuciones](#). Los registros de revisión están en [auditorias/](#).

1. Propósito y alcance

Aprender desde cero a entender, utilizar y administrar GNU/Linux y escribir scripts de Bash. Cada práctica debe poder explicarse, repetirse sin copiar y revisarse ante errores. No se presupone experiencia con terminal, programación ni administración.

Esta entrega contiene el diseño completo y únicamente el primer módulo desarrollado. Los módulos 2-35 son un plan de trabajo; sus títulos no implican que sus lecciones o prácticas ya estén redactadas o probadas.

La base documental es la v2 del repositorio y el informe de NotebookLM adjunto a la conversación «Analizar enlace NotebookLM». Se conserva la intención pedagógica del informe y se corrigen sus generalizaciones. No se importan como evidencia sus referencias numéricas sin una bibliografía identificable.

2. Conservación y organización

La v2 permanece en `linux/manuales/manual-maestro-linux-shell-scripting-2026-v2.md`. Esta edición se guarda en otra carpeta: `linux/manuales/v3/`.

Archivo nuevo	Función
00-indice-arquitectura.md	Alcance, índice, progresión y reglas editoriales
modulo-01-gnu-linux-kernel-distribuciones.md	Primera lección completa, práctica y evaluación
02-auditoria-fuentes.md	Correcciones, fuentes, límites de verificación y pendientes

Se recomienda esta organización modular: facilita estudiar una lección y revisar commits pequeños. Un archivo único facilita imprimir, pero crece rápidamente y dificulta revisar cambios. La compilación integral se preparará cuando existan más módulos; no se duplican hoy contenidos en dos versiones que puedan divergir.

La numeración cambia: el Módulo 0 conceptual de v2 pasa a ser Módulo 1 de v3. El Módulo 1 de v2, dedicado a terminal, corresponde al Módulo 2 de v3. La navegación de v2 se desglosa en los módulos 3-5. No se interpreta esta renumeración como eliminación del original.

3. Método de cada lección

- Objetivo observable y conocimientos previos.
- Qué es el concepto y para qué sirve.
- Una forma de uso y un ejemplo pequeño.
- Explicación de cada línea, opción y argumento nuevo.
- Resultado esperado, distinguiendo ejemplo ilustrativo de salida real.
- Errores frecuentes: causa, corrección mínima y comprobación.
- Ejercicio guiado, variante independiente y detección de un error.
- Mini evaluación sin conceptos nuevos y recuperación posterior.
- Fuentes, entorno compatible y riesgos de la práctica.

Estados del aprendizaje: **NO VISTO → EN APRENDIZAJE → PRACTICADO → DOMINADO**. Una respuesta correcta no basta: se requiere explicación propia, resolución autónoma y detección de errores en más de una ocasión. El estado de redacción de un módulo se registra por separado del aprendizaje del estudiante.

4. Índice v3

Los resultados de la última columna son metas de evaluación, no logros ya obtenidos.

Fase I — Fundamentos y línea de comandos

Módulo	Contenido	Evidencia de aprendizaje
1	GNU, Linux, kernel y distribuciones; laboratorio seguro	Distinguir núcleo, distribución y herramienta; identificar el entorno
2	Terminal, CLI, shell, Bash, prompt y ayuda	Diferenciar interfaz e intérprete; leer un error sin improvisar
3	Navegación inicial: <code>pwd</code> , <code>ls</code> , <code>cd</code>	Ubicarse y cambiar de carpeta conscientemente
4	Rutas absolutas y relativas; <code>~</code> , <code>.</code> , <code>..</code> ; nombres con espacios	Predecir qué ruta se utilizará antes de ejecutar
5	Árbol de archivos y FHS; <code>/etc</code> , <code>/usr</code> , <code>/var</code> , <code>/tmp</code> , <code>/proc</code> , <code>/sys</code>	Separar datos, configuración y vistas del kernel; reconocer enlaces

Módulo	Contenido	Evidencia de aprendizaje
6	Crear, copiar, mover y renombrar; rm y rmdir	Trabajar en el laboratorio sin sobrescribir originales
7	Lectura: cat, less, head, tail	Elegir una herramienta según tamaño y propósito
8	Flujos estándar y redirecciones	Explicar >, >>, 2> y el riesgo de truncamiento
9	Tuberías y procesamiento de texto	Conectar salida y entrada sin confundirlas con archivos
10	grep, find, locate; introducción a patrones	Distinguir búsqueda de texto, recorrido e índice

Fase II — Identidad, permisos y procesos

Módulo	Contenido	Evidencia de aprendizaje
11	Usuarios y grupos; whoami, id	Interpretar identidad y pertenencia a grupos
12	Permisos, propietarios, chmod, chown, umask, sudo; introducción a ACL	Explicar los permisos de un objeto antes de cambiarlos
13	Procesos; ps, top; htop opcional	Identificar un proceso propio y sus recursos
14	Trabajos, jobs, fg, bg y señales	Distinguir suspender, continuar y terminar

Fase III — Sistemas, servicios y redes

Módulo	Contenido	Evidencia de aprendizaje
15	Paquetes, repositorios y actualizaciones	Identificar el gestor adecuado y revisar una operación
16	Diferencias Debian, Ubuntu, Fedora y RHEL	Seleccionar instrucciones para la distribución y versión reales
17	Servicios y arranque: systemd como itinerario principal; otros init	Separar consulta, inicio, habilitación y configuración
18	Registros: journalctl, archivos de log; introducción a auditoría	Buscar un evento sin publicar datos sensibles
19	Redes básicas: IP, DNS, rutas, ip, ss	Distinguir problemas de dirección, resolución y conectividad
20	SSH en equipos propios o autorizados	Explicar identidad del servidor, autenticación y cuidado de claves
21	Cortafuegos: nftables, UFW y firewalld según entorno	Diseñar reglas y recuperación antes de aplicarlas
22	Edición con vi/Vim desde cero	Guardar una copia de práctica y salir sin perder trabajo

Fase IV — Shell scripting con Bash

Módulo	Contenido	Evidencia de aprendizaje
23	Primer script, intérprete, shebang y ejecución	Explicar bash archivo.sh y ejecución directa
24	Variables, entrada, argumentos, expansiones y comillas	Preservar texto y límites de argumentos, incluido "\$@"
25	Códigos de salida y composición de órdenes	Interpretar éxito y fallo según el comando
26	Decisiones: if, test, [], [[]], case y aritmética	Elegir Bash o portabilidad POSIX y justificarlo
27	Bucles, lectura de líneas y nombres de archivo	Recorrer datos con espacios sin analizar la salida de ls
28	Funciones, parámetros y ámbito	Separar tareas y validar entradas
29	Errores, trap, mktemp, límites de set -e/-u y ShellCheck	Probar fallos controlados y explicar límites de la limpieza

Fase V — Automatización y administración defensiva

Módulo	Contenido	Evidencia de aprendizaje
30	cron y temporizadores de systemd	Probar manualmente antes de programar una tarea
31	Archivos y compresión: tar, gzip, xz	Inspeccionar un archivo antes de extraerlo
32	Copias y restauración con rsync	Verificar una restauración y explicar riesgos de sincronizar
33	Git y GitHub para scripts; enlace al itinerario específico de Git	Revisar diferencias y guardar commits sin secretos

Módulo	Contenido	Evidencia de aprendizaje
34	Almacenamiento: lsblk, df, du; montaje y fstab	Separar inspección de cambios de alto impacto
35	Defensa, actualizaciones, mínimo privilegio, auditoría y AIDE	Documentar un laboratorio propio con evidencia y recuperación

Ampliaciones posteriores, fuera del núcleo inicial: namespaces, cgroups, contenedores, eBPF, Rust en el kernel y EEFVDF en profundidad. Las herramientas rg, fd, bat o eza serán complementarias, sin exigir instalaciones al principiante.

5. Dependencias y ajustes pedagógicos

Las fases se cursan en orden. Cada módulo parte del anterior, con estas precisiones:

- El Módulo 1 incluye una preparación guiada mínima del laboratorio. No exige dominar todos sus comandos; los módulos 2-6 retoman cada uno con práctica más amplia.
- Las rutas anteceden a la descripción extensa del árbol para reducir abstracción antes de trabajar con archivos.
- El Módulo 6 podrá introducir un editor gráfico disponible para crear notas; el dominio de Vim se reserva al 22.
- Los códigos de salida anteceden a if, porque las condiciones de la shell se apoyan en el resultado de comandos.
- La conservación en GitHub empieza desde esta entrega, gestionada por el tutor. El estudiante no necesita dominar Git para que se conserve su trabajo. El Módulo 33 formaliza su autonomía.
- La lectura precede siempre a cambios en servicios, red, almacenamiento y seguridad.

6. Contrato de seguridad

Las prácticas se limitan a equipos propios o expresamente autorizados. Se usa un usuario normal y el laboratorio ~/linux-lab. Esa carpeta **no es una barrera de aislamiento**: una ruta absoluta, un enlace o un comando mal escrito puede alcanzar otros lugares.

Antes de modificar: explicar efecto, comprobar ubicación con pwd, inspeccionar con ls y comprobar también el destino exacto. Esos dos comandos ayudan, pero no garantizan por sí solos seguridad. Si una orden falla, detenerse antes de continuar con la siguiente.

No se exige sudo en prácticas iniciales. No se usa rm -rf como rutina. No se editan archivos del sistema ni se cambian discos, servicios o cortafuegos en el Módulo 1. Las prácticas posteriores con impacto necesitan copia o instantánea, alcance explícito y procedimiento de recuperación.

Cada práctica llevará una etiqueta: **consulta, creación en laboratorio, modificación o borrado**. El riesgo se evalúa por orden y opciones: un nombre de programa no basta. Un comando de consulta puede revelar información privada si se comparte su salida.

7. Compatibilidad y referencias de versión

- GNU Coreutils **9.11** es la referencia editorial solicitada y corroborada. No se afirma que esa sea la versión instalada ni que deba actualizarse el equipo para empezar. [Anuncio oficial](#).
- Bash se enseña indicando qué construcciones son propias de Bash y cuáles pertenecen a POSIX. No se asume que /bin/sh sea Bash. [Manual GNU Bash](#).
- Para prácticas de RHEL 10 se prioriza la documentación de **RHEL 10**; RHEL 8 se mantiene como referencia contextual si ese es el sistema estudiado. No se trasladan recetas entre versiones automáticamente. [Guía oficial de seguridad de RHEL 10](#).
- Debian y Ubuntu tienen fichas separadas para cortafuegos y configuración. Fedora y RHEL también requieren verificar su versión; pertenecer a una familia no vuelve idénticos todos los procedimientos.
- systemd será el recorrido principal de servicios; se explicará que existen otros sistemas de inicio y entornos donde no actúa como gestor del sistema.

8. Puerta de revisión antes de cada entrega

Comprobar fuentes, versión y aplicabilidad; revisar cada ejemplo y su explicación; verificar enlaces internos; buscar contradicciones con módulos anteriores; comprobar que no se incluyen credenciales ni datos personales; conservar originales; guardar cambios pequeños. La prueba documental, la ejecución real y el dominio del estudiante se informan por separado.

Siguiente entrega prevista: Módulo 2, después de recibir el diagnóstico del estudiante. No se declara completado ni se redacta anticipadamente en este archivo.

Módulo 1 — GNU, Linux, kernel y distribuciones

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Primera entrega.

1. Qué aprenderás

Al terminar podrás distinguir un núcleo, una distribución y una shell; identificar qué sistema estás usando; y explicar para qué servirá `~/linux-lab`.

No necesitas saber programar. Antes de la práctica responde: ¿trabajas en Linux instalado, una máquina virtual, una distribución dentro de WSL o todavía solo en Windows? ¿Has escrito alguna orden en una terminal? Si no lo sabes, estudia primero los conceptos y deja pendiente la ejecución. No hace falta instalar nada para leer este módulo.

En Windows, PowerShell no interpreta las órdenes de Bash de la misma forma. Abre la terminal de tu distribución Linux ya preparada. Si no tienes una, la preparación del entorno será una actividad guiada aparte; no improvises una instalación ni un particionado. Git Bash sirve para ciertas prácticas de shell, pero no equivale a disponer de un sistema Linux completo.

2. Qué es un sistema operativo

Es el conjunto de programas que permite usar los recursos del equipo y ejecutar aplicaciones. Coordina tareas como reservar memoria, acceder a archivos y comunicarse con dispositivos.

Para qué te sirve saberlo: al aparecer un problema, podrás distinguir si está en una aplicación, una herramienta o una parte del sistema.

Ejemplo: al abrir una nota, el editor presenta el texto; otras partes del sistema permiten leer los datos del almacenamiento. El editor no es por sí solo todo el sistema operativo.

Error frecuente: llamar «sistema operativo» a la ventana de terminal. La terminal es solo una interfaz para interactuar con programas.

Tu ejercicio: menciona una aplicación que uses y una tarea del sistema que necesite para funcionar.

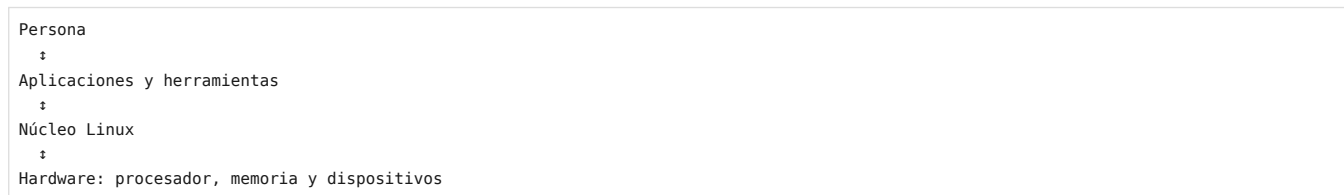
3. Qué es Linux y qué significa kernel

Kernel significa núcleo. Linux es un núcleo: participa en la gestión de procesador, memoria, procesos y dispositivos. Un proceso es un programa en ejecución. El núcleo ofrece mecanismos que utilizan los programas; no es el escritorio ni la terminal.

Para qué sirve: permite que varias aplicaciones compartan los recursos del equipo bajo las reglas del sistema.

Ejemplo: si escuchas audio y editas texto, hay varios procesos que necesitan tiempo de procesador. No necesitas conocer todavía el algoritmo que organiza esa ejecución.

Esquema conceptual simplificado:



Las flechas representan interacción entre capas. El esquema omite detalles como bibliotecas y controladores para empezar con lo esencial.

Error frecuente: creer que una versión del kernel identifica por sí sola a Ubuntu o Debian. Distintas distribuciones pueden usar núcleos relacionados y mantener cambios propios.

Tu ejercicio: explica con tus palabras por qué una aplicación y el kernel no son lo mismo. [Referencia conceptual de Debian](#).

4. Qué es GNU

GNU es un proyecto de software libre y un sistema operativo desarrollado mediante muchos componentes. Entre sus programas están Bash y las utilidades GNU Coreutils. Cuando un sistema combina componentes de GNU con el núcleo Linux se suele denominar **GNU/Linux**. En el uso cotidiano también se emplea «Linux» para hablar de distribuciones completas; aquí aclararemos cuándo nos referimos específicamente al núcleo. [Terminología del proyecto GNU](#).

Para qué sirve distinguirlos: la documentación del núcleo y la de un comando no responden necesariamente a la misma pregunta. Para estudiar una opción de `ls`, se consulta la documentación de la implementación de esa utilidad, por ejemplo GNU Coreutils.

Ejemplo: Bash interpreta órdenes; `ls` muestra entradas de directorio; Linux es el núcleo. Ninguno de esos nombres es sinónimo de los otros.

Error frecuente: creer que todos los sistemas que usan el núcleo Linux incluyen exactamente las mismas herramientas. Hay distintas combinaciones; este curso se centra en entornos GNU/Linux habituales.

Tu ejercicio: clasifica Linux como núcleo y Bash como herramienta de interacción. Explica qué relación tienen dentro de un sistema.

5. Qué es una distribución

Una distribución selecciona, integra y distribuye componentes: núcleo, herramientas, paquetes, configuraciones y mecanismos de actualización. Puede ofrecer instalación gráfica o funcionar sin escritorio.

Para qué sirve: evita tener que reunir y mantener por tu cuenta cada componente del sistema.

Debian, Ubuntu, Fedora y Red Hat Enterprise Linux (RHEL) son ejemplos de distribuciones. Comparten conceptos, pero difieren en decisiones de integración, versiones y administración. Se estudian en detalle en los módulos 15 y 16; hoy no instalarás paquetes.

Término	Qué identifica	Ejemplo
Núcleo	Una parte central del sistema	Linux
Distribución	Un sistema integrado y mantenido	Debian
Shell	Un intérprete de órdenes	Bash
Terminal	Una interfaz de entrada y salida de texto	La ventana donde escribes órdenes

Error frecuente: copiar una guía de otra distribución porque también dice «Linux». Primero se comprueba el sistema y después la compatibilidad de la instrucción.

Tu ejercicio: ¿por qué dos equipos con núcleo Linux podrían necesitar instrucciones diferentes para instalar un programa? No necesitas nombrar todavía ningún gestor.

6. Terminal y shell: solo lo necesario para empezar

La terminal muestra texto y recibe lo que escribes. La shell interpreta la orden. Bash es una shell, pero no la única. CLI significa interfaz de línea de comandos. En el Módulo 2 se profundizará en estas diferencias.

Una orden sencilla suele tener esta forma:

```
comando opción argumento
```

El comando indica la acción; una opción cambia cómo actúa; un argumento puede señalar el objeto sobre el que trabaja. No todas las órdenes necesitan las tres partes.

En los bloques siguientes copia únicamente la orden. No añadas los símbolos \$ o # que algunas guías muestran como indicador de entrada. Ejecuta una línea, lee la respuesta y solo después pasa a la siguiente. Las salidas y los mensajes pueden aparecer en otro idioma.

7. Práctica A — Identificar el entorno sin cambiar su configuración

Tipo: consulta. **Privilegios:** usuario normal. **Requisito:** terminal de Linux, no PowerShell. Las órdenes consultan información; no actualizan ni instalan software.

Paso 1. Nombre del núcleo

```
uname -s
```

- `uname` muestra información del sistema.
- `-s` pide el nombre del núcleo.
- En el entorno Linux previsto se espera `Linux`.

Si ves otra cosa, conserva el mensaje y revisa el entorno antes de seguir. La salida anterior es esperada, no una medición de tu computadora. [GNU: `uname`](#).

Paso 2. Versión de ejecución del núcleo

```
uname -r
```

- Se utiliza otra vez `uname`.
- `-r` solicita la identificación de la versión de ejecución del núcleo.
- Anota el resultado tal como aparece; puede incluir letras y sufijos del proveedor. No se espera un número fijo para aprobar.

Esto no indica por sí solo la versión de la distribución. [GNU: `uname`](#).

Paso 3. Identidad de la distribución

```
cat /etc/os-release
```

- `cat` muestra el contenido del archivo indicado.
- `/etc/os-release` es una ruta absoluta: empieza por `/`, la raíz del árbol de archivos.

- Busca `PRETTY_NAME`, un nombre legible del sistema; `ID` identifica la distribución y `VERSION_ID`, cuando existe, su versión.

No ejecutes como órdenes las líneas que aparecen en la salida. Solo léelas. En un contenedor, esos datos describen su entorno de usuarios; no bastan para deducir la distribución del equipo anfitrión. [Especificación oficial de os-release](#).

Si `/etc/os-release` no existe, consulta la ruta alternativa oficial `/usr/lib/os-release` (solo lectura). No crees ni descargues estos archivos: registra el resultado. La práctica deberá adaptarse al entorno. Su presencia tampoco prueba por sí sola que `systemd` esté actuando como gestor del sistema.

Ejercicio corto: indica qué orden usarías para conocer la distribución y cuál para consultar el núcleo. Explica por qué son dos consultas diferentes.

8. Práctica B — Preparar `~/linux-lab`

Tipo: navegación y creación de una carpeta. **Privilegios:** usuario normal. Esta práctica crea un directorio si no existe; no borra ni vacía contenidos. Aun así, lee cada paso antes de ejecutarlo.

Un directorio es una carpeta. `~` representa tu directorio personal en Bash cuando se usa como en los ejemplos. `~/linux-lab` será nuestra carpeta de ejercicios. No es un contenedor ni una protección automática contra errores.

Paso 1. Ir al directorio personal

```
cd ~
```

`cd` cambia el directorio de trabajo de la shell. `~` señala tu directorio personal. No mueve ni cambia archivos. Si falla, detente. No pegues los pasos restantes como un bloque automático.

Paso 2. Comprobar dónde estás

```
pwd
```

`pwd` muestra el directorio de trabajo actual. Una ruta ilustrativa es `/home/estudiante`; en tu equipo puede ser diferente. No copies esa ruta ilustrativa como si fuera la tuya.

Si estás trabajando como administrador del sistema o la ubicación no corresponde a tu usuario normal, pausa la práctica y revisa la sesión. No uses `sudo` para resolverlo.

Paso 3. Inspeccionar el contenido

```
ls
```

`ls` muestra las entradas no ocultas del directorio actual. Una salida vacía no implica un fallo. Los archivos ocultos se estudiarán después.

Si aparece `linux-lab`, inspecciona esa entrada antes de continuar:

```
ls -ld ~/linux-lab
```

`-l` solicita detalles y `-d` muestra la entrada de la carpeta en lugar de listar su interior. El primer carácter `d` identifica un directorio, `l` un enlace y `-` un archivo ordinario. Si es un enlace o no es un directorio, detente: no lo reemplaces. Si no comprendes la salida, pide ayuda antes de continuar. Los enlaces se explicarán en el Módulo 5.

Paso 4. Crear la carpeta de trabajo

```
mkdir -p ~/linux-lab
```

- `mkdir` crea directorios.
- `-p` permite crear componentes necesarios y no falla únicamente porque el directorio ya exista.
- `~/linux-lab` fija el destino en tu carpeta personal; no depende de escribir un nombre de usuario.

Si ya existe una carpeta normal con ese nombre, sus archivos se conservan. Si hay un archivo ordinario con ese nombre, la orden falla: no lo borres. Si hay un error de permisos, detente y revisa usuario y ruta; no añadas `sudo` automáticamente. [GNU: mkdir](#).

Paso 5. Entrar y verificar

Ejecuta una línea cada vez:

```
cd ~/linux-lab
pwd
ls
```

1. `cd ~/linux-lab` cambia a la carpeta del laboratorio. Si falla, no continúes.
2. `pwd` permite verificar la ubicación. Debe corresponder al laboratorio de tu usuario; si hay una diferencia inesperada, pausa y revísala.
3. `ls` permite observar su contenido. Puede estar vacío o conservar trabajos anteriores. No elimines nada para que coincida con un ejemplo.

Resultado esperado: puedes explicar dónde está el laboratorio y cómo comprobar que has entrado. No necesitas que esté vacío.

Error de comillas: no escribas `cd "~/linux-lab"` en Bash esperando la misma expansión. Las comillas impiden que ese `~` inicial se expanda. En este caso usamos `cd ~/linux-lab`. Más adelante aprenderás a citar variables de ruta según su contexto. [GNU Bash: expansión de tilde](#).

9. Errores frecuentes y corrección mínima

Situación	Causa posible	Qué hacer
command not found	Orden mal escrita o entorno diferente	Revisar la escritura y qué terminal se abrió; no instalar al azar
No such file or directory	Ruta ausente o mal escrita	Comparar la ruta con la lección; detener la secuencia
Permission denied	Usuario o ubicación inadecuados	Revisar sesión y destino; no anteponer sudo automáticamente
mkdir informa que existe un archivo	El nombre puede estar ocupado por un archivo	Conservarlo y revisar; no borrarlo para continuar
uname -r no muestra «Ubuntu»	Se consultó el núcleo	Usar la consulta de distribución, sin modificar nada
El laboratorio tiene archivos	Puede contener prácticas previas	Conservarlas; no es requisito dejarlo vacío

10. Práctica independiente

Primero responde sin volver a los ejemplos:

1. Explica núcleo, distribución y shell usando tus propias palabras.
2. Identifica qué consulta permite conocer tu distribución. Ejecútala solo si el entorno está preparado.
3. Entra al laboratorio y comprueba tu ubicación sin copiar la secuencia.
4. Detecta el problema en esta afirmación: «Si `cd` falla, puedo ejecutar igualmente el siguiente comando de modificación».
5. Explica por qué una carpeta de laboratorio no impide afectar otros archivos.

Pistas si te bloqueas: la orden que cambia de directorio empieza con `c`; la que muestra dónde estás empieza con `p`. Si no puedes continuar, vuelve al paso concreto y registra qué no comprendiste.

11. Mini evaluación

- ¿Linux y Bash son el mismo tipo de componente?
- ¿Conocer el núcleo basta para elegir instrucciones de administración para cualquier distribución?
- ¿`mkdir -p` vacía una carpeta existente?
- ¿Por qué debes leer el resultado de una orden antes de ejecutar la siguiente?

No se incluyen respuestas junto a las preguntas para que puedas intentarlo primero. La corrección debe señalar aciertos, explicar cada error y proponer una variante. En la siguiente sesión se repetirá una comprobación con otro ejemplo; una ejecución correcta hoy no certifica dominio.

12. Registro de aprendizaje

Puedes responder en el chat con esta ficha. No hace falta crear archivos adicionales ni aprender un editor todavía.

Entorno: Linux / máquina virtual / WSL / pendiente de identificar
Distribución y versión, si las pude consultar:
Nombre y versión del núcleo, si los pude consultar:
En mis palabras, un núcleo es:
En mis palabras, una distribución es:
En mis palabras, una shell es:
¿Pude entrar al laboratorio y comprobarlo?:
Un error que sé reconocer:
Qué necesito practicar otra vez:
Estado: EN APRENDIZAJE / PRACTICADO

Antes de compartir o guardar la ficha, omite nombres personales, rutas que identifiquen personas, nombres privados de equipos, direcciones de red y cualquier secreto. No copies automáticamente toda la salida de tu terminal. El tutor revisará la ficha antes de conservarla en GitHub con un nombre nuevo, sin sobrescribir ejercicios previos.

13. Fuentes y límites de esta lección

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Qué se conserva: todas las explicaciones, prácticas y preguntas originales de esta entrega.

Evaluación formativa: el estudiante debe explicar los conceptos con sus palabras, ejecutar una práctica segura en su propio entorno cuando corresponda, interpretar la salida y reconocer al menos un error sin copiar la solución. Una respuesta correcta aislada no acredita dominio.

Registro: EN APRENDIZAJE / PRACTICADO / DOMINADO (solo tras varias evidencias revisadas). La revisión documental del texto no equivale a práctica realizada.

Fuentes primarias para contrastar esta lección: - GNU Bash Reference Manual: <https://www.gnu.org/software/bash/manual/> - GNU Coreutils Manual: <https://www.gnu.org/software/coreutils/manual/> - Debian Reference: <https://www.debian.org/doc/manuals/debian-reference/> - Linux kernel documentation: <https://docs.kernel.org/>

Las fuentes se consultan según el tema tratado; no se presume que todos los enlaces respalden cada afirmación del módulo.

Estado de esta lección: revisada documentalmente. Las consultas específicas de Linux y la práctica en la distribución del estudiante deben validarse en ese entorno; no se presentan como ejecutadas en su equipo.

Módulo 2 — Terminal, CLI, shell, Bash, prompt y ayuda

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Segunda entrega.

1. Qué aprenderás

Al terminar podrás distinguir la ventana de terminal del programa que interpreta tus órdenes, reconocer dónde escribir y elegir una ayuda adecuada. No necesitas memorizar opciones ni dominar la navegación todavía.

Antes de ejecutar, explica con tus palabras la diferencia entre núcleo y distribución. Confirma que trabajas en una terminal de Linux y que el laboratorio del Módulo 1 está preparado. Si todavía no conoces tu entorno, puedes estudiar la parte conceptual y dejar pendiente la práctica.

Seguridad: usuario normal, sin sudo, instalaciones, borrados, cambios de permisos ni edición de configuración. Las órdenes previstas consultan información o muestran texto. La shell puede registrar lo que escribes en su historial: no introduzcas contraseñas, tokens ni otros secretos como práctica.

La lección está escrita para Bash. Windows Terminal puede alojar PowerShell o una distribución WSL: el nombre de la ventana no identifica el intérprete. No pegues estos ejemplos en PowerShell esperando el mismo comportamiento.

2. Terminal: la interfaz donde entra y sale texto

Una terminal permite enviar entrada y recibir salida textual de programas. En un escritorio, normalmente utilizas un **emulador de terminal**, una aplicación que ofrece esa interfaz en una ventana. También existen consolas de texto sin escritorio gráfico.

Para qué sirve: permite trabajar con órdenes y programas interactivos. Abrir la ventana no significa que hayas entrado al kernel ni que tengas privilegios de administrador.

Ejemplo conceptual: la ventana muestra tu orden, la salida y después otra invitación para escribir. La ventana y el programa que interpreta la orden cumplen funciones diferentes.

Error típico: «Cambié el color de la terminal, por tanto cambié de shell». El aspecto de la interfaz no determina el intérprete.

Ejercicio corto: describe qué parte ves en pantalla y qué parte crees que interpreta el texto.

3. CLI: una forma de interacción

CLI significa *Command Line Interface*, interfaz de línea de comandos. Describe una forma de dar instrucciones mediante texto. No es el nombre de una aplicación concreta ni de una distribución.

Para qué sirve: permite expresar acciones de manera precisa y reproducible. Una interfaz gráfica permite acciones mediante ventanas, botones y menús; ambas formas pueden coexistir.

Ejemplo: escribir una orden para consultar una versión es una interacción de CLI. Pulsar un botón «Acerca de» es una interacción gráfica. No hay que rechazar la interfaz gráfica para aprender Linux.

Error típico: creer que «CLI» significa «Bash». Bash permite interacción por línea de comandos, pero el concepto es más amplio.

Ejercicio corto: menciona una diferencia entre una acción escrita y una acción realizada con un botón.

4. Shell y Bash

Una **shell** interpreta un lenguaje de órdenes. Puede realizar acciones internas o ejecutar otros programas. **Bash** es una implementación de shell; existen otras, con diferencias de sintaxis y comportamiento.

Para qué sirve distinguirlas: una receta válida en Bash no tiene por qué funcionar sin cambios en todas las shells. Tampoco debe asumirse que /bin/sh es Bash. [GNU: qué es una shell](#).

Ejemplo: cd necesita cambiar el directorio de trabajo de la shell; Bash ofrece esa operación como orden interna. Un programa externo es un ejecutable separado que la shell puede iniciar. Más adelante estudiarás alias y funciones, que también pueden intervenir al resolver un nombre.

Concepto	Papel
Terminal	Facilita entrada y salida textual
CLI	Forma de interacción mediante órdenes
Shell	Interpreta órdenes
Bash	Una shell concreta
Kernel	Núcleo del sistema, presentado en el Módulo 1

Error típico: utilizar bash --version como prueba de que la shell actual es Bash. Esa orden ejecuta el Bash encontrado y consulta su versión; no identifica por sí sola el intérprete desde el que la lanzaste. Tampoco el contenido de la variable SHELL garantiza identificar la shell que está ejecutándose ahora.

Para esta práctica utiliza una sesión que sepas que está configurada con Bash. Si no está confirmado, pide ayuda para identificarla; no cambies la shell predeterminada ni edites archivos de inicio. [GNU: invocación de Bash](#).

Ejercicio corto: explica por qué una misma aplicación de terminal podría mostrar sesiones con shells diferentes.

5. Prompt: la invitación para escribir

El prompt es texto que muestra la shell para indicar que espera entrada. Este ejemplo es ilustrativo:

```
estudiante@equipo:~/linux-lab$
```

Puede mostrar una etiqueta de usuario, equipo y directorio. Su contenido es configurable; no todas las terminales muestran esta estructura. El símbolo `$` suele usarse en ejemplos de usuario normal y `#` en ejemplos de administrador. **No prueban por sí solos los privilegios de una sesión.**

En la línea ilustrativa siguiente, solo `pwd` es la orden:

```
estudiante@equipo:~/linux-lab$ pwd
```

Escribe únicamente:

```
pwd
```

`pwd` consulta el directorio de trabajo. No copies `estudiante@equipo`, el signo `$` ni una salida del ejemplo. Reconocerás esta orden del Módulo 1.

Bash diferencia el prompt principal de otro que solicita continuar una orden incompleta. Este último suele aparecer como `>`, por ejemplo cuando falta cerrar una comilla. Ese símbolo mostrado por Bash no es una orden que debas copiar. [GNU: comportamiento interactivo](#).

Error típico: interpretar cualquier `>` de un tutorial como parte de un comando. Su significado depende del contexto: prompt de continuación o carácter escrito en la orden. Las redirecciones se explican en el Módulo 8.

Ejercicio corto: en el ejemplo anterior, separa la invitación, la orden y la información que esperas recibir.

6. Primera interacción: escribir, ejecutar y leer

En Bash, ejecuta esta línea y pulsa Enter:

```
echo "Estoy aprendiendo a usar la terminal"
```

- `echo` muestra sus argumentos en la salida.
- Las comillas dobles agrupan este texto con espacios; no forman parte del mensaje mostrado.
- El ejemplo utiliza texto literal sencillo, sin opciones ni barras invertidas. No se presenta `echo` como herramienta universal para cualquier dato; más adelante se estudiará `printf`.

Salida esperada:

```
Estoy aprendiendo a usar la terminal
```

Después normalmente reaparece el prompt. No confundas la salida con una segunda orden. Esta salida es ilustrativa, no una captura de tu equipo. [GNU: órdenes internas](#).

Ejercicio corto: muestra otra frase sencilla. Antes de pulsar Enter, señala dónde comienza y termina el texto entre comillas.

Recuperarte de una línea incompleta

Si olvidaste cerrar una comilla y Bash espera más texto, pulsa **Ctrl+C** para cancelar esa entrada y volver a intentarlo. No añadas comandos nuevos a una línea cuyo estado no comprendes. No provoques el error con órdenes de modificación: basta observarlo si ocurre en esta práctica de texto.

Ctrl+C no deshace efectos ya producidos. Mientras corre un programa, normalmente solicita una interrupción y el programa puede reaccionar de distintas formas. Tampoco es equivalente a cerrar la ventana. Si no sabes qué se está ejecutando, lee primero la pantalla. [GNU: señales](#).

Ctrl+D puede terminar una shell si se pulsa con la línea vacía en la configuración habitual; no lo uses como sustituto de «borrar un carácter». En esta lección no necesitas cerrar la sesión para corregir una orden.

7. Ayuda: elegir el punto de consulta

No hace falta adivinar ni instalar software ante cada duda. Empieza por la documentación disponible en tu propio entorno. Estas tres vías se complementan:

Necesidad	Primera consulta	Condición
Una orden interna de Bash	<code>help cd</code>	Estar usando Bash

Necesidad	Primera consulta	Condición
Ayuda breve de GNU <code>ls</code>	<code>ls --help</code>	Tener esa implementación disponible
Página de manual instalada	<code>man ls</code>	Disponer de <code>man</code> y de esa página

`--help` no es una opción universal. No se añade automáticamente a cualquier orden. Si una ayuda falta, usa la referencia oficial enlazada, sin instalar nada todavía.

A. Ayuda integrada de Bash

```
help cd
```

`help` consulta ayuda integrada; `cd` es el tema. Leer esa ayuda no cambia de carpeta. Busca la descripción inicial y una opción, sin intentar memorizar todo.

```
type cd
```

`type` indica cómo se interpreta un nombre de orden; aquí examina `cd`, sin ejecutarlo. En una sesión Bash habitual dirá que es una orden interna. Si aparece un alias o función, registra la diferencia y consulta antes de asumir el comportamiento. [GNU: help y type](#).

B. Ayuda de una utilidad

```
ls --help
```

`ls` es la utilidad; `--help` solicita su ayuda en GNU Coreutils. Lee la descripción y localiza `-a`. No necesitas usarla todavía: el objetivo es encontrar información. El texto y el idioma pueden variar. [GNU: opciones comunes](#).

C. Página de manual

```
man ls
```

`man` busca documentación instalada; `ls` es el tema. No ejecuta un listado de archivos. Busca NAME (nombre), SYNOPSIS (forma de uso) y DESCRIPTION (descripción). Si el visualizador es `less`, Espacio avanza, /palabra busca y q sale; con otro visualizador las teclas pueden variar.

En un esquema como orden [opción]... archivo, los corchetes de la sinopsis señalan elementos opcionales y los puntos indican repetición. No copies esa notación literalmente. Esta convención documental no explica todas las apariciones de corchetes en código.

Si aparece «No manual entry» o falta `man`, registra el mensaje y utiliza la ayuda anterior o la documentación oficial. La ausencia de la página no demuestra que el programa esté ausente. [Manual de man-db](#).

Ejercicio corto: elige la ayuda para `cd`, lee su descripción y explica qué aprendiste. Después consulta `ls` mediante otra vía y compara la presentación.

8. Práctica guiada en el laboratorio

No se crea ni sobrescribe ningún archivo en esta práctica. El directorio debe existir desde el Módulo 1. Ejecuta una línea cada vez:

```
cd ~/linux-lab
pwd
```

1. `cd` cambia la ubicación de trabajo de la shell al laboratorio. Si falla, detente y vuelve a la preparación del Módulo 1.
2. `pwd` comprueba esa ubicación. No se espera una ruta idéntica entre personas.

Luego completa estas acciones por separado:

1. Reconoce el prompt y explica qué parte no debes copiar.
2. Muestra una frase con `echo`.
3. Consulta `help cd` y explica una línea que entiendas.
4. Consulta `type cd`; distingue consultar de ejecutar `cd`.
5. Lee `ls --help` y, si está disponible, `man ls`.
6. Sal del visualizador con su tecla correspondiente y comprueba que vuelve el prompt.

No publiques una captura completa: el prompt puede revelar nombres personales o de equipos. Basta describir el resultado sin esos datos.

9. Errores frecuentes y corrección mínima

Error o confusión	Por qué puede ocurrir	Corrección
Escribir <code>\$ pwd</code> copiando el prompt	Se confundió presentación con orden	Escribir solo <code>pwd</code>
<code>help</code> no se reconoce	Puede ser otra shell o entorno	Comprobar el entorno; no instalar un paquete al azar

Error o confusión	Por qué puede ocurrir	Corrección
Bash muestra > tras una frase incompleta	Puede faltar una comilla	Cancelar la entrada con Ctrl+C y escribir la frase completa
man ls no encuentra la página	Documentación no instalada o no localizada	Usar ls --help o la referencia oficial
Pulsar q en el prompt esperando salir del manual	El manual ya no estaba abierto	Distinguir pantalla del visualizador y prompt antes de pulsar teclas
Interpretar \$ como prueba de usuario normal	El prompt se puede personalizar	No deducir privilegios solo por su aspecto
Pensar que Ctrl+C restaura cambios anteriores	Interrumpir no es deshacer	Revisar el efecto de cada orden antes de ejecutarla

10. Práctica independiente y mini evaluación

1. Explica terminal, CLI, shell y Bash con tus palabras.
2. En una línea que contiene un prompt seguido de help cd, identifica qué escribirías realmente.
3. Elige cómo consultar una orden interna de Bash y cómo consultar una utilidad GNU.
4. Explica qué harías si man ls no encuentra una página.
5. Corrige esta afirmación: «El resultado de bash --version demuestra qué shell estoy usando ahora».
6. Muestra una frase nueva sin copiar el ejemplo y explica sus partes.

Pistas: una interfaz y un intérprete no cumplen el mismo papel; consultar una página no ejecuta el programa documentado. Antes de pedir la solución completa, señala qué parte de la pregunta no entiendes.

En otra sesión repite la distinción conceptual con un ejemplo diferente y detecta un error sin ayuda. Se considera **PRACTICADO** después de realizar la actividad; **DOMINADO** requiere evidencia repetida. La redacción terminada del módulo no significa que el estudiante ya lo domine.

11. Registro de aprendizaje

Puedes responder en el chat; no hace falta crear archivos:

```
En mis palabras, la terminal es:
En mis palabras, la shell es:
La diferencia entre CLI y Bash es:
Qué ayuda usé para cd y qué aprendí:
Qué parte del prompt no debo copiar:
Un error que reconocí y cómo lo corregí:
Qué necesito repetir:
Estado: EN APRENDIZAJE / PRACTICADO
```

No pegues capturas completas del prompt ni rutas personales sin revisarlas. La redacción del manual y el aprendizaje del estudiante se registran por separado.

12. Fuentes y límites de esta lección

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Revisión de esta entrega

Se revisaron progresión, sintaxis de ejemplos, comillas, coherencia con M1, advertencias y fuentes oficiales. Las referencias GNU se contrastaron mediante documentación oficial indexada; no se infiere una versión instalada a partir de ellas. La práctica interactiva en el Linux del estudiante sigue pendiente.

Decisiones de precisión: prompt configurable; help específico del entorno Bash; --help no universal; documentación ausente distinta de ejecutable ausente; interrupción distinta de deshacer; consulta de versión distinta de identificación de la shell actual.

Criterios de evaluación y fuentes

Qué se conserva: todas las explicaciones, prácticas y preguntas originales de esta entrega.

Evaluación formativa: el estudiante debe explicar los conceptos con sus palabras, ejecutar una práctica segura en su propio entorno cuando corresponda, interpretar la salida y reconocer al menos un error sin copiar la solución. Una respuesta correcta aislada no acredita dominio.

Registro: EN APRENDIZAJE / PRACTICADO / DOMINADO (solo tras varias evidencias revisadas). La revisión documental del texto no equivale a práctica realizada.

Fuentes primarias para contrastar esta lección: - GNU Bash Reference Manual: <https://www.gnu.org/software/bash/manual/> - GNU Coreutils Manual: <https://www.gnu.org/software/coreutils/manual/> - Debian Reference: <https://www.debian.org/doc/manuals/debian-reference/> - Linux kernel documentation: <https://docs.kernel.org/>

Las fuentes se consultan según el tema tratado; no se presume que todos los enlaces respalden cada afirmación del módulo.

Módulo 3 — Navegación inicial: pwd, ls y cd

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Tercera entrega.

1. Qué aprenderás

Aprenderás a responder tres preguntas: **¿dónde estoy?**, **¿qué hay aquí?**, **¿cómo cambio de carpeta?** Usarás una orden para cada pregunta y comprobarás el resultado antes de continuar.

Requisitos: distinguir prompt y orden, utilizar Bash en un entorno Linux preparado y tener el directorio normal `~/linux-lab` del Módulo 1. Si falta, vuelve a su preparación; no crees carpetas del sistema ni añadas `sudo` para resolver un error.

Diagnóstico breve: explica qué hace la shell y cómo consultarías ayuda sobre `cd`. Si no lo recuerdas, repasa el Módulo 2 antes de ejecutar la práctica.

Alcance: solo consultas y cambios del directorio de trabajo de la shell. No se crean, mueven, modifican ni borran archivos. La shell puede guardar las órdenes en su historial. No practiques con nombres o datos secretos. El laboratorio no es una barrera de aislamiento.

2. Directorio de trabajo: tu ubicación actual

Un directorio es una carpeta. La shell mantiene una ubicación de trabajo desde la cual interpreta muchas de las rutas que escribes. Cambiar esa ubicación no mueve físicamente los archivos.

Imagina este árbol ilustrativo, no una descripción de tu equipo:

```
directorio personal
└─ linux-lab
```

Si la shell está en `linux-lab`, esa es su ubicación de trabajo. Otra terminal puede estar en una carpeta diferente: cambiar de directorio en una sesión no cambia automáticamente las demás.

Para qué sirve: permite entender a qué carpeta te refieres cuando una orden no incluye una ruta completa.

Error típico: pensar que «estar en una carpeta» significa que todos los comandos solo pueden afectar esa carpeta. Una orden puede recibir otra ruta como argumento.

Ejercicio: explica la diferencia entre cambiar tu ubicación y mover una carpeta a otro lugar.

3. pwd: preguntar dónde estás

Escribe únicamente la orden y pulsa Enter:

```
pwd
```

`pwd` muestra la ruta del directorio de trabajo. No necesita un nombre de archivo. Una respuesta ilustrativa podría ser `/home/estudiante/linux-lab`; no tiene que coincidir con tu equipo.

Lee la ruta antes de actuar. No basta con reconocer la última palabra: comprueba que se refiere al laboratorio de tu usuario.

Error típico: ejecutar `pwd` y no mirar la salida. La comprobación útil consiste en interpretar el resultado, no solo en escribir la orden.

Ejercicio: consulta tu ubicación y explica qué información obtuviste. No publiques la ruta completa si contiene datos personales.

Nota de precisión: qué versión de pwd consultas

Bash tiene una orden interna `pwd`; también existe una utilidad externa de GNU con ese nombre. No se deben trasladar sus valores predeterminados sin comprobarlos. En Bash, `help pwd` explica la orden interna. Cuando hay enlaces simbólicos, una ruta lógica y una física pueden diferir. Se estudiará esa distinción después; si una ruta sorprende, pausa la práctica en lugar de asumir que es un error que requiere mover archivos. [GNU: pwd](#).

4. ls: observar sin cambiar de carpeta

```
ls
```

`ls` lista las entradas no ocultas del directorio de trabajo con su comportamiento habitual. Puede mostrar nombres en columnas. Una salida vacía no demuestra que la carpeta no contenga entradas ocultas.

Para qué sirve: orientarte antes de decidir qué quieres consultar. No abre cada archivo ni entra en las carpetas que muestra. El color depende de la configuración; no lo uses como única prueba del tipo de objeto.

Ejercicio: consulta el contenido del laboratorio. Explica qué puedes concluir si no aparece ningún nombre y qué todavía no puedes concluir.

Mostrar entradas ocultas

```
ls -a
```

`ls` sigue siendo la orden; `-a` es una opción que incluye nombres que comienzan por punto. También muestra normalmente `.` y `..`: representan el directorio actual y su padre. No se han creado archivos por añadir esta opción. [GNU: selección de entradas](#).

Error típico: creer que oculto equivale a protegido. Un nombre que empieza por punto afecta su presentación habitual, no establece por sí solo permisos de seguridad.

Ejercicio: compara `ls` y `ls -a`. Si solo aparecen `.` y `..`, explica por qué eso no es un fallo.

Pedir detalles

```
ls -l
```

La letra de `-l` es una *ele* minúscula, no el número uno. Pide un listado largo con datos de cada entrada. Hoy solo necesitas reconocer el primer carácter: `d` señala un directorio, `-` un archivo ordinario y `l` un enlace simbólico. Los permisos y demás campos se estudiarán después.

```
ls -la
```

Aquí se combinan las opciones anteriores: detalles e inclusión de entradas ocultas. No memorices todas las columnas ni supongas que el tamaño mostrado para una carpeta es la suma del contenido de sus archivos. [GNU: información del listado](#).

Ejercicio: identifica una entrada de directorio sin depender del color. No hace falta abrirla ni cambiar permisos.

Consultar otra carpeta sin entrar

```
ls ~/linux-lab
```

`ls` consulta y `~/linux-lab` indica el destino de la consulta. Esa ruta del Módulo 1 utiliza `~` para tu directorio personal. Ejecuta `pwd` después: verás que listar otra carpeta no ha cambiado tu ubicación.

Si quieres inspeccionar la entrada de la carpeta misma:

```
ls -ld ~/linux-lab
```

`-l` pide detalles y `-d` muestra el directorio como entrada, en vez de listar su interior. Si aparece como enlace, detente antes de usarlo como laboratorio y revisa su destino con ayuda. Esta orden ya se presentó en M1.

5. cd: cambiar tu ubicación de trabajo

```
cd ~/linux-lab
```

`cd` cambia el directorio de trabajo de la shell; `~/linux-lab` indica a dónde quieres ir. Si tiene éxito, normalmente no muestra texto. Comprueba la ubicación con `pwd`: silencio no es una comprobación suficiente.

No escribe dentro de los archivos del directorio ni los mueve. Si falla, no asumas que llegaste al destino: normalmente sigues en la ubicación anterior. Lee el error y detén la secuencia.

Volver al directorio personal

```
cd ~
```

Este uso de `~` ya se explicó en M1. No es la raíz `/`: es el directorio personal de tu usuario. No encierres literalmente `~` entre comillas esperando la misma expansión.

Subir un nivel

```
cd ..
```

Hay un espacio entre `cd` y los dos puntos. `..` indica el padre en la navegación habitual. Desde un laboratorio normal dentro de tu directorio personal, vuelve al personal. En la raíz no hay un nivel superior al que seguir subiendo. Las rutas y los enlaces se explicarán con más detalle en los módulos 4 y 5.

Regresar a la ubicación anterior

```
cd -
```

El guion solicita a Bash volver al directorio anterior registrado. Normalmente también muestra la ruta al cambiar. Si todavía no existe ese registro, puede aparecer un error sobre `OLDPWD`. No es una lista de todas las carpetas visitadas ni significa subir un nivel.

Ejercicio: explica la diferencia entre volver al directorio personal, subir al padre y volver al anterior. Puedes apoyarte en un dibujo de dos carpetas. [GNU Bash: cd y directorios de trabajo](#).

6. Práctica guiada: predecir, ejecutar, comprobar

Realiza cada paso por separado. Antes de pulsar Enter, di qué esperas que ocurra. Ante un error, detente. No pegues toda la tabla como si fuera un script.

Paso	Orden	Explicación y comprobación
1	<code>cd ~</code>	Ir al directorio personal conocido
2	<code>pwd</code>	Confirmar dónde estás antes de continuar
3	<code>ls -ld ~/linux-lab</code>	Revisar que el laboratorio existente es un directorio normal
4	<code>ls ~/linux-lab</code>	Ver su contenido sin entrar
5	<code>pwd</code>	Confirmar que el paso 4 no cambió la ubicación
6	<code>cd ~/linux-lab</code>	Entrar; detenerse si falla
7	<code>pwd</code>	Confirmar que estás en el laboratorio
8	<code>ls -a</code>	Observar también las entradas ocultas
9	<code>cd ..</code>	Volver al padre del laboratorio
10	<code>pwd</code>	Comprobar el retorno esperado al directorio personal
11	<code>cd -</code>	Regresar al laboratorio, que acaba de ser la ubicación anterior
12	<code>pwd</code>	Comprobar el destino final

Si el directorio personal o el laboratorio incluyen enlaces y las rutas no coinciden con tus expectativas, no intentes «arreglar» el equipo. Conserva la observación y pide ayuda. La práctica presupone el laboratorio normal preparado en M1.

Resultado esperado: has realizado consultas y navegación, sin crear ni borrar materiales de práctica. No se exige que tu carpeta esté vacía ni que la salida coincida palabra por palabra con otra persona.

7. Errores frecuentes: entender antes de corregir

Situación	Explicación	Corrección mínima
Escribes <code>cd ..</code>	Falta el espacio que separa la orden del argumento	Escribir <code>cd ..</code> y después comprobar con <code>pwd</code>
Escribes <code>CD</code> esperando <code>cd</code>	En el entorno habitual, los nombres de órdenes distinguen mayúsculas	Usar la escritura documentada
Escribes <code>ls -l</code> esperando detalles	El uno pide una entrada por línea en GNU <code>ls</code> ; no es la ele	Usar <code>ls -l</code> para el formato largo
«No such file or directory»	No se encontró la ruta indicada	Revisar escritura y existencia; no crear ni borrar para ocultar el error
«Not a directory»	El destino puede ser un archivo	Inspeccionar la entrada; <code>cd</code> requiere un directorio
«Permission denied»	No tienes el acceso necesario	Detenerse; no usar <code>sudo</code> ni cambiar permisos en esta lección
Esperabas que <code>ls</code> carpeta entrara en ella	Listar y navegar son operaciones distintas	Usar <code>cd</code> solo después de comprobar el destino
<code>cd -</code> vuelve a un lugar inesperado	Anterior no es lo mismo que padre	Reconstruir los cambios recientes y consultar <code>pwd</code>

Si tu sesión tiene alias, funciones o configuraciones de navegación personalizadas, puede diferir de los ejemplos. Puedes usar `type ls` o `type cd`, aprendidos en M2, para pedir información antes de continuar. No modifiques esa configuración como parte del ejercicio.

8. Práctica independiente

Sin copiar la tabla anterior:

1. Consulta dónde estás.
2. Lista el laboratorio sin cambiar de carpeta y demuestra que no cambió tu ubicación.
3. Entra en él y verifica el resultado.
4. Muestra también sus entradas ocultas.
5. Vuelve al directorio personal y regresa a la ubicación anterior.
6. Explica en qué momentos comprobarías un fallo antes de continuar.

Si necesitas una pista: una orden informa, otra lista y otra cambia la ubicación. Si recurres a la tabla, anota qué paso te costó y vuelve a intentarlo más tarde con otro punto de partida conocido.

9. Mini evaluación y corrección de errores

- ¿Qué cambia después de un `cd` correcto y qué no cambia?
- ¿Una salida vacía de `ls` demuestra que no hay entradas ocultas?
- ¿Por qué `ls -l` y `ls -L` no son intercambiables?
- ¿Qué diferencia existe entre `cd ..` y `cd -`?
- ¿Qué harías si `cd ~/linux-lab` devuelve un error?
- Detecta el problema en esta explicación: «Ya ejecuté `ls ~/linux-lab`, así que cualquier orden posterior se ejecutará desde el laboratorio».

Responde antes de pedir soluciones. El tutor señalará primero tus aciertos, explicará cada error y propondrá una variante. En otra sesión, repite la navegación sin copiar y explica una situación distinta. Una sola respuesta correcta no acredita dominio.

10. Registro de aprendizaje

Puedes responder en el chat; no hace falta crear archivos:

```
La diferencia entre pwd, ls y cd es:  
Cómo comprobé que listar no cambió mi ubicación:  
Qué significa ...:  
Qué significa el guion en cd -:  
Un error que reconocí y cómo lo corregí:  
Qué necesito repetir:  
Estado: EN APRENDIZAJE / PRACTICADO
```

No pegues nombres de usuario, rutas personales, archivos privados ni capturas completas sin revisarlos. La redacción del manual y el aprendizaje del estudiante se registran por separado.

11. Fuentes y límites de esta lección

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Revisión de esta entrega

Esta entrega tiene revisión documental de sintaxis, orden pedagógico, fuentes, errores y alcance. Se contrastaron referencias oficiales de GNU mediante resultados indexados. No se afirma haber ejecutado la práctica en la distribución del estudiante. La comparación lógica/física y las reglas completas de rutas quedan para módulos posteriores.

Las referencias están enlazadas junto a sus explicaciones: GNU Coreutils para `ls` y la utilidad externa `pwd`; GNU Bash para las órdenes internas. Coreutils 9.11 sigue siendo la referencia editorial, sin exigir esa versión instalada.

Criterios de evaluación y fuentes

Qué se conserva: todas las explicaciones, prácticas y preguntas originales de esta entrega.

Evaluación formativa: el estudiante debe explicar los conceptos con sus palabras, ejecutar una práctica segura en su propio entorno cuando corresponda, interpretar la salida y reconocer al menos un error sin copiar la solución. Una respuesta correcta aislada no acredita dominio.

Registro: EN APRENDIZAJE / PRACTICADO / DOMINADO (solo tras varias evidencias revisadas). La revisión documental del texto no equivale a práctica realizada.

Fuentes primarias para contrastar esta lección: - GNU Bash Reference Manual: <https://www.gnu.org/software/bash/manual/> - GNU Coreutils Manual: <https://www.gnu.org/software/coreutils/manual/> - Debian Reference: <https://www.debian.org/doc/manuals/debian-reference/> - Linux kernel documentation: <https://docs.kernel.org/>

Las fuentes se consultan según el tema tratado; no se presume que todos los enlaces respalden cada afirmación del módulo.

Módulo 4 — Rutas absolutas y relativas; ~, ., .. y nombres con espacios

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Cuarta entrega.

1. Qué aprenderás

Podrás predecir qué ubicación señala una ruta antes de usarla, distinguir raíz y directorio personal, navegar entre carpetas y conservar como un solo argumento un nombre que contiene espacios.

Antes de empezar, explica para qué sirven `pwd`, `ls` y `cd`. Después indica por qué debes detenerte si falla `cd`. Si necesitas consultar la respuesta, repasa M3; no hace falta memorizar más opciones todavía.

Entorno previsto: Bash sobre Linux, usuario normal y laboratorio `~/linux-lab` preparado. No se supone que tu directorio personal esté necesariamente en `/home/estudiante`.

Seguridad: la práctica creará únicamente tres carpetas de ejercicios si sus nombres están disponibles. No borra ni sobrescribe archivos ni modifica permisos. Ejecuta una orden cada vez y revisa su resultado. No uses `sudo` para solucionar errores de esta lección.

2. Una ruta indica cómo localizar un objeto

Una ruta es una secuencia de nombres separados por `/`. Puede señalar un archivo o un directorio. La barra separa componentes; no es un carácter que pueda formar parte de un único nombre de archivo en Linux.

Ejemplo conceptual:

```
/home/estudiante/linux-lab
```

Este ejemplo empieza en la raíz `/`, pasa por `home`, después por `estudiante` y finalmente por `linux-lab`. No lo copies como si fuera tu ruta real.

Para qué sirve: muchos comandos necesitan saber qué objeto consultar o modificar. Escribir una ruta no crea automáticamente el objeto que nombra ni demuestra que exista.

Error típico: pensar que toda ruta termina en un archivo. La del ejemplo termina en un directorio.

Ejercicio: separa los componentes del ejemplo y explica qué representa la primera barra.

3. Rutas absolutas: comenzar en la raíz

Una ruta absoluta empieza por `/`. Su interpretación parte de la raíz del entorno de archivos del proceso, no de la carpeta de trabajo actual. [Linux: resolución de rutas](#).

Ejemplo de consulta ya visto:

```
ls /etc
```

`ls` lista y `/etc` identifica el directorio consultado. Esta orden no te cambia de carpeta ni modifica la configuración de `/etc`. En esta lección no necesitas ejecutarla: sirve para reconocer la forma de una ruta absoluta.

Error típico: llamar «absoluta» a una ruta solo porque es larga. La diferencia es dónde empieza a resolverse, no su longitud.

Ejercicio: ¿`/tmp` es absoluta? ¿Y carpeta/subcarpeta/archivo.txt? Justifica la respuesta sin contar caracteres.

Una ruta absoluta tampoco garantiza seguridad: puede apuntar fuera del laboratorio. En un contenedor o entorno aislado, `/` puede representar una vista distinta de la del sistema anfitrión.

4. Rutas relativas: comenzar desde una base

Para las consultas ordinarias de esta lección, una ruta relativa se interpreta desde tu directorio de trabajo. Por eso necesitas conocer primero la salida de `pwd`.

Ejemplo conceptual: si estás en `/home/estudiante`, `linux-lab` se refiere a una carpeta dentro de esa ubicación. Si ya estás en `/home/estudiante/linux-lab`, el mismo texto buscaría otra carpeta llamada `linux-lab` dentro de ella.

```
Ubicación actual: /home/estudiante
Ruta relativa:   linux-lab
Destino:         /home/estudiante/linux-lab
```

Esto es un razonamiento sobre una estructura sencilla sin enlaces; no una salida de tu equipo.

Para qué sirve: permite trabajar con carpetas cercanas sin repetir toda la ruta. **Error típico:** reutilizar una ruta relativa después de cambiar de ubicación y asumir que conserva el destino.

Ejercicio: si estás en el laboratorio y escribes `ls apuntes`, ¿en qué carpeta se buscará apuntes?

Nota de precisión: Bash puede tener configurada una búsqueda especial para `cd` mediante `CDPATH`. Usaremos `./nombre` cuando queramos dejar explícito que el destino empieza en la carpeta actual. No cambies tu configuración para hacer esta práctica. [GNU Bash: cd](#).

5. Punto y doble punto

Dentro de una ruta, `.` representa el directorio actual y `..` el padre. Para este primer recorrido se usan directorios normales sin enlaces simbólicos.

```
ls .
```

`ls` consulta y `.` señala la carpeta actual. Es una forma explícita de indicar el lugar que ya se listaba con `ls` sin ruta.

```
cd ./apuntes
```

`cd` cambia de carpeta; `./apuntes` empieza en la ubicación actual y busca un directorio llamado `apuntes`. Solo ejecútalo cuando esa carpeta exista en la práctica.

```
cd ..
```

En el recorrido normal, sube al padre. No equivale a `cd -`, que vuelve al directorio anterior registrado. Tampoco hay que cambiarlo por tres puntos: `...` no es una abreviatura especial para subir dos niveles.

Ejercicio: partiendo de una carpeta `apuntes` que está dentro del laboratorio, explica qué señalarían `.` y `...`.

No simplifiques automáticamente toda ruta como si fuera una suma: con enlaces simbólicos, navegación lógica y física pueden producir diferencias. En M5 se introducirá su interpretación. Si el recorrido real no coincide con lo esperado, pausa y revisa.

6. Tilde: el directorio personal

En los ejemplos de Bash, un `~` inicial sin citar se expande al directorio personal. Así, `~/linux-lab` llega a tu laboratorio aunque estés trabajando en otra carpeta.

No confundas estos tres elementos:

Escritura	Significado en estos ejemplos
<code>/</code>	Raíz del árbol de archivos
<code>~</code>	Abreviatura que Bash expande al directorio personal
<code>.</code>	Directorio de trabajo actual

`~/linux-lab` es una expresión de la shell: después de su expansión habitual resulta una ruta absoluta. No es una ruta relativa ordinaria por el mero hecho de no empezar visualmente con `/`. Otras aplicaciones no tienen por qué expandir `~`. [GNU Bash: expansión de tilde](#).

Error típico: escribir `cd "~/linux-lab"` esperando que Bash expanda la tilde. Al estar citada, se conserva literalmente. En nuestros ejemplos se usa `cd ~/linux-lab`.

Ejercicio: explica por qué `/`, `~` y `.` pueden señalar lugares diferentes en una misma sesión.

7. Nombres con espacios: agrupar un argumento

Un espacio sin proteger separa palabras de la orden. Si un nombre es `mis notas`, debes indicar que sus dos palabras forman un solo argumento.

Ejemplo de consulta, para cuando exista esa carpeta:

```
ls "mis notas"
```

- `ls` es la orden.
- `"mis notas"` se transmite como un argumento con un espacio interno.
- Las comillas de agrupación no se convierten en parte del nombre.

Para este nombre literal sencillo también sirve `ls 'mis notas'`. Las comillas simples y dobles no son equivalentes en todos los contextos: las dobles permiten ciertas expansiones que las simples impiden. Hoy usa nombres literales sencillos; la regla completa se estudiará en M24. [GNU Bash: comillas y comillas dobles](#).

Combinar tilde y espacios

```
ls ~/linux-lab/modulo-04-rutas/"mis notas"
```

La tilde inicial queda sin citar y se expande. El segmento con espacios queda entre comillas. Como no hay separadores sin proteger entre los segmentos, Bash construye un único argumento de ruta.

Error típico: `ls mis notas` consulta dos nombres, `mis` y `notas`; no equivale al nombre `mis notas`. Que la orden no falle no demuestra que haya consultado el destino que querías.

Ejercicio: explica cuántos argumentos de ruta recibe `ls` en cada una de las dos escrituras anteriores. No necesitas ejecutar el ejemplo incorrecto.

8. Preparación guiada del recorrido

Esta sección crea carpetas; lee el efecto antes de ejecutar. No se crearán archivos dentro de ellas. Si un nombre ya está ocupado, no elimines ni renombres lo existente para continuar.

Paso A. Entrar y comprobar

Ejecuta una línea cada vez:

```
cd ~/linux-lab
pwd
ls -la
```

La primera entra al laboratorio; si falla, detente. La segunda comprueba la ubicación. La tercera permite inspeccionar también entradas ocultas. Reutiliza los conocimientos de M3 y confirma que el laboratorio es el directorio normal previsto.

Paso B. Crear un directorio nuevo de práctica

Si no aparece una entrada llamada `modulo-04-rutas`, ejecuta:

```
mkdir modulo-04-rutas
```

`mkdir`, presentado en M1, crea una carpeta; aquí el nombre es relativo a la ubicación comprobada. No usamos `-p`: si el nombre ya existe, queremos revisar la situación. Un fallo no autoriza a borrar lo que lo ocupa.

Si ya existía, inspecciónalo con `ls -ld ./modulo-04-rutas`. Si no es un directorio normal, pausa. Si lo es, revisa su contenido con `ls -la ./modulo-04-rutas` y conserva todo: puedes reutilizarlo solo después de entender qué contiene.

Paso C. Entrar y preparar dos destinos

```
cd ./modulo-04-rutas
pwd
ls -la
```

Cada línea sirve para entrar, confirmar e inspeccionar. Si los siguientes nombres están disponibles, créalos por separado:

```
mkdir apuntes
mkdir "mis notas"
```

La primera crea apuntes; la segunda crea una sola carpeta cuyo nombre contiene un espacio. Si alguna ya existe, inspecciónala antes de decidir reutilizarla. Ante enlaces, archivos ordinarios o dudas, detente. No se exige que las carpetas estén vacías.

Árbol esperado, si acabas de crearlas:

```
linux-lab/
├── modulo-04-rutas/
│   ├── apuntes/
│   └── mis notas/
```

9. Navegación: anticipar el destino

Empieza dentro de `modulo-04-rutas`, comprobado con `pwd`. Realiza una orden por vez; si falla, no continúes.

Paso	Orden	Qué debes explicar
1	<code>cd ./apuntes</code>	Entrar en un hijo del directorio actual
2	<code>pwd</code>	Confirmar el destino real
3	<code>ls ..</code>	Listar el padre sin cambiar de ubicación
4	<code>cd ../"mis notas"</code>	Subir al padre y entrar en el otro directorio
5	<code>pwd</code>	Ver una ruta cuyo nombre final contiene un espacio
6	<code>cd .</code>	Permanecer en la ubicación actual
7	<code>pwd</code>	Comprobar que no se cambió de lugar
8	<code>cd ~/linux-lab</code>	Volver al laboratorio mediante la abreviatura del directorio personal
9	<code>pwd</code>	Confirmar dónde terminó el recorrido

Ahora compara una ruta relativa y una absoluta al mismo destino. Dentro de modulo-04-rutas, ls ./apuntes consulta el hijo. Para escribir la alternativa absoluta, toma la ruta real mostrada allí por pwd, añade /apuntes y encierra **toda esa ruta real** entre comillas dobles si contiene espacios. No uses literalmente la palabra «ruta» ni la dirección ilustrativa de otra persona.

Si no estás seguro de haberla construido bien, muéstrala al tutor con los datos personales sustituidos y deja la consulta pendiente. El propósito es entender el recorrido, no acertar por ensayo y error.

Al terminar, deja las carpetas creadas tal como están. No hay un paso de limpieza con borrado.

10. Errores frecuentes y corrección mínima

Problema	Causa posible	Corrección
cd "~/linux-lab" no llega al laboratorio	La tilde citada no se expande	Usar la forma del ejemplo con ~ sin citar
cd mis notas falla	Se han entregado palabras separadas	Agrupar el nombre: cd "mis notas", desde el padre correcto
Una ruta relativa encuentra otro destino	Cambió el directorio de trabajo	Consultar pwd y reconstruir la ruta
Se utilizó /linux-lab por ~/linux-lab	Raíz y directorio personal se confundieron	Revisar el punto de partida; no crear carpetas en /
mkdir informa que existe una entrada	El nombre está ocupado	Inspeccionar y conservar; no borrar para repetir la práctica
Aparece «Permission denied»	Falta acceso al destino	Detenerse y revisar ubicación, sin sudo ni cambios de permisos

Un mensaje de error no implica por sí solo que debas reinstalar herramientas. Tampoco una orden exitosa demuestra que la ruta elegida era la correcta: comprueba intención y resultado.

11. Práctica independiente y mini evaluación

Sin copiar la tabla, parte del laboratorio, entra a apuntes, consulta el padre y navega a mis notas. Regresa al laboratorio y explica qué parte del recorrido fue relativa y cuál utilizó expansión de tilde. Comprueba cada cambio con pwd.

Responde después:

1. ¿Qué diferencia fundamental existe entre una ruta absoluta y una relativa?
2. Si estás en modulo-04-rutas/apuntes, ¿qué señala ../"mis notas" en nuestro árbol?
3. ¿Por qué escribir comillas no crea un nombre diferente en el ejemplo correcto?
4. ¿Es ~ una abreviatura del directorio actual o del personal?
5. ¿Por qué no puedes sustituir cd .. por cd - en todos los casos?
6. ¿Qué harías si el nombre de práctica estuviera ocupado por un archivo?

Pista si te bloqueas: dibuja el árbol, marca dónde estás y recorre los componentes de izquierda a derecha. No se considera dominado por un único intento. En otra sesión repite con un punto de partida diferente, explica el razonamiento y detecta un error por tu cuenta.

12. Registro de aprendizaje

Puedes enviar una respuesta breve al chat: qué entendiste de las rutas, un ejemplo explicado, un error reconocido y qué necesitas repetir. No pegues rutas personales o nombres privados sin revisarlos. La ficha del estudiante se conservará cuando exista y haya sido revisada; no se inventan respuestas ni avances.

13. Fuentes y límites de esta lección

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Revisión de esta entrega

Revisión documental: sintaxis de ejemplos, separación de argumentos, expansión de tilde, destinos del árbol y coherencia con M1-M3. Las referencias GNU se consultaron mediante documentación oficial indexada; la resolución de rutas se contrastó con Linux man-pages. La práctica en el Linux del estudiante sigue pendiente. No se afirma validación de ejecuciones reales.

Criterios de evaluación y fuentes

Qué se conserva: todas las explicaciones, prácticas y preguntas originales de esta entrega.

Evaluación formativa: el estudiante debe explicar los conceptos con sus palabras, ejecutar una práctica segura en su propio entorno cuando corresponda, interpretar la salida y reconocer al menos un error sin copiar la solución. Una respuesta correcta aislada no acredita dominio.

Registro: EN APRENDIZAJE / PRACTICADO / DOMINADO (solo tras varias evidencias revisadas). La revisión documental del texto no equivale a práctica realizada.

Fuentes primarias para contrastar esta lección: - GNU Bash Reference Manual: <https://www.gnu.org/software/bash/manual/> - GNU Coreutils Manual: <https://www.gnu.org/software/coreutils/manual/> - Debian Reference: <https://www.debian.org/doc/manuals/debian-reference/> - Linux kernel documentation: <https://docs.kernel.org/>

Las fuentes se consultan según el tema tratado; no se presume que todos los enlaces respalden cada afirmación del módulo.

Módulo 5 — Árbol de archivos, FHS, /etc, /usr, /var, /tmp, /proc, /sys y enlaces

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Quinta entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué significa que Linux presente un árbol de archivos con raíz /;
- distinguir la función general de /etc, /usr, /var, /tmp, /proc y /sys;
- reconocer que una distribución puede organizar algunos directorios de forma distinta sin dejar de ser GNU/Linux;
- diferenciar un directorio real de una vista virtual proporcionada por el kernel;
- reconocer un enlace simbólico y explicar que puede apuntar a otro nombre o ruta;
- inspeccionar estos elementos sin modificar la configuración del sistema.

Conocimientos previos: Módulo 3 (pwd, ls, cd) y Módulo 4 (rutas absolutas, relativas, ~, ., .. y espacios).

Seguridad: este módulo es principalmente de lectura. No editarás archivos de /etc, no escribirás en /proc ni /sys, no borrarás nada y no usarás sudo. Algunas entradas de /proc y /sys pueden ser escribibles y modificar el sistema; por eso aquí solo se consultan rutas y metadatos.

2. Un solo árbol que empieza en /

En GNU/Linux, los archivos y directorios se presentan dentro de un árbol jerárquico. La parte superior se llama **raíz** y se representa con:

```
/
```

No confundas:

- / = raíz del árbol;
- ~ = abreviatura de Bash para el directorio personal;
- . = directorio actual.

Ejemplo conceptual simplificado:

```
/
├─ etc/
├─ home/
├─ tmp/
├─ usr/
├─ var/
├─ proc/
└─ sys/
```

El esquema no muestra todos los directorios posibles ni garantiza que tu sistema tenga exactamente esa presentación. Una distribución, un contenedor o un entorno especializado puede diferir.

Para qué sirve este modelo: cuando una orden usa una ruta como /etc/os-release, sabes que comienza desde la raíz y no desde tu carpeta actual.

Error típico: pensar que / significa “mi carpeta personal”. El directorio personal suele estar en otra ubicación y se representa cómodamente con ~ en Bash.

Ejercicio: explica con tus palabras la diferencia entre / y ~.

3. Qué es FHS

FHS significa *Filesystem Hierarchy Standard*. Es una especificación que describe propósitos y ubicaciones convencionales dentro de jerarquías de sistemas tipo Unix/Linux.

La versión publicada de referencia de la serie 3 es **FHS 3.0**, publicada en 2015. Sigue siendo útil como mapa conceptual, pero no debes interpretarla como una fotografía exacta de todas las distribuciones actuales.

La FHS describe, entre otros, directorios como:

- /etc: configuración específica del sistema;
- /usr: jerarquía secundaria con programas y datos de usuario del sistema;
- /var: datos variables;
- /tmp: archivos temporales.

También contempla que ciertos nombres de la raíz puedan ser enlaces a otros directorios.

Regla del manual: aprenderemos primero el propósito general y después comprobaremos cómo lo implementa la distribución real. No memorizaremos una lista completa como si fuera universal.

Referencia: [Filesystem Hierarchy Standard 3.0](#).

4. /etc — configuración del sistema

La FHS define /etc como ubicación para **configuración específica del sistema anfitrión**.

Ejemplos que ya has visto:

```
/etc/os-release
/etc/passwd
/etc/group
```

Eso no significa que debas editar esos archivos.

Consulta segura

Puedes observar los nombres del directorio con:

```
ls /etc
```

Esta orden lista entradas; no modifica la configuración.

Para volver a consultar la identidad de la distribución:

```
cat /etc/os-release
```

Ya estudiaste esta lectura en el Módulo 1.

Error típico: pensar que, porque un archivo está en /etc, debes abrirlo con privilegios de administrador. Leer y modificar son operaciones distintas. En este módulo no hacemos cambios.

Ejercicio: ¿por qué /etc/os-release es más apropiado para identificar la distribución que para conocer únicamente la versión del kernel?

5. /usr — programas y datos de la jerarquía de usuario del sistema

En FHS, /usr es una jerarquía secundaria que contiene gran parte de los programas, bibliotecas y datos distribuidos con el sistema.

Subdirectorios frecuentes:

```
/usr/bin
/usr/lib
/usr/share
```

No confundas /usr con el directorio personal de un usuario. El nombre es histórico; no significa “tu carpeta de usuario”.

En sistemas modernos puede existir un diseño llamado **merged-usr**, donde nombres históricos como /bin o /sbin son enlaces hacia ubicaciones dentro de /usr. No asumas que todos los sistemas tienen exactamente la misma forma: primero inspecciona.

Consulta segura

```
ls -ld /usr
ls -ld /bin
```

- -l muestra detalles.
- -d muestra la entrada nombrada en vez de listar su interior.
- Si el primer carácter es d, observas un directorio.
- Si es l, observas un enlace simbólico.

No importa si /bin es directorio o enlace en tu equipo; el objetivo es reconocer el resultado sin cambiar nada.

6. /var — datos que cambian durante el funcionamiento

/var contiene **datos variables**: información cuyo contenido puede cambiar mientras el sistema funciona.

Dependiendo de la distribución y los servicios instalados, puedes encontrar zonas relacionadas con:

- registros;
- cachés;
- colas;
- estado de aplicaciones;
- datos de servicios.

Ejemplos comunes:

```
/var/log
/var/cache
```

No todas las distribuciones ni aplicaciones almacenan exactamente lo mismo en cada subdirectorio.

Consulta segura

```
ls -ld /var
ls /var
```

No necesitas abrir registros privados ni copiar su contenido al chat. Más adelante, en el módulo de logs, aprenderás a consultar eventos sin exponer datos sensibles.

Error típico: pensar que “variable” significa una variable de programación. Aquí significa que son datos que pueden cambiar con el uso del sistema.

7. /tmp — archivos temporales

/tmp es una ubicación convencional para archivos temporales.

No debes utilizarla para guardar algo irremplazable. La política de limpieza depende del sistema y de su configuración: algunos entornos eliminan contenido al reiniciar o después de cierto tiempo; otros aplican reglas distintas.

Consulta segura

```
ls -ld /tmp
```

En muchos sistemas verás permisos especiales (por ejemplo, una t al final, como en drwxrwxrwt), pero todavía no necesitas interpretarlos. Los permisos básicos se estudian en el Módulo 12; el **sticky bit**, que explica esa t, queda fuera del núcleo inicial y se pospone.

Regla: temporal no significa “seguro para guardar secretos”. Un archivo temporal debe diseñarse con cuidado. Más adelante aprenderás mktemp.

Error típico: asumir que todo lo almacenado en /tmp se elimina inmediatamente o siempre en cada reinicio. No existe esa garantía universal.

8. /proc — una vista del kernel y de los procesos

/proc normalmente es un **pseudo-sistema de archivos** proporcionado por el kernel. No debe imaginarse simplemente como una carpeta ordinaria llena de documentos almacenados en disco.

Incluye, entre otras cosas:

- información del sistema;
- datos asociados a procesos;
- subdirectorios cuyo nombre puede ser un PID;
- parámetros del kernel bajo determinadas rutas.

Ejemplo:

```
ls -ld /proc
ls /proc
```

La segunda salida puede cambiar mientras el sistema está funcionando porque los procesos aparecen y desaparecen.

Otra consulta de lectura:

```
cat /proc/version
```

La salida depende del kernel y de cómo fue construido. No necesitas memorizarla.

Advertencia importante: algunas rutas bajo /proc/sys permiten modificar parámetros del kernel. En este curso no escribirás ahí durante los módulos iniciales.

Referencia: [Linux Kernel Documentation — The /proc Filesystem](#).

Error típico: creer que cada entrada de /proc corresponde a un archivo permanente guardado en el disco.

9. /sys — objetos y dispositivos expuestos por el kernel

/sys suele ser el punto de montaje de **sysfs**, una interfaz que expone objetos del kernel, dispositivos y atributos relacionados.

Puedes inspeccionar su estructura:

```
ls -ld /sys
ls /sys
```

Es común encontrar categorías como:

```
/sys/class
/sys/devices
/sys/bus
```

Su presencia exacta depende del entorno.

Advertencia: algunas entradas de sysfs son escribibles y pueden cambiar el comportamiento de dispositivos o del sistema. En este módulo solo observamos nombres y rutas.

Error típico: pensar que todo lo visible como “archivo” dentro de /sys es un archivo de texto normal almacenado en el disco.

Referencia general: [Linux Kernel Documentation — sysfs](#).

10. Directorio real, pseudo-sistema de archivos y punto de montaje

Ya puedes separar tres ideas:

1. Un directorio como ~/linux-lab pertenece al árbol normal de tu espacio de archivos.
2. /proc y /sys suelen presentar información dinámica mediante sistemas de archivos virtuales.
3. Un directorio puede funcionar como **punto de montaje**, donde se integra otro sistema de archivos dentro del árbol.

No aprenderás todavía a montar discos. Esa operación puede afectar el acceso a datos y se reserva para el Módulo 34.

Ejercicio: explica por qué /proc puede cambiar aunque no hayas creado ni borrado archivos manualmente.

11. Qué es un enlace simbólico

Un **enlace simbólico** (*symbolic link* o *symlink*) es un tipo especial de archivo que contiene una ruta hacia otro nombre.

Modelo mental:

```
enlace → destino
```

No es una copia del destino.

Un enlace simbólico puede:

- apuntar a un archivo;
- apuntar a un directorio;
- cruzar entre sistemas de archivos;
- quedar “colgando” si su destino ya no existe.

Referencia: [Linux man-pages — symlink\(7\)](#).

Reconocer un enlace

```
ls -l nombre
```

Una salida ilustrativa puede verse así:

```
lrwxrwxrwx ... acceso -> destino
```

Lo importante ahora:

- el primer carácter l indica enlace simbólico;
- la flecha muestra el texto del destino en una salida típica de ls -l.

No copies la salida ilustrativa como si fuera la de tu equipo.

12. Enlace simbólico y enlace duro: diferencia inicial

Linux también tiene **enlaces duros** (*hard links*), pero no los practicaremos todavía.

Idea mínima:

- un enlace simbólico apunta mediante un nombre o ruta;
- un enlace duro es otra entrada de directorio que referencia el mismo objeto subyacente dentro de las restricciones del sistema de archivos.

Un enlace duro ordinario no cruza sistemas de archivos. Los enlaces simbólicos sí pueden hacerlo.

No necesitas aprender inodos todavía. Solo recuerda que “enlace simbólico” y “enlace duro” no son sinónimos.

13. Práctica guiada — reconocer y crear un symlink dentro del laboratorio

Esta práctica modifica únicamente tu carpeta de ejercicios. No utiliza sudo, no borra nada y no toca /etc, /proc ni /sys.

Paso A. Entrar al laboratorio

Ejecuta una línea cada vez:

```
cd ~/linux-lab
pwd
ls -la
```

Si cd falla, detente.

Paso B. Crear la carpeta del módulo

Comprueba primero si existe:

```
ls -ld ./modulo-05-fhs
```

Si responde que no existe, crea únicamente esa carpeta:

```
mkdir modulo-05-fhs
```

Si ya existe, inspecciónala. No borres ni sobrescribas su contenido para repetir la práctica.

Después:

```
cd ./modulo-05-fhs
pwd
ls -la
```

Paso C. Crear un destino de práctica

Si el nombre está libre:

```
mkdir destino
```

Comprueba:

```
ls -ld destino
```

El primer carácter debería corresponder a un directorio.

Paso D. Crear el enlace simbólico

Solo si no existe una entrada llamada acceso-destino:

```
ln -s destino acceso-destino
```

Explicación:

- ln crea enlaces;
- -s solicita un enlace simbólico;
- destino es el texto del objetivo;
- acceso-destino es el nombre nuevo del enlace.

Ahora inspecciona:

```
ls -ld destino acceso-destino
```

Debes poder distinguir cuál es el directorio y cuál es el enlace.

Paso E. Comprobar que el enlace se puede recorrer

```
cd acceso-destino
pwd
```

Bash puede conservar una ruta lógica que incluya el nombre del enlace. Para observar la ruta física resuelta:

```
pwd -P
```

No necesitas memorizar -P todavía. Solo observa que un enlace puede cambiar cómo se expresa el camino lógico y físico.

Regresa al laboratorio:

```
cd ~/linux-lab
pwd
```

No borres la práctica al terminar. La conservaremos para ejercicios posteriores.

14. Recorrido de inspección del sistema

Ahora harás únicamente consultas.

Ejecuta una línea, observa y explica antes de pasar a la siguiente:

```
ls -ld /etc
ls -ld /usr
ls -ld /var
ls -ld /tmp
ls -ld /proc
ls -ld /sys
ls -ld /bin
```

Preguntas:

- 1. ¿Cuáles aparecen como directorios?
- 2. ¿Alguno aparece como enlace simbólico?
- 3. ¿Puedes saber solo con esos resultados qué distribución usas? No necesariamente.
- 4. ¿Cuál de esas rutas no usarías para guardar tus ejercicios personales? Todas; tus ejercicios siguen en ~/linux-lab.

No ejecutes sudo, no cambies permisos y no intentes “arreglar” un enlace del sistema.

15. Errores frecuentes y corrección mínima

Situación	Problema	Corrección
“/usr es mi carpeta personal”	Confusión por el nombre	El directorio personal se consulta con ~ o pwd; /usr cumple otra función
“Todo lo de /proc está guardado en disco”	Se ignora que procfs es virtual	Tratar /proc como interfaz dinámica del kernel
“Puedo escribir en /sys porque parece una carpeta”	Apariencia no implica operación segura	En módulos iniciales, solo lectura
“/tmp siempre se borra al reiniciar”	Generalización dependiente de política	No guardar ahí información irremplazable
“Un symlink es una copia”	El enlace apunta a otro nombre	Verificar el destino; no asumir duplicación de datos
“Si /bin es un enlace, Linux está dañado”	Sistemas modernos pueden usar merged-/usr	Inspeccionar y documentar; no modificar
“FHS obliga a que todos los sistemas sean idénticos”	FHS es un estándar de jerarquía, no una clonación exacta	Consultar distribución y versión reales

16. Práctica independiente

Sin copiar los párrafos anteriores, responde:

- 1. ¿Qué tipo de información esperarías encontrar en /etc?
- 2. ¿Por qué /var recibe ese nombre?
- 3. ¿Qué precaución debes tener con /tmp?
- 4. ¿Qué diferencia conceptual existe entre ~/linux-lab y /proc?
- 5. ¿Qué indica el primer carácter l en una salida larga de ls?
- 6. ¿Por qué un enlace simbólico puede quedar colgando?
- 7. Si /bin aparece como enlace, ¿debes reemplazarlo por un directorio? Explica por qué no.

Después dibuja un árbol pequeño con:

```
/
├─ etc
├─ usr
├─ var
├─ tmp
├─ proc
└─ sys
```

y escribe una frase al lado de cada nombre indicando su función general.

17. Mini evaluación

1. ¿Cuál es la raíz del árbol de archivos?
 - A. ~
 - B. /
 - C. .
 - D. /home
2. ¿Qué ruta se asocia principalmente con configuración específica del sistema?
 - A. /etc
 - B. /tmp
 - C. /proc
 - D. /sys
3. ¿Qué afirmación sobre /proc es correcta?
 - A. Es siempre una carpeta ordinaria de archivos permanentes.
 - B. Suele ser una interfaz virtual hacia información del kernel y procesos.
 - C. Es el directorio personal del administrador.
 - D. Solo contiene programas ejecutables.
4. ¿Un enlace simbólico es una copia completa del destino?
 - A. Sí.
 - B. No.
5. ¿FHS garantiza que todas las distribuciones tengan una estructura idéntica en cada detalle?
 - A. Sí.
 - B. No.
6. ¿Debemos escribir valores en /proc/sys o /sys durante esta lección?
 - A. Sí.
 - B. No.

No revises las respuestas hasta haber razonado cada punto. El tutor corregirá primero lo acertado y después cada error.

18. Registro de aprendizaje

Puedes responder en el chat:

```
En mis palabras, la raíz / es:
Para mí, /etc sirve principalmente para:
Para mí, /usr sirve principalmente para:
Para mí, /var sirve principalmente para:
Para mí, /tmp sirve principalmente para:
La diferencia entre /proc y una carpeta normal es:
Un enlace simbólico es:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

No pegues contenidos sensibles de /etc, /proc o /sys. Si una salida contiene nombres privados del equipo, usuarios, rutas personales o datos de red, redáctalos antes de compartirla.

19. Fuentes y límites de esta lección

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- [Filesystem Hierarchy Standard 3.0](#).
- [Linux Kernel Documentation — /proc](#).
- [Linux Kernel Documentation — sysfs](#).
- [Linux man-pages — symlink\(7\)](#).
- [Linux man-pages — path_resolution\(7\)](#).

FHS 3.0 es una referencia estructural y no se usa para afirmar que cada distribución actual sea idéntica. Las prácticas concretas deben comprobarse en la distribución real del estudiante.

Estado de la lección: redactada y revisada documentalmente. La práctica en el equipo del estudiante todavía debe ejecutarse y evaluarse.

Módulo 6 — Crear, copiar, mover, renombrar y borrar con seguridad

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Sexta entrega.

1. Qué aprenderás

Al terminar este módulo podrás crear archivos y directorios de práctica, copiar archivos, moverlos o renombrarlos y distinguir correctamente `rm` de `rmdir`.

También aprenderás una regla esencial: **antes de borrar, primero comprueba ubicación, objetivo y tipo de objeto**.

Conocimientos previos: - `pwd`, `ls`, `cd`; - rutas absolutas y relativas; - laboratorio `~/linux-lab`; - reconocimiento básico de archivos, directorios y enlaces.

Seguridad: en este módulo todas las operaciones se limitan a una carpeta nueva dentro de `~/linux-lab`. No se utiliza `sudo`. No se usa `rm -rf`. No se modifica `/etc`, `/usr`, `/var`, `/proc` ni `/sys`.

Cómo leer este módulo: las secciones 2 a 14 explican cada orden con ejemplos. No las ejecutes todavía; la práctica empieza en la sección 15, dentro de la carpeta del laboratorio.

2. Crear un archivo con `touch`

`touch` se utiliza principalmente para modificar marcas de tiempo de archivos. Si el archivo indicado no existe, normalmente crea un archivo vacío.

Ejemplo:

```
touch notas.txt
```

Si `notas.txt` no existe, se crea un archivo vacío.

Comprueba:

```
ls -l notas.txt
```

Importante: si el archivo ya existe, `touch notas.txt` no lo vacía ni reemplaza su contenido; actualiza marcas de tiempo según el comportamiento normal de `touch`.

Error típico: pensar que `touch` siempre “crea desde cero”. Si el archivo ya existe, el efecto es diferente.

Referencia: GNU Coreutils, `touch`.

3. Crear directorios con `mkdir`

Ya utilizaste `mkdir`.

Ejemplo:

```
mkdir documentos
```

Esto crea un directorio llamado `documentos` en la ubicación actual.

Comprueba:

```
ls -ld documentos
```

Si el nombre ya existe, `mkdir` normalmente informa un error. En este curso ese error es útil: evita que sobrescribas o confundas una práctica existente.

Opción `-p`

```
mkdir -p proyecto/notas
```

`-p` permite crear directorios padre necesarios y no falla únicamente porque ya exista la jerarquía prevista.

No uses `-p` automáticamente cuando quieras detectar nombres ya ocupados.

4. Copiar con `cp`

`cp` copia archivos y, con opciones apropiadas, directorios.

Ejemplo:

```
cp original.txt copia.txt
```

Aquí: - original.txt es la fuente; - copia.txt es el destino.

La copia es independiente del archivo original.

Comprueba:

```
ls -l original.txt copia.txt
```

Referencia: GNU Coreutils, cp.

5. Riesgo de sobrescritura con cp

Si el destino ya existe, cp puede reemplazar su contenido dependiendo de la operación y opciones utilizadas.

Por eso este manual no enseña a copiar “a ciegas”.

Antes:

```
ls -l
```

Después confirma que el destino que planeas usar no contiene trabajo que quieras conservar.

En prácticas iniciales puedes usar:

```
cp -i original.txt copia.txt
```

-i solicita confirmación antes de sobrescribir un archivo destino existente.

Esto ayuda a aprender, pero **no sustituye comprender qué estás copiando**.

6. Copiar directorios

Por defecto, cp no copia directorios de forma recursiva.

Para copiar un directorio y su contenido se utiliza una opción recursiva, por ejemplo:

```
cp -R carpeta-origen carpeta-copia
```

En este módulo solo la usarás dentro del laboratorio.

No necesitas aprender todavía todas las diferencias entre -R, -r y -a. Más adelante veremos preservación de metadatos y copias más completas.

7. Mover y renombrar con mv

mv sirve para mover o renombrar archivos y directorios.

Renombrar:

```
mv borrador.txt notas.txt
```

Mover a otra carpeta:

```
mv notas.txt documentos/
```

En el primer caso cambia el nombre. En el segundo cambia la ubicación.

Referencia: GNU Coreutils, mv.

8. Diferencia entre cp y mv

Modelo mental:

```
cp = conservar original + crear copia  
mv = cambiar nombre o ubicación del objeto
```

Ejemplo:

```
cp uno.txt dos.txt
```

Después pueden existir ambos.

En cambio:

```
mv uno.txt dos.txt
```

el nombre uno.txt deja de representar el objeto original en esa ubicación si la operación termina correctamente.

Error típico: usar mv esperando conservar una copia.

9. Riesgo de sobrescritura con mv

mv también puede reemplazar un destino existente en determinadas circunstancias.

Para prácticas iniciales:

```
mv -i origen.txt destino.txt
```

-i puede pedir confirmación antes de reemplazar.

Pero primero debes inspeccionar:

```
pwd
ls -la
```

No conviertas -i en una excusa para dejar de verificar las rutas.

10. Borrar archivos con rm

rm elimina archivos.

Ejemplo:

```
rm archivo.txt
```

A diferencia de una interfaz gráfica, rm no debe asumirse como “mover a la papelera”.

Antes de usarlo en este curso:

```
pwd
ls -l archivo.txt
```

Después decide si ese archivo es realmente el objetivo.

Opción interactiva

Durante el aprendizaje puedes usar:

```
rm -i archivo.txt
```

-i pregunta antes de eliminar cada archivo.

Referencia: GNU Coreutils, rm.

11. rmdir: solo directorios vacíos

rmdir elimina directorios vacíos.

Ejemplo:

```
rmdir carpeta-vacia
```

Si la carpeta contiene archivos o subdirectorios, la operación falla.

Ese fallo es útil para un principiante porque evita eliminar contenido accidentalmente.

Referencia: GNU Coreutils, rmdir.

12. rm y rmdir no son equivalentes

Recuerda:

```
rm archivo.txt
rmdir carpeta-vacia
```

No enseñaremos todavía borrado recursivo como práctica rutinaria.

GNU `rm` dispone de opciones recursivas como `-r` o `-R`, pero implican un alcance mucho mayor porque pueden eliminar árboles completos de directorios.

13. Por qué no usamos `rm -rf` como hábito

`rm -rf` combina dos comportamientos de alto impacto:

- `-r`: eliminación recursiva;
- `-f`: fuerza la operación y elimina varias confirmaciones.

Eso puede borrar muchos archivos si la ruta es incorrecta.

Por eso:

- no aparece como comando normal de principiante;
- no se usa para “limpiar” ejercicios;
- no se añade `sudo`;
- no se prueba sobre directorios del sistema.

Más adelante podrás estudiar qué significa técnicamente, pero no necesitas ejecutarlo para aprender Linux correctamente.

14. Los tres controles antes de borrar

Antes de una eliminación revisa:

Control 1 — ubicación

```
pwd
```

¿Estás donde crees?

Control 2 — objetivo

```
ls -ld nombre
```

¿El nombre existe y es el objeto esperado?

Control 3 — alcance

Pregunta:

¿Voy a eliminar un archivo, un directorio vacío o algo que contiene más elementos?

Si la respuesta no está clara, no ejecutes la eliminación.

`pwd` y `ls` ayudan, pero no son una garantía automática: también debes interpretar la ruta y el tipo de objeto.

15. Práctica guiada — preparar el laboratorio

Entra al laboratorio:

```
cd ~/linux-lab
pwd
ls -la
```

Si `cd` falla, detente.

Comprueba si existe:

```
ls -ld ./modulo-06-archivos
```

Si informa que no existe:

```
mkdir modulo-06-archivos
```

Después:

```
cd ./modulo-06-archivos
pwd
ls -la
```

No borres una carpeta previa para repetir la práctica. Si existe, inspecciona primero su contenido.

16. Crear archivos de práctica

Crea dos archivos vacíos:

```
touch original.txt
touch temporal.txt
```

Comprueba:

```
ls -l
```

Debes identificar ambos archivos.

17. Copiar

Ejecuta:

```
cp original.txt copia.txt
```

Después:

```
ls -l
```

Responde:

- ¿sigue existiendo original.txt?
- ¿aparece copia.txt?

La respuesta esperada es sí a ambas si la copia terminó correctamente.

18. Renombrar

Ahora:

```
mv copia.txt respaldo.txt
```

Comprueba:

```
ls -l
```

Debes poder explicar que: - copia.txt ya no aparece con ese nombre; - respaldo.txt representa el objeto movido/renombrado.

19. Crear y mover a un directorio

Crea:

```
mkdir documentos
```

Después mueve:

```
mv respaldo.txt documentos/
```

Comprueba:

```
ls -l
ls -l documentos
```

Explica la diferencia entre la ubicación anterior y la nueva.

20. Borrado controlado de un archivo de práctica

Solo eliminaremos temporal.txt, que se creó específicamente para este ejercicio.

Primero:

```
pwd
ls -l temporal.txt
```

Si ambas comprobaciones coinciden con la práctica:

```
rm -i temporal.txt
```

Lee la pregunta que muestre `rm` y confirma solo si el nombre coincide exactamente.

Después:

```
ls -l
```

No elimines `original.txt` ni la carpeta `documentos`.

21. Práctica de `rmdir`

Crea una carpeta vacía:

```
mkdir vacia
```

Comprueba:

```
ls -ld vacia
```

Después:

```
rmdir vacia
```

Comprueba otra vez:

```
ls -ld vacia
```

Ahora debería aparecer un mensaje indicando que ya no existe.

Ese mensaje, después de una eliminación deliberada y verificada, puede ser el resultado esperado.

22. Provocar un error seguro con `rmdir`

Crea:

```
mkdir no-vacia  
touch no-vacia/nota.txt
```

Intenta:

```
rmdir no-vacia
```

Debe fallar porque contiene `nota.txt`.

No corrijas el error borrando el archivo.

Déjala así para recordar qué protege `rmdir`.

23. Errores frecuentes

Error	Por qué ocurre	Corrección mínima
<code>cp</code> deja dos archivos y creías que movería	<code>cp</code> copia	Usa <code>mv</code> solo cuando quieras mover/renombrar
<code>mv</code> hace desaparecer el nombre original	Se movió o renombró	Comprueba el destino antes de repetir
<code>rmdir</code> falla en una carpeta	No está vacía	Inspecciona; no borres contenido automáticamente
<code>rm</code> elimina el archivo equivocado	Ruta o nombre incorrecto	Verifica <code>pwd</code> , objetivo y alcance
Aparece "Permission denied"	No tienes permiso	No añadas <code>sudo</code> ; revisa la ubicación
Un nombre empieza con <code>-</code>	Puede interpretarse como opción	No improvises; aprende después el uso de <code>--</code> o rutas explícitas
Quieres usar <code>rm -rf</code> para limpiar rápido	Estás aumentando demasiado el alcance	Conserva la práctica o elimina de forma controlada más adelante

24. Práctica independiente

Sin copiar la secuencia guiada:

- Dentro de `~/linux-lab`, crea una carpeta nueva para una segunda práctica.
- Crea `archivo1.txt`.

3. Cópialo como `archivo2.txt`.
4. Renombra la copia como `respaldo.txt`.
5. Crea un directorio guardados.
6. Mueve `respaldo.txt` dentro.
7. Comprueba cada paso con `ls`.
8. Explica antes de borrar qué objeto podrías eliminar de forma segura.
9. No uses `rm -rf`.

Si algo falla, conserva el estado y explica el error; no reinicies borrando todo.

25. Mini evaluación

1. ¿Qué hace normalmente `cp`?
 - A. Copia.
 - B. Borra.
 - C. Cambia permisos.
 - D. Reinicia.
2. ¿Qué hace `mv`?
 - A. Solo copia.
 - B. Mueve o renombra.
 - C. Solo lista.
 - D. Cambia usuario.
3. ¿Qué elimina `rmdir`?
 - A. Cualquier árbol de archivos.
 - B. Directorios vacíos.
 - C. Procesos.
 - D. Paquetes.
4. ¿`rm` debe suponerse equivalente a una papelera gráfica?
 - A. Sí.
 - B. No.
5. ¿Cuál es el orden correcto antes de borrar?
 - A. Borrar y después comprobar.
 - B. Comprobar ubicación, objetivo y alcance.
 - C. Añadir `sudo`.
 - D. Usar siempre `-f`.
6. ¿Por qué no usamos `rm -rf` como práctica rutinaria?
 - A. Porque no existe.
 - B. Porque puede eliminar recursivamente con pocas protecciones si la ruta es incorrecta.
 - C. Porque solo funciona en Windows.
 - D. Porque crea archivos.

26. Registro de aprendizaje

Puedes responder:

```
cp sirve para:
mv sirve para:
rm sirve para:
rmdir sirve para:
Antes de borrar debo comprobar:
Un error que provoqué de forma segura:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

No se considera dominado porque una práctica salga una vez. En otra sesión deberás realizar un ejercicio equivalente sin copiar y detectar un error por tu cuenta.

27. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Coreutils — `cp`: https://www.gnu.org/software/coreutils/manual/html_node/cp-invocation.html
- GNU Coreutils — `mv`: https://www.gnu.org/software/coreutils/manual/html_node/mv-invocation.html
- GNU Coreutils — `rm`: https://www.gnu.org/software/coreutils/manual/html_node/rm-invocation.html
- GNU Coreutils — `rmdir`: https://www.gnu.org/software/coreutils/manual/html_node/rmdir-invocation.html
- GNU Coreutils — `touch`: https://www.gnu.org/software/coreutils/manual/html_node/touch-invocation.html

Esta lección usa una selección pequeña de opciones. No intenta cubrir todas las posibilidades de GNU Coreutils.

Vigencia (E05): la lección se redactó con GNU Coreutils 9.11. El 9 de octubre de 2026 el manual oficial en línea documenta la versión 9.12. Las opciones usadas aquí son básicas, pero tu sistema puede tener otra versión: compruébala con `cp --version`.

Estado de la lección: redactada y revisada documentalmente. La práctica real del estudiante sigue pendiente.

Módulo 7 — Leer archivos con cat, less, head y tail

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Séptima entrega.

1. Qué aprenderás

Al terminar este módulo podrás elegir una herramienta adecuada para leer archivos de texto sin modificarlos:

- `cat` para mostrar o concatenar contenido;
- `less` para navegar por archivos largos de forma interactiva;
- `head` para ver las primeras líneas;
- `tail` para ver las últimas líneas;
- `tail -f` para seguir contenido que va creciendo, entendiendo sus límites.

También aprenderás a distinguir **leer de editar**, a reconocer cuándo un archivo es demasiado grande para mostrarlo completo y a evitar copiar información sensible al chat o a un repositorio.

Conocimientos previos: - navegación con `pwd`, `ls` y `cd`; - rutas; - creación de archivos dentro de `~/linux-lab`; - operaciones básicas con archivos.

Seguridad: este módulo se centra en lectura. No modificarás archivos del sistema. No uses `sudo` para abrir contenido que normalmente no puedes leer. Si un archivo contiene información privada, credenciales, tokens, nombres personales o datos de red, no lo pegues completo en el chat ni en GitHub.

2. Leer no es editar

Una orden de lectura obtiene contenido sin que su objetivo principal sea modificar el archivo.

Ejemplo:

```
cat notas.txt
```

Esto muestra el contenido de `notas.txt`.

No es lo mismo que abrir un editor y cambiar el archivo.

Modelo mental:

```
leer      = observar contenido
editar    = modificar contenido
borrar     = eliminar contenido u objeto
```

Error típico: pensar que cualquier comando que “abre” un archivo puede modificarlo. Aquí las herramientas se usan en modo de lectura.

3. cat — mostrar y concatenar

`cat` pertenece a GNU Coreutils en los entornos GNU habituales.

Su nombre proviene de **concatenate**: concatenar.

Ejemplo simple:

```
cat notas.txt
```

Si el archivo contiene:

```
uno
dos
tres
```

la salida esperada será:

```
uno
dos
tres
```

¿Para qué sirve cat?

En este curso lo usaremos inicialmente para:

- mostrar archivos pequeños;
- comprobar rápidamente contenido;
- concatenar varios archivos hacia la salida estándar.

Ejemplo conceptual:

```
cat parte1.txt parte2.txt
```

cat muestra primero el contenido del primer archivo y luego el del segundo.

Importante: en este ejemplo no se crea automáticamente un archivo nuevo. La redirección para guardar salidas se estudiará en el Módulo 8.

Referencia: GNU Coreutils — cat.

4. Cuándo NO conviene usar cat

Si un archivo tiene miles de líneas, cat archivo-grande.txt puede llenar rápidamente la terminal.

No suele dañar el archivo, pero puede hacer difícil revisar el contenido.

En ese caso es mejor un paginador como less.

Regla inicial:

```
archivo pequeño → cat puede ser cómodo
archivo largo   → less suele ser más práctico
```

No necesitas calcular un número exacto de líneas. Elige según la facilidad de lectura.

5. less — navegar sin cargar toda la experiencia visual de golpe

less es un paginador interactivo. Permite revisar contenido por partes.

Ejemplo:

```
less notas.txt
```

Al abrirlo, la terminal entra en una vista interactiva.

Controles iniciales:

- Space o Page Down: avanzar;
- b: retroceder;
- flechas: desplazamiento;
- / seguido de texto: buscar hacia adelante;
- n: repetir la búsqueda;
- q: salir.

No memorices todos los atajos. Para empezar basta con:

```
Space = avanzar
/      = buscar
q      = salir
```

Error típico: creer que quedaste “atrapado” en less. Pulsa q para salir.

6. less no es un editor

Aunque puedes moverte, buscar y examinar texto, el uso básico de less en este módulo no está destinado a editar el archivo.

Por eso es útil para inspeccionar:

- documentación;
- archivos de texto largos;
- salidas guardadas;
- ciertos registros, cuando tienes permiso.

No uses less como excusa para abrir archivos sensibles que no necesitas revisar.

7. head — primeras líneas

head muestra el principio de un archivo.

Ejemplo:

```
head notas.txt
```

Por defecto, GNU head muestra las primeras 10 líneas.

Si quieres una cantidad concreta:

```
head -n 5 notas.txt
```

Explicación:

- head = muestra el comienzo;
- -n = indica que especificarás una cantidad de líneas;
- 5 = número solicitado;
- notas.txt = archivo.

Resultado: como máximo verás las primeras cinco líneas disponibles.

Referencia: GNU Coreutils — head.

8. tail — últimas líneas

tail muestra el final de un archivo.

Ejemplo:

```
tail notas.txt
```

Por defecto, GNU tail muestra las últimas 10 líneas.

Para una cantidad concreta:

```
tail -n 5 notas.txt
```

Esto muestra las últimas cinco líneas.

Uso típico: revisar la parte más reciente de un archivo cuyo contenido se añade al final.

Referencia: GNU Coreutils — tail.

9. Diferencia entre head y tail

Modelo mental:

```
head → principio  
tail → final
```

Si un archivo tiene:

```
línea 1  
línea 2  
línea 3  
línea 4  
línea 5
```

entonces:

```
head -n 2 archivo.txt
```

mostraría:

```
línea 1  
línea 2
```

y:

```
tail -n 2 archivo.txt
```

mostraría:

```
línea 4  
línea 5
```

Ejercicio mental: sin ejecutar nada, explica qué mostraría head -n 1 y qué mostraría tail -n 1.

10. tail -f — seguir un archivo que crece

tail -f intenta continuar mostrando contenido nuevo cuando el archivo aumenta.

Ejemplo:

```
tail -f actividad.log
```

La terminal puede permanecer esperando cambios.

Para terminar normalmente:

```
Ctrl + C
```

Esto envía una interrupción al proceso de tail que está ejecutándose en primer plano.

¿Para qué sirve?

Es útil para observar contenido que una aplicación va añadiendo, por ejemplo en un archivo de registro dentro de un laboratorio propio.

Límite importante

tail -f no significa “monitorizar cualquier cosa automáticamente”. Sigue el archivo según su comportamiento y opciones. Rotación de logs, reemplazos del archivo o permisos pueden cambiar lo que ves.

No lo uses todavía sobre registros sensibles del sistema. Primero practicaremos con un archivo creado por nosotros.

11. No confundas salida con contenido almacenado

Cuando ejecutas:

```
cat notas.txt
```

el texto aparece en la terminal.

Eso no significa que el archivo haya sido copiado a otro lugar.

Más adelante aprenderás que la salida estándar puede:

- mostrarse en pantalla;
- enviarse a otro comando;
- redirigirse a un archivo.

Ese será el tema de los Módulos 8 y 9.

12. Preparar la práctica

Entra al laboratorio:

```
cd ~/linux-lab
pwd
ls -la
```

Si cd falla, detente.

Comprueba si existe:

```
ls -ld ./modulo-07-lectura
```

Si no existe:

```
mkdir modulo-07-lectura
```

Después:

```
cd ./modulo-07-lectura
pwd
ls -la
```

No borres una práctica anterior si ya existe.

13. Crear un archivo de texto para leer

Todavía no hemos enseñado redirecciones formalmente. Para que el módulo sea autocontenido, usaremos una sola vez printf con > como herramienta controlada para generar texto dentro del laboratorio.

Primero comprueba que el nombre está libre:

```
ls -l lectura.txt
```

Si responde que no existe, crea el archivo:

```
printf '%s\n' \  
'línea 1' 'línea 2' 'línea 3' 'línea 4' 'línea 5' 'línea 6' \  
'línea 7' 'línea 8' 'línea 9' 'línea 10' 'línea 11' 'línea 12' \  
> lectura.txt
```

Importante: esta orden utiliza una redirección >, tema que todavía no se ha explicado formalmente. Aquí se usa solo para preparar el archivo de práctica. No la copies para trabajar con archivos importantes. En el Módulo 8 aprenderás exactamente qué hace y por qué puede sobrescribir.

Después comprueba:

```
ls -l lectura.txt
```

No necesitas interpretar todavía todos los campos de `ls -l`.

14. Leer con cat

Ejecuta:

```
cat lectura.txt
```

Debes ver las doce líneas.

Pregunta:

¿cat cambió el archivo?

Respuesta esperada: no, en esta práctica solo lo mostró.

15. Leer con head

Ejecuta:

```
head lectura.txt
```

Después:

```
head -n 3 lectura.txt
```

Compara ambas salidas.

Debes poder explicar: - cuántas líneas muestra la primera por defecto; - cuántas solicitaste explícitamente en la segunda.

16. Leer con tail

Ejecuta:

```
tail lectura.txt
```

Después:

```
tail -n 3 lectura.txt
```

Compara con `head -n 3`.

Pregunta: ¿por qué las líneas son diferentes si ambos reciben el mismo número 3?

17. Abrir con less

Ejecuta:

```
less lectura.txt
```

Dentro de less:

1. desplázate;
2. escribe /línea 10;
3. pulsa Enter;

4. observa la coincidencia;
5. pulsa q para salir.

Si la búsqueda no funciona como esperabas, sal con q y vuelve a intentarlo. No cierres la terminal a la fuerza.

18. Práctica controlada de tail -f

Primera terminal:

```
cd ~/linux-lab/modulo-07-lectura
ls -l seguimiento.txt
```

Si seguimiento.txt no existe, créalo antes de seguirlo:

```
touch seguimiento.txt
```

Después, en la misma terminal:

```
tail -f seguimiento.txt
```

En una segunda terminal, entra a la misma carpeta y añade una línea con:

```
printf 'evento de prueba\n' >> seguimiento.txt
```

La primera terminal debería mostrar la nueva línea.

Nota: >> también es una redirección y se explicará formalmente en el Módulo 8. Aquí se utiliza solo para demostrar el comportamiento de tail -f.

Termina tail -f con:

```
Ctrl + C
```

No cierres el proceso con señales fuertes. La interrupción normal es suficiente.

19. Archivos binarios y texto ilegible

No todos los archivos están pensados para mostrarse con cat o less.

Un ejecutable, imagen o archivo comprimido puede contener bytes que no representan texto legible.

Regla: antes de mostrar un archivo desconocido, no asumas que es texto.

Más adelante aprenderás herramientas para identificar tipos de archivo.

En esta práctica solo usamos archivos de texto creados por nosotros.

20. Privacidad al leer archivos

Antes de compartir una salida, revisa si contiene:

- nombres de usuario;
- rutas personales;
- nombres de host;
- direcciones IP;
- tokens;
- contraseñas;
- cookies;
- claves;
- datos personales;
- contenido privado de aplicaciones.

No copies archivos completos del sistema al chat “para que los revise” sin inspeccionarlos primero.

Cuando necesites ayuda, comparte el fragmento mínimo relevante y oculta datos sensibles.

21. Errores frecuentes

Situación	Qué ocurrió	Corrección
cat llena la terminal con muchas líneas	Archivo demasiado largo para una lectura cómoda	Usa less o selecciona una parte con head/tail
No sabes salir de less	Estás dentro del paginador	Pulsa q
head -n 5 muestra menos de 5 líneas	El archivo tiene menos contenido	No es un error

Situación	Qué ocurrió	Corrección
<code>tail -f</code> parece “quedarse detenido”	Está esperando contenido nuevo	Usa <code>Ctrl+C</code> cuando termines
“Permission denied”	No tienes acceso	No añadas <code>sudo</code> ; revisa si realmente necesitas ese archivo
Pegaste un log completo	Puede contener información privada	Comparte solo el fragmento necesario
Usas <code>cat</code> sobre un archivo binario	No era texto legible	Detente y usa herramientas apropiadas más adelante

22. Práctica independiente

Dentro de una nueva carpeta de práctica:

1. crea un archivo de al menos 15 líneas;
2. muestra el archivo completo;
3. muestra solo las primeras 4 líneas;
4. muestra solo las últimas 4;
5. ábrelo con un paginador;
6. busca una palabra dentro del paginador;
7. sal correctamente;
8. explica qué herramienta elegirías si el archivo tuviera 20 000 líneas.

No copies la secuencia guiada. Escribe las órdenes basándote en lo aprendido.

23. Mini evaluación

1. ¿Qué herramienta es apropiada para mostrar rápidamente un archivo pequeño?
 - A. `cat`
 - B. `mkdir`
 - C. `rm`
 - D. `chmod`
2. ¿Qué herramienta permite navegar por un archivo largo de forma interactiva?
 - A. `tail`
 - B. `less`
 - C. `touch`
 - D. `mv`
3. ¿Qué muestra `head -n 3 archivo.txt`?
 - A. Las últimas tres líneas.
 - B. Las primeras tres líneas.
 - C. Tres archivos.
 - D. Tres directorios.
4. ¿Qué muestra `tail -n 3 archivo.txt`?
 - A. Las últimas tres líneas.
 - B. Las primeras tres.
 - C. Los permisos.
 - D. El propietario.
5. ¿Cómo sales normalmente de `less`?
 - A. `q`
 - B. `rm`
 - C. `sudo`
 - D. `exit --force`
6. ¿Cómo terminas normalmente una ejecución interactiva de `tail -f` en primer plano?
 - A. `Ctrl+C`
 - B. `rm -rf`
 - C. Reiniciando el equipo.
 - D. Borrando el archivo.
7. ¿Debes pegar un archivo completo del sistema al chat sin revisar su contenido?
 - A. Sí.
 - B. No.

24. Registro de aprendizaje

Puedes responder:

```
cat sirve para:
less sirve para:
head sirve para:
tail sirve para:
Para salir de less uso:
Para detener tail -f normalmente uso:
Si un archivo es muy largo elegiría:
Un error que ya sé reconocer:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

Una ejecución correcta no basta para marcar el tema como dominado. En otra sesión debes elegir la herramienta adecuada sin que se te indique cuál usar.

25. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Coreutils — cat: https://www.gnu.org/software/coreutils/manual/html_node/cat-invocation.html
- GNU Coreutils — head: https://www.gnu.org/software/coreutils/manual/html_node/head-invocation.html
- GNU Coreutils — tail: https://www.gnu.org/software/coreutils/manual/html_node/tail-invocation.html
- Documentación de less: <https://www.greenwoodsoftware.com/less/>

Esta lección usa las funciones básicas necesarias para principiantes. No cubre todavía: - numeración avanzada; - seguimiento por nombre frente a descriptor; - rotación de logs; - combinaciones de tail con múltiples archivos; - opciones avanzadas de less; - lectura de archivos binarios.

Estado de la lección: redactada y revisada documentalmente. La práctica real del estudiante sigue pendiente.

Módulo 8 — stdin, stdout, stderr y redirecciones

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Octava entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué son la entrada estándar, la salida estándar y la salida de error;
- reconocer los descriptores 0, 1 y 2;
- distinguir mostrar salida en pantalla de guardarla en un archivo;
- usar `>`, `>>` y `2>` dentro del laboratorio;
- explicar por qué `>` puede truncar o reemplazar el contenido previo de un archivo;
- separar la salida normal de los mensajes de error;
- comprobar el destino antes de redirigir.

Conocimientos previos: - navegación con `pwd`, `ls` y `cd`; - creación y lectura de archivos; - uso básico de `cat`, `head`, `tail` y `less`.

Seguridad: todas las prácticas se limitan a `~/linux-lab`. No se redirigirá hacia `/etc`, `/usr`, `/var`, `/proc` ni `/sys`. No uses `sudo` para forzar una redirección. Antes de usar `>`, verifica la ruta y decide si puedes perder el contenido existente del archivo destino.

2. La idea central: los programas reciben y producen datos

Muchos programas trabajan con flujos.

Modelo simplificado:

```
entrada → programa → salida
                  ↘ errores
```

En Unix y GNU/Linux se utilizan tres flujos estándar:

- **stdin**: entrada estándar;
- **stdout**: salida estándar;
- **stderr**: salida estándar de error.

No todos los programas usan los tres de la misma manera, pero este modelo es fundamental para comprender la shell.

3. stdin — entrada estándar

stdin es el flujo de entrada estándar.

Su descriptor tradicional es:

```
0
```

En muchos comandos interactivos, `stdin` recibe lo que escribes con el teclado.

Ejemplo conceptual:

```
cat
```

Si ejecutas `cat` sin indicar un archivo, puede leer desde la entrada estándar y mostrar lo recibido.

Para terminar una entrada interactiva de este tipo suele indicarse el fin de la entrada (EOF) con `Ctrl+D` en una terminal tipo Unix. **Ctrl+D no es una señal**: en las condiciones habituales de lectura desde la terminal, permite que el programa reciba fin de archivo.

No necesitamos practicar esto todavía si te resulta confuso; lo importante es entender que un programa puede leer datos sin que provengan necesariamente de un archivo.

4. stdout — salida estándar

stdout es la salida estándar.

Descriptor:

```
1
```

Ejemplo:

```
printf 'hola\n'
```

Normalmente verás:

```
hola
```

El programa escribió en stdout y la terminal mostró ese flujo.

Importante: que algo aparezca en pantalla no significa que se haya guardado en un archivo.

5. stderr — salida de error

stderr es un flujo separado para diagnósticos y errores.

Descriptor:

```
2
```

Ejemplo dentro del laboratorio, usando un nombre que no exista:

```
ls archivo-que-no-existe
```

Puedes recibir un mensaje de error.

Ese mensaje normalmente va a stderr, no al mismo flujo lógico que la salida normal.

Para qué sirve separarlos: permite guardar o procesar la salida correcta sin mezclarla necesariamente con los errores.

6. Los tres descriptores básicos

Memoriza este mapa:

```
0 = stdin
1 = stdout
2 = stderr
```

No significa que un proceso solo pueda abrir tres archivos. Significa que esos tres descriptores tienen funciones estándar convencionales.

Ejercicio: explica qué descriptor usarías para referirte a la salida de error.

7. Redirección: cambiar el destino de un flujo

La shell puede redirigir un flujo.

Ejemplo:

```
printf 'hola\n' > saludo.txt
```

Aquí stdout ya no se muestra principalmente en pantalla: la shell abre `saludo.txt` como destino de la salida.

Después:

```
cat saludo.txt
```

verás:

```
hola
```

8. > — escribir reemplazando el contenido previo

El operador:

```
>
```

redirige stdout hacia un archivo.

Ejemplo:

```
printf 'primera versión\n' > ejemplo.txt
```

Después:

```
printf 'segunda versión\n' > ejemplo.txt
```


El contenido previo puede quedar reemplazado.

Eso se denomina **truncamiento** del archivo destino en este contexto.

Regla crítica

Antes de usar >:

1. comprueba dónde estás con `pwd`;
2. comprueba si el destino ya existe;
3. decide si puedes perder su contenido;
4. solo entonces ejecuta la redirección.

No uses > sobre archivos importantes para “probar”.

9. >> — agregar al final

El operador:

```
>>
```

agrega `stdout` al final del archivo en lugar de reemplazar todo su contenido.

Ejemplo:

```
printf 'línea 1\n' > lista.txt
printf 'línea 2\n' >> lista.txt
```

Después:

```
cat lista.txt
```

Salida esperada:

```
línea 1
línea 2
```

Diferencia clave:

```
> = reemplazar/truncar el destino
>> = agregar al final
```

10. > no significa “guardar de forma segura”

El hecho de usar redirección no crea automáticamente una copia de seguridad.

Si escribes:

```
comando > archivo-importante.txt
```

y el destino existe, puedes perder su contenido anterior.

Por eso en este curso las primeras prácticas usan nombres creados específicamente para el laboratorio.

11. 2> — redirigir `stderr`

Puedes redirigir la salida de error con:

```
comando 2> errores.txt
```

Ejemplo seguro:

```
ls archivo-inexistente 2> errores.txt
```

Después:

```
cat errores.txt
```

Deberías ver el diagnóstico que antes aparecía en pantalla.

Explicación:

- 2 identifica `stderr`;

- > indica redirección;
- errores.txt es el destino.

12. 1> y > suelen representar stdout

Puedes escribir explícitamente:

```
printf 'hola\n' 1> salida.txt
```

En estos ejemplos, equivale a:

```
printf 'hola\n' > salida.txt
```

porque > sin descriptor explícito dirige stdout.

Para principiantes usaremos normalmente > para stdout y 2> para stderr.

13. Separar salida normal y errores

Supón que una orden produce: - información válida; - mensajes de error.

Puedes enviarlos a archivos distintos:

```
comando > salida.txt 2> errores.txt
```

Modelo:

```
stdout → salida.txt  
stderr → errores.txt
```

Esto es especialmente útil en scripts y diagnósticos.

14. El orden de las redirecciones puede importar

Las redirecciones se procesan en el orden escrito por la shell.

Esto se vuelve importante con construcciones como:

```
2>&1
```

Todavía no necesitas dominarla.

Solo conserva esta idea:

■ Redirigir stdout y stderr no siempre es intercambiable si se cambia el orden.

La combinación avanzada de flujos se estudiará más adelante, después de dominar cada descriptor por separado.

15. stdin desde un archivo con <

También existe la redirección de entrada:

```
<
```

Ejemplo conceptual:

```
comando < datos.txt
```

La shell conecta el contenido del archivo con stdin del programa.

Un ejemplo simple:

```
cat < notas.txt
```

puede mostrar el contenido, aunque para cat es más sencillo escribir:

```
cat notas.txt
```

El objetivo es comprender el flujo, no complicar una tarea simple.

16. Preparar la práctica

Entra al laboratorio:

```
cd ~/linux-lab
pwd
ls -la
```

Comprueba si existe:

```
ls -ld ./modulo-08-redirecciones
```

Si no existe:

```
mkdir modulo-08-redirecciones
```

Después:

```
cd ./modulo-08-redirecciones
pwd
ls -la
```

No borres prácticas anteriores.

17. Práctica A — stdout con >

Primero verifica que el nombre de destino esté libre:

```
ls -l salida.txt
```

Si no existe, crea contenido con:

```
printf 'uno\n' > salida.txt
```

Comprueba:

```
cat salida.txt
```

Ahora ejecuta:

```
printf 'dos\n' > salida.txt
```

Comprueba otra vez:

```
cat salida.txt
```

Explica qué ocurrió con uno.

18. Práctica B — agregar con >>

Ahora:

```
printf 'tres\n' >> salida.txt
```

Después:

```
cat salida.txt
```

Debes ver:

```
dos
tres
```

Explica por qué dos no desapareció esta vez.

19. Práctica C — stderr con 2>

Provoca un error seguro:

```
ls nombre-que-no-existe 2> errores.txt
```

Comprueba:

```
cat errores.txt
```

No importa que el texto exacto del error cambie según idioma o versión. Lo importante es que el diagnóstico se guardó en `errores.txt`.

20. Práctica D — separar stdout y stderr

Ejecuta una orden que produzca al menos una salida normal y un error. Una forma segura es pedir a `ls` un archivo que existe y otro que no:

```
touch existe.txt
ls existe.txt no-existe.txt > correctos.txt 2> fallos.txt
```

Después:

```
cat correctos.txt
cat fallos.txt
```

Debes poder explicar qué flujo terminó en cada archivo.

21. Práctica E — entrada con <

Comprueba primero que el nombre está libre con `ls -l entrada.txt`. Si no existe, crea el archivo:

```
printf 'alpha\nbeta\ngamma\n' > entrada.txt
```

Después:

```
cat < entrada.txt
```

Compara con:

```
cat entrada.txt
```

La salida puede verse igual, pero el modo en que `cat` recibe los datos no es conceptualmente idéntico.

22. Qué pasa si el comando falla

Una redirección puede ocurrir incluso cuando el comando no produce la salida que esperabas.

Por ejemplo, si el archivo destino de stdout se abre antes y el comando después falla, el archivo puede haberse creado o truncado.

Por eso:

■ Nunca uses un archivo importante como destino de prueba solo porque “el comando probablemente falle”.

Este detalle es una razón más para trabajar dentro de `~/linux-lab`.

23. sudo y redirecciones: error de concepto frecuente

Una construcción como:

```
sudo echo "dato" > /ruta/protegida
```

no significa que la redirección `>` se ejecute automáticamente con privilegios elevados.

La shell que procesa la redirección intenta abrir el archivo en su propio contexto.

En este curso no usaremos esto para escribir archivos del sistema. Cuando llegue el momento de administración, aprenderás mecanismos apropiados y sus riesgos.

Regla: no añadas `sudo` a ciegas para “arreglar” una redirección que falla.

24. Redirección y permisos

Si no tienes permiso para escribir en un destino, puedes recibir:

```
Permission denied
```

No es una invitación automática a usar privilegios.

Primero revisa: - si estás en la ruta correcta; - si el archivo pertenece al laboratorio; - si realmente debes modificarlo; - si se trata de un archivo del sistema.

25. Privacidad

Redirigir salida a un archivo puede guardar datos sensibles que antes solo veías en pantalla.

Antes de conservar o subir un archivo generado, revisa si contiene:

- nombres personales;
- rutas privadas;
- direcciones IP;
- tokens;
- contraseñas;
- cookies;
- claves;
- información de servicios;
- logs con datos sensibles.

No subas esos archivos a GitHub.

26. Errores frecuentes

Error	Qué pasó	Corrección
Usaste > pensando que agregaba	Truncaste el archivo	Usa >> solo cuando realmente quieras añadir
El error sigue apareciendo en pantalla	Solo redirigiste stdout	Usa 2> si necesitas redirigir stderr
2> crea un archivo vacío	No hubo error o stderr no produjo contenido	Comprueba la orden y el flujo esperado
Pusiste sudo delante de echo y la redirección falló	La shell procesa > fuera del sudo del comando	No fuerces; aprende el mecanismo apropiado más adelante
Redirigiste hacia el archivo equivocado	Ruta mal verificada	Revisa pwd, destino y existencia antes
Guardaste un log con secretos	No revisaste la salida	Sanitiza o elimina del flujo de publicación; no lo subas a GitHub

27. Práctica independiente

Dentro de una carpeta nueva del laboratorio:

1. crea un archivo con una línea usando >;
2. reemplaza su contenido con otra línea;
3. agrega una tercera línea con >>;
4. provoca un error seguro con ls sobre un nombre inexistente;
5. guarda ese error con 2>;
6. muestra ambos archivos con cat;
7. explica qué flujo representa cada archivo;
8. no uses sudo.

28. Mini evaluación

1. ¿Qué descriptor corresponde a stdin?
 - A. 0
 - B. 1
 - C. 2
2. ¿Qué descriptor corresponde a stdout?
 - A. 0
 - B. 1
 - C. 2
3. ¿Qué descriptor corresponde a stderr?
 - A. 0
 - B. 1
 - C. 2
4. ¿Qué hace > normalmente en estos ejemplos?
 - A. Agrega al final.
 - B. Redirige stdout y puede trunca el destino.
 - C. Cambia permisos.
 - D. Borra directorios.
5. ¿Qué hace >>?
 - A. Agrega al final.
 - B. Redirige stderr.
 - C. Cambia de usuario.

- D. Reinicia la shell.
6. ¿Qué hace 2>?
- A. Redirige stdin.
 - B. Redirige stdout.
 - C. Redirige stderr.
7. ¿Debes usar sudo automáticamente si una redirección falla por permisos?
- A. Sí.
 - B. No.
8. ¿> puede crear o truncar el archivo destino aunque el comando no produzca la salida esperada?
- A. Sí, dependiendo de cómo se procese la orden.
 - B. No, nunca.

29. Registro de aprendizaje

Puedes responder:

```
stdin es:
stdout es:
stderr es:
0 significa:
1 significa:
2 significa:
> sirve para:
>> sirve para:
2> sirve para:
El principal riesgo de > es:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

30. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Bash Reference Manual — Redirections: https://www.gnu.org/software/bash/manual/html_node/Redirections.html
- GNU Bash Reference Manual — Shell Operation: https://www.gnu.org/software/bash/manual/html_node/Shell-Operation.html
- The Open Group — POSIX Shell Command Language, sección Redirection:
https://pubs.opengroup.org/onlinepubs/9799919799/utilities/V3_chap02.html#tag_19_07

Esta lección cubre solo los fundamentos. Se posponen:

- 2>&1;
- &>;
- here-documents;
- here-strings;
- process substitution;
- redirecciones avanzadas con descriptores personalizados;
- noclobber;
- agrupación compleja de comandos.

Estado de la lección: redactada y revisada documentalmente. La práctica real del estudiante sigue pendiente.

Módulo 9 — Tuberías (pipes) y composición de comandos

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Novena entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué hace una tubería |;
- conectar la salida estándar de un comando con la entrada estándar de otro;
- distinguir una tubería de una redirección hacia archivo;
- construir cadenas pequeñas de comandos sin crear archivos intermedios;
- reconocer que stderr no entra automáticamente en la tubería normal;
- leer una tubería de izquierda a derecha;
- detectar cuándo una tubería añade complejidad innecesaria.

Conocimientos previos: - stdin, stdout y stderr; - descriptores 0, 1 y 2; - >, >> y 2>; - lectura con cat, head, tail y less.

Seguridad: las prácticas se realizan en ~/linux-lab y usan datos creados por nosotros. No canalices a una shell código descargado o desconocido. Una tubería puede encadenar operaciones rápidamente; por eso debes comprender cada comando por separado antes de unirlos.

2. Qué es una tubería

Una tubería, o **pipe**, conecta la salida estándar de un comando con la entrada estándar del siguiente.

Símbolo:

```
|
```

Modelo:

```
comando A stdout → stdin comando B
```

Ejemplo:

```
printf 'uno\ndos\ntres\n' | head -n 2
```

Aquí:

1. printf produce tres líneas en stdout.
2. | conecta esa salida con stdin de head.
3. head -n 2 conserva las primeras dos líneas que recibe.

Salida esperada:

```
uno
dos
```

3. La tubería no guarda automáticamente en un archivo

Compara:

```
printf 'hola\n' | cat
```

con:

```
printf 'hola\n' > saludo.txt
```

En el primer caso, stdout de printf pasa a stdin de cat.

En el segundo, stdout se redirige a un archivo.

Modelo:

```
| = conectar programas
> = enviar salida a un archivo y puede truncarlo
>> = agregar salida a un archivo
```

Error típico: pensar que una tubería crea un archivo. No lo hace por sí sola.

4. Leer una tubería de izquierda a derecha

Ejemplo:

```
printf 'rojo\nverde\nazul\n' | tail -n 1
```

Léelo así:

“Genera tres líneas y entrega esa salida a tail, que conserva la última.”

Salida:

```
azul
```

No intentes entender una cadena larga de golpe. Separa cada etapa.

5. Cada comando debe tener sentido por separado

Antes de unir:

```
comandoA | comandoB
```

pregunta:

- ¿qué produce comandoA en stdout?;
- ¿qué espera comandoB en stdin?;
- ¿el formato producido por A tiene sentido para B?

Una tubería no “traduce” automáticamente datos incompatibles.

6. Un ejemplo con un archivo

Supón que lectura.txt contiene muchas líneas.

Puedes escribir:

```
cat lectura.txt | head -n 3
```

Esto funciona como demostración del flujo:

```
archivo → cat → stdout → pipe → head
```

Pero para esta tarea concreta existe una forma más simple:

```
head -n 3 lectura.txt
```

Lección: que una tubería funcione no significa que sea la solución más clara.

Evita añadir comandos que no aportan valor.

7. Contar líneas con wc -l

wc significa *word count*. Una de sus opciones es:

```
-l
```

que cuenta líneas.

Ejemplo:

```
printf 'a\nb\nc\n' | wc -l
```

Salida esperada:

```
3
```

No necesitas dominar todavía todas las funciones de wc. Aquí se usa para mostrar cómo un programa puede consumir el flujo producido por otro.

Referencia: GNU Coreutils — wc.

8. Varias etapas

Una tubería puede tener más de dos comandos.

Ejemplo:

```
printf 'uno\ndos\ntres\ncuatro\n' | head -n 3 | tail -n 1
```

Paso a paso:

1. `printf` produce cuatro líneas.
2. `head -n 3` deja: uno, dos, tres.
3. `tail -n 1` toma la última de esas tres.
4. Resultado:

```
tres
```

Regla pedagógica

Empieza con dos etapas.

Añade una tercera solo cuando puedas explicar las dos anteriores.

9. `stdout` sí; `stderr` no entra por defecto en |

La tubería normal conecta **`stdout`** del comando izquierdo con `stdin` del derecho.

Ejemplo conceptual:

```
stdout → |
stderr → terminal
```

Si el comando izquierdo produce un error, ese mensaje puede seguir apareciendo en la terminal aunque `stdout` esté conectado a otra orden.

No introduciremos todavía combinaciones avanzadas como `2>&1` o `|&`. Primero domina la separación de flujos.

10. Tubería frente a archivo intermedio

Sin tubería podrías imaginar:

```
comando A → archivo temporal → comando B
```

Con pipe:

```
comando A → comando B
```

Ventaja: - evita crear un archivo intermedio cuando no hace falta.

Pero no significa que todos los datos existan mágicamente “a la vez”. Los procesos pueden producir y consumir datos conforme avanzan.

No necesitamos estudiar todavía buffers, bloqueos ni detalles internos del kernel.

11. La shell participa en la tubería

En Bash, la shell reconoce el operador `|` y crea la conexión entre los procesos o comandos de la pipeline.

Por eso:

```
printf 'hola\n' | cat
```

no entrega el carácter `|` como argumento ordinario a `printf`.

La shell interpreta primero la estructura de la orden.

12. Pipeline no significa ejecución “segura”

Una tubería puede conectar comandos inocuos, pero también podría conectar una fuente de datos con una operación destructiva o con un intérprete.

Por eso este manual aplica una regla estricta:

| No canales contenido descargado o desconocido directamente hacia `bash`, `sh`, `python` u otro intérprete.

Primero debes conocer el origen, revisar el contenido y entender qué ejecutaría.

En los laboratorios iniciales usaremos únicamente texto creado por nosotros y comandos de lectura/procesamiento.

13. No confundas | con ||

Estos símbolos son distintos:

```
| = tubería
|| = operador lógico de control
```

| se estudiará más adelante con códigos de salida y control de flujo.

No escribas dos barras cuando solo quieres conectar stdout con stdin.

14. Preparar la práctica

Entra al laboratorio:

```
cd ~/linux-lab
pwd
ls -la
```

Comprueba si existe:

```
ls -ld ./modulo-09-pipes
```

Si no existe:

```
mkdir modulo-09-pipes
```

Después:

```
cd ./modulo-09-pipes
pwd
ls -la
```

No borres una práctica anterior si ya existe.

15. Práctica A — primera tubería

Ejecuta:

```
printf 'uno\ndos\ntres\n' | head -n 2
```

Antes de mirar la salida intenta predecirla.

Después explica:

- qué comando produce datos;
- qué comando recibe datos;
- qué hace |.

16. Práctica B — principio y final

Ejecuta:

```
printf '1\n2\n3\n4\n5\n' | tail -n 2
```

Salida esperada:

```
4
5
```

Ahora cambia solo el consumidor:

```
printf '1\n2\n3\n4\n5\n' | head -n 2
```

Explica por qué la fuente es la misma pero el resultado cambia.

17. Práctica C — contar líneas

Ejecuta:

```
printf 'alpha\nbeta\ngamma\ndelta\n' | wc -l
```

Debes obtener:

```
4
```

Pregunta:

¿wc -l leyó un archivo?

En esta práctica, no. Recibió su entrada desde la tubería.

18. Práctica D — tres etapas

Ejecuta:

```
printf 'a\nb\nc\nd\ne\n' | head -n 4 | tail -n 2
```

Razonamiento:

1. fuente: cinco líneas;
2. primera etapa: quedan cuatro;
3. segunda etapa: quedan las dos últimas de esas cuatro.

Predice el resultado antes de ejecutarlo.

19. Práctica E — usar un archivo existente

Crea un archivo de práctica:

```
printf 'enero\nfebrero\nmarzo\nabril\nmayo\n' > meses.txt
```

Ya conoces el riesgo de >: hazlo solo si meses.txt es un nombre destinado a esta práctica y su contenido puede ser reemplazado.

Después:

```
cat meses.txt | tail -n 2
```

Comprueba también la versión más simple:

```
tail -n 2 meses.txt
```

Las salidas deberían coincidir.

Conclusión: la tubería es útil, pero no debe añadirse sin necesidad.

20. Práctica F — stderr no se canaliza normalmente

Ejecuta:

```
ls archivo-inexistente | wc -l
```

Observa dos cosas:

- el diagnóstico de ls puede aparecer en la terminal porque va por stderr;
- wc -l recibe stdout de ls, que en este caso puede estar vacío.

No necesitas memorizar el número exacto mostrado por wc; entiende la separación de flujos.

21. Pipes y less

Un caso útil:

```
printf 'uno\ndos\ntres\ncuatro\ncinco\n' | less
```

less puede recibir datos por stdin.

Pulsa:

```
q
```

para salir.

Esto muestra que un paginador no necesita obligatoriamente recibir un nombre de archivo.

22. Errores frecuentes

Error	Qué ocurre	Corrección
Crees que <code> </code> crea un archivo	Pipe conecta flujos	Usa redirección solo cuando realmente quieras un archivo
Confundes <code> </code> con <code> </code>	Son operadores distintos	Una barra para pipe; <code> </code> se estudiará después
Esperas que <code>stderr</code> pase por el pipe	Pipe normal toma <code>stdout</code>	Mantén separados los flujos por ahora
Añades <code>cat</code> innecesariamente	La tubería funciona pero complica	Usa la forma directa cuando el comando acepta archivo
Una cadena larga no se entiende	Hay demasiadas etapas	Prueba y explica cada etapa por separado
Canalizas código desconocido a una shell	Ejecutarías contenido sin revisarlo	No lo hagas; inspecciona y comprende antes
Un resultado es distinto de lo previsto	Alguna etapa transforma datos de otra forma	Ejecuta etapas por separado y compara

23. Método de diagnóstico de una pipeline

Si esta cadena falla:

```
A | B | C
```

no cambies opciones al azar.

Haz:

1. ejecuta A y observa `stdout`;
2. conecta A con B;
3. comprueba el resultado;
4. solo después agrega C;
5. revisa errores por separado.

Este método reduce la cantidad de cosas que intentas depurar al mismo tiempo.

24. Práctica independiente

Sin copiar los ejemplos exactos:

1. genera seis líneas con `printf`;
2. usa un pipe para conservar las primeras cuatro;
3. añade una segunda etapa para conservar las últimas dos de esas cuatro;
4. predice el resultado antes de ejecutar;
5. crea otro flujo y cuenta sus líneas con `wc -l`;
6. explica cuál comando fue productor y cuál consumidor en cada caso;
7. identifica una situación donde un pipe sería innecesario.

No uses intérpretes, comandos destructivos ni datos descargados.

25. Mini evaluación

1. ¿Qué hace `|?`
 - A. Borra un archivo.
 - B. Conecta `stdout` del comando izquierdo con `stdin` del derecho.
 - C. Cambia permisos.
 - D. Crea un usuario.
2. ¿Una tubería normal envía `stderr` automáticamente al comando siguiente?
 - A. Sí.
 - B. No.
3. ¿Qué diferencia principal existe entre `|` y `>`?
 - A. Ninguna.
 - B. `|` conecta comandos; `>` redirige salida hacia un archivo.
 - C. `|` siempre borra archivos.
 - D. `>` siempre conecta dos programas.
4. ¿Qué hace `wc -l` en los ejemplos?
 - A. Cuenta líneas.
 - B. Borra líneas.
 - C. Cambia de carpeta.
 - D. Reinicia.
5. En: `printf 'a\nb\nc\n' | head -n 1` ¿qué comando produce los datos?
 - A. `head`

- B. printf
6. ¿Es buena práctica canalizar un script descargado y desconocido directamente hacia bash?
- A. Sí.
 - B. No.
7. Si una pipeline larga falla, ¿qué estrategia es mejor?
- A. Añadir más comandos.
 - B. Probar cada etapa progresivamente.
 - C. Usar sudo.
 - D. Borrar el laboratorio.

26. Registro de aprendizaje

Puedes responder:

```
Una tubería sirve para:  
El comando de la izquierda entrega:  
El comando de la derecha recibe:  
La diferencia entre | y > es:  
stderr en una tubería normal:  
wc -l sirve para:  
Un ejemplo donde no necesito pipe:  
Un error que ya sé diagnosticar:  
Algo que todavía confundo:  
Estado: EN APRENDIZAJE / PRACTICADO
```

No se considera dominado con una sola pipeline correcta. Debes poder predecir resultados, construir una combinación nueva y detectar una etapa incorrecta.

27. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Bash Reference Manual — Pipelines: https://www.gnu.org/software/bash/manual/html_node/Pipelines.html
- GNU Bash Reference Manual — Redirections: https://www.gnu.org/software/bash/manual/html_node/Redirections.html
- GNU Coreutils — wc: https://www.gnu.org/software/coreutils/manual/html_node/wc-invocation.html
- GNU Coreutils — head y tail: https://www.gnu.org/software/coreutils/manual/html_node/head-invocation.html
https://www.gnu.org/software/coreutils/manual/html_node/tail-invocation.html

Se posponen para módulos posteriores:

- estado de salida de una pipeline;
- pipefail;
- PIPESTATUS;
- || y &&;
- |&;
- combinación avanzada de stderr con stdout;
- procesos y concurrencia internos de pipelines.

Estado de la lección: redactada y revisada documentalmente. La práctica real del estudiante sigue pendiente.

Módulo 10 — grep, find y locate: buscar texto y archivos

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Décima entrega.

1. Qué aprenderás

Al terminar este módulo podrás distinguir tres tareas diferentes:

- buscar **texto dentro de contenido** con grep;
- buscar **archivos y directorios recorriendo una ruta** con find;
- buscar **nombres mediante una base de datos indexada** con una implementación de locate, cuando esté disponible.

También aprenderás a:

- usar grep -i y grep -n;
- combinar grep con pipes;
- usar find con -type y -name;
- entender por qué conviene citar patrones como "*.txt";
- reconocer que locate puede mostrar resultados desactualizados;
- evitar búsquedas innecesarias sobre todo el sistema.

Conocimientos previos: - rutas; - lectura de archivos; - stdout, stderr y redirecciones; - tuberías.

Seguridad: todas las prácticas se limitan a ~/linux-lab. No recorreremos / como ejercicio de principiante. Buscar sobre todo el sistema puede producir muchas salidas, errores de permisos y exposición innecesaria de rutas privadas. No uses sudo para “ver más resultados”.

2. Tres preguntas distintas

Antes de elegir una herramienta, identifica qué quieres encontrar.

Pregunta A — “¿En qué líneas aparece esta palabra?”

Herramienta:

```
grep
```

Pregunta B — “¿Dónde está un archivo con este nombre?”

Herramienta:

```
find
```

Pregunta C — “¿Existe rápidamente algún nombre parecido en el índice del sistema?”

Posible herramienta:

```
locate
```

No son sinónimos.

3. grep — buscar patrones dentro del texto

grep examina texto y selecciona líneas que coinciden con un patrón.

Ejemplo:

```
grep 'error' registro.txt
```

Si registro.txt contiene:

```
inicio correcto
error al abrir archivo
proceso completado
```

la salida sería:

```
error al abrir archivo
```

Modelo mental:

```
texto → grep patrón → líneas coincidentes
```

Referencia principal: GNU Grep Manual.

4. Patrón no significa siempre “texto literal simple”

grep trabaja con patrones y, por defecto, interpreta una forma de expresiones regulares.

En este módulo usaremos patrones sencillos como:

```
error
Linux
usuario
```

No estudiaremos todavía expresiones regulares en profundidad.

Regla: si solo quieres aprender búsqueda básica, empieza con texto simple y no agregues símbolos especiales al azar.

5. grep -i — ignorar mayúsculas y minúsculas

Ejemplo:

```
grep -i 'linux' notas.txt
```

Puede coincidir con:

```
linux
Linux
LINUX
```

-i hace una comparación que ignora diferencias de mayúsculas/minúsculas según el comportamiento y configuración local de la herramienta.

Error típico: creer que grep 'linux' y grep -i 'linux' siempre producen exactamente lo mismo.

6. grep -n — mostrar número de línea

Ejemplo:

```
grep -n 'error' registro.txt
```

Una salida ilustrativa:

```
4:error al abrir archivo
```

El número indica la línea donde apareció la coincidencia.

Esto es útil para localizar rápidamente la posición dentro de un archivo.

7. Combinar opciones

Puedes combinar:

```
grep -in 'linux' notas.txt
```

o escribir opciones separadas:

```
grep -i -n 'linux' notas.txt
```

En estos ejemplos ambas formas expresan la misma intención: ignorar mayúsculas/minúsculas y mostrar números de línea.

Para aprender, prioriza claridad sobre abreviar todo.

8. grep y stdin

grep también puede recibir texto desde stdin.

Ejemplo:

```
printf 'rojo\nverde\nazul\n' | grep 'verde'
```

Salida:

```
verde
```

Aquí no leyó un archivo nombrado: recibió texto mediante el pipe.

Esto conecta directamente con el Módulo 9.

9. grep no busca nombres de archivos por defecto

Si escribes:

```
grep 'reporte' archivo.txt
```

grep busca el patrón dentro del contenido de archivo.txt.

No está buscando automáticamente un archivo llamado reporte.

Para nombres y rutas usaremos find.

10. find — recorrer una ruta real

find recorre una jerarquía de directorios desde un punto inicial y evalúa criterios.

Ejemplo:

```
find ~/linux-lab -type f
```

Explicación:

- find = herramienta;
- ~/linux-lab = ruta inicial;
- -type f = selecciona archivos ordinarios.

Puede mostrar muchas rutas si tu laboratorio ya tiene varias prácticas.

Referencia principal: GNU Findutils Manual.

11. Buscar solo directorios

```
find ~/linux-lab -type d
```

-type d selecciona directorios.

Comparación:

```
-type f = archivos ordinarios  
-type d = directorios
```

No necesitas memorizar todavía todos los tipos posibles.

12. find -name — buscar por nombre

Ejemplo:

```
find ~/linux-lab -type f -name "*.txt"
```

Esto recorre el laboratorio y selecciona archivos ordinarios cuyo nombre coincide con el patrón *.txt.

¿Por qué usamos comillas?

Porque queremos que el patrón:

```
*.txt
```

llegue a find para que **find** lo interprete.

Si lo dejas sin citar, la shell podría expandirlo antes dependiendo de los nombres presentes en tu directorio actual.

Regla de este módulo:

```
-name "*.txt"
```

es preferible para que el patrón llegue completo a find.

13. El asterisco en -name

Dentro del patrón de find:

```
*
```

puede representar una secuencia de caracteres en el nombre.

Ejemplo:

```
*.txt
```

puede coincidir con:

```
notas.txt  
reporte.txt  
a.txt
```

No significa “buscar dentro del contenido”.

14. find es sensible a la ruta inicial

Compara:

```
find .
```

con:

```
find ~/linux-lab
```

. significa “desde aquí”.

~/linux-lab fija explícitamente el laboratorio.

Para principiantes, la segunda forma suele ser más clara cuando queremos limitar el alcance.

15. Evitar find /

Esta orden:

```
find /
```

inicia desde la raíz del sistema.

Puede:

- recorrer una cantidad enorme de rutas;
- producir errores de permisos;
- mostrar rutas privadas;
- tardar mucho;
- consumir recursos innecesariamente.

No la necesitamos para aprender find.

Regla: practica con una ruta concreta y pequeña.

16. find y errores de permisos

Aunque una búsqueda sea de lectura, puede intentar entrar en lugares para los que tu usuario no tiene acceso.

Si aparece:

```
Permission denied
```

no añadas sudo automáticamente.

Pregunta primero:

¿Necesito realmente buscar en esa ruta?

En nuestras prácticas, la respuesta será normalmente no.

17. Buscar por nombre exacto

Ejemplo:

```
find ~/linux-lab -type f -name "notas.txt"
```

Esto busca archivos ordinarios llamados exactamente `notas.txt` según el patrón proporcionado.

No examina el contenido de esos archivos.

18. Buscar texto dentro de varios archivos encontrados

Más adelante aprenderás formas robustas de combinar `find` con otras herramientas.

Por ahora no construiremos pipelines complejas con nombres de archivo, porque los nombres pueden contener:

- espacios;
- saltos de línea;
- guiones iniciales;
- otros caracteres especiales.

No uses como regla general:

```
for archivo in $(find ...)
```

Ese patrón puede romper nombres válidos.

En este nivel, primero aprende a obtener rutas correctamente.

19. locate — búsqueda por índice

Una herramienta tipo `locate` puede buscar nombres usando una base de datos previamente construida.

Ejemplo conceptual:

```
locate ejemplo.txt
```

La ventaja principal suele ser velocidad: no necesita recorrer todas las rutas en tiempo real en cada búsqueda.

Pero existe una diferencia fundamental:

```
find  = recorre rutas actuales
locate = consulta un índice
```

20. locate puede no estar instalado

No todas las distribuciones instalan una herramienta `locate` por defecto.

Además, la implementación puede variar.

En sistemas actuales podrías encontrar, por ejemplo, una implementación como `plocate`, que normalmente proporciona el comando `locate`.

No instales nada solo para completar este módulo.

Comprueba primero:

```
command -v locate
```

Si no produce una ruta, registra:

```
locate no disponible en mi entorno
```

y continúa con `grep` y `find`.

21. La base de datos puede estar desactualizada

Si acabas de crear:

```
~/linux-lab/archivo-nuevo.txt
```

`find` puede localizarlo inmediatamente porque recorre la jerarquía actual.

`locate` puede no mostrarlo todavía si su índice no se ha actualizado.

Esa es una de las diferencias más importantes del módulo.

22. updatedb: concepto, no práctica administrativa

Las implementaciones de `locate` suelen disponer de un mecanismo para actualizar la base de datos, tradicionalmente mediante una herramienta llamada `updatedb`.

Actualizar una base del sistema puede depender de:

- distribución;
- implementación;
- permisos;
- servicios o temporizadores configurados.

No ejecutaremos `sudo updatedb` como receta universal.

Para este módulo basta entender por qué un índice puede no reflejar cambios recientes.

23. locate no sustituye find

Usa `find` cuando necesites:

- controlar exactamente la ruta recorrida;
- buscar el estado actual;
- filtrar por tipo;
- construir criterios sobre la jerarquía real.

Usa `locate`, cuando esté disponible, para una búsqueda rápida basada en nombres indexados.

24. Preparar el laboratorio

Entra:

```
cd ~/linux-lab
pwd
ls -la
```

Comprueba:

```
ls -ld ./modulo-10-busquedas
```

Si no existe:

```
mkdir modulo-10-busquedas
```

Después:

```
cd ./modulo-10-busquedas
pwd
ls -la
```

No borres una práctica anterior.

25. Crear datos de práctica

Primero comprueba cada nombre en el laboratorio con `ls -ld documentos`, `ls -ld registros` y `ls -ld notas`. Si alguno ya existe, **detente y revisa su contenido**; no sobrescribas prácticas anteriores.

Solo si los tres nombres están libres, crea los subdirectorios:

```
mkdir documentos
mkdir registros
mkdir notas
```

Verifica con `ls -ld documentos registros notas` antes de crear archivos.

Crea archivos:

```
printf 'Linux es el núcleo\nBash es una shell\n' > documentos/conceptos.txt
printf 'inicio correcto\nERROR de prueba\nfin correcto\n' > registros/app.log
printf 'aprender grep\naprender find\naprender pipes\n' > notas/tareas.txt
```

Estas redirecciones reemplazan los archivos destino si ya existen.

Por eso ejecútalas solo si acabas de crear esta práctica o ya verificaste que el contenido puede reemplazarse.

26. Práctica A — grep simple

Ejecuta:

```
grep 'Bash' documentos/conceptos.txt
```

Predice la línea antes de mirar.

Después prueba:

```
grep 'bash' documentos/conceptos.txt
```

Compara el resultado.

27. Práctica B — grep -i

```
grep -i 'bash' documentos/conceptos.txt
```

Explica por qué ahora puede coincidir aunque el archivo tenga Bash con mayúscula.

28. Práctica C — grep -n

```
grep -n 'ERROR' registros/app.log
```

Debes poder identificar:

- número de línea;
- contenido coincidente.

No memorices el formato como una regla universal para todas las opciones; interpreta esta salida concreta.

29. Práctica D — grep mediante pipe

```
cat notas/tareas.txt | grep 'find'
```

Después usa la forma más directa:

```
grep 'find' notas/tareas.txt
```

Explica por qué la segunda es más simple para este caso.

30. Práctica E — find por tipo

Desde modulo-10-busquedas:

```
find . -type f
```

Después:

```
find . -type d
```

Identifica qué resultados corresponden a archivos y cuáles a directorios.

31. Práctica F — find por extensión

```
find . -type f -name "*.txt"
```

Debe encontrar los archivos .txt de la práctica, pero no app.log, porque su nombre no termina en .txt.

Explica por qué las comillas alrededor de *.txt son importantes.

32. Práctica G — nombre exacto

```
find . -type f -name "tareas.txt"
```

El resultado debe incluir la ruta relativa a notas/tareas.txt.

Ahora explica la diferencia entre:

```
grep 'tareas' archivo.txt
```

y:

```
find . -name "tareas.txt"
```

33. Práctica H — comprobar locate

```
command -v locate
```

Si muestra una ruta, puedes probar dentro de tu propio laboratorio:

```
locate modulo-10-busquedas
```

Si no muestra resultados, no significa necesariamente que la carpeta no exista. El índice puede no estar actualizado o puede aplicar reglas de visibilidad.

Si locate no está instalado, no hagas nada más.

34. grep recursivo: introducción limitada

GNU grep puede buscar recursivamente con opciones como:

```
grep -r 'aprender' .
```

Dentro de nuestra pequeña carpeta de práctica, esto puede ser útil.

Pero no uses:

```
grep -r palabra /
```

como ejercicio: recorrer todo el sistema genera ruido, errores de permisos y posibles exposiciones.

En módulos posteriores veremos filtros y estrategias más precisas.

35. grep y archivos binarios

Una búsqueda recursiva puede encontrar archivos que no son texto.

grep tiene comportamientos y opciones para esos casos, pero no los estudiaremos todavía.

Por ahora limita la práctica a archivos de texto creados por nosotros.

36. Errores frecuentes

Error	Problema	Corrección
Usas grep para localizar un nombre de archivo	grep busca texto/patrones	Usa find para rutas/nombres
Usas find esperando buscar contenido	find evalúa objetos y rutas	Usa grep para contenido
locate no encuentra un archivo recién creado	Índice desactualizado o no disponible	Usa find para el estado actual
Escribes -name *.txt sin comillas	La shell puede expandir el patrón antes	Usa -name "*.txt"
Ejecutas find / y aparecen cientos de errores	Alcance demasiado amplio	Limita la ruta
Añades sudo por errores de permisos	Estás ampliando acceso sin necesidad	Revisa si debes buscar ahí
grep no coincide por mayúsculas	La búsqueda normal distingue caso	Usa -i si esa es la intención

37. Método para elegir herramienta

Hazte estas preguntas:

1. ¿Busco texto dentro de contenido?
 - grep.
2. ¿Busco un objeto por ruta, nombre o tipo?
 - find.
3. ¿Quiero una búsqueda rápida por índice y acepto que pueda estar desactualizada?
 - locate, si existe.

Esta decisión es más importante que memorizar muchas opciones.

38. Práctica independiente

Sin copiar la secuencia exacta:

1. crea dos carpetas;
2. crea al menos cuatro archivos de texto;
3. incluye una palabra común en dos de ellos;
4. usa `grep` para encontrar las líneas;
5. repite ignorando mayúsculas/minúsculas;
6. usa `find` para listar solo archivos `.txt`;
7. usa `find` para localizar un nombre exacto;
8. explica qué pasaría si `locate` no muestra un archivo recién creado;
9. no busques desde `/` y no uses `sudo`.

39. Mini evaluación

1. ¿Qué herramienta busca patrones dentro del texto?
 - A. `grep`
 - B. `find`
 - C. `cd`
2. ¿Qué herramienta recorre rutas y puede filtrar por nombre o tipo?
 - A. `grep`
 - B. `find`
 - C. `cat`
3. ¿Qué significa `grep -i`?
 - A. Ignorar diferencias de mayúsculas/minúsculas.
 - B. Instalar `grep`.
 - C. Buscar solo directorios.
4. ¿Qué aporta `grep -n`?
 - A. Número de línea.
 - B. Nombre del usuario.
 - C. Tamaño del disco.
5. En `find . -type f -name "*.txt"`, ¿por qué citamos `*.txt`?
 - A. Para que el patrón llegue a `find` sin expansión prematura de la shell.
 - B. Porque `find` exige comillas en todos los argumentos.
 - C. Para usar `sudo`.
6. ¿`locate` consulta necesariamente el sistema de archivos en tiempo real?
 - A. Sí.
 - B. No.
7. Si acabas de crear un archivo y necesitas encontrarlo con certeza dentro de una ruta, ¿qué elegirías primero?
 - A. `find`
 - B. `locate`
8. ¿Debes ejecutar `find /` con `sudo` como práctica básica?
 - A. Sí.
 - B. No.

40. Registro de aprendizaje

Puedes responder:

```
grep sirve para:
find sirve para:
locate sirve para:
grep -i significa:
grep -n significa:
-type f significa:
-type d significa:
¿Por qué cito "*.txt"?:
Si locate no encuentra algo recién creado:
Un error que ya sé identificar:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

No se marca como dominado con una sola búsqueda correcta. Debes poder escoger la herramienta sin que el tutor te diga cuál corresponde.

41. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Grep Manual: <https://www.gnu.org/software/grep/manual/grep.html>

- GNU Findutils Manual: https://www.gnu.org/software/findutils/manual/html_mono/find.html
- plocate: <https://plocate.sesse.net/>

Se posponen:

- expresiones regulares detalladas;
- `grep -E`, `grep -F`, contexto y colores;
- `find -exec`;
- `-print0` y procesamiento NUL;
- criterios por permisos, fechas y tamaños;
- diferencias detalladas entre `locate`, `mlocate` y `plocate`;
- búsquedas administrativas del sistema.

Estado de la lección: redactada y revisada documentalmente. La práctica real del estudiante sigue pendiente.

Módulo 11 — Usuarios y grupos: whoami, id, UID, GID e identidad

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Undécima entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué representa un usuario en GNU/Linux;
- distinguir nombre de usuario de UID;
- distinguir nombre de grupo de GID;
- reconocer grupo primario y grupos suplementarios;
- consultar tu identidad con `whoami` e `id`;
- interpretar una salida básica de `id`;
- reconocer que `root` y UID 0 requieren especial cuidado;
- diferenciar consultar identidad de modificar usuarios o grupos.

Conocimientos previos: - terminal y Bash; - lectura de archivos; - rutas; - `grep` básico.

Seguridad: este módulo es de consulta. No crearemos, eliminaremos ni modificaremos usuarios o grupos. No cambiaremos contraseñas ni archivos de cuentas. No uses `sudo` para estas prácticas. No publiques listas completas de usuarios, nombres reales, rutas personales ni otros datos de identidad en GitHub o en el chat.

2. Qué es un usuario

Un **usuario** es una identidad que el sistema utiliza para asociar procesos, archivos, permisos y otras decisiones de acceso.

No debes pensar únicamente en “una persona que inicia sesión”.

También pueden existir cuentas utilizadas por:

- servicios;
- demonios;
- aplicaciones;
- tareas del sistema.

Por eso un equipo Linux puede tener más cuentas que personas.

3. Nombre de usuario y UID

Una cuenta suele tener un nombre legible, por ejemplo:

```
alumno
```

Pero internamente el sistema utiliza un identificador numérico:

```
UID
```

UID significa **User Identifier**.

Modelo:

```
nombre de usuario → UID
alumno           → 1000 (ejemplo ilustrativo)
```

El número 1000 es solo un ejemplo. No asumas que tu UID será ese.

Idea importante: los permisos del sistema se basan en identificadores y metadatos, no únicamente en el texto visible del nombre.

4. Qué es un grupo

Un **grupo** reúne identidades para facilitar decisiones de acceso.

En lugar de asignar determinados permisos individualmente a muchas cuentas, un sistema puede usar pertenencia a grupos.

Los grupos también tienen:

- un nombre;
- un identificador numérico llamado **GID**.

GID significa **Group Identifier**.

Modelo:


```
nombre de grupo → GID
estudiantes      → 1000 (ejemplo ilustrativo)
```

De nuevo, el número es solo ilustrativo.

5. Grupo primario y grupos suplementarios

Un usuario puede tener:

- un **grupo primario**;
- cero o más **grupos suplementarios**.

El grupo primario participa, entre otras cosas, en la asociación de grupo que puede recibir un archivo nuevo según el contexto y las reglas aplicables.

Los grupos suplementarios amplían pertenencias disponibles para decisiones de acceso.

No estudiaremos todavía todos los detalles de creación de archivos ni de ACL; el Módulo 12 introduce los permisos y las ACL básicas. El bit SGID de directorios queda fuera del núcleo inicial y se pospone.

6. whoami — quién soy en esta sesión

Ejecuta:

```
whoami
```

En GNU Coreutils, `whoami` muestra el nombre asociado al **usuario efectivo** del proceso actual.

Es equivalente conceptualmente a:

```
id -un
```

para esta consulta.

No confundas `whoami` con “mostrar todas las cuentas del sistema”. Solo responde quién eres desde el punto de vista de la identidad efectiva actual.

Referencia: GNU Coreutils — `whoami`.

7. id — información de identidad

Ejecuta:

```
id
```

Una salida ilustrativa podría parecerse a:

```
uid=1000(alumno) gid=1000(alumno) groups=1000(alumno),27(ejemplo)
```

Tu salida será diferente.

Partes básicas:

- `uid=` → UID del usuario;
- `gid=` → GID del grupo primario;
- `groups=` → grupos asociados.

No copies una salida ilustrativa como si fuera la tuya.

8. Consultar solo el UID

```
id -u
```

Muestra el UID numérico de la identidad actual.

Para solicitar el nombre en vez del número:

```
id -un
```

Aquí:

- `-u` selecciona el identificador de usuario;
- `-n` solicita el nombre correspondiente.

Por eso `id` -un se relaciona con `whoami`.

9. Consultar el GID primario

```
id -g
```

Muestra el GID primario numérico.

Para el nombre:

```
id -gn
```

No confundas UID y GID:

```
UID = usuario  
GID = grupo
```

10. Consultar grupos

```
id -G
```

muestra identificadores numéricos de los grupos.

Con nombres:

```
id -Gn
```

También puede existir la utilidad:

```
groups
```

para mostrar nombres de grupos asociados.

En este manual priorizamos `id` porque reúne varias consultas relacionadas en una sola herramienta.

11. Por qué importan los identificadores numéricos

Supón un archivo con propietario:

```
alumno
```

Visualmente ves un nombre.

Pero el sistema mantiene identificadores numéricos asociados a propietario y grupo.

Eso ayuda a comprender situaciones como:

- archivos copiados entre sistemas;
- cuentas recreadas;
- contenedores;
- volúmenes compartidos;
- discrepancias entre nombres y números.

Todavía no resolveremos esos escenarios. Solo necesitamos entender que nombre e ID son conceptos relacionados, no idénticos.

12. root y UID 0

Tradicionalmente, la cuenta administrativa principal se llama:

```
root
```

y utiliza UID:

```
0
```

UID 0 recibe tratamiento privilegiado especial en muchos controles del sistema.

Regla de seguridad: no trabajes como root para prácticas ordinarias.

Para aprender navegación, lectura, búsqueda y Bash básico debes utilizar tu usuario normal.

Más adelante estudiaremos privilegios mínimos y `sudo` de forma controlada.

13. root no significa “modo mágico”

Aunque root tiene privilegios muy amplios, eso no elimina:

- errores humanos;
- rutas incorrectas;
- comandos destructivos;
- pérdida de datos;
- restricciones externas;
- mecanismos modernos de seguridad;
- límites de contenedores o entornos.

Tener más privilegios aumenta el impacto posible de un error.

Por eso:

▮ privilegio elevado se usa solo cuando una tarea realmente lo exige.

14. sudo y la identidad

sudo puede permitir ejecutar una orden con otra identidad de acuerdo con una política configurada.

En muchos sistemas, el caso habitual es ejecutar una orden autorizada con privilegios elevados.

No significa que debas anteponer `sudo` a cualquier comando que falle.

En este módulo no ejecutaremos ninguna orden administrativa.

15. /etc/passwd — información de cuentas

Tradicionalmente, información básica de cuentas se expone en:

```
/etc/passwd
```

A pesar de su nombre, en sistemas modernos no suele contener las contraseñas secretas en texto claro.

Una línea típica tiene campos separados por `:`.

No necesitas memorizar todos los campos todavía.

Puedes consultar de forma limitada:

```
head -n 5 /etc/passwd
```

pero **no pegues la salida completa** en el chat o en GitHub: puede revelar nombres de cuentas y estructura del sistema.

16. /etc/shadow no es material de práctica

Las credenciales y datos sensibles relacionados con autenticación pueden almacenarse en archivos protegidos como:

```
/etc/shadow
```

No intentes abrirlo con `sudo` para completar el módulo.

No necesitamos su contenido para aprender usuarios y grupos.

Regla: nunca publiques hashes, credenciales ni archivos de autenticación.

17. /etc/group — información de grupos

Información tradicional sobre grupos se encuentra en:

```
/etc/group
```

Puedes observar unas pocas líneas:

```
head -n 5 /etc/group
```

De nuevo, no compartas el archivo completo si no es necesario.

La herramienta `id` es mejor para consultar tus pertenencias actuales sin revisar manualmente grandes archivos.

18. Nombre visible frente a identidad efectiva

Una sesión puede tener conceptos de identidad más complejos que “el nombre con el que inicié sesión”.

Por eso `whoami` se define alrededor del **usuario efectivo**.

Más adelante, al estudiar procesos y privilegios, veremos con más detalle:

- identidad real;
- identidad efectiva;
- cambios de privilegio;
- procesos.

Por ahora basta con saber que `whoami` responde la identidad efectiva que la herramienta observa.

19. Preparar la práctica

Entra al laboratorio:

```
cd ~/linux-lab
pwd
```

No necesitas crear archivos para consultar tu identidad.

Si quieres registrar tus respuestas, crea una carpeta de práctica:

```
ls -ld ./modulo-11-identidad
```

Si no existe:

```
mkdir modulo-11-identidad
```

No guardes automáticamente en ella la salida completa de archivos de cuentas.

20. Práctica A — `whoami`

Ejecuta:

```
whoami
```

Anota para ti el nombre mostrado.

Pregunta:

¿esto representa el UID numérico?

No. Representa el nombre de la identidad efectiva.

21. Práctica B — `id`

Ejecuta:

```
id
```

Sin copiar la salida completa al chat, identifica:

- UID;
- nombre del usuario;
- GID primario;
- nombre del grupo primario;
- grupos adicionales, si aparecen.

22. Práctica C — consultas específicas

Ejecuta una por una:

```
id -u
id -un
id -g
id -gn
id -G
id -Gn
```

Relaciona:

```
-u → usuario
-g → grupo primario
-G → grupos
-n → nombres en vez de IDs numéricos
```

No memorices las opciones sin entender qué dato cambia.

23. Práctica D — comparar whoami con id -un

Ejecuta:

```
whoami
id -un
```

En una sesión normal deberían referirse al mismo usuario efectivo.

Si no comprendes un resultado inesperado, no intentes corregir usuarios o privilegios: registra la diferencia y revísala.

24. Práctica E — inspección limitada de cuentas

Ejecuta:

```
head -n 5 /etc/passwd
```

Después:

```
head -n 5 /etc/group
```

Objetivo:

- reconocer que existen archivos tradicionales de información de cuentas y grupos;
- no memorizar cada línea;
- no modificar nada.

No subas estas salidas a GitHub.

25. Práctica F — buscar tu propio nombre de usuario de forma controlada

Primero obtén tu nombre con:

```
whoami
```

Luego consulta tu propia cuenta. La orden siguiente usa `$(id -un)`: Bash ejecuta primero `id -un` y coloca su resultado en ese lugar, así no necesitas escribir tu nombre. Esta sintaxis se estudia a fondo en el Módulo 24.

```
getent passwd "$(id -un)"
```

`id -un` obtiene el nombre de usuario actual; `getent passwd` consulta la base de cuentas mediante NSS, que también puede integrar servicios de directorio como LDAP o SSSD. Buscar solo en `/etc/passwd` con `grep` no permite concluir que una cuenta no existe si la búsqueda queda vacía.

Si la línea aparece, no la publiques completa. Úsala solo para reconocer la relación entre nombre de cuenta y registro del sistema.

26. Usuarios de servicio

Si observas `/etc/passwd`, puedes encontrar nombres que no correspondan a personas.

Eso es normal.

Muchos servicios usan cuentas separadas para limitar privilegios y aislar responsabilidades.

No elimines una cuenta porque “no reconoces su nombre”.

Modificar cuentas de servicio sin entenderlas puede romper aplicaciones o el sistema.

27. UID bajo o alto no define por sí solo “humano” o “seguro”

Las distribuciones suelen reservar rangos para diferentes tipos de cuentas, pero los rangos concretos dependen de políticas y distribución.

No memorices una regla universal como:

UID menor que X = servicio
UID mayor que X = persona

sin consultar la política real del sistema.

Para identificar tu propia cuenta usa herramientas como `id`, no una suposición basada solo en el número.

28. Errores frecuentes

Error	Problema	Corrección
Confundes nombre con UID	Uno es etiqueta legible; otro identificador numérico	Usa <code>id -un</code> frente a <code>id -u</code>
Confundes UID con GID	Usuario y grupo son conceptos distintos	Recuerda U=user, G=group
Crees que solo existen cuentas humanas	Servicios también usan cuentas	No elimines cuentas desconocidas
Ejecutas todo con root	Amplías innecesariamente el impacto	Usa usuario normal
Antepones <code>sudo</code> si algo falla	Privilegios no corrigen errores conceptuales	Revisa la tarea y la ruta
Intentas leer <code>/etc/shadow</code>	Es información sensible y no necesaria	No lo hagas
Publicas <code>/etc/passwd</code> completo	Revela información del sistema	Comparte solo el mínimo necesario

29. Método para interpretar `id`

Si ves una salida compleja de `id`:

1. localiza `uid=`;
2. identifica número y nombre;
3. localiza `gid=`;
4. identifica grupo primario;
5. localiza `groups=`;
6. separa cada pertenencia;
7. no cambies nada solo porque haya grupos que no reconozcas.

30. Práctica independiente

Sin copiar la tabla de opciones:

1. muestra tu nombre efectivo;
2. consulta tu UID numérico;
3. consulta el nombre asociado a ese usuario;
4. consulta el GID primario;
5. consulta el nombre del grupo primario;
6. muestra tus grupos por nombre;
7. explica la diferencia entre UID y GID;
8. explica por qué no debemos practicar como root.

No uses `sudo`.

31. Mini evaluación

1. ¿Qué significa UID?
 - A. User Identifier.
 - B. Universal Internet Directory.
 - C. User Installation Driver.
2. ¿Qué significa GID?
 - A. General Internet Data.
 - B. Group Identifier.
 - C. GNU Internal Disk.
3. ¿Qué muestra `whoami` en GNU Coreutils?
 - A. La identidad efectiva por nombre.
 - B. Todos los usuarios.
 - C. Los archivos del usuario.
 - D. La contraseña.
4. ¿Qué muestra `id -u`?
 - A. UID numérico.
 - B. GID.
 - C. Nombre del host.
 - D. Ruta actual.
5. ¿Qué muestra `id -g`?
 - A. UID.

- B. GID primario.
 - C. Todos los procesos.
 - D. La shell.
6. ¿Puede un usuario pertenecer a varios grupos?
- A. Sí.
 - B. No.
7. ¿Debemos abrir /etc/shadow con sudo para esta lección?
- A. Sí.
 - B. No.
8. ¿Trabajar como root reduce el impacto de errores?
- A. Sí.
 - B. No.

32. Registro de aprendizaje

Puedes responder:

```
Un usuario es:
Un grupo es:
UID significa:
GID significa:
whoami muestra:
id -u muestra:
id -g muestra:
id -Gn muestra:
¿Por qué no debo trabajar como root?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

Antes de compartir respuestas, elimina nombres de cuenta privados si no son necesarios.

33. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Coreutils — whoami: https://www.gnu.org/software/coreutils/manual/html_node/whoami-invocation.html
- GNU Coreutils — id: https://www.gnu.org/software/coreutils/manual/html_node/id-invocation.html
- Linux man-pages — passwd(5): <https://man7.org/linux/man-pages/man5/passwd.5.html>
- Linux man-pages — group(5): <https://man7.org/linux/man-pages/man5/group.5.html>
- Linux man-pages — credentials: <https://man7.org/linux/man-pages/man7/credentials.7.html>

Se posponen:

- creación y eliminación de cuentas;
- useradd, usermod, userdel;
- groupadd y administración de grupos;
- contraseñas y passwd;
- PAM;
- NSS;
- LDAP y directorios;
- capacidades de Linux;
- identidad real, efectiva y saved IDs en profundidad;
- sudoers;
- ACL.

Estado de la lección: redactada y revisada documentalmente. La práctica real del estudiante sigue pendiente.

Módulo 12 — Permisos, propietarios, chmod, chown, umask, sudo y ACL básica

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Duodécima entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- interpretar los permisos básicos mostrados por `ls -l`;
- distinguir propietario, grupo y otros;
- explicar qué significan `r`, `w` y `x` en archivos y directorios;
- cambiar permisos de un archivo propio con `chmod`;
- interpretar los modos numéricos básicos como 600, 640, 644, 700, 750 y 755;
- explicar qué hace `chown` y por qué no debe usarse sin necesidad;
- explicar `umask` sin confundirlo con una resta decimal;
- comprender qué problema resuelven las ACL;
- explicar para qué sirve `sudo` y por qué no debe anteponerse automáticamente a una orden que falla.

Conocimientos previos: - usuarios, UID, grupos y GID; - `ls -l`; - creación de archivos y directorios; - navegación segura en `~/linux-lab`.

Seguridad: modificar permisos puede impedir el acceso a archivos o exponerlos a otros usuarios. Todas las prácticas de escritura se realizan solo sobre objetos creados expresamente dentro de `~/linux-lab/modulo-12-permisos`. No cambiamos permisos del sistema. No usamos `sudo chmod`, `sudo chown` ni `chmod -R` en este módulo.

2. Leer la primera columna de ls -l

Ejecuta sobre un archivo propio:

```
ls -l archivo.txt
```

Una salida ilustrativa podría empezar así:

```
-rw-r-----
```

Los primeros diez caracteres se interpretan así:

```
- rw- r-- ---
| | | |
| | | |  otros
| | | |  grupo
| | | |  propietario
| | | |  tipo de objeto
```

El primer carácter no es un permiso. Indica el tipo de entrada.

Ejemplos frecuentes:

```
- = archivo ordinario
d = directorio
l = enlace simbólico
```

Los nueve caracteres restantes se dividen en tres grupos de tres.

3. Las tres clases de permisos

Las tres clases básicas son:

```
u = user   = propietario
g = group  = grupo
o = others = otros
```

También existe:

```
a = all
```

que se refiere a todas las clases cuando se usa en expresiones simbólicas de `chmod`.

No confundas:

- **propietario** con “cualquier usuario”;
- **grupo** con todos los grupos del usuario;
- **otros** con un grupo llamado others.

4. r, w y x en un archivo

Para un archivo ordinario:

```
r = read    = leer contenido
w = write   = modificar contenido
x = execute = intentar ejecutarlo como programa/script
```

Ejemplo:

```
rw-
```

significa lectura y escritura, sin ejecución.

Importante: tener x no garantiza que el contenido sea un programa válido. Solo es una condición de permisos para determinadas formas de ejecución.

5. r, w y x en un directorio

En un directorio, las letras tienen efectos distintos:

- **r**: permite leer/listar nombres de entradas, sujeto a otras condiciones;
- **w**: permite modificar entradas del directorio, como crear, borrar o renombrar, sujeto a otras reglas;
- **x**: permite atravesar el directorio y acceder a entradas conocidas, si los demás permisos lo permiten.

Por eso:

El significado práctico de x depende de si el objeto es un archivo o un directorio.

No estudiaremos todavía todos los casos con sticky bit, ACL, montajes y políticas adicionales.

6. Propietario y grupo

Una salida larga puede verse conceptualmente así:

```
-rw-r----- 1 alumno proyecto ... informe.txt
```

Aquí:

- **alumno** sería el propietario;
- **proyecto** sería el grupo asociado.

Los nombres son ilustrativos.

Para consultar un archivo real propio:

```
ls -l archivo.txt
```

No necesitas cambiar propietario para aprender a leer esta información.

7. chmod — cambiar bits de modo

chmod cambia permisos o, más precisamente, bits del modo del archivo según las reglas y opciones aplicables.

Hay dos formas principales que aprenderemos:

1. simbólica;
2. numérica u octal.

Primero usaremos la simbólica porque muestra con claridad qué clase cambia.

Referencia principal: GNU Coreutils — chmod.

8. chmod simbólico

Ejemplo:

```
chmod u+x script.sh
```

Se lee:

- u: propietario;
- +: añadir;
- x: permiso de ejecución.

Otro ejemplo:

```
chmod g-w archivo.txt
```

Se lee:

- grupo;
- quitar;
- escritura.

Y:

```
chmod o-r archivo.txt
```

quita lectura a otros.

9. Operadores +, - y =

En modo simbólico:

```
+ = añadir permisos indicados
- = quitar permisos indicados
= = establecer exactamente los permisos indicados para esa clase
```

Ejemplo:

```
chmod u=rw archivo.txt
```

establece para el propietario lectura y escritura, sin ejecución.

Cuidado: = reemplaza los permisos de esa clase por los indicados.

10. Notación numérica: r=4, w=2, x=1

Cada clase puede representarse mediante la suma de:

```
r = 4
w = 2
x = 1
```

Ejemplos:

```
7 = 4 + 2 + 1 = rwx
6 = 4 + 2      = rw-
5 = 4 + 1      = r-x
4 = 4          = r--
0 = 0          = ---
```

Tres dígitos representan:

```
propietario | grupo | otros
```

11. Modos numéricos frecuentes

Ejemplos:

```
600 = rw-----
640 = rw-r-----
644 = rw-r--r--
700 = rwx-----
750 = rwxr-x---
755 = rwxr-xr-x
```

No memorices números sin traducirlos a letras.

Ejemplo

```
chmod 640 informe.txt
```

significa:

- propietario: rw-;
- grupo: r--;
- otros: ---.

12. Por qué no usamos chmod 777 como solución universal

```
777 = rwxrwxrwx
```

Concede lectura, escritura y ejecución a propietario, grupo y otros.

Eso suele ser **mucho más acceso del necesario**.

Usar `chmod 777` para “hacer que funcione” puede ocultar el problema real y ampliar innecesariamente el acceso.

Regla: aplica el mínimo permiso necesario.

13. chmod recursivo aumenta el alcance

Una opción como:

```
chmod -R ...
```

puede cambiar permisos de un árbol completo.

No se practica en este módulo.

Un error en la ruta podría afectar muchos archivos y directorios.

Aprende primero a modificar un solo objeto creado expresamente para la práctica.

14. chown — cambiar propietario y/o grupo

`chown` puede cambiar el propietario y, según la sintaxis, el grupo.

Ejemplo conceptual:

```
chown usuario archivo
chown usuario:grupo archivo
```

Cambiar propietario normalmente requiere privilegios adecuados.

En este módulo no ejecutaremos `chown`.

¿Por qué?

- no necesitamos cambiar propietario para aprender permisos;
- un cambio equivocado puede dejar archivos inaccesibles para la cuenta esperada;
- practicarlo con `sudo` enseñaría a elevar privilegios antes de comprender el impacto.

Referencia: GNU Coreutils — `chown`.

15. chgrp

Existe también `chgrp` para cambiar el grupo de un archivo.

Un usuario puede tener ciertas posibilidades de cambio de grupo según sus pertenencias y las reglas del sistema.

No lo practicaremos todavía.

Primero aprenderemos a interpretar correctamente propietario, grupo y permisos.

16. umask — máscara de creación

`umask` influye en los permisos iniciales que reciben nuevos archivos y directorios.

Consulta:

```
umask
```

Puede mostrar, por ejemplo:

```
0022
```

Tu valor puede ser diferente.

Regla importante

No enseñaremos `umask` como una simple “resta decimal”.

El modelo correcto es:

una máscara **elimina bits de permisos** del conjunto que la aplicación solicita al crear el objeto.

Para muchas demostraciones sencillas se parte conceptualmente de:

```
archivos:    0666
directorios: 0777
```

y se eliminan los bits marcados por la máscara.

Pero una aplicación puede solicitar modos diferentes, y otras reglas también pueden influir.

17. Ejemplo conceptual de `umask 022`

Si una aplicación solicita para un archivo:

```
0666 = rw-rw-rw-
```

y la máscara es:

```
0022
```

se eliminan escritura de grupo y escritura de otros:

```
0644 = rw-r--r--
```

Para un directorio solicitado con:

```
0777
```

la misma máscara suele producir:

```
0755 = rwxr-xr-x
```

No es “666 menos 22” como aritmética decimal.

18. Demostración segura de `umask` sin cambiar tu sesión principal

Cambiar `umask` afecta a la shell y a procesos descendientes. Para no alterar la configuración de tu sesión principal, usaremos una **subshell temporal**.

No necesitas dominar todavía la sintaxis de paréntesis; se estudiará más adelante.

Dentro de una carpeta de práctica vacía y verificada:

```
( umask 027; touch prueba-umask.txt; mkdir prueba-umask-dir; ls -ld prueba-umask.txt prueba-umask-dir )
```

Todo lo que está entre paréntesis se ejecuta en una subshell.

Al terminar, el `umask 027` de esa subshell desaparece con ella; tu shell principal conserva su valor anterior.

La práctica crea dos objetos dentro del laboratorio, pero no cambia permisos del sistema.

19. Interpretar `umask 027`

Conceptualmente:

```
0 = no enmascarar bits del propietario
2 = quitar escritura al grupo
7 = quitar lectura, escritura y ejecución a otros
```

Para un archivo solicitado con 0666:

```
0666
máscara 0027
→ típicamente 0640
```

Para un directorio solicitado con 0777:

```
0777
máscara 0027
→ típicamente 0750
```

Después comprueba con `ls -ld`.

20. sudo — ejecutar una orden según una política de privilegios

`sudo` permite ejecutar órdenes como otro usuario según la configuración de seguridad.

En muchos sistemas se utiliza para tareas administrativas autorizadas.

No significa:

```
"si falla, añade sudo"
```

Antes de sudo, pregunta

1. ¿la tarea realmente requiere privilegios?
2. ¿estoy usando la ruta correcta?
3. ¿entiendo qué modificaré?
4. ¿existe una alternativa sin privilegios?
5. ¿puedo revertir el cambio?

En este módulo no se ejecutan cambios administrativos mediante `sudo`.

21. sudo no corrige permisos mal entendidos

Si:

```
cat archivo.txt
```

falla por permisos, no debes concluir automáticamente:

```
sudo cat archivo.txt
```

Primero determina si tienes derecho y necesidad de leer ese archivo.

Un error de permisos puede ser una protección correcta, no un problema que debas eliminar.

22. ACL — listas de control de acceso

Los permisos tradicionales tienen tres clases:

```
propietario
grupo
otros
```

A veces eso no basta.

Una **ACL** puede permitir reglas más específicas, por ejemplo permisos para usuarios o grupos adicionales.

Conceptualmente:

```
permisos tradicionales
+
entradas ACL adicionales
```

En Linux, herramientas comunes son:

```
getfacl
setfacl
```

Su disponibilidad depende del sistema y paquetes instalados.

23. getfacl — inspección

Comprueba primero:

```
command -v getfacl
```

Si existe, sobre un archivo propio:

```
getfacl archivo.txt
```

Puede mostrar:

- propietario;
- grupo;
- entradas de usuario;
- entradas de grupo;
- máscara ACL;
- otros.

No necesitas memorizar todas las líneas todavía.

Si `getfacl` no está disponible, no instales nada para completar esta lección.

24. setfacl — modificación, solo concepto por ahora

`setfacl` modifica ACL.

Puede añadir o quitar permisos específicos.

No ejecutaremos `setfacl` en esta lección porque una entrada incorrecta puede cambiar el acceso de otros usuarios y porque todavía no hemos estudiado la máscara ACL con suficiente profundidad.

Primero aprendemos a inspeccionar.

25. La máscara ACL no es umask

No confundas:

- **umask**: influye en permisos iniciales al crear objetos;
- **mask de una ACL**: limita los permisos efectivos de determinadas entradas ACL.

Tienen nombres parecidos pero funciones diferentes.

26. El signo + en ls -l

En sistemas con soporte de ACL, `ls -l` puede mostrar un indicador adicional, por ejemplo:

```
-rw-r--r--+
```

El + puede indicar que existen datos de ACL extendidos.

No asumas que su ausencia o presencia se comporta exactamente igual en todos los sistemas y herramientas.

Si aparece, puedes investigar con `getfacl`.

27. Permisos no son toda la seguridad

Aunque un archivo muestre ciertos bits, el acceso efectivo también puede depender de:

- ACL;
- permisos de los directorios del camino;
- identidad efectiva;
- capacidades;
- políticas MAC como SELinux o AppArmor;
- sistema de archivos;
- montajes;
- atributos adicionales.

No necesitas dominar todo eso hoy.

La lección correcta es:

rwx es fundamental, pero no representa por sí solo todo el modelo de seguridad de Linux.

28. Preparar la práctica

Entra al laboratorio:

```
cd ~/linux-lab
pwd
ls -la
```

Comprueba:

```
ls -ld ./modulo-12-permisos
```

Si no existe:

```
mkdir modulo-12-permisos
```

Después:

```
cd ./modulo-12-permisos
pwd
ls -la
```

No reutilices nombres sin inspeccionarlos.

29. Crear objetos de práctica

Si los nombres están libres:

```
touch privado.txt
touch compartido.txt
mkdir carpeta-privada
```

Comprueba:

```
ls -ld privado.txt compartido.txt carpeta-privada
```

Anota los permisos iniciales. Pueden variar según tu umask.

30. Práctica A — chmod simbólico

Primero:

```
chmod u=rw,g=,o= privado.txt
```

Esto intenta dejar:

```
rw-----
```

Comprueba:

```
ls -l privado.txt
```

Explica:

- propietario: lectura y escritura;
- grupo: sin permisos;
- otros: sin permisos.

31. Práctica B — añadir lectura al grupo

Ejecuta:

```
chmod g+r privado.txt
```

Comprueba:

```
ls -l privado.txt
```

Ahora deberías poder reconocer:

```
rw-r-----
```

si no hay otros factores que alteren la presentación.

Después quita de nuevo la lectura:

```
chmod g-r privado.txt
```

32. Práctica C — modo numérico

Sobre compartido.txt:

```
chmod 640 compartido.txt
```

Comprueba:

```
ls -l compartido.txt
```

Traduce 640:

```
6 = rw-
4 = r--
0 = ---
```

No avances hasta poder explicar cada dígito.

33. Práctica D — permisos de directorio

Sobre la carpeta creada:

```
chmod 700 carpeta-privada
```

Comprueba:

```
ls -ld carpeta-privada
```

Interpreta:

```
rwX-----
```

En un directorio, x permite atravesarlo, no “ejecutar la carpeta”.

34. Práctica E — consultar umask

Ejecuta:

```
umask
```

Anota el valor para ti.

No lo cambies en la shell principal.

Después, si los nombres prueba-umask.txt y prueba-umask-dir están libres, realiza la demostración aislada:

```
( umask 027; touch prueba-umask.txt; mkdir prueba-umask-dir; ls -ld prueba-umask.txt prueba-umask-dir )
```

Al terminar, vuelve a ejecutar:

```
umask
```

Debería mostrar el mismo valor que antes en tu shell principal.

35. Práctica F — inspección ACL opcional

Comprueba:

```
command -v getfacl
```

Si existe:


```
getfacl compartido.txt
```

Solo interpreta:

- owner;
- group;
- user;
- group;
- other.

No modifiques ACL todavía.

36. Práctica G — leer propietario y grupo

Ejecuta:

```
ls -l compartido.txt
id
```

Compara:

- propietario del archivo;
- grupo del archivo;
- tu usuario;
- tus grupos.

No uses `chown` aunque los nombres no coincidan como esperabas. Primero entiende por qué.

37. Qué no haremos

No ejecutes en esta lección:

```
sudo chmod ...
sudo chown ...
chmod -R ...
chmod 777 ...
chown -R ...
setfacl ...
```

No porque todas esas formas sean siempre inválidas, sino porque aumentan el impacto o requieren contexto que aún no hemos estudiado.

38. Errores frecuentes

Error	Problema	Corrección
Lees <code>rwX</code> igual en archivo y directorio	<code>x</code> y otros bits tienen efectos distintos	Interpreta según el tipo de objeto
Usas <code>chmod 777</code> para resolver permisos	Das acceso excesivo	Aplica mínimo privilegio
Usas <code>chmod -R</code> sin revisar ruta	Amplías el alcance	Practica primero sobre un objeto
Intentas <code>chown</code> con <code>sudo</code> para aprender	Cambias propiedad sin necesidad	Solo estudia sintaxis en este módulo
Tratas <code>umask</code> como resta decimal	Modelo incorrecto	Piensa en bits enmascarados
Confundes <code>umask</code> con máscara ACL	Son mecanismos diferentes	Separa creación inicial de ACL efectiva
Añades <code>sudo</code> ante "Permission denied"	Saltas el diagnóstico	Revisa necesidad, ruta y permisos
Crees que <code>rwX</code> explica todo	Ignoras ACL y otros controles	Recuerda que es la capa básica

39. Método para diagnosticar un problema de permisos

Antes de cambiar nada:

1. ejecuta `whoami`;
2. ejecuta `id`;
3. inspecciona el objeto con `ls -ld`;
4. identifica propietario y grupo;
5. traduce los permisos de propietario, grupo y otros;
6. revisa permisos de los directorios del camino si procede;
7. solo después decide si un cambio es necesario.

No empieces con `sudo chmod`.

40. Práctica independiente

Dentro de una nueva carpeta de tu laboratorio:

1. crea un archivo;
2. consulta sus permisos iniciales;
3. establece 600;
4. traduce el resultado a letras;
5. cambia simbólicamente para añadir lectura al grupo;
6. comprueba el resultado;
7. crea un directorio con 700;
8. explica qué significa x en ese directorio;
9. consulta umask;
10. explica por qué no usarías chmod 777.

No uses sudo, chown, chmod -R ni setfacl.

41. Mini evaluación

1. En -rw-r-----, ¿quién tiene escritura?
 - A. Solo el propietario.
 - B. Grupo y otros.
 - C. Todos.
2. ¿Qué representa chmod 640 archivo?
 - A. rw-r-----
 - B. rwxrwxrwx
 - C. r-----
3. En un directorio, ¿qué permite principalmente x?
 - A. Atravesarlo/acceder a entradas conocidas según los demás permisos.
 - B. Convertirlo en programa.
 - C. Borrarlo automáticamente.
4. ¿chmod 777 es una solución general recomendada?
 - A. Sí.
 - B. No.
5. ¿umask debe explicarse como resta decimal simple?
 - A. Sí.
 - B. No.
6. ¿Qué herramienta cambia propietario?
 - A. chown
 - B. grep
 - C. pwd
7. ¿Debemos practicar sudo chown sobre archivos del sistema?
 - A. Sí.
 - B. No.
8. ¿Qué herramienta inspecciona ACL cuando está disponible?
 - A. getfacl
 - B. tail
 - C. mkdir
9. ¿umask y la máscara de ACL son lo mismo?
 - A. Sí.
 - B. No.

42. Registro de aprendizaje

Puedes responder:

```
r en un archivo significa:
w en un archivo significa:
x en un archivo significa:
x en un directorio significa:
640 significa:
700 significa:
chmod sirve para:
chown sirve para:
umask sirve para:
sudo NO debe usarse cuando:
Una ACL sirve para:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

43. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Coreutils — permisos de archivo: https://www.gnu.org/software/coreutils/manual/html_node/File-permissions.html
- GNU Coreutils — chmod: https://www.gnu.org/software/coreutils/manual/html_node/chmod-invocation.html
- GNU Coreutils — chown: https://www.gnu.org/software/coreutils/manual/html_node/chown-invocation.html
- GNU Bash Reference Manual — umask como builtin: https://www.gnu.org/software/bash/manual/html_node/Bourne-Shell-Builtins.html
- sudo manual: <https://www.sudo.ws/docs/man/sudo.man/>
- Linux man-pages — ACL: <https://man7.org/linux/man-pages/man5/acl.5.html>

Se posponen:

- setuid, setgid y sticky bit en profundidad;
- ACL por defecto;
- setfacl práctico;
- capacidades de Linux;
- SELinux y AppArmor;
- políticas sudoers;
- permisos recursivos;
- cambios de propietario administrativos.

Estado de la lección: redactada y revisada documentalmente. La práctica real del estudiante sigue pendiente.

Módulo 13 — Procesos: ps, top y htop opcional

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Decimotercera entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué es un proceso;
- distinguir programa de proceso;
- reconocer PID y PPID;
- consultar procesos con `ps`;
- seleccionar columnas útiles con `ps -o`;
- interpretar de forma inicial usuario, estado, CPU, memoria y comando;
- usar `top` para observar procesos en tiempo real;
- salir correctamente de `top`;
- reconocer `htop` como herramienta opcional, sin instalarla si no está disponible;
- diferenciar observar procesos de administrarlos o terminarlos.

Conocimientos previos: - usuarios e identidad; - permisos básicos; - lectura de comandos y salidas; - tuberías y búsqueda.

Seguridad: este módulo es de observación. No terminaremos procesos, no cambiaremos prioridades y no usaremos `sudo`. Los comandos `kill`, `killall`, `renice` y señales se reservan para el Módulo 14.

2. Programa y proceso no son lo mismo

Un **programa** es código almacenado que puede ejecutarse.

Un **proceso** es una instancia de un programa que está ejecutándose y tiene estado asociado por el sistema.

Ejemplo conceptual:

```
programa: /usr/bin/bash
proceso: una ejecución concreta de Bash
```

Puedes tener varias instancias del mismo programa ejecutándose al mismo tiempo; cada una será un proceso distinto.

3. PID — identificador de proceso

Cada proceso tiene un identificador numérico llamado:

```
PID
```

PID significa **Process ID**.

Ejemplo ilustrativo:

```
PID 4231
```

Ese número no es permanente. Puede cambiar entre ejecuciones y reutilizarse después de que un proceso termine.

Regla: nunca memorices un PID como si identificara para siempre a una aplicación.

4. PPID — proceso padre

Muchos procesos se crean a partir de otro proceso.

El identificador del proceso padre se denomina:

```
PPID
```

Ejemplo conceptual:

```
terminal
└─ shell
    └─ ps
```

Cuando ejecutas `ps` desde una shell, el proceso de `ps` normalmente tiene como padre a la shell que lo inició.

No necesitas estudiar todavía `fork()`, `exec()` ni creación de procesos a nivel de programación.

5. El usuario de un proceso

Cada proceso ejecuta con identidades asociadas.

Esto importa porque los permisos y decisiones de acceso dependen de la identidad con la que actúa.

En una salida de procesos puedes encontrar una columna como:

```
USER
```

No asumas que todos los procesos pertenecen a tu cuenta. Los servicios del sistema suelen usar otras identidades.

6. ps — fotografía de procesos

ps muestra información sobre procesos.

Ejecuta:

```
ps
```

La salida habitual, sin opciones, suele limitarse a procesos asociados a la terminal/sesión según la implementación y entorno.

Una salida ilustrativa:

PID	TTY	TIME	CMD
1234	pts/0	00:00:00	bash
5678	pts/0	00:00:00	ps

No copies esos números como si fueran los de tu sistema.

Referencia principal: ps(1) de procpd-ng.

7. Columnas básicas de ps

En una salida sencilla puedes ver:

- PID: identificador del proceso;
- TTY: terminal asociada, cuando existe;
- TIME: tiempo de CPU acumulado según el formato mostrado;
- CMD o COMMAND: nombre o comando asociado.

La selección exacta de columnas depende de las opciones y del estilo usado.

8. ps -f — formato más completo

Ejecuta:

```
ps -f
```

Puede añadir columnas como:

- UID o usuario;
- PID;
- PPID;
- hora de inicio;
- comando.

El objetivo no es memorizar cada encabezado hoy, sino reconocer PID y PPID.

9. Elegir columnas con ps -o

Una forma clara de aprender es pedir exactamente las columnas que quieres:

```
ps -o pid,ppid,user,stat,comm
```

Esto solicita:

- pid: PID;
- ppid: PPID;
- user: usuario;
- stat: estado resumido;
- comm: nombre del ejecutable/comando.

Esta forma evita depender de una salida predeterminada diferente entre opciones.

10. Estado de un proceso

Una columna como STAT resume el estado y posibles indicadores adicionales.

Estados comunes pueden incluir letras como:

```
R = running/runnable
S = sleeping interrumpible
D = espera no interrumpible
T = detenido o trazado
Z = zombie
```

No memorices todavía todos los modificadores.

Importante: “sleeping” no significa necesariamente un problema. Muchos procesos esperan eventos la mayor parte del tiempo.

11. Qué es un proceso zombie

Un proceso en estado Z es un proceso que ya terminó, pero cuya información de terminación todavía no ha sido recogida por su padre.

No significa que esté “ejecutándose muerto” ni que debas intentar eliminarlo con comandos al azar.

Este tema se retomará con señales y procesos padre.

Por ahora solo reconoce la letra Z si aparece.

12. ps -e — más procesos

```
ps -e
```

solicita una vista de todos los procesos accesibles en ese contexto, no solo los asociados a tu terminal.

Puede producir bastante salida.

Puedes combinar una selección de columnas:

```
ps -e -o pid,ppid,user,stat,comm
```

No necesitas usar sudo para esta práctica.

13. ps aux — estilo BSD, útil pero diferente

También verás frecuentemente:

```
ps aux
```

Es una sintaxis muy extendida que usa opciones de estilo BSD sin guion para esas letras.

Puede mostrar columnas como:

- USER;
- PID;
- %CPU;
- %MEM;
- VSZ;
- RSS;
- STAT;
- START;
- COMMAND.

No mezcles opciones de distintos estilos sin saber qué significan. Para aprender de forma gradual, este manual prioriza ps -o cuando queremos columnas concretas.

14. CPU y memoria: no confundir métricas

En listados de procesos puedes encontrar:

```
%CPU
%MEM
VSZ
RSS
```

Interpretación inicial:

- %CPU: uso de CPU según el cálculo de la herramienta;
- %MEM: proporción de memoria física según el cálculo mostrado;
- VSZ: tamaño del espacio de memoria virtual del proceso;
- RSS: memoria residente aproximada en RAM.

No interpretes VSZ como “RAM realmente consumida” sin contexto.

La memoria de procesos incluye conceptos compartidos, virtuales y residentes que se estudiarán más adelante.

15. Filtrar ps con grep: cuidado con la coincidencia

Un ejemplo educativo:

```
ps -e -o pid,user,comm | grep 'bash'
```

Puede mostrar procesos cuyo texto coincida con bash.

Pero grep también puede aparecer en ciertos patrones de búsqueda, y buscar texto no es igual que consultar procesos de forma estructurada.

En módulos posteriores conocerás herramientas como pgrep.

Por ahora usa grep solo para practicar lectura, no para automatizar decisiones sobre procesos.

16. top — vista dinámica

top muestra una vista interactiva que se actualiza periódicamente.

Ejecuta:

```
top
```

Puedes observar:

- carga del sistema;
- número de tareas/procesos;
- CPU;
- memoria;
- tabla de procesos.

Para salir:

```
q
```

No cierres la terminal por la fuerza para salir.

17. top no modifica procesos solo por abrirlo

Abrir top para observar no cambia automáticamente procesos.

Sin embargo, top también posee funciones interactivas capaces de enviar señales o cambiar prioridades.

En este módulo no usamos esas funciones.

Solo observamos y salimos con q.

18. Interpretar la zona superior de top

La cabecera de top puede mostrar información como:

- tiempo de actividad;
- carga promedio;
- número de tareas;
- distribución de CPU;
- memoria.

El formato y etiquetas pueden variar entre versiones.

Qué significa load average

En Linux, los tres valores de carga promedio representan promedios de **1, 5 y 15 minutos**.

La documentación de `/proc/loadavg` los describe a partir del número de tareas que están:

- en estado R: ejecutándose o listas para ejecutarse;
- en estado D: espera no interrumpible, normalmente asociada a I/O.

Ejemplo conceptual:

```
load average: 0.40, 0.25, 0.20
               1m   5m  15m
```

Esto **no es un porcentaje de CPU**.

```
load average ≠ %CPU
```

Un valor de carga tampoco puede interpretarse correctamente sin contexto. Debes considerar, entre otras cosas:

- cuántas CPU lógicas tiene el sistema;
- cuánto tiempo dura la carga;
- si existen tareas en espera no interrumpible;
- uso de memoria;
- I/O;
- contexto de la aplicación.

Por ejemplo, una carga de 4.0 no significa automáticamente «400 % de CPU» ni demuestra por sí sola que exista un problema.

También evita una simplificación opuesta:

❗ «load average solo cuenta procesos usando CPU».

En Linux puede incluir tareas en estado D, por lo que una carga elevada puede estar relacionada con espera de I/O y no únicamente con trabajo activo de CPU.

No conviertas un solo número alto observado durante un instante en un diagnóstico.

El diagnóstico de rendimiento en profundidad queda fuera de esta introducción.

19. Ordenar en top

top suele permitir ordenar procesos por diferentes métricas mediante teclas interactivas.

No necesitamos memorizar todas.

Para esta lección:

1. abre top;
2. observa columnas;
3. identifica PID, usuario, CPU, memoria y comando;
4. pulsa q.

Más adelante aprenderás navegación avanzada.

20. htop — herramienta opcional

htop es un visor interactivo de procesos con una interfaz diferente.

Comprueba si existe:

```
command -v htop
```

Si muestra una ruta, puedes ejecutarlo:

```
htop
```

Normalmente puedes salir con:

```
q
```

o la tecla indicada por su interfaz.

Si no está instalado, **no instales nada solo para completar este módulo**.

htop es opcional; ps y top son suficientes para los objetivos básicos.

21. /proc y los procesos

En el Módulo 5 aprendiste que /proc expone información del kernel y procesos.

Muchos directorios numéricos bajo `/proc` corresponden a PID existentes.

Ejemplo conceptual:

```
/proc/1234/
```

No abras archivos sensibles de procesos ajenos ni recorras `/proc` sin necesidad.

Las herramientas `ps` y `top` presentan información de forma más adecuada para esta etapa.

22. Un proceso puede terminar entre dos consultas

Si ejecutas:

```
ps -e
```

y unos segundos después repites la orden, la lista puede cambiar.

Esto es normal:

- procesos terminan;
- procesos nuevos aparecen;
- PID se asignan y eventualmente se reutilizan.

Por eso una “fotografía” de procesos envejece rápidamente.

23. Preparar la práctica

En este módulo no necesitas crear archivos, pero puedes conservar notas dentro de:

```
~/linux-lab/modulo-13-procesos
```

Comprueba primero:

```
cd ~/linux-lab
pwd
ls -ld ./modulo-13-procesos
```

Si no existe:

```
mkdir modulo-13-procesos
```

Después:

```
cd ./modulo-13-procesos
pwd
```

No guardes automáticamente listados completos de procesos en GitHub porque pueden contener nombres de usuario, rutas o comandos privados.

24. Práctica A — ps básico

Ejecuta:

```
ps
```

Identifica:

- el proceso de tu shell, si aparece;
- el propio proceso de `ps`;
- sus PID.

No memorices los PID.

25. Práctica B — PID y PPID

Ejecuta:

```
ps -o pid,ppid,user,stat,comm
```

Busca la fila de tu shell y la de `ps`.

Pregunta:

¿Qué relación observas entre el PPID de `ps` y el PID de la shell desde la que lo ejecutaste?

En una situación normal, el padre de `ps` será la shell que lo lanzó.

26. Práctica C — lista amplia

Ejecuta:

```
ps -e -o pid,ppid,user,stat,comm
```

No pegues toda la salida en el chat.

Observa:

- procesos de distintos usuarios;
- variedad de estados;
- nombres de comandos.

Si aparecen procesos que no reconoces, no los termines.

27. Práctica D — buscar Bash

Ejecuta:

```
ps -e -o pid,user,comm | grep 'bash'
```

Interpreta el resultado.

Si no aparece nada, no significa necesariamente que el sistema esté mal: tu shell puede ser otra o el nombre mostrado puede variar según entorno.

Consulta tu shell actual según lo aprendido en módulos anteriores si necesitas contexto.

28. Práctica E — top

Ejecuta:

```
top
```

Observa durante unos segundos:

- PID;
- USER;
- %CPU;
- %MEM;
- COMMAND.

Después sal con:

```
q
```

No envíes señales ni cambies prioridades desde `top`.

29. Práctica F — htop opcional

```
command -v htop
```

Si existe:

```
htop
```

Solo observa y sal.

Si no existe:

```
htop no disponible; práctica omitida
```

Eso es suficiente.

30. Práctica G — observar un proceso temporal de forma segura

Abre una primera terminal y ejecuta:

```
sleep 60
```

`sleep` espera durante aproximadamente 60 segundos.

Mientras sigue ejecutándose, abre una segunda terminal y usa:

```
ps -e -o pid,ppid,user,stat,comm | grep 'sleep'
```

Observa el proceso.

Cuando termine el tiempo desaparecerá por sí mismo.

Si quieres terminar la espera antes, vuelve a la primera terminal y pulsa:

```
Ctrl+C
```

No uses `kill` todavía; se estudiará en el Módulo 14.

31. Qué significa “sleeping”

Mientras `sleep 60` espera, puede aparecer en un estado de espera como `S`.

Eso no significa que el proceso esté dañado.

Un proceso puede estar esperando:

- tiempo;
- entrada;
- red;
- disco;
- eventos.

El estado debe interpretarse según el propósito del proceso.

32. No terminar procesos desconocidos

Si ves un proceso con mucho CPU o memoria:

no lo termines solo por ese dato.

Primero se necesita:

- identificar qué es;
- confirmar quién lo inició;
- entender su función;
- comprobar si el valor es sostenido;
- saber el impacto de detenerlo.

El Módulo 14 enseñará señales con un proceso creado expresamente para laboratorio.

33. Errores frecuentes

Error	Problema	Corrección
Confundes programa con proceso	Archivo ejecutable e instancia en ejecución no son lo mismo	Distingue código almacenado de ejecución
Memorizas un PID	Los PID cambian y pueden reutilizarse	Consulta el PID actual
Creas que <code>S</code> significa fallo	Muchos procesos esperan normalmente	Interpreta estado según contexto
Lees <code>VSZ</code> como RAM real exacta	Memoria virtual y residente son métricas distintas	Usa contexto; no simplifiques
Confundes load average con %CPU	Son métricas diferentes	Interpreta carga como tareas R/D promediadas en 1, 5 y 15 min
Creas que load average solo refleja CPU	También puede incluir tareas en espera no interrumpible	Considera I/O y estado D
Terminas procesos desconocidos	Puedes afectar servicios o datos	Solo observa en este módulo
Usas <code>sudo top</code>	Privilegios innecesarios	Ejecuta como usuario normal
Instalas <code>htop</code> solo por la práctica	Es opcional	Omite si no está disponible
Pegas todo <code>ps -e</code> en GitHub	Puede exponer datos del sistema	Comparte solo fragmentos revisados

34. Método para analizar un proceso

Cuando encuentres un proceso:

1. anota su PID actual;
2. identifica USER;
3. identifica COMMAND/COMM;
4. observa STAT;
5. observa CPU y memoria solo como indicadores;
6. no actúes todavía;
7. busca contexto antes de tomar una decisión.

Diagnóstico antes que intervención.

35. Práctica independiente

Sin copiar exactamente los ejemplos:

1. muestra tus procesos asociados a la sesión;
2. pide columnas PID, PPID, usuario, estado y comando;
3. amplía la consulta a más procesos;
4. identifica al menos un proceso de tu usuario;
5. abre top, localiza PID y CPU y sal correctamente;
6. comprueba si htop está disponible;
7. explica por qué no debes terminar un proceso que no reconoces.

No uses sudo, kill, pkill, killall ni cambios de prioridad.

36. Mini evaluación

1. ¿Qué es un proceso?
 - A. Una instancia de un programa en ejecución.
 - B. Un directorio.
 - C. Un permiso.
2. ¿Qué significa PID?
 - A. Process ID.
 - B. Program Installation Directory.
 - C. Permission Identity Data.
3. ¿Qué significa PPID?
 - A. PID del proceso padre.
 - B. Permiso privado.
 - C. Puerto de red.
4. ¿ps muestra una fotografía del estado de procesos?
 - A. Sí.
 - B. No.
5. ¿top actualiza la vista periódicamente?
 - A. Sí.
 - B. No.
6. ¿Cómo sales de top en la práctica?
 - A. q
 - B. rm
 - C. sudo
7. ¿Debes instalar htop obligatoriamente?
 - A. Sí.
 - B. No.
8. ¿Debes terminar un proceso desconocido solo porque usa CPU?
 - A. Sí.
 - B. No.
9. ¿load average es lo mismo que porcentaje de CPU?
 - A. Sí.
 - B. No.
10. ¿los tres valores de load average representan aproximadamente 1, 5 y 15 minutos?
 - A. Sí.
 - B. No.
11. ¿en Linux la carga puede incluir tareas en estado D además de tareas R?
 - A. Sí.
 - B. No.
12. ¿VSZ equivale siempre a RAM física realmente ocupada?

- A. Sí.
- B. No.

37. Registro de aprendizaje

Puedes responder:

```
Un proceso es:
PID significa:
PPID significa:
ps sirve para:
top sirve para:
load average significa:
Sus intervalos son:
¿Por qué no equivale a %CPU?:
htop es:
STAT representa:
Un estado S puede significar:
¿Por qué no debo terminar procesos desconocidos?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

No compartas listados completos de procesos si contienen nombres, rutas o comandos privados.

38. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- procps-ng / ps(1): <https://man7.org/linux/man-pages/man1/ps.1.html>
- procps-ng upstream — top(1): <https://gitlab.com/procps-ng/procps/-/blob/master/man/top.1>
- Linux man-pages project — proc_loadavg(5), en el libro PDF de la edición 6.17 (el 9 de octubre de 2026 la edición más reciente publicada es la 6.19): <https://www.kernel.org/pub/linux/docs/man-pages/book/man-pages-6.17.pdf>
- Linux kernel documentation — /proc filesystem: <https://docs.kernel.org/filesystems/proc.html>
- Linux man-pages — proc(5): <https://man7.org/linux/man-pages/man5/proc.5.html>
- htop: <https://htop.dev/>

Puntos verificados documentalmente:

- top muestra la carga media del sistema para 1, 5 y 15 minutos;
- /proc/loadavg define sus tres primeros campos a partir de tareas en cola de ejecución (estado R) o en espera no interrumpible de I/O (estado D) promediadas en 1, 5 y 15 minutos;
- load average no es un porcentaje de CPU;
- una carga elevada requiere contexto de CPU, duración, I/O, memoria y aplicación antes de diagnosticar un problema.

Se posponen:

- señales;
- kill, pkill, killall;
- jobs, fg y bg;
- prioridades, nice y renice;
- cgroups;
- namespaces;
- afinidad de CPU;
- diagnóstico profundo de memoria;
- procesos zombie y huérfanos en profundidad.

Estado de la lección: redactada y revisada documentalmente. La práctica real del estudiante sigue pendiente.

Módulo 14 — jobs, fg, bg, señales y kill

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Decimocuarta entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué es un trabajo (*job*) de la shell;
- distinguir foreground y background;
- usar `jobs` para consultar trabajos de la shell actual;
- suspender temporalmente un trabajo de laboratorio con `Ctrl+Z`;
- reanudarlo con `bg` o devolverlo al primer plano con `fg`;
- distinguir número de job de PID;
- explicar qué es una señal;
- usar `kill -TERM` sobre un proceso propio creado expresamente para la práctica;
- comprobar si el proceso terminó;
- explicar por qué `SIGKILL` no debe ser la primera opción.

Conocimientos previos: - procesos, PID y PPID; - `ps`; - terminal y Bash; - señales de interrupción vistas con `Ctrl+C`.

Seguridad: solo controlaremos procesos creados por nosotros, como `sleep`. No enviaremos señales a procesos del sistema, servicios, procesos de otros usuarios ni aplicaciones con trabajo no guardado. No usaremos `sudo`. No practicaremos `kill -9`.

2. Foreground y background

Un proceso en **foreground** está asociado al primer plano de la terminal y normalmente recibe la interacción del teclado.

Ejemplo:

```
sleep 120
```

Mientras está en primer plano, esa shell queda esperando a que `sleep` termine o sea interrumpido.

Un proceso en **background** puede seguir ejecutándose mientras la shell vuelve a aceptar órdenes.

Modelo:

```
foreground → ocupa el primer plano de la shell
background → continúa mientras recuperas el prompt
```

3. El símbolo &

En Bash puedes iniciar un comando en background agregando:

```
&
```

Ejemplo:

```
sleep 120 &
```

La shell suele mostrar algo parecido a:

```
[1] 4321
```

Esto ilustra dos identificadores distintos:

- `[1]` = número de job de esa shell;
- `4321` = PID del proceso, ejemplo ilustrativo.

No memorices esos números.

4. Job no es lo mismo que proceso

Un **job** es la forma en que una shell interactiva agrupa y controla una pipeline o comando lanzado desde esa shell.

Un job puede estar asociado a uno o varios procesos.

Por eso:

```
job number ≠ PID
```

Ejemplo:

```
%1    → job 1 de la shell
4321  → PID
```

No intercambies uno por otro sin saber qué herramienta espera cada forma.

5. jobs — trabajos de la shell actual

Ejecuta:

```
jobs
```

Puede mostrar trabajos:

- ejecutándose;
- detenidos;
- terminados recientemente.

jobs se refiere a la shell interactiva actual. No es un listado global de todos los procesos del sistema.

6. Suspender con Ctrl+Z

Si tienes en foreground:

```
sleep 120
```

puedes pulsar:

```
Ctrl+Z
```

El **controlador de terminal** envía **SIGTSTP** al grupo de procesos en primer plano cuando pulsas Ctrl+Z; después, la shell recupera el prompt. **SIGTSTP** puede gestionarse por el programa, a diferencia de **SIGSTOP**, que no puede capturarse ni ignorarse.

Después:

```
jobs
```

puede mostrar el trabajo como detenido.

Importante: suspender no es lo mismo que terminar. El proceso sigue existiendo, pero está detenido.

7. bg — continuar en background

Si el trabajo suspendido es el job actual, puedes ejecutar:

```
bg
```

o de forma explícita:

```
bg %1
```

si el trabajo es %1.

Esto reanuda el trabajo en background.

Después:

```
jobs
```

debería mostrarlo como ejecutándose.

8. fg — traer al foreground

Para devolver un trabajo al primer plano:

```
fg %1
```

La shell vuelve a asociarlo al foreground.

Si `sleep` sigue esperando, la terminal quedará ocupada por ese trabajo hasta que:

- termine;
- lo suspendas;
- lo interrumpas de forma normal.

9. Ctrl+C y SIGINT

En una terminal interactiva, Ctrl+C normalmente provoca que el terminal entregue una señal de interrupción al grupo de procesos en foreground.

La señal suele ser:

```
SIGINT
```

No todos los programas reaccionan exactamente igual: una aplicación puede manejar o ignorar ciertas señales.

Para nuestro sleep de práctica, Ctrl+C normalmente lo termina.

10. Qué es una señal

Una **señal** es un mecanismo mediante el cual el sistema puede notificar a un proceso sobre un evento.

Ejemplos comunes:

```
SIGINT → interrupción
SIGTERM → solicitud de terminación
SIGKILL → terminación forzada por el kernel
SIGSTOP → detener (no se puede capturar ni ignorar)
SIGTSTP → suspensión desde el teclado (Ctrl+Z)
SIGCONT → continuar
```

No necesitas memorizar todos los números de señal.

En este manual preferimos los nombres, porque son más claros.

11. SIGTERM — primera opción para terminar

Cuando necesitas solicitar a un proceso propio que termine, la primera señal que estudiaremos es:

```
SIGTERM
```

Ejemplo:

```
kill -TERM PID
```

Sustituye PID por el PID real de un proceso **creado por ti para la práctica**.

SIGTERM permite que un programa tenga oportunidad de realizar limpieza si está preparado para manejarla.

Por eso se prefiere antes que una terminación forzada.

12. kill no significa siempre “matar”

El nombre kill puede confundir.

La utilidad envía una señal.

Ejemplo:

```
kill -TERM 4321
```

significa:

■ enviar SIGTERM al proceso con PID 4321.

La señal predeterminada de muchas implementaciones de kill es TERM, pero en este manual la escribimos explícitamente al aprender.

13. Verificar antes de enviar una señal

Antes de ejecutar:

```
kill -TERM PID
```

haz tres comprobaciones:

1. confirma el PID;
2. confirma el usuario;
3. confirma el comando.

Ejemplo:

```
ps -o pid,user,stat,comm -p PID
```

Solo después decide si es el proceso de laboratorio correcto.

14. Comprobar después

Después de SIGTERM:

```
ps -p PID
```

Si el proceso terminó, normalmente ya no aparecerá como proceso activo en esa consulta.

No envíes señales repetidamente sin comprobar el resultado.

15. SIGKILL — último recurso

SIGKILL fuerza la terminación y el proceso no puede capturarla ni ejecutar su manejo normal de esa señal.

Eso significa que puede perder la oportunidad de:

- guardar estado;
- cerrar archivos de forma ordenada;
- liberar recursos de aplicación;
- retirar archivos temporales propios.

Por eso:

SIGKILL no es la primera opción.

En este módulo **no practicamos**:

```
kill -9  
kill -KILL
```

Solo debes reconocer que existe para situaciones donde una terminación normal no funciona y el contexto justifica una acción forzada.

16. Por qué kill -9 no es “más profesional”

Usar `kill -9` inmediatamente no demuestra mayor dominio.

El enfoque correcto es:

1. identificar el proceso;
2. entender su función;
3. solicitar terminación normal;
4. esperar y comprobar;
5. escalar solo si existe una razón técnica.

La fuerza sin diagnóstico aumenta el riesgo de pérdida de trabajo o estado inconsistente.

17. SIGSTOP y SIGCONT

Conceptualmente:

```
SIGSTOP → detener  
SIGCONT → continuar
```

El control de jobs de la shell utiliza mecanismos relacionados para suspender y continuar trabajos.

No enviaremos manualmente estas señales en la práctica inicial porque `Ctrl+Z`, `bg` y `fg` permiten comprender el flujo con menos complejidad.

18. Jobspec con %

Bash permite referirse a jobs mediante sintaxis como:

```
%1  
%2
```

Ejemplo:

```
fg %1
```

Esto no significa “PID 1”.

El símbolo % indica que estás usando una referencia de job para la shell.

19. jobs -l

Puedes solicitar que jobs muestre PID además de información del job:

```
jobs -l
```

Esto ayuda a relacionar:

- número de job;
- PID;
- estado;
- comando.

Aun así, recuerda que una pipeline puede implicar más de un proceso.

20. Un job puede ser una pipeline

Ejemplo conceptual:

```
comandoA | comandoB &
```

Eso es una pipeline lanzada como job en background.

Puede involucrar más de un proceso.

No practicaremos señales sobre pipelines en este módulo. Trabajaremos con un único sleep.

21. Preparar el laboratorio

Entra:

```
cd ~/linux-lab  
pwd
```

Comprueba:

```
ls -ld ./modulo-14-jobs-senales
```

Si no existe:

```
mkdir modulo-14-jobs-senales
```

Después:

```
cd ./modulo-14-jobs-senales  
pwd
```

No necesitas crear archivos para esta práctica.

22. Práctica A — job en background

Ejecuta:

```
sleep 120 &
```

Observa la línea que devuelve la shell.

Después:

```
jobs -l
```

Identifica:

- job number;

- PID;
- estado;
- comando.

No envíes ninguna señal todavía.

23. Práctica B — foreground

Si tu job es %1:

```
fg %1
```

Ahora sleep está en foreground.

Pulsa:

```
Ctrl+Z
```

Después:

```
jobs
```

Debe aparecer detenido si la suspensión funcionó.

24. Práctica C — bg

Reanuda:

```
bg %1
```

Después:

```
jobs -l
```

Comprueba que vuelva a estar ejecutándose en background.

25. Práctica D — terminar normalmente desde foreground

Trae el trabajo:

```
fg %1
```

Y pulsa:

```
Ctrl+C
```

Después:

```
jobs
```

El trabajo debería haber terminado y puede desaparecer o mostrarse temporalmente como terminado según la shell.

Esta práctica usa la interrupción normal del foreground, no kill.

26. Práctica E — SIGTERM con PID

Crea un nuevo proceso:

```
sleep 180 &
```

Consulta:

```
jobs -l
```

Copia **solo el PID de ese sleep de práctica**.

Verifica:

```
ps -o pid,user,stat,comm -p PID
```

Sustituye PID por el número real.

Si confirma que es tu proceso sleep:

```
kill -TERM PID
```

Comprueba:

```
ps -p PID
jobs
```

No envíes otra señal si ya terminó.

27. No practiques con servicios del sistema

No uses como objetivo:

- PID 1;
- gestores de sesión;
- procesos de red;
- procesos gráficos;
- servicios;
- procesos de otros usuarios;
- procesos cuyo propósito desconozcas.

El objetivo pedagógico es aprender señales, no provocar fallos.

28. pkill y killall — existen, pero se posponen

Herramientas como:

```
pkill
killall
```

pueden seleccionar procesos por nombre u otros criterios.

Eso amplía el alcance y puede coincidir con más procesos de lo previsto.

Por ahora no las usamos.

Primero debes dominar:

- PID;
- verificación;
- SIGTERM;
- comprobación posterior.

29. Señal no garantiza una acción idéntica

SIGTERM es una solicitud de terminación, pero un programa puede:

- capturarla;
- realizar limpieza;
- retrasar su salida;
- ignorarla en ciertos diseños o condiciones.

SIGKILL, en cambio, no puede ser capturada o ignorada por el proceso.

Esta diferencia es precisamente la razón para intentar TERM primero.

30. Procesos detenidos y recursos

Un proceso suspendido deja de ejecutar instrucciones de usuario mientras permanece detenido, pero sigue existiendo y puede conservar:

- memoria;
- archivos abiertos;
- otros recursos.

No confundas “detenido” con “eliminado”.

31. Salir de una terminal con jobs activos

Cerrar una shell con trabajos activos puede producir comportamientos que dependen de:

- shell;

- configuración;
- señales;
- job control;
- cómo se lanzó el proceso.

No estudiaremos todavía `nohup`, `disown`, `terminal multiplexers` ni servicios persistentes.

Para esta práctica termina tus procesos `sleep` de laboratorio antes de cerrar.

32. Errores frecuentes

Error	Problema	Corrección
Confundes %1 con PID 1	Job y PID son espacios distintos	%1 es jobspec de la shell
Ctrl+Z “terminó” el programa	En realidad lo suspendiste	Consulta <code>jobs</code>
Usas <code>bg</code> esperando verlo en foreground	<code>bg</code> reanuda en segundo plano	Usa <code>fg</code> para traerlo
Ejecutas <code>kill -9</code> primero	Fuerzas salida sin oportunidad de limpieza	Usa <code>TERM</code> primero
Envías señal sin verificar PID	PID puede ser otro proceso	Usa <code>ps -p PID</code>
Usas <code>sudo kill</code>	Amplías privilegios innecesariamente	Practica solo con procesos propios
Usas <code>pkill</code> por comodidad	Puede coincidir con más procesos	Selecciona un PID verificado
Crees que un job detenido no consume recursos	Sigue existiendo	Reanúdalo o términalo correctamente

33. Método seguro para terminar un proceso propio

Usa esta secuencia mental:

1. identificar;
2. verificar;
3. solicitar terminación normal;
4. esperar;
5. comprobar;
6. solo escalar con una razón técnica.

Para nuestro laboratorio:

```
jobs -l
↓
ps -p PID
↓
kill -TERM PID
↓
ps -p PID
```

34. Práctica independiente

Solo con procesos `sleep` creados por ti:

1. inicia uno en background;
2. consulta su job y PID;
3. tráelo a foreground;
4. suspéndelo;
5. consulta su estado;
6. reanúdalo en background;
7. verifica su PID con `ps`;
8. términalo con `SIGTERM`;
9. confirma que terminó;
10. explica por qué no usaste `SIGKILL`.

No uses `sudo`, `pkill`, `killall` ni `kill -9`.

35. Mini evaluación

1. ¿Qué muestra `jobs`?
 - A. Trabajos de la shell actual.
 - B. Todos los archivos.
 - C. Usuarios.
2. ¿Qué hace Ctrl+Z normalmente sobre el job en foreground?
 - A. Lo suspende.
 - B. Lo borra.
 - C. Reinicia Linux.

3. ¿Qué hace `bg`?
 - A. Reanuda un job en background.
 - B. Borra un grupo.
 - C. Cambia permisos.
4. ¿Qué hace `fg`?
 - A. Lleva un job al foreground.
 - B. Finaliza obligatoriamente el proceso.
 - C. Cambia el GID.
5. ¿%1 es necesariamente PID 1?
 - A. Sí.
 - B. No.
6. ¿Qué señal debemos intentar primero para una terminación normal?
 - A. SIGTERM.
 - B. SIGKILL.
7. ¿Practicamos `kill -9` como rutina?
 - A. Sí.
 - B. No.
8. ¿Debes verificar PID y comando antes de enviar una señal?
 - A. Sí.
 - B. No.
9. ¿Un proceso suspendido deja de existir?
 - A. Sí.
 - B. No.

36. Registro de aprendizaje

Puedes responder:

```
Un job es:
Foreground significa:
Background significa:
jobs sirve para:
Ctrl+Z hace:
bg sirve para:
fg sirve para:
SIGTERM sirve para:
SIGKILL se reserva para:
¿Por qué verifico el PID antes de kill?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

37. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Bash Reference Manual — Job Control: https://www.gnu.org/software/bash/manual/html_node/Job-Control.html
- GNU Bash Reference Manual — Job Control Builtins: https://www.gnu.org/software/bash/manual/html_node/Job-Control-Builtins.html
- Linux man-pages — `kill(2)`: <https://man7.org/linux/man-pages/man2/kill.2.html>
- Linux man-pages — `signal(7)`: <https://man7.org/linux/man-pages/man7/signal.7.html>
- Linux man-pages — `kill(1)`: <https://man7.org/linux/man-pages/man1/kill.1.html>

Se posponen:

- `pkill` y `killall` prácticos;
- signal masks;
- signal handlers en programación;
- `nohup`;
- `disown`;
- `SIGHUP`;
- terminal multiplexers;
- procesos daemon;
- prioridades con `nice/renice`;
- `systemd` y servicios.

Estado de la lección: redactada y revisada documentalmente. La práctica real del estudiante sigue pendiente.

Módulo 15 — Paquetes, repositorios y actualizaciones

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Decimoquinta entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué es un paquete;
- explicar qué es un repositorio;
- distinguir paquete instalado, paquete disponible y actualización;
- comprender qué son dependencias y metadatos;
- distinguir APT de dpkg en Debian/Ubuntu;
- distinguir DNF de RPM en Fedora/RHEL;
- identificar qué familia de herramientas corresponde a tu distribución;
- explicar la diferencia entre actualizar el índice y actualizar paquetes;
- consultar información de paquetes sin modificar el sistema;
- reconocer por qué añadir repositorios de terceros requiere revisión;
- diferenciar una actualización normal de paquetes de una actualización mayor de distribución.

Conocimientos previos: - distribución y versión; - usuarios y privilegios; - archivos y rutas; - procesos; - lectura de documentación.

Seguridad: instalar, eliminar o actualizar paquetes modifica el sistema. En esta lección las prácticas obligatorias son de consulta. Los comandos administrativos se muestran para comprender su propósito, pero no deben ejecutarse hasta confirmar distribución, versión, respaldo apropiado, energía estable y el impacto previsto. No añadas `sudo` automáticamente.

2. Qué es un paquete

Un **paquete** es una unidad de distribución de software preparada para que el sistema pueda instalarla, actualizarla o retirarla de forma controlada.

Puede incluir:

- programas;
- bibliotecas;
- documentación;
- archivos de configuración;
- metadatos;
- scripts de mantenimiento.

Dos formatos importantes en las familias que estudiaremos son:

```
.deb → Debian, Ubuntu y derivadas
.rpm → RHEL, Fedora y otras distribuciones de la familia RPM
```

El formato del paquete y la herramienta de alto nivel no son exactamente lo mismo.

3. Qué es un repositorio

Un **repositorio** es una fuente organizada de paquetes y metadatos.

Modelo simplificado:

```
repositorio
├─ paquetes
├─ versiones
├─ dependencias
└─ metadatos
```

El gestor de paquetes consulta repositorios configurados para saber:

- qué software existe;
- qué versiones están disponibles;
- qué dependencias necesita;
- qué actualizaciones hay.

No confundas repositorio de paquetes con repositorio Git. Ambos almacenan contenido organizado, pero resuelven problemas diferentes.

4. Dependencias

Un paquete puede necesitar otros paquetes para funcionar.

Ejemplo conceptual:

```
aplicación A
├── biblioteca B
└── biblioteca C
```

Un gestor de paquetes de alto nivel puede resolver dependencias automáticamente según los repositorios configurados.

Eso es una diferencia importante frente a copiar ejecutables manualmente sin comprender sus requisitos.

5. Metadatos

Los gestores no descargan siempre cada paquete completo solo para saber qué existe.

Utilizan **metadatos**, que pueden describir:

- nombres;
- versiones;
- arquitectura;
- dependencias;
- repositorio;
- descripción;
- firmas o datos relacionados con verificación.

Por eso existe una diferencia entre:

```
actualizar información sobre paquetes
```

y:

```
actualizar los paquetes instalados
```

6. Identifica primero tu distribución

Antes de copiar un comando de Internet:

```
cat /etc/os-release
```

Busca campos como:

```
ID=
VERSION_ID=
PRETTY_NAME=
```

No publiques la salida completa si contiene personalizaciones privadas.

La herramienta correcta depende de la distribución y versión reales.

7. Familias que estudiaremos

En esta edición:

```
Debian / Ubuntu → APT + dpkg
Fedora / RHEL   → DNF + RPM
```

Esto es una guía curricular, no una afirmación de que solo existan esas familias.

Otras distribuciones pueden usar gestores como:

- pacman;
- zypper;
- apk;
- emerge.

No mezclaremos instrucciones entre familias.

8. APT en Debian y Ubuntu

APT significa **Advanced Package Tool**.

En Ubuntu, la documentación oficial recomienda APT para administrar paquetes Debian y repositorios configurados.

Comandos conceptuales frecuentes:


```
apt update
apt install
apt remove
apt upgrade
apt search
apt show
```

No todos modifican el sistema de la misma manera.

9. apt update NO actualiza los programas instalados

Este punto es fundamental.

```
sudo apt update
```

actualiza el **índice local de paquetes** consultando los repositorios configurados.

No significa:

■ “instala automáticamente todas las versiones nuevas de mis programas”.

Después de actualizar el índice, APT puede saber qué actualizaciones están disponibles.

En este módulo no ejecutamos `sudo apt update` como práctica obligatoria, porque escribe metadatos locales y requiere acceso administrativo en configuraciones habituales.

10. apt upgrade sí modifica paquetes instalados

Ejemplo administrativo:

```
sudo apt upgrade
```

puede descargar e instalar versiones nuevas de paquetes instalados según la resolución de APT.

Eso cambia software del sistema.

Antes de hacerlo en un equipo real se debe considerar:

- qué paquetes cambiarán;
- cuánto se descargará;
- servicios que podrían reiniciarse;
- espacio disponible;
- energía y conectividad;
- necesidad de reinicio;
- respaldo y recuperación cuando corresponda.

No se ejecuta como práctica inicial.

11. apt install

Ejemplo conceptual:

```
sudo apt install NOMBRE_PAQUETE
```

Instala un paquete y las dependencias necesarias según la resolución del gestor.

No copies nombres de paquetes al azar.

Antes de instalar:

1. identifica el paquete;
2. comprueba su origen;
3. revisa la descripción;
4. confirma qué dependencias propone;
5. verifica que corresponde a tu distribución y versión.

12. apt remove y purge

`apt remove` retira paquetes según las reglas de APT.

Una opción como `--purge` puede retirar también archivos de configuración administrados por el paquete.

Por eso **no debe tratarse como una limpieza inocua**.

En este módulo no eliminamos software.

13. apt search y apt show

Estas operaciones son útiles para investigación.

Ejemplo:

```
apt search bash
```

Busca paquetes relacionados con el término.

Para información de un paquete:

```
apt show bash
```

Dependiendo del estado de los índices, los resultados reflejan la información disponible localmente.

Estas consultas no instalan el paquete.

14. apt-cache

apt-cache permite consultar información del caché de APT.

Ejemplo seguro:

```
apt-cache policy bash
```

Puede mostrar:

- versión instalada;
- versión candidata;
- orígenes conocidos.

Es una buena herramienta para aprender sin modificar paquetes.

15. dpkg — nivel más bajo en sistemas Debian

dpkg administra paquetes Debian a nivel local.

Puede:

- consultar la base de paquetes;
- instalar un archivo .deb;
- retirar paquetes;
- listar archivos de un paquete.

Pero por sí solo no ofrece la misma resolución automática de repositorios y dependencias que APT.

Modelo:

```
APT → gestión de alto nivel, repositorios y dependencias
dpkg → gestión local de paquetes .deb y base instalada
```

16. Consultas seguras con dpkg

Puedes comprobar si bash aparece en la base de paquetes:

```
dpkg-query -W bash
```

Otra consulta:

```
dpkg-query -W -f='${Package}\t${Version}\n' bash
```

No necesitas memorizar todavía el formato -f; se muestra para distinguir nombre y versión.

Si tu sistema no es Debian/Ubuntu, no uses esta sección como práctica.

17. DNF en Fedora y RHEL

DNF es la herramienta de alto nivel que estudiaremos para la familia Fedora/RHEL.

La documentación oficial de RHEL 10 utiliza DNF para:

- buscar paquetes;
- consultar repositorios;
- instalar software;
- comprobar actualizaciones;
- actualizar paquetes.

Ejemplos conceptuales:

```
dnf search
dnf info
dnf list
dnf repolist
dnf check-update
dnf install
dnf upgrade
dnf remove
```

Nota sobre versiones: RHEL 10 documenta DNF, mientras que Fedora 41 y posteriores emplean DNF5 (cambio [SwitchToDnf5](#), comprobado el 9 de octubre de 2026). La documentación de DNF5 denomina `check-upgrade` a la comprobación de actualizaciones; no presupongas que `check-update` es un alias compatible en todas las instalaciones. Antes de aplicar ejemplos, identifica la versión con `dnf --version` y consulta la ayuda de tu distribución. Referencia: [DNF5 check-upgrade](#).

18. dnf search

Consulta:

```
dnf search bash
```

Busca el término en metadatos de paquetes disponibles para DNF.

No instala el paquete.

Si DNF necesita consultar o refrescar metadatos, puede acceder a repositorios según su configuración.

19. dnf info

Ejemplo:

```
dnf info bash
```

Puede mostrar información de paquete, como:

- nombre;
- versión;
- arquitectura;
- repositorio;
- descripción.

El formato depende de la versión de DNF.

20. dnf repolist

Consulta:

```
dnf repolist
```

muestra repositorios habilitados según la configuración.

No añadas ni habilites repositorios solo para que la salida “se vea completa”.

El número y los nombres correctos dependen de la distribución, versión, suscripción y configuración.

21. dnf check-update

La documentación de RHEL 10 utiliza:

```
dnf check-update
```

para comprobar actualizaciones disponibles.

Esta operación **consulta** actualizaciones; no las instala.

La documentación oficial de DNF define estos estados de salida para `check-update`:

```
0 → no hay actualizaciones disponibles
100 → hay actualizaciones disponibles
1 → ocurrió un error
```

Por tanto, en este comando específico:

```
estado distinto de 0 ≠ necesariamente error
```

El valor 100 es un resultado esperado cuando DNF encuentra actualizaciones. Esto conecta directamente con el **Módulo 25 — Códigos de salida**: siempre debes interpretar un estado según la documentación del programa que lo produjo.

Ejemplo de observación:

```
dnf check-update
estado=$?
printf 'Estado de dnf check-update: %s\n' "$estado"
```

No confundas “hay actualizaciones disponibles” con “ya fueron instaladas”.

22. dnf upgrade

Ejemplo administrativo:

```
sudo dnf upgrade
```

actualiza paquetes instalados según los repositorios y la resolución de DNF.

Es una operación que modifica el sistema.

No se ejecuta en este módulo.

En RHEL, las actualizaciones del kernel tienen manejo específico: las versiones nuevas pueden instalarse junto a kernels anteriores según las reglas de paquetes `installonly`.

23. RPM — nivel de paquete local

RPM es tanto un formato de paquete como una herramienta/base de gestión en sistemas de la familia RPM.

Modelo simplificado:

```
DNF → repositorios + dependencias + operaciones de alto nivel
RPM → paquete local y base RPM
```

No instalaremos archivos RPM manualmente en este módulo.

24. Consultas seguras con rpm

En un sistema RPM puedes consultar si un paquete está instalado:

```
rpm -q bash
```

Para mostrar información:

```
rpm -qi bash
```

Estas consultas no instalan ni eliminan paquetes.

25. Repositorios oficiales y de terceros

Los repositorios oficiales de una distribución forman parte de su modelo de mantenimiento.

Un repositorio de terceros puede ser legítimo, pero añade confianza adicional:

- quién publica el software;
- quién controla la clave de firma;
- qué paquetes puede sustituir;
- qué frecuencia de actualización tiene;
- qué soporte ofrece;
- si es compatible con tu versión.

No añadas un repositorio porque una página diga simplemente:

```
copia y pega estos comandos
```

Primero investiga su procedencia.

26. Firmas y confianza

Los sistemas de paquetes modernos utilizan mecanismos criptográficos para verificar procedencia e integridad de metadatos o paquetes según el diseño de la distribución.

Pero una firma válida no significa automáticamente:

“este software es seguro para cualquier propósito”.

Una firma ayuda a comprobar autenticidad/integridad respecto a una clave confiada; no sustituye revisar qué software instalas.

27. No uses curl | bash para instalar software sin revisar

Una instrucción de Internet del tipo:

```
curl ... | bash
```

descarga contenido y lo entrega directamente a una shell.

Eso combina dos riesgos:

1. ejecutas contenido remoto;
2. puedes no revisarlo antes.

En este curso no se utiliza como método normal de instalación.

Prioriza:

- repositorios oficiales;
- documentación oficial;
- paquetes identificados;
- revisión antes de ejecución.

28. Actualización de paquetes vs actualización de distribución

No confundas:

```
actualizar paquetes de la versión actual
```

con:

```
migrar a una nueva versión de la distribución
```

Por ejemplo, pasar de una versión mayor de Ubuntu o RHEL a otra puede requerir procedimientos específicos de migración.

No se hace simplemente porque exista un comando `upgrade`.

29. Reinicios y servicios

Una actualización puede modificar:

- bibliotecas;
- servicios;
- kernel;
- componentes de seguridad.

Algunas actualizaciones pueden requerir:

- reiniciar servicios;
- cerrar aplicaciones;
- reiniciar el sistema.

Por eso una actualización en un servidor o equipo importante se planifica.

30. Preparar la práctica

Primero identifica tu entorno:

```
cat /etc/os-release
```

Después comprueba qué herramientas existen:

```
command -v apt
command -v dpkg-query
command -v dnf
command -v rpm
```

No significa que debas usar todas.

Registra solo:

```
APT disponible: sí/no
dpkg-query disponible: sí/no
DNF disponible: sí/no
RPM disponible: sí/no
```

No pegues rutas personales ni datos innecesarios.

31. Práctica A — si usas Debian o Ubuntu

Solo si `/etc/os-release` confirma Debian, Ubuntu o una distribución compatible cuya documentación indique APT:

```
apt-cache policy bash
```

Después:

```
dpkg-query -W bash
```

Objetivo:

- identificar versión instalada;
- reconocer que son consultas;
- no modificar el sistema.

No ejecutes `sudo apt update`, `upgrade`, `install` o `remove` como parte de esta práctica.

32. Práctica B — búsqueda APT

En Debian/Ubuntu:

```
apt search bash
```

La salida puede ser larga.

Puedes limitarla para lectura:

```
apt search bash | head
```

Al enviar su salida a una tubería, `apt` puede mostrar el aviso `WARNING: apt does not have a stable CLI interface. Es una advertencia para quien escribe scripts, no un error de tu práctica. Una alternativa pensada para este uso es apt-cache search bash | head.`

No interpretes cada coincidencia como un paquete que debes instalar.

33. Práctica C — si usas Fedora o RHEL

Solo si tu distribución corresponde a esta familia:

```
dnf info bash
```

Después:

```
rpm -q bash
```

Objetivo:

- comparar información de alto nivel con la base RPM;
- no instalar ni actualizar nada.

34. Práctica D — repositorios DNF

En Fedora/RHEL:

```
dnf repolist
```

Solo observa los repositorios habilitados.

No ejecutes opciones para habilitar, deshabilitar o añadir repositorios.

35. Práctica E — identifica qué comando modificaría el sistema

Clasifica estos ejemplos:

```
apt-cache policy bash
apt search bash
sudo apt install paquete
sudo apt upgrade
dnf info bash
dnf repolist
sudo dnf install paquete
sudo dnf upgrade
rpm -q bash
```

Debes separar:

- consultas;
- instalaciones;
- actualizaciones.

La meta no es ejecutar los comandos administrativos.

36. No mezcles gestores

Ejemplo incorrecto de razonamiento:

“En una guía dice `apt install`, así que lo usaré en RHEL.”

No.

Primero identifica:

- distribución;
- versión;
- herramienta soportada;
- repositorio apropiado.

La sintaxis parecida entre gestores no implica compatibilidad.

37. No instales por resolver `command not found` automáticamente

Si aparece:

```
command not found
```

no respondas inmediatamente con una instalación.

Pregunta:

1. ¿escribí bien el comando?
2. ¿debería existir en esta distribución?
3. ¿la herramienta ya tiene un equivalente instalado?
4. ¿necesito realmente ese programa?
5. ¿cuál es el paquete oficial?
6. ¿qué cambios producirá instalarlo?

Esto evita llenar el sistema de herramientas innecesarias.

38. Paquetes y seguridad

Mantener software soportado y actualizado es importante para seguridad.

Pero “actualizar todo inmediatamente sin revisar” tampoco es una política universal para sistemas críticos.

En un equipo personal sencillo el proceso puede ser directo.

En producción se consideran:

- compatibilidad;

- pruebas;
- ventanas de mantenimiento;
- respaldo;
- rollback;
- dependencia entre servicios.

Este manual enseñará administración de forma gradual.

39. Historial y registros

Los gestores pueden dejar registros de operaciones.

En Ubuntu, las acciones de paquetes se registran, entre otros lugares, en archivos relacionados con dpkg/APT.

No revisaremos logs completos todavía porque pueden contener información del sistema.

El módulo de registros enseñará cómo consultar fragmentos de forma segura.

40. Errores frecuentes

Error	Problema	Corrección
Creas que apt update actualiza programas	Actualiza el índice	Distingue índice de paquetes instalados
Ejecutas upgrade sin revisar	Modifica muchos paquetes	Revisa transacción y contexto
Mezclas APT y DNF	Pertenecen a familias distintas	Identifica distribución primero
Instalas un paquete por nombre parecido	Puede ser otro software	Consulta descripción y origen
Añades repositorio de terceros sin verificar	Amplías la cadena de confianza	Investiga proveedor y compatibilidad
Copias curl ... bash	Ejecutas contenido remoto sin revisión	Evita ejecución directa
Usas sudo para consultas	Privilegios innecesarios	Consulta como usuario normal
Confundes paquete con repositorio	Uno es software empaquetado; otro es fuente	Separa conceptos
Confundes actualización de paquetes con versión mayor de distribución	Son procesos distintos	Sigue procedimiento específico

41. Método seguro antes de instalar o actualizar

Antes de una operación administrativa:

1. identifica distribución y versión;
2. identifica el gestor soportado;
3. confirma repositorios;
4. consulta el paquete;
5. revisa la transacción propuesta;
6. considera espacio, energía y conectividad;
7. considera reinicios y servicios;
8. confirma respaldo/recuperación si el impacto lo justifica;
9. ejecuta solo cuando comprendes el cambio;
10. verifica el resultado.

42. Práctica independiente

Sin modificar el sistema:

1. identifica tu distribución;
2. detecta si tienes APT o DNF;
3. consulta información del paquete Bash con la herramienta apropiada;
4. consulta si Bash está instalado mediante dpkg-query o rpm -q, según corresponda;
5. explica qué es un repositorio;
6. explica qué es una dependencia;
7. explica la diferencia entre actualizar metadatos y actualizar paquetes;
8. indica qué comandos del módulo requieren privilegios y por qué no los ejecutaste.

43. Mini evaluación

1. ¿Qué es un paquete?
 - A. Una unidad gestionable de software y metadatos.
 - B. Un usuario.
 - C. Un proceso.
2. ¿Qué es un repositorio?
 - A. Una fuente organizada de paquetes y metadatos.

- B. Un permiso.
 - C. Un PID.
3. En Ubuntu, ¿qué herramienta de alto nivel se documenta para paquetes .deb?
- A. APT.
 - B. DNF.
 - C. RPM.
4. En RHEL 10, ¿qué herramienta de alto nivel documenta Red Hat?
- A. DNF.
 - B. APT.
 - C. pacman.
5. ¿apt update instala todas las actualizaciones disponibles?
- A. Sí.
 - B. No.
6. ¿apt upgrade modifica paquetes instalados?
- A. Sí.
 - B. No.
7. ¿dnf check-update es lo mismo que instalar actualizaciones?
- A. Sí.
 - B. No.
8. Si dnf check-update devuelve 100, ¿significa necesariamente un error?
- A. Sí.
 - B. No; significa que hay actualizaciones disponibles.
9. Si dnf check-update devuelve 0, ¿qué significa?
- A. No hay actualizaciones disponibles.
 - B. La instalación falló.
10. Si dnf check-update devuelve 1, ¿qué significa?
- A. Ocurrió un error.
 - B. Hay actualizaciones disponibles.
11. ¿Debes añadir repositorios de terceros sin verificar su origen?
- A. Sí.
 - B. No.
12. ¿Es recomendable ejecutar directamente código remoto con curl ... | bash sin revisión?
- A. Sí.
 - B. No.
13. ¿Una actualización de paquetes equivale siempre a migrar de versión mayor de la distribución?
- A. Sí.
 - B. No.

44. Registro de aprendizaje

Puedes responder:

```
Un paquete es:
Un repositorio es:
Una dependencia es:
APT se usa principalmente en:
DNF se usa principalmente en:
dpkg se relaciona con:
RPM se relaciona con:
apt update hace:
apt upgrade hace:
dnf check-update hace:
Estado 0 de dnf check-update significa:
Estado 100 de dnf check-update significa:
Estado 1 de dnf check-update significa:
¿Por qué no debo tratar todo estado no-cero como error sin leer la documentación del comando?:
¿Por qué no debo instalar desde un repositorio desconocido?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

45. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales verificadas para esta edición:

- Ubuntu Server — Install and manage packages: <https://ubuntu.com/server/docs/how-to/software/package-management/>

- Debian — APT User's Guide y manuales de usuario: <https://www.debian.org/doc/user-manuals>
- DNF Project — Command Reference, `check-update`: https://dnf.readthedocs.io/en/latest/command_ref.html#check-update-command
- Red Hat Enterprise Linux 10 — Managing software with the DNF tool:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/managing_software_with_the_dnf_tool/index
- Red Hat Enterprise Linux 10 — DNF commands list:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/managing_software_with_the_dnf_tool/dnf-commands-list

Se posponen:

- instalación real de paquetes;
- eliminación real de paquetes;
- actualizaciones del sistema;
- claves y firmas en profundidad;
- pinning y prioridades;
- repositorios personalizados;
- PPAs;
- módulos de DNF;
- snapshots y rollback;
- actualizaciones automáticas;
- migraciones de versión mayor.

Estado de la lección: redactada y revisada documentalmente. Las prácticas administrativas reales permanecen pendientes y deben realizarse únicamente cuando se conozca la distribución y el impacto.

Módulo 16 — Diferencias entre Debian, Ubuntu, Fedora y RHEL

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Decimosexta entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- distinguir Debian, Ubuntu, Fedora y RHEL sin tratarlas como sistemas idénticos;
- reconocer qué distribuciones usan paquetes DEB y cuáles RPM;
- relacionar APT/dpkg con Debian/Ubuntu;
- relacionar DNF/RPM con Fedora/RHEL tradicionales;
- comprender diferencias generales de ciclo de publicación y soporte;
- reconocer diferencias de políticas y herramientas de seguridad;
- evitar copiar instrucciones de una distribución a otra sin verificar;
- identificar tu distribución y versión antes de administrar paquetes, firewall o seguridad.

Conocimientos previos: - distribución y versión; - paquetes y repositorios; - usuarios y permisos; - procesos; - lectura de `/etc/os-release`.

Seguridad: este módulo es de identificación y comparación. Las prácticas son de lectura. No instalaremos paquetes, no cambiaremos repositorios, no modificaremos firewall, SELinux ni AppArmor, y no realizaremos migraciones de versión.

2. Cuatro distribuciones, no cuatro “sabores idénticos”

Todas pueden ejecutar un kernel Linux y muchas herramientas GNU, pero difieren en aspectos como:

- proyecto y gobernanza;
- ciclo de publicación;
- política de soporte;
- selección de versiones de software;
- repositorios;
- formato y herramientas de paquetes;
- políticas de seguridad;
- documentación;
- soporte comercial;
- convenciones administrativas.

Por eso un comando válido en una distribución no debe copiarse automáticamente a otra.

3. Debian

Debian es una distribución comunitaria con varias ramas de desarrollo y mantenimiento.

La documentación oficial distingue al menos:

```
stable
testing
unstable
```

En octubre de 2026, la rama estable actual es:

```
Debian 13 "trixie"
```

Debian recomienda `stable` como la versión de producción para la mayoría de usuarios.

Fuentes: - <https://www.debian.org/releases/> - <https://www.debian.org/doc/manuals/debian-reference/>

4. Debian stable, testing y unstable

Modelo simplificado:

```
unstable → desarrollo activo
  ↓
testing  → preparación de la siguiente estable
  ↓
stable   → publicación estable
```

No interpretes `testing` como una simple “versión estable pero más nueva”.

Debian advierte que mezclar suites sin una política clara puede producir problemas de dependencias y versiones.

Para este curso, las prácticas de administración se diseñan pensando en una versión estable soportada.

5. Paquetes en Debian

Debian utiliza paquetes:

```
.deb
```

Herramientas importantes:

```
APT → gestión de alto nivel
dpkg → gestión local/base de paquetes Debian
```

En la referencia Debian actual se recomienda `apt` para operaciones interactivas y `apt-get` para scripts y determinados casos automatizados.

No reemplaces `apt` por `DNF` ni `RPM`.

6. Ubuntu

Ubuntu deriva de Debian, pero es una distribución distinta con:

- su propio ciclo de publicación;
- repositorios propios;
- selección de paquetes propia;
- políticas de soporte propias;
- herramientas y documentación específicas;
- soporte comercial de Canonical.

Que Ubuntu use paquetes DEB no significa que puedas mezclar repositorios Debian y Ubuntu.

7. Ubuntu LTS e interim

Ubuntu publica nuevas versiones aproximadamente cada seis meses.

Existen dos tipos principales:

```
LTS → Long Term Support
interim → publicación de soporte corto
```

Las versiones LTS aparecen cada dos años y reciben cinco años de mantenimiento de seguridad estándar para el componente principal; las versiones interim tienen soporte mucho más corto, actualmente nueve meses.

En 2026 existe Ubuntu 26.04 LTS.

Fuente: - <https://ubuntu.com/project/docs/release-team/ubuntu-releases/>

8. Ubuntu no es “Debian con otro fondo de pantalla”

Aunque comparten formato DEB y APT, pueden diferir en:

- versiones de paquetes;
- repositorios;
- parches;
- calendarios;
- defaults;
- integración del escritorio;
- kernel y hardware enablement;
- mecanismos de soporte;
- componentes de repositorio.

No añadas un repositorio Debian a Ubuntu ni uno Ubuntu a Debian salvo que documentación especializada y compatible lo indique explícitamente. Para principiantes, se consideran ecosistemas separados.

9. AppArmor en Ubuntu

Ubuntu documenta **AppArmor instalado y cargado por defecto**.

AppArmor es un Linux Security Module que aplica control de acceso obligatorio mediante perfiles.

Consulta de estado, si la herramienta existe:

```
command -v aa-status
```

y, si está disponible:

```
aa-status
```

No deshabilites AppArmor para “resolver” un problema sin diagnóstico.

Fuente: - <https://ubuntu.com/server/docs/security-apparmor/>

10. Debian y AppArmor: estado predeterminado y verificación

Debian habilita AppArmor de forma predeterminada desde Debian 10, según su documentación oficial. Esto no significa que todas las instalaciones tengan los mismos perfiles cargados ni que el servicio esté activo en un sistema modificado. Para comprobarlo, consulta `aa-status` si la herramienta está disponible; no actives ni cambies perfiles durante la práctica.

Fuente: <https://wiki.debian.org/AppArmor/HowToUse>

Debian dispone de AppArmor y documentación relacionada, pero este manual **no afirmará que todas las instalaciones Debian tengan exactamente el mismo estado o conjunto de perfiles por defecto**.

La configuración puede depender de:

- versión;
- instalación;
- paquetes;
- entorno;
- decisiones del administrador.

Regla:

consulta el sistema real; no heredes automáticamente un “default” de Ubuntu hacia Debian.

11. Fedora

Fedora Linux es una distribución comunitaria de evolución rápida.

En sistemas Fedora tradicionales, el software del host se distribuye principalmente en paquetes RPM y se administra con DNF.

Fedora también posee variantes basadas en imágenes, como Silverblue/CoreOS, donde el modelo de actualización del host puede usar `rpm-ostree` en lugar del flujo tradicional de DNF para el sistema base.

Por eso incluso dentro de “Fedora” debes identificar la variante antes de administrar paquetes.

Fuentes: - <https://docs.fedoraproject.org/> - <https://docs.fedoraproject.org/en-US/atomic-desktops/> (Fedora Atomic Desktops User Guide; la dirección anterior `/fedora-silverblue/` redirige a esta guía, comprobado el 9 de octubre de 2026)

12. Fedora tradicional y paquetes

Modelo simplificado:

```
formato/base de paquetes → RPM
gestión de alto nivel      → DNF
```

Ejemplos de consulta:

```
command -v dnf
command -v rpm
```

No instales nada por el simple hecho de que esos comandos existan.

13. Fedora image-based

En variantes image-based, el sistema host puede administrarse con un modelo transaccional o basado en imágenes.

Ejemplo de herramienta:

```
rpm-ostree
```

Eso significa que una guía para Fedora Workstation tradicional no debe copiarse automáticamente a Fedora Silverblue o CoreOS.

Primero identifica:

```
cat /etc/os-release
command -v rpm-ostree
```

Las consultas son seguras; no ejecutes rebase, deploy, rollback o upgrade en esta lección.

14. Fedora y SELinux

Fedora utiliza SELinux ampliamente y su documentación de actualización contempla políticas y relabeling de SELinux.

Consulta, si la herramienta existe:

```
command -v getenforce
```

Después:

```
getenforce
```

Puede mostrar estados como:

```
Enforcing
Permissive
Disabled
```

No cambies el modo de SELinux en este módulo.

15. RHEL

RHEL significa:

```
Red Hat Enterprise Linux
```

Es una distribución empresarial de Red Hat con:

- ciclo de vida largo;
- documentación empresarial;
- soporte mediante suscripción según el producto;
- certificaciones e integración con ecosistemas empresariales;
- políticas de estabilidad y mantenimiento distintas a Fedora.

RHEL 10 es la rama principal actual documentada en esta edición.

Fuente: - https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10

16. Ciclo de vida de RHEL

Red Hat documenta un ciclo de vida de diez años para RHEL 8, 9 y 10 a través de fases de soporte, seguido de opciones de soporte extendido.

Esto contrasta con distribuciones de cadencia mucho más rápida.

No significa que “RHEL sea viejo”: significa que prioriza previsibilidad, compatibilidad y mantenimiento empresarial dentro de una rama.

Fuente: - <https://access.redhat.com/support/policy/updates/errata>

17. RHEL y paquetes

RHEL usa:

```
RPM → formato/base de paquetes
DNF → herramienta de alto nivel
```

La documentación actual de RHEL 10 utiliza DNF para:

- buscar paquetes;
- consultar repositorios;
- instalar;
- actualizar;
- eliminar.

No debes asumir que un repositorio Fedora es apropiado para RHEL aunque ambos usen RPM.

18. Fedora y RHEL no son intercambiables

Comparten tecnologías importantes, pero no son “la misma distribución”.

Pueden diferir en:

- versiones de paquetes;
- repositorios;
- soporte;
- ciclos de publicación;
- políticas de actualización;
- certificación;
- disponibilidad de software;
- compatibilidad ABI/API según componentes.

Regla:

un paquete RPM no es automáticamente válido para cualquier distribución que use RPM.

19. SELinux en RHEL

RHEL 10 documenta SELinux como parte central de su modelo de seguridad.

También integra decisiones de SELinux con systemd y servicios.

Consulta segura:

```
getenforce
```

si la herramienta está instalada.

No ejecutes:

```
setenforce 0
```

como forma automática de “arreglar permisos”.

Primero diagnostica contexto, etiquetas y política.

Fuente: - https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/using_selinux/

20. firewalld y nftables en RHEL

RHEL 10 documenta:

- firewalld;
- nftables;
- otros mecanismos de filtrado según el caso.

No estudiaremos reglas todavía.

Consulta segura:

```
command -v firewall-cmd
```

No abras puertos ni cambies zonas en este módulo.

Fuente: - documentación RHEL 10 de firewalls y packet filters.

21. Ubuntu y firewall: no generalizar a Debian

Ubuntu documenta UFW como herramienta de firewall sencilla.

Eso **no permite afirmar** que “Debian y Ubuntu usan UFW por defecto de la misma manera”.

En este manual:

```
Ubuntu → UFW se estudia según documentación Ubuntu
Debian → verificar herramientas/configuración reales
Fedora/RHEL → firewalld/nftables según documentación y entorno
```

La administración práctica llegará en el módulo de firewall.

22. systemd

Las versiones actuales principales de Debian, Ubuntu, Fedora y RHEL usan systemd ampliamente para servicios y arranque.

Herramienta:

```
systemctl
```

Pero:

- unidades disponibles;
- presets;
- nombres de servicios;
- políticas;
- configuración;

pueden variar entre distribuciones.

No copies un nombre de servicio sin verificarlo.

23. Tabla comparativa principal

Área	Debian	Ubuntu	Fedora tradicional	RHEL
Paquete principal	DEB	DEB	RPM	RPM
Alto nivel	APT	APT	DNF	DNF
Bajo nivel	dpkg	dpkg	RPM	RPM
Ciclo	stable/testing/unstable	LTS/interim	rápido	empresarial de largo ciclo
Seguridad MAC destacada	AppArmor habilitado por defecto desde Debian 10; verificar estado real	AppArmor documentado por defecto	SELinux	SELinux
Firewall	verificar entorno	UFW documentado por Ubuntu	firewalld/nftables según entorno	firewalld/nftables
Soporte comercial principal	no como producto empresarial único	Canonical	comunidad/proveedores diversos	Red Hat
Variante image-based destacada	no es el foco de esta tabla	no es el foco	Silverblue/CoreOS	existen tecnologías empresariales, fuera de este módulo

La tabla es una orientación. Nunca sustituye consultar la versión concreta.

24. Misma orden, distinto contexto

Ejemplo:

```
systemctl status ssh
```

puede funcionar en un sistema y no en otro si el servicio se llama distinto, no está instalado o usa otra configuración.

Otro ejemplo:

```
apt install ...
```

no corresponde a RHEL.

Y:

```
dnf install ...
```

no corresponde a Debian estándar.

25. No copies rutas de repositorios entre distribuciones

Ejemplos conceptuales:

```
Debian/Ubuntu → configuración APT
Fedora/RHEL → configuración DNF/repositorios RPM
```

Incluso dentro de una misma familia las rutas y formatos pueden evolucionar.

Ubuntu moderno documenta fuentes en formato deb822 bajo:

```
/etc/apt/sources.list.d/
```

Debian también admite diferentes archivos de sources.

No edites esos archivos en esta lección.

26. No mezcles paquetes por extensión

Un archivo termina en:

```
.rpm
```

Eso no responde:

¿es compatible con mi RHEL/Fedora, versión y arquitectura?

Igualmente:

```
.deb
```

no significa:

funciona igual en cualquier Debian/Ubuntu.

La compatibilidad depende de más factores que la extensión.

27. Arquitectura también importa

Una distribución puede ofrecer paquetes para arquitecturas como:

```
x86_64 / amd64
aarch64 / arm64
otras
```

Los nombres exactos de arquitectura pueden variar entre ecosistemas.

Consulta:

```
uname -m
```

y usa la herramienta de paquetes correspondiente para confirmar arquitectura de paquetes.

No descargues un paquete solo porque “parece Linux”.

28. Versiones de software

Fedora suele incorporar software nuevo con rapidez.

RHEL mantiene versiones dentro de una política empresarial y puede proporcionar correcciones de seguridad mediante backports sin cambiar necesariamente al número de versión más reciente que veas en upstream.

Debian stable también prioriza estabilidad durante su ciclo.

Ubuntu LTS equilibra estabilidad y mantenimiento prolongado.

Por eso:

número de versión más alto no significa automáticamente sistema más seguro.

29. Backport

Un **backport** toma una corrección o cambio y lo adapta a una versión mantenida sin adoptar necesariamente toda la versión upstream nueva.

Esto es importante cuando compares:

```
versión aparente
```

con:

```
estado real de parches de seguridad
```

No declares una vulnerabilidad solo porque un paquete tenga un número de versión “antiguo”; consulta los avisos de seguridad de la distribución.

30. Fuentes de documentación correctas

Prioridad:

```
Debian → debian.org
Ubuntu → ubuntu.com / documentation.ubuntu.com
Fedora → docs.fedoraproject.org
RHEL → docs.redhat.com / access.redhat.com
```

Una respuesta de foro puede ayudar, pero no debe reemplazar la documentación oficial para cambios de sistema.

31. Preparar la práctica

Ejecuta:

```
cat /etc/os-release
```

Localiza:

```
ID
ID_LIKE
VERSION_ID
PRETTY_NAME
```

ID_LIKE puede ayudar a conocer relación con otras familias, pero no significa que ambas distribuciones sean intercambiables.

32. Práctica A — identificar herramientas

Ejecuta:

```
command -v apt
command -v dpkg-query
command -v dnf
command -v rpm
command -v rpm-ostree
```

No instales herramientas ausentes.

Completa:

```
Mi distribución:
Mi versión:
APT:
dpkg-query:
DNF:
RPM:
rpm-ostree:
```

33. Práctica B — identificar seguridad MAC

Consulta únicamente disponibilidad:

```
command -v aa-status
command -v getenforce
```

Si existe aa-status, puedes consultarlo sin cambiar perfiles.

Si existe getenforce, puedes consultar el estado de SELinux.

No ejecute herramientas que cambien el modo.

34. Práctica C — identificar firewall sin modificar

```
command -v ufw
command -v firewall-cmd
command -v nft
```

El objetivo es reconocer herramientas presentes.

No ejecute comandos para habilitar, deshabilitar, abrir puertos, cambiar zonas o recargar reglas.

35. Práctica D — clasificar comandos por distribución

Clasifica:

```
apt-cache policy bash
dpkg-query -W bash
dnf info bash
rpm -q bash
aa-status
getenforce
firewall-cmd
ufw
```

No todos son exclusivos de una sola distribución, pero debes asociarlos con el ecosistema donde este manual los estudia principalmente.

36. Práctica E — detectar una guía incorrecta

Supón:

```
Sistema: Ubuntu
Guía: sudo dnf install paquete
```

Problema:

se copió una instrucción de otra familia sin verificar.

Ahora:

```
Sistema: RHEL
Guía: sudo apt install paquete
```

mismo error conceptual.

La solución no es traducir comandos al azar: es buscar la documentación de la distribución y versión.

37. Práctica F — versión y soporte

Responde sin modificar nada:

1. ¿tu distribución tiene una versión estable/LTS/major específica?
2. ¿esa versión sigue soportada?
3. ¿qué fuente oficial lo confirma?

No confíes solo en el número de versión que recuerdes.

38. Errores frecuentes

Error	Problema	Corrección
“Ubuntu = Debian”	Comparten raíces y DEB, pero políticas y repositorios difieren	Trátalos como distribuciones distintas
“Fedora = RHEL”	Comparten tecnologías, no ciclo ni repositorios	Verifica distribución concreta
Mezclas repositorios	Riesgo de dependencias incompatibles	Usa repositorios de tu distribución
Paquete DEB/RPM parece compatible	Extensión no garantiza compatibilidad	Verifica distro, versión y arquitectura
Asumes UFW en Debian	Generalización desde Ubuntu	Inspecciona el sistema real
Deshabilitas SELinux/AppArmor ante un error	Eliminas control de seguridad sin diagnosticar	Investiga política y etiquetas/perfiles
Copias un nombre de servicio	Puede variar	Consulta unidades instaladas
“Versión vieja = vulnerable”	Puede haber backports	Consulta avisos oficiales
Usas instrucciones de otra release	Defaults y sintaxis pueden cambiar	Verifica versión

39. Método antes de seguir una guía Linux

Antes de copiar una orden:

1. identifica distribución;
2. identifica versión;
3. identifica variante/edición;
4. comprueba arquitectura si aplica;
5. identifica gestor de paquetes;
6. revisa la fuente de la guía;
7. verifica que la documentación corresponda a tu versión;
8. entiende si la orden consulta o modifica;
9. revisa privilegios;
10. solo entonces ejecuta.

40. Práctica independiente

Sin modificar tu sistema:

1. identifica distro y versión;
2. clasifícala como Debian/Ubuntu/Fedora/RHEL u otra;
3. identifica el gestor de paquetes disponible;
4. identifica formato DEB o RPM cuando corresponda;
5. consulta qué mecanismo MAC está disponible;
6. comprueba qué herramientas de firewall existen;
7. busca en documentación oficial el ciclo de soporte de tu versión;
8. explica por qué no mezclarías repositorios de otra distribución.

41. Mini evaluación

1. ¿Debian y Ubuntu usan normalmente paquetes DEB?
 - A. Sí.
 - B. No.
2. ¿Eso los hace intercambiables?
 - A. Sí.
 - B. No.
3. ¿Fedora tradicional y RHEL usan RPM/DNF?
 - A. Sí.
 - B. No.
4. ¿Eso permite instalar cualquier RPM de Fedora en RHEL sin revisar?
 - A. Sí.
 - B. No.
5. ¿Ubuntu documenta AppArmor cargado por defecto?
 - A. Sí.
 - B. No.
6. ¿RHEL 10 documenta SELinux?
 - A. Sí.
 - B. No.
7. ¿Debemos asumir UFW en Debian solo porque Ubuntu lo usa?
 - A. Sí.
 - B. No.
8. ¿Una variante Fedora image-based puede usar rpm-ostree?
 - A. Sí.
 - B. No.
9. ¿Un número de versión de paquete aparentemente antiguo demuestra por sí solo que falta un parche?
 - A. Sí.
 - B. No.
10. ¿Qué debes identificar antes de copiar un comando administrativo?
 - A. Distribución y versión.
 - B. Solo que sea Linux.

42. Registro de aprendizaje

Puedes responder:

```
Debian se caracteriza por:
Ubuntu se caracteriza por:
Fedora se caracteriza por:
RHEL se caracteriza por:
DEB se relaciona con:
RPM se relaciona con:
APT se relaciona con:
DNF se relaciona con:
Ubuntu documenta como MAC:
RHEL documenta como MAC:
¿Por qué no debo mezclar repositorios?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

43. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales verificadas para esta edición:

Corte de versiones: 8 de octubre de 2026. Debian 13 «trixie» es la rama estable (actualización puntual 13.7 del 12 de septiembre de 2026); Ubuntu 26.04 LTS se publicó el 23 de abril de 2026; Red Hat documenta la rama RHEL 10, incluida la versión 10.2 publicada en mayo de 2026. Estos datos deben revisarse antes de reutilizar la guía en otra fecha. La versión mayor, la actualización puntual y el estado de soporte no son equivalentes.

- Debian Releases: <https://www.debian.org/releases/>
- Debian — anuncio de 13.7: <https://lists.debian.org/debian-announce/2026/msg00009.html>
- Ubuntu 26.04 LTS — notas oficiales: <https://documentation.ubuntu.com/release-notes/26.04/>
- Red Hat — fechas de versiones RHEL: <https://access.redhat.com/articles/red-hat-enterprise-linux-release-dates>
- Debian Reference: <https://www.debian.org/doc/manuals/debian-reference/>
- Ubuntu Releases: <https://ubuntu.com/project/docs/release-team/ubuntu-releases/>
- Ubuntu Package Management: <https://ubuntu.com/server/docs/how-to/software/package-management/>
- Ubuntu AppArmor: <https://ubuntu.com/server/docs/security-apparmor/>
- Fedora Documentation: <https://docs.fedoraproject.org/>
- Fedora software/package documentation: <https://docs.fedoraproject.org/>
- RHEL 10 Documentation: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10
- RHEL 10 SELinux: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/using_selinux/
- RHEL lifecycle: <https://access.redhat.com/support/policy/updates/errata>

Se posponen:

- migraciones entre distribuciones;
- conversiones DEB/RPM;
- pinning;
- repositorios de terceros;
- SELinux práctico;
- AppArmor práctico;
- firewall práctico;
- rpm-ostree administrativo;
- soporte empresarial y suscripciones en profundidad.

Estado de la lección: redactada y revisada documentalente. Las prácticas son de identificación y consulta; no modifican configuración.

Módulo 17 — systemd, unidades y servicios; otros sistemas init en contexto

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Decimoséptima entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué función cumple un sistema init;
- reconocer a systemd como sistema y gestor de servicios ampliamente usado en distribuciones Linux actuales;
- explicar qué es una unidad de systemd;
- distinguir una unidad `.service` de otros tipos de unidad;
- usar `systemctl` para consultar estado sin modificar servicios;
- distinguir `start` de `enable`;
- distinguir `stop` de `disable`;
- comprender `restart`, `reload` y `daemon-reload`;
- comprender qué significa que una unidad esté `masked`;
- reconocer que SysV init, OpenRC, runit y otros sistemas existen y que systemd no es universal;
- evitar modificar servicios sin identificar antes la unidad, el impacto y la distribución.

Conocimientos previos: - procesos; - usuarios y privilegios; - paquetes y distribuciones; - lectura de archivos; - comandos de consulta.

Seguridad: las prácticas obligatorias son de lectura. No iniciaremos, detendremos, reiniciaremos, habilitaremos, deshabilitaremos ni enmascaramos servicios del sistema. No editaremos archivos de unidades. No usaremos `sudo` en los ejercicios de consulta.

2. Qué problema resuelve un sistema init

Durante el arranque del sistema, después de que el kernel inicia el espacio de usuario, debe existir un proceso encargado de coordinar servicios y otras tareas del sistema.

Tradicionalmente ese papel se asocia al sistema **init**.

En un sistema Linux con systemd como init del sistema, systemd normalmente ocupa:

```
PID 1
```

Puedes consultar PID 1 sin modificar nada:

```
ps -p 1 -o pid,comm,args
```

No asumas que todos los sistemas Linux usan systemd. Contenedores, sistemas embebidos y distribuciones específicas pueden usar otros diseños.

3. systemd es más que “arrancar servicios”

systemd funciona como un **system and service manager**.

Entre sus responsabilidades puede incluir:

- gestión de servicios;
- dependencias entre unidades;
- sockets;
- timers;
- puntos de montaje;
- sesiones y scopes;
- seguimiento de procesos mediante cgroups;
- integración con logging mediante journald;
- objetivos de arranque.

No necesitas dominar todas estas funciones en una sola lección.

4. Qué es una unidad

systemd organiza recursos y tareas mediante **units**.

Una unidad se identifica mediante un nombre como:

```
ssh.service
cron.service
systemd-journald.service
multi-user.target
example.timer
example.socket
```

Los nombres reales dependen del sistema.

Modelo:

```
nombre.tipo
```

La extensión indica el tipo de unidad.

5. Tipos frecuentes de unidad

Algunos tipos comunes:

```
.service → servicio/proceso administrado
.socket  → socket para activación
.timer   → temporizador
.target  → agrupación/sincronización de unidades
.path    → observa cambios en rutas
.mount   → punto de montaje
.automount → automontaje
.swap    → swap
.device  → dispositivo
.scope   → procesos creados externamente agrupados por systemd
.slice   → agrupación jerárquica de recursos
```

No memorices todos ahora. Para empezar debes reconocer sobre todo:

```
.service
.target
.timer
```

6. systemctl

systemctl permite consultar y controlar el gestor systemd.

Consulta:

```
systemctl --version
```

Puede mostrar:

- versión de systemd;
- características compiladas.

Esto no demuestra por sí solo que systemd sea PID 1. Para verificar el init real puedes consultar:

```
ps -p 1 -o comm=
```

7. Ver estado general

Consulta:

```
systemctl status
```

Puede mostrar un resumen del estado del sistema systemd.

En algunos entornos, la salida puede abrirse en un paginador.

Para salir del paginador:

```
q
```

No confundas un estado degradado con una instrucción automática de reiniciar o reparar; primero identifica qué unidad falló.

8. Listar unidades cargadas

Para listar unidades actualmente cargadas:

```
systemctl list-units
```

Para limitar a servicios:

```
systemctl list-units --type=service
```

Esto muestra unidades que systemd tiene cargadas según los filtros y estado.

No es lo mismo que listar todos los archivos de unidades instalados.

9. list-unit-files

Para consultar archivos de unidades conocidos:

```
systemctl list-unit-files
```

Solo servicios:

```
systemctl list-unit-files --type=service
```

Puede mostrar estados como:

```
enabled
disabled
masked
static
indirect
generated
```

No todos esos estados significan “está ejecutándose” o “está detenido”.

10. Estado activo y estado de habilitación son conceptos distintos

Dos preguntas diferentes:

```
¿está activo ahora?
¿está configurado para activarse según enlaces/dependencias al arranque u otros mecanismos?
```

Herramientas:

```
systemctl is-active NOMBRE.service
systemctl is-enabled NOMBRE.service
```

Una unidad puede estar:

- activa pero disabled;
- inactiva pero enabled;
- activa y enabled;
- inactiva y disabled.

No deduzcas una condición a partir de la otra.

11. start frente a enable

Conceptualmente:

```
systemctl start servicio.service
```

intenta iniciar la unidad **ahora**.

En cambio:

```
systemctl enable servicio.service
```

configura la habilitación según la sección [Install] y las relaciones definidas para futuras activaciones, habitualmente relacionadas con el arranque.

start no implica enable. enable no implica necesariamente start inmediato.

Existen combinaciones como `enable --now`, pero no las practicaremos todavía.

12. stop frente a disable

Conceptualmente:

```
systemctl stop servicio.service
```

intenta detener la unidad **ahora**.

```
systemctl disable servicio.service
```

retira la habilitación configurada para futuras activaciones según su instalación.

stop no implica disable. disable no implica necesariamente detener un servicio que ya está corriendo.

Esta diferencia es fundamental.

13. restart

```
systemctl restart servicio.service
```

detiene e inicia de nuevo según el comportamiento de la unidad.

Un restart puede:

- interrumpir conexiones;
- provocar una breve indisponibilidad;
- afectar aplicaciones dependientes;
- reinicializar estado.

Por eso no se ejecuta sin entender el servicio.

14. reload

```
systemctl reload servicio.service
```

solicita al servicio que recargue su configuración sin realizar necesariamente un reinicio completo.

No todos los servicios soportan reload.

Por eso reload no es un sustituto universal de restart.

15. daemon-reload no reinicia servicios

```
systemctl daemon-reload
```

hace que el gestor systemd vuelva a cargar archivos de unidad y regenere ciertas dependencias internas.

No significa:

```
reiniciar todos los servicios
```

Puede ser necesario después de modificar archivos de unidades, pero en esta lección no editaremos unidades ni ejecutaremos `daemon-reload`.

16. status de una unidad

Una de las consultas más útiles:

```
systemctl status NOMBRE.service
```

Puede mostrar:

- estado cargado;
- estado activo;
- PID principal;
- información reciente;
- fragmento de logs;
- ruta de la unidad.

No pegues la salida completa en GitHub si contiene nombres de host, rutas privadas o información sensible.

17. systemctl show

Para datos estructurados:

```
systemctl show NOMBRE.service
```

Puede producir muchas propiedades.

Puedes pedir solo una:

```
systemctl show -p ActiveState NOMBRE.service
```

o varias:

```
systemctl show -p ActiveState -p SubState -p FragmentPath NOMBRE.service
```

Esto es útil para aprender a separar propiedades concretas.

18. systemctl cat

Para ver la definición efectiva de una unidad y sus fragmentos:

```
systemctl cat NOMBRE.service
```

Es preferible para principiantes a adivinar dónde vive el archivo.

Los archivos de unidades pueden proceder de ubicaciones como:

```
/etc/systemd/system/  
/run/systemd/system/  
/usr/lib/systemd/system/
```

y, según distribución/compatibilidad, pueden encontrarse rutas relacionadas como:

```
/lib/systemd/system/
```

No edites directamente archivos del proveedor para “probar”.

19. /etc frente a archivos del proveedor

Regla conceptual:

```
/usr/lib o /lib → unidades proporcionadas por paquetes/proveedor  
/etc/systemd → configuración administrativa local y overrides  
/run/systemd → unidades/configuración de tiempo de ejecución
```

La disposición exacta depende de la distribución.

Cuando llegue el módulo avanzado, aprenderás a usar overrides en lugar de modificar archivos del paquete.

20. Qué significa masked

Una unidad **masked** queda impedida de iniciarse normalmente mediante el mecanismo de systemd porque su nombre se enlaza a `/dev/null` o se aplica una máscara equivalente según el contexto.

Conceptualmente:

```
disabled → no habilitada para activación configurada al arranque  
masked → inicio bloqueado de forma más fuerte
```

No enmascaras unidades para experimentar.

21. static no significa “rota”

Una unidad marcada como:

```
static
```

normalmente no contiene instrucciones de instalación para habilitarla directamente con el mecanismo estándar de `enable`.

Puede ser activada por:

- dependencias;
- sockets;
- timers;
- otras unidades;
- activación explícita.

No intentes “arreglarla” porque no diga enabled.

22. Targets

Los `.target` agrupan y sincronizan unidades.

Ejemplos comunes:

```
multi-user.target
graphical.target
rescue.target
```

Cumplen un papel conceptual parecido a puntos de sincronización o agrupación del estado del sistema.

No los confundas de forma simplista con los antiguos runlevels: existen relaciones de compatibilidad, pero systemd usa un modelo de dependencias más amplio.

23. Target predeterminado

Consulta:

```
systemctl get-default
```

Puede devolver algo como:

```
graphical.target
```

o:

```
multi-user.target
```

No cambies el target predeterminado en esta lección.

24. Timers

Una unidad:

```
algo.timer
```

puede activar otra unidad según un horario o evento temporal.

Consulta segura:

```
systemctl list-timers
```

Los timers se estudiarán con detalle en el Módulo 30 junto con cron.

25. Sockets

Una unidad:

```
algo.socket
```

puede permitir activación de servicios al recibir actividad en un socket.

Consulta:

```
systemctl list-units --type=socket
```

No necesitas entender todavía todos los mecanismos de socket activation.

26. Dependencias

Las unidades pueden declarar relaciones como:

```
Requires=
Wants=
After=
Before=
```

Estas relaciones no son todas equivalentes.

Por ejemplo:

- After= describe orden;
- Requires= expresa una dependencia más fuerte;
- Wants= expresa una dependencia más débil.

No reduzcas todo a “A depende de B” sin distinguir orden y requisito.

27. systemctl list-dependencies

Consulta:

```
systemctl list-dependencies NOMBRE.service
```

Permite observar relaciones.

No modifiques unidades después de ver una dependencia inesperada.

La relación puede existir por razones de arranque, montaje, sockets, targets u otros mecanismos.

28. Un servicio puede estar active (exited)

No todos los servicios activos tienen un proceso principal ejecutándose continuamente.

Puedes encontrar:

```
active (running)
active (exited)
```

Una unidad oneshot, por ejemplo, puede ejecutar una tarea y permanecer considerada activa según su configuración.

Por eso:

■ active no equivale siempre a “hay un daemon corriendo”.

29. Estado failed

Para consultar unidades fallidas:

```
systemctl --failed
```

Que una unidad aparezca como failed no implica automáticamente que debas:

```
restart
reset-failed
reinstall
disable
```

Primero debes investigar el motivo.

El Módulo 18 enseñará journalctl para logs.

30. systemctl --user

systemd también puede gestionar unidades a nivel del usuario:

```
systemctl --user
```

Estas unidades no son lo mismo que los servicios del gestor de sistema.

Consulta:

```
systemctl --user list-units --type=service
```

En algunos entornos sin una sesión de usuario systemd completa puede no funcionar.

No es un error universal del sistema.

31. Práctica A — identificar PID 1

Ejecuta:

```
ps -p 1 -o pid,comm,args
```

Pregunta:

¿el proceso 1 de tu entorno es systemd?

Si estás en un contenedor, WSL u otro entorno, el resultado puede ser diferente.

No intentes cambiar el init.

32. Práctica B — versión

Si systemd está disponible:

```
systemctl --version
```

Anota:

- versión;
- no todas las características.

No compares versiones de distintas distribuciones como si “más alto” significara automáticamente “mejor”.

33. Práctica C — servicios cargados

```
systemctl list-units --type=service
```

No pegues toda la lista.

Identifica para ti:

- una unidad active/running;
- una active/exited, si existe;
- una inactiva, si repites la consulta con --all: `systemctl list-units --type=service --all`.

34. Práctica D — archivos de unidades

```
systemctl list-unit-files --type=service
```

Localiza ejemplos de:

```
enabled
disabled
static
masked
```

Puede que no estén presentes todos.

No cambies sus estados.

35. Práctica E — consultar una unidad conocida

Elige una unidad que **ya aparezca** en tu listado.

No inventes el nombre.

Después:

```
systemctl status NOMBRE.service
```

y:

```
systemctl show -p ActiveState -p SubState -p FragmentPath NOMBRE.service
```

Solo lectura.

36. Práctica F — definición de la unidad

Sobre la misma unidad:

```
systemctl cat NOMBRE.service
```

Identifica:

- nombre;
- secciones como [Unit], [Service] o [Install] si existen;
- ruta del fragmento.

No edites el archivo.

37. Práctica G — target predeterminado

```
systemctl get-default
```

Explica con tus palabras qué target aparece.

No uses set-default.

38. Práctica H — timers

```
systemctl list-timers
```

Observa:

- próxima ejecución;
- última ejecución;
- timer;
- unidad activada.

No crees ni modifiques timers todavía.

39. start, stop, enable y disable: clasificación

Clasifica sin ejecutar:

```
systemctl start demo.service
systemctl stop demo.service
systemctl enable demo.service
systemctl disable demo.service
systemctl status demo.service
systemctl is-active demo.service
systemctl is-enabled demo.service
```

Debes separar:

```
consultas
cambios de estado actual
cambios de habilitación
```

40. No uses sudo systemctl por reflejo

Algunas operaciones de sistema requieren privilegios y mecanismos de autorización.

Pero:

```
Permission denied
```

o una solicitud de autenticación no significa que debas elevar privilegios sin analizar el cambio.

Primero pregunta:

- ¿qué unidad es?;
- ¿qué hace?;
- ¿quién depende de ella?;
- ¿se perderán conexiones?;
- ¿afectará el arranque?;
- ¿tengo autorización?.

41. Otros sistemas init

systemd no es el único sistema init existente.

Otros ejemplos:

```
SysV init
OpenRC
runit
s6
```

También existen sistemas minimalistas o diseños propios en entornos embebidos y contenedores.

Este manual se centra en systemd porque es dominante en Debian, Ubuntu, Fedora y RHEL actuales, pero mantiene contexto de otros sistemas.

42. SysV init en contexto histórico y de compatibilidad

SysV init utilizaba tradicionalmente:

- scripts de inicio;
- runlevels;
- directorios como `/etc/init.d/`.

Muchas distribuciones modernas usan systemd, pero pueden conservar compatibilidad con algunos scripts SysV.

No enseñaremos SysV como mecanismo principal para administración nueva, pero sí debes reconocerlo cuando aparezca en sistemas heredados.

43. OpenRC

OpenRC es un sistema init y supervisor de servicios usado por algunas distribuciones, especialmente asociado a Gentoo y otras opciones.

Sus herramientas y conceptos no son iguales a `systemctl`.

No intentes traducir automáticamente:

```
systemctl start
```

a una orden OpenRC sin consultar su documentación.

44. runit

runit utiliza un enfoque distinto basado en supervisión de servicios y directorios de servicio.

De nuevo:

“Linux” no implica automáticamente “systemctl”.

Primero identifica el init real.

45. Contenedores y WSL

Dentro de un contenedor, PID 1 puede ser:

- la aplicación;
- un shell;
- un init mínimo;
- systemd si el contenedor fue diseñado para ello.

En WSL moderno, systemd puede estar habilitado según configuración y versión, pero no debes asumirlo sin verificar.

Por eso:

```
ps -p 1 -o comm=
```

es más fiable que asumir.

46. Errores frecuentes

Error	Problema	Corrección
“systemd = servicio”	systemd gestiona muchos tipos de unidad	Distingue gestor y unidad
“unit = service”	Hay timers, sockets, targets, mounts, etc.	Mira la extensión

Error	Problema	Corrección
start = enable	Estado actual y habilitación son diferentes	Aprende cada acción
stop = disable	Detener ahora no cambia necesariamente el arranque	Separa conceptos
reload = restart	No son equivalentes	Revisa soporte del servicio
daemon-reload reinicia servicios	Recarga definición del gestor	No confundir con restart
static = roto	Puede activarse por dependencia	No intentes habilitarlo a ciegas
disabled = detenido	Puede estar activo aunque disabled	Consulta is-active
"Linux siempre usa systemd"	Existen otros init	Comprueba PID 1
Reinicias servicios desconocidos	Puedes interrumpir funciones	Investiga antes

47. Método seguro antes de controlar un servicio

Antes de cualquier cambio:

1. identifica distribución y entorno;
2. confirma que systemd sea el gestor relevante;
3. identifica la unidad exacta;
4. consulta status;
5. consulta cat o propiedades relevantes;
6. identifica dependencias;
7. entiende impacto de start/stop/restart;
8. determina si necesitas habilitación al arranque;
9. revisa logs si existe un fallo;
10. solo entonces realiza cambios con autorización.

48. Práctica independiente

Sin modificar servicios:

1. identifica PID 1;
2. consulta versión de systemd si aplica;
3. lista servicios cargados;
4. lista archivos de unidades de servicio;
5. elige una unidad existente y consulta su estado;
6. consulta ActiveState, SubState y FragmentPath;
7. muestra su definición con systemctl cat;
8. consulta el target predeterminado;
9. lista timers;
10. explica la diferencia entre start y enable.

No uses sudo, start, stop, restart, enable, disable, mask ni unmask.

49. Mini evaluación

1. ¿systemd puede actuar como PID 1?
 - A. Sí.
 - B. No.
2. ¿toda unidad systemd es un servicio?
 - A. Sí.
 - B. No.
3. ¿qué extensión identifica normalmente un servicio?
 - A. .service
 - B. .txt
 - C. .deb
4. ¿systemctl start equivale a systemctl enable?
 - A. Sí.
 - B. No.
5. ¿systemctl stop equivale a systemctl disable?
 - A. Sí.
 - B. No.
6. ¿daemon-reload significa reiniciar todos los servicios?
 - A. Sí.
 - B. No.
7. ¿una unidad static está necesariamente rota?
 - A. Sí.
 - B. No.
8. ¿un servicio disabled puede estar activo ahora?

- A. Sí.
- B. No.

9. ¿systemd es el único init posible en Linux?

- A. Sí.
- B. No.

10. Antes de reiniciar un servicio, ¿debes conocer su función e impacto?

- A. Sí.
- B. No.

50. Registro de aprendizaje

Puedes responder:

```
Un sistema init sirve para:
systemd es:
Una unidad es:
.service significa:
start hace:
enable hace:
stop hace:
disable hace:
reload hace:
daemon-reload hace:
masked significa:
¿Por qué systemd no es el único init?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

51. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales verificadas:

- systemd / systemctl manual: <https://www.freedesktop.org/software/systemd/man/latest/systemctl.html>
- systemd unit manual: <https://www.freedesktop.org/software/systemd/man/latest/systemd.unit.html>
- Debian systemd documentation: <https://wiki.debian.org/systemd/documentation>
- Debian systemd overview: <https://wiki.debian.org/systemd/>
- RHEL 10 — Managing system services with systemctl:
[https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/using_systemd_unit_files_to_customize_and_optimize_your_sy](https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/using_systemd_unit_files_to_customize_and_optimize_your_system-services-with-systemctl)
system-services-with-systemctl

Se posponen:

- edición de unit files;
- overrides con systemctl edit;
- creación de servicios propios;
- daemon-reload práctico;
- start/stop/restart reales;
- enable/disable/mask/unmask reales;
- dependencias avanzadas;
- journald;
- timers prácticos;
- resource control;
- cgroups;
- boot troubleshooting.

Estado de la lección: redactada y revisada documentalmente. Las prácticas son de consulta; no cambian servicios ni configuración.

Módulo 18 — Logs y diagnóstico inicial con journalctl

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Decimoctava entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué es un log;
- distinguir evento, mensaje y registro;
- explicar qué hace `systemd-journald`;
- usar `journalctl` para consultar el journal sin modificar el sistema;
- limitar resultados por arranque, unidad, prioridad y tiempo;
- seguir mensajes nuevos con `journalctl -f`;
- diferenciar journal de archivos tradicionales en `/var/log`;
- reconocer que la persistencia del journal depende de configuración y distribución;
- aplicar un método básico de diagnóstico antes de reiniciar o modificar servicios;
- reconocer que los logs pueden contener datos sensibles.

Conocimientos previos: - procesos; - systemd y unidades; - `systemctl status`; - pipes, grep, less, head y tail.

Seguridad y privacidad: las prácticas son de lectura. No borraremos logs, no cambiaremos retención, no modificaremos `journald.conf`, no reiniciaremos servicios y no usaremos `sudo` para ampliar acceso. No publiques logs completos: pueden contener nombres de usuario, hostnames, rutas, direcciones IP, identificadores, comandos, errores de aplicaciones o datos privados.

2. Qué es un log

Un **log** es un registro de eventos generado por un sistema, servicio o aplicación.

Puede registrar, por ejemplo:

- inicio y detención de servicios;
- errores;
- advertencias;
- autenticaciones;
- eventos del kernel;
- actividad de aplicaciones;
- arranque del sistema.

Un log no es automáticamente “un archivo de texto”. En systemd, el journal puede almacenarse en un formato estructurado administrado por `systemd-journald`.

3. Evento, mensaje y entrada

Modelo simple:

```
evento → genera información → entrada de log
```

Una entrada puede incluir campos como:

- fecha y hora;
- host;
- proceso;
- PID;
- unidad systemd;
- prioridad;
- mensaje;
- otros metadatos.

La cantidad exacta de campos depende de la fuente y del sistema de logging.

4. systemd-journald

`systemd-journald` es el servicio de systemd que recopila y almacena datos de logging.

Puede recibir mensajes de fuentes como:

- kernel;
- servicios systemd;
- `stdout/stderr` de determinados servicios;
- syslog;

- otras interfaces de logging.

En algunas distribuciones, journald trabaja junto con un sistema tradicional como rsyslog.

No asumas que “journal” reemplaza siempre todos los archivos de /var/log.

5. journalctl

journalctl consulta el journal.

Ejemplo:

```
journalctl
```

Puede producir una gran cantidad de salida.

En muchos entornos se abre mediante un paginador.

Para salir:

```
q
```

Para principiantes, es mejor aplicar filtros que leer todo el journal.

6. No confundir consulta con modificación

Comandos como:

```
journalctl -b
journalctl -u nombre.service
journalctl -p err
journalctl --since today
```

son consultas.

En cambio, acciones relacionadas con:

- vaciado;
- rotación forzada;
- límites de tamaño;
- configuración de almacenamiento;
- cambio de permisos;

pueden modificar el sistema y no pertenecen a esta práctica inicial.

7. Journal del arranque actual con -b

Consulta:

```
journalctl -b
```

-b limita la consulta al arranque actual.

Esto es útil cuando quieres responder:

¿qué ocurrió desde que arrancó este sistema?

No significa que el journal tenga necesariamente datos de arranques anteriores.

8. Listar arranques conocidos

Consulta:

```
journalctl --list-boots
```

Si el journal conserva información de varios arranques, puede mostrarlos con identificadores e índices.

Ejemplo conceptual:

```
-1 → arranque anterior
0 → arranque actual
```

La disponibilidad de arranques anteriores depende de la persistencia y retención del journal.

9. Arranque anterior

Si existe información persistida:

```
journalctl -b -1
```

consulta el arranque anterior.

Si no devuelve datos útiles, no significa necesariamente que el sistema esté roto. Puede deberse a:

- journal no persistente;
- rotación;
- limpieza;
- instalación reciente;
- configuración particular.

10. Filtrar por unidad con -u

Ejemplo:

```
journalctl -u NOMBRE.service
```

Muestra entradas asociadas a una unidad.

Para limitar al arranque actual:

```
journalctl -u NOMBRE.service -b
```

Este patrón es muy útil para investigar un servicio que falló al arrancar.

11. Elegir primero una unidad real

No inventes nombres.

Primero:

```
systemctl list-units --type=service
```

Elige una unidad que exista en tu sistema.

Después consulta:

```
systemctl status NOMBRE.service
```

y solo entonces:

```
journalctl -u NOMBRE.service -b
```

12. Prioridades

Los mensajes pueden usar niveles de prioridad compatibles con syslog.

Niveles comunes, de más grave a menos grave:

```
emerg
alert
crit
err
warning
notice
info
debug
```

No necesitas memorizar todos inmediatamente.

Para comenzar:

```
err    → errores y más graves
warning → advertencias y más graves
info   → información y más graves
```

13. Filtrar por prioridad

Ejemplo:

```
journalctl -p err -b
```

muestra mensajes del arranque actual con prioridad err y, según la semántica del filtro, niveles más importantes.

Otra consulta:

```
journalctl -p warning -b
```

Puede producir más resultados.

Advertencia: un mensaje de prioridad alta no demuestra por sí solo que exista un problema actual. Debes interpretar contexto y tiempo.

14. Filtrar por tiempo con --since

Ejemplos:

```
journalctl --since today
```

```
journalctl --since "1 hour ago"
```

```
journalctl --since "2026-10-07 10:00:00"
```

Las expresiones aceptadas dependen del parser temporal de systemd.

Para diagnóstico real, usa intervalos lo bastante estrechos para no mezclar eventos irrelevantes.

15. --until

Puedes limitar el final del intervalo:

```
journalctl --since "10:00" --until "10:15"
```

Esto ayuda a investigar:

¿qué ocurrió durante esos 15 minutos?

No necesitas consultar horas de datos si el fallo ocurrió en un intervalo pequeño y conocido.

16. Seguir mensajes con -f

```
journalctl -f
```

muestra entradas recientes y continúa esperando mensajes nuevos.

Se parece conceptualmente a:

```
tail -f archivo.log
```

pero consulta el journal.

Para detener el seguimiento y volver al prompt:

```
Ctrl+C
```

No modifica el journal.

17. Seguir una unidad concreta

Ejemplo:

```
journalctl -u NOMBRE.service -f
```

Esto puede ser útil mientras reproduces un problema controlado.

No reinicies o fuerces fallos de servicios solo para generar mensajes.

18. Mostrar mensajes recientes

Puedes limitar por cantidad:

```
journalctl -n 20
```

Muestra las últimas entradas según el filtro aplicado.

Combinación:

```
journalctl -u NOMBRE.service -n 20
```

Esto suele ser más manejable que abrir miles de entradas.

19. Kernel con -k

Consulta:

```
journalctl -k -b
```

muestra mensajes del kernel para el arranque actual cuando están disponibles en el journal.

No confundas cada mensaje del kernel con un fallo.

Muchos son informativos.

20. Formatos de salida

journalctl admite varios formatos.

Ejemplo:

```
journalctl -n 5 -o short
```

Otros formatos pueden mostrar más metadatos o datos estructurados.

En esta etapa usaremos la salida predeterminada o short.

No necesitas JSON todavía.

21. Buscar texto dentro de la salida

Puedes combinar con herramientas conocidas:

```
journalctl -b | grep -i 'error'
```

Pero esta técnica tiene límites:

- “error” puede aparecer en mensajes no críticos;
- errores reales pueden no contener la palabra error;
- pierdes campos estructurados si solo buscas texto.

Siempre que exista un filtro nativo adecuado, priorízalo.

22. status y journalctl se complementan

systemctl status NOMBRE.service ofrece una vista resumida:

- estado;
- PID;
- fragmento de logs reciente.

journalctl -u NOMBRE.service permite profundizar.

Método:

```
status → identifica estado
journalctl → investiga historial
```

No reinicies el servicio antes de leer la evidencia, porque podrías cambiar el contexto del problema.

23. Journal persistente y volátil

systemd puede almacenar el journal en ubicaciones como:

```
/run/log/journal/
```

para almacenamiento volátil, o:

```
/var/log/journal/
```

para persistencia, según configuración y entorno.

No generalices un único comportamiento para todas las distribuciones.

La política real depende de:

- Storage=;
- existencia de rutas;
- distribución;
- configuración;
- límites y rotación.

24. RHEL 10 y persistencia

La documentación de RHEL 10 describe una configuración predeterminada donde el journal puede ser volátil en `/run/log/journal` y explica cómo configurar persistencia (comprobado en la documentación de RHEL 10 el 9 de octubre de 2026).

Eso es una política de RHEL documentada, no una regla universal de systemd en todas las distribuciones.

Por eso este manual separa:

```
comportamiento de systemd
```

de:

```
default de una distribución concreta
```

25. /var/log sigue siendo importante

Servicios y sistemas pueden escribir archivos tradicionales bajo:

```
/var/log/
```

Ejemplos posibles según distribución:

```
/var/log/messages  
/var/log/syslog  
/var/log/auth.log  
/var/log/secure
```

No todos existen en todas las distribuciones.

No uses una ruta de Ubuntu como si fuera universal para RHEL o viceversa.

26. Rsyslog

rsyslog es un sistema de logging tradicional/moderno basado en syslog que puede coexistir con journald.

En RHEL, la documentación describe integración donde journald recopila mensajes y rsyslog puede procesarlos y escribir archivos bajo `/var/log`.

No asumiremos esa arquitectura exacta para todas las distribuciones.

27. Permisos de lectura de logs

No todos los usuarios pueden ver todas las entradas.

Si una consulta omite mensajes por permisos:

- no añadas `sudo` automáticamente;
- pregunta si necesitas realmente esos datos;
- usa el mínimo acceso requerido.

En un entorno administrado, el acceso al journal puede controlarse por grupos y políticas.

28. Logs contienen información sensible

Un log puede revelar:

- nombre de usuario;
- hostname;
- IP;
- rutas;
- nombres de servicios;
- procesos;
- identificadores;
- errores de autenticación;
- nombres de archivos;
- datos introducidos por aplicaciones.

Antes de compartir:

1. recorta solo las líneas relevantes;
2. elimina secretos;
3. oculta identificadores privados cuando no sean necesarios;
4. conserva timestamps si ayudan al diagnóstico;
5. no alteres el significado técnico.

29. Nunca publiques secretos

Si un log contiene:

- contraseña;
- token;
- cookie;
- API key;
- clave privada;
- secreto de aplicación;
- credencial;

no lo subas a GitHub ni lo pegues en el chat.

Detén la publicación y elimina/rota el secreto según el sistema afectado.

30. Diagnóstico: correlación temporal

Una buena investigación comienza preguntando:

¿a qué hora ocurrió el problema?

Después:

```
momento del fallo
↓
unidad afectada
↓
logs del intervalo
↓
mensajes anteriores y posteriores
```

No busques todo el journal sin límite si conoces la hora aproximada.

31. Diagnóstico: correlación entre servicio y sistema

Un servicio puede fallar por algo externo:

- red;
- disco;
- permisos;
- dependencia;
- montaje;
- configuración;
- memoria.

Por eso, además de:


```
journalctl -u NOMBRE.service
```

puede ser útil revisar el arranque o prioridad correspondiente.

No concluyas demasiado pronto.

32. No borres logs para “limpiar el error”

Borrar logs no corrige la causa.

Además, elimina evidencia útil.

En este módulo no practicamos:

- `journalctl --vacuum=*`;
- eliminación manual de archivos de log;
- truncado de logs;
- rotación forzada.

Primero diagnostica.

33. No edites logs

Los logs son evidencia del comportamiento del sistema.

No los abras en un editor para “corregir” mensajes.

Si necesitas guardar una copia de laboratorio, copia solo un fragmento ya revisado y sin datos sensibles.

34. `journalctl --disk-usage`

Consulta segura:

```
journalctl --disk-usage
```

muestra cuánto espacio ocupa el journal accesible para la consulta.

No uses esta cifra como razón automática para borrar logs.

Primero revisa:

- políticas de retención;
- espacio del sistema;
- configuración;
- necesidad de auditoría.

35. Preparar la práctica

No necesitas crear archivos.

Comprueba primero:

```
ps -p 1 -o comm=
```

Si `systemd` es relevante:

```
command -v journalctl
```

Si `journalctl` no existe o `systemd` no es el `init` del entorno, registra esa condición y no fuerces la práctica.

36. Práctica A — journal del arranque

```
journalctl -b -n 20
```

Objetivo:

- ver solo 20 entradas;
- reconocer timestamps;
- reconocer proceso/unidad;
- no compartir toda la salida.

37. Práctica B — mensajes de prioridad alta

```
journalctl -b -p err -n 20
```

No asumas que cualquier resultado requiere reparación.

Elige una entrada y responde para ti:

- ¿cuándo ocurrió?
- ¿qué componente la generó?
- ¿sigue ocurriendo?
- ¿está relacionada con un síntoma real?

38. Práctica C — unidad existente

Primero:

```
systemctl list-units --type=service
```

Elige una unidad real.

Después:

```
systemctl status NOMBRE.service
```

y:

```
journalctl -u NOMBRE.service -b -n 20
```

No reinicies la unidad.

39. Práctica D — tiempo

Consulta:

```
journalctl --since "30 minutes ago" -n 30
```

Si tu versión acepta la expresión, revisa solo ese intervalo.

Si no la acepta, usa un timestamp explícito documentado por tu sistema.

40. Práctica E — kernel

```
journalctl -k -b -n 20
```

Identifica mensajes informativos sin asumir que son errores.

41. Práctica F — seguimiento

```
journalctl -f
```

Observa unos segundos.

Después:

```
Ctrl+C
```

No necesitas provocar errores para ver actividad.

42. Práctica G — arranques conocidos

```
journalctl --list-boots
```

Si solo aparece el arranque actual, explica al menos dos razones posibles sin concluir que existe un fallo.

43. Método inicial de diagnóstico

Cuando un servicio falla:

1. registra hora aproximada;

2. consulta `systemctl status UNIDAD`;
3. consulta `journalctl -u UNIDAD -b`;
4. limita por tiempo si es necesario;
5. busca mensajes inmediatamente anteriores al fallo;
6. distingue error raíz de errores secundarios;
7. revisa dependencias;
8. no reinicies todavía;
9. formula una hipótesis;
10. cambia una sola cosa cuando llegue el momento de intervenir.

44. Error raíz frente a error secundario

Ejemplo conceptual:

```
archivo de configuración inválido
↓
servicio no inicia
↓
aplicación dependiente pierde conexión
↓
aparecen más errores
```

El último error visible no siempre es la causa inicial.

Por eso es útil leer cronológicamente alrededor del primer fallo.

45. auditd — introducción conceptual

Algunas distribuciones utilizan el subsistema de auditoría de Linux y un servicio como `auditd` para registrar eventos de seguridad y cumplimiento.

Esto **no es lo mismo que journald**.

Modelo:

```
journald → logging general de systemd y sistema
audit    → auditoría de eventos según reglas/políticas
```

No configuraremos reglas de auditoría en este módulo.

46. No asumir auditd en todos los sistemas

La presencia, configuración y uso de `auditd` depende de:

- distribución;
- instalación;
- política;
- entorno;
- requisitos de cumplimiento.

Consulta únicamente:

```
command -v ausearch
systemctl status auditd.service
```

solo si la unidad existe.

No habilites ni modifiques `auditd` para completar la lección.

47. Errores frecuentes

Error	Problema	Corrección
Lees todo <code>journalctl</code> sin filtros	Exceso de ruido	Filtra por boot, unidad, tiempo o prioridad
Reinicias antes de revisar logs	Cambias evidencia y contexto	Lee primero
“err = causa raíz”	Puede ser consecuencia	Correlaciona eventos
Publicas logs completos	Expones datos sensibles	Comparte solo fragmentos sanitizados
Asumes persistencia universal	Depende de configuración	Comprueba <code>--list-boots</code> y entorno
Asumes rutas de <code>/var/log</code> universales	Cambian por distro	Verifica documentación
Usas sudo automáticamente	Amplías acceso	Aplica mínimo privilegio

Error	Problema	Corrección
Borras logs para liberar espacio	Pierdes evidencia	Investiga retención antes
Confundes journald y auditd	Objetivos distintos	Separa logging y auditoría
Grep "error" = diagnóstico completo	Busca texto, no contexto estructurado	Usa filtros nativos y tiempo

48. Práctica independiente

Sin modificar el sistema:

1. muestra las últimas 15 entradas del arranque actual;
2. consulta errores del arranque;
3. elige una unidad real;
4. muestra 15 entradas de esa unidad;
5. limita una consulta por tiempo;
6. consulta 10 mensajes del kernel;
7. lista arranques conocidos;
8. consulta uso en disco del journal;
9. explica si tu journal parece persistente o no, sin cambiarlo;
10. explica qué datos eliminarías antes de compartir un fragmento.

49. Mini evaluación

1. ¿Qué hace `journalctl`?
 - A. Consulta el journal.
 - B. Instala paquetes.
 - C. Cambia permisos.
2. ¿Qué hace `journalctl -b`?
 - A. Limita al arranque seleccionado/actual.
 - B. Borra logs.
 - C. Reinicia.
3. ¿Qué hace `journalctl -u servicio.service`?
 - A. Filtra por unidad.
 - B. Actualiza el servicio.
 - C. Desinstala el servicio.
4. ¿Qué hace `journalctl -f`?
 - A. Sigue entradas nuevas.
 - B. Fuerza un reinicio.
 - C. Filtra archivos.
5. ¿La persistencia del journal es idéntica en todas las distribuciones?
 - A. Sí.
 - B. No.
6. ¿Debes reiniciar un servicio antes de revisar sus logs?
 - A. Siempre.
 - B. No.
7. ¿Los logs pueden contener datos sensibles?
 - A. Sí.
 - B. No.
8. ¿Borrar logs corrige la causa de un fallo?
 - A. Sí.
 - B. No.
9. ¿`journald` y `auditd` son exactamente lo mismo?
 - A. Sí.
 - B. No.
10. ¿Un mensaje `err` demuestra siempre la causa raíz?
 - A. Sí.
 - B. No.

50. Registro de aprendizaje

Puedes responder:

```
Un log es:
journald sirve para:
journalctl sirve para:
-b significa:
-u sirve para:
-p sirve para:
-f sirve para:
--since sirve para:
La persistencia depende de:
¿Por qué no reinicio antes de leer logs?:
¿Qué datos no compartiría?:
auditd se diferencia de journald porque:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

51. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- systemd journalctl manual: <https://www.freedesktop.org/software/systemd/man/latest/journalctl.html>
- systemd-journald manual: <https://www.freedesktop.org/software/systemd/man/latest/systemd-journald.service.html>
- RHEL 10 — Troubleshooting problems by using log files:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/risk_reduction_and_recovery_operations/troubleshooting-problems-by-using-log-files
- RHEL 10 — systemd journal configuration:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/automating_system_administration_by_using_rhel_system_roles/the-systemd-journal-by-using-rhel-system-roles

Se posponen:

- configuración de journald.conf;
- persistencia práctica;
- límites de almacenamiento;
- vacuum/rotación;
- forwarding remoto;
- rsyslog avanzado;
- auditd y reglas de auditoría;
- logrotate;
- observabilidad centralizada;
- SIEM.

Estado de la lección: redactada y revisada documentalente. Las prácticas son de consulta y diagnóstico; no modifican logs ni servicios.

Módulo 19 — Redes básicas: IP, DNS, rutas, ip y ss

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Decimonovena entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué es una interfaz de red;
- reconocer una dirección IPv4 e IPv6;
- distinguir dirección IP, máscara/prefijo, gateway y DNS;
- explicar qué representa una ruta;
- consultar interfaces y direcciones con `ip`;
- consultar la tabla de rutas con `ip route`;
- interpretar de forma inicial una ruta por defecto;
- explicar qué hace DNS;
- realizar consultas de resolución de nombres con herramientas de lectura;
- consultar sockets con `ss`;
- distinguir un socket escuchando de una conexión establecida;
- reconocer que observar tu propio equipo no equivale a escanear otras redes.

Conocimientos previos: - procesos; - archivos; - pipes; - permisos; - logs; - systemd básico.

Seguridad y alcance: este módulo es de observación sobre tu propio equipo y tu propia red. No cambiaremos direcciones IP, rutas, DNS, interfaces ni firewall. No haremos escaneos de redes o puertos externos. No usaremos `ip link set`, `ip addr add/del`, `ip route add/del`, herramientas de escaneo ni `sudo` para ampliar acceso.

2. Qué es una red

Una red permite que dispositivos intercambien datos mediante protocolos.

Modelo simplificado:

```
equipo A — red — equipo B
```

En Internet y redes locales, una parte fundamental de la comunicación utiliza la familia de protocolos IP.

No necesitas estudiar todavía Ethernet, ARP, TCP, UDP y DNS a nivel profundo. En este módulo construiremos el mapa básico.

3. Interfaz de red

Una **interfaz de red** es el punto lógico mediante el cual el sistema participa en una red.

Puede corresponder a:

- Ethernet;
- Wi-Fi;
- loopback;
- interfaces virtuales;
- VPN;
- bridges;
- contenedores.

Nombres posibles:

```
lo
eth0
enp3s0
wlan0
wlp2s0
```

Los nombres reales dependen del sistema.

4. Loopback

La interfaz:

```
lo
```

es la interfaz de loopback.

Direcciones típicas:

```
127.0.0.1 → IPv4 loopback
::1       → IPv6 loopback
```

Permiten que el equipo se comunique consigo mismo.

No significan “la IP de Internet” del equipo.

5. Dirección IP

Una dirección IP identifica una interfaz o punto de comunicación dentro de un contexto de red.

Ejemplo IPv4 ilustrativo:

```
192.0.2.10
```

Ejemplo IPv6 reservado para documentación:

```
2001:db8::10
```

Estos ejemplos pertenecen a rangos reservados para documentación y no representan tu red real.

6. IPv4

IPv4 usa direcciones de 32 bits.

Se escriben habitualmente como cuatro números decimales:

```
192.0.2.10
```

Cada octeto va de 0 a 255.

No todas las direcciones IPv4 tienen el mismo propósito.

7. Rangos privados IPv4

RFC 1918 reserva tres bloques de direcciones IPv4 para redes privadas:

```
10.0.0.0/8
172.16.0.0/12
192.168.0.0/16
```

Estas direcciones **no son globalmente únicas**. Pueden enrutarse dentro de una red privada —o entre redes que hayan acordado coordinar ese espacio—, pero la información de enrutamiento de estos bloques no debe propagarse por el Internet público.

Por eso no debes interpretar «privada» como «imposible de enrutar». La diferencia principal es el **ámbito**: una dirección RFC 1918 tiene significado dentro de la red privada que la utiliza, no como dirección pública global.

Para que un equipo con una dirección privada acceda a servicios externos, una red puede utilizar un gateway y mecanismos como NAT, según su diseño. NAT es un tema posterior y no es una propiedad obligatoria de RFC 1918.

Referencia primaria: RFC 1918 — *Address Allocation for Private Internets* (RFC Editor/IETF).

Una dirección privada no debe confundirse con la dirección pública que pueda utilizar un router o gateway para comunicarse hacia otras redes.

8. IPv6

IPv6 usa direcciones de 128 bits.

Ejemplo documental:

```
2001:db8::1
```

IPv6 utiliza representación hexadecimal y permite abreviar grupos de ceros.

No estudiaremos todavía reglas completas de abreviación, SLAAC, DHCPv6, NDP o privacidad de direcciones.

9. Prefijo CIDR

Una dirección puede mostrarse así:

```
192.0.2.10/24
```

El:

```
/24
```

indica la longitud del prefijo de red.

En IPv6 es frecuente ver valores como:

```
/64
```

No debes interpretar el prefijo como “número de equipos” directamente.

El cálculo de subredes se estudiará aparte.

10. ip — herramienta principal del módulo

La utilidad `ip` forma parte de `iproute2`.

Puede mostrar o modificar:

- direcciones;
- enlaces;
- rutas;
- vecinos;
- reglas;
- otros objetos de red.

En este módulo **solo utilizamos operaciones de consulta**.

La sintaxis general es:

```
ip [opciones] OBJETO COMANDO
```

11. ip link show

Consulta:

```
ip link show
```

Muestra interfaces/enlaces conocidos por el kernel.

Puedes encontrar datos como:

- nombre;
- estado;
- MTU;
- flags;
- dirección de enlace.

No publiques direcciones MAC completas sin necesidad.

12. ip -br link

Una vista más compacta:

```
ip -br link
```

`-br` significa **brief**.

Es útil para principiantes porque reduce el ruido.

Ejemplo conceptual:

```
lo          UNKNOWN
enp...      UP
```

No asumas que UNKNOWN en loopback representa una falla.

13. ip address

Consulta:

```
ip address show
```


Forma abreviada:

```
ip addr
```

Muestra direcciones asociadas a interfaces.

Puede incluir:

- IPv4;
- IPv6;
- scope;
- prefijo;
- lifetime.

14. Vista breve de direcciones

```
ip -br address
```

puede ser más fácil de leer.

Antes de compartir una salida, elimina:

- direcciones públicas;
- IPv6 globales;
- MAC;
- nombres de interfaces que revelen información privada si no son necesarios.

15. Estado UP no garantiza Internet

Una interfaz puede aparecer:

```
UP
```

y aun así no tener acceso a Internet.

Para conectividad completa también pueden intervenir:

- dirección;
- ruta;
- gateway;
- DNS;
- firewall;
- enlace físico;
- proveedor;
- red remota.

No diagnostiques solo mirando una palabra.

16. Ruta

Una **ruta** indica cómo debe tratar el sistema tráfico destinado a determinadas redes o direcciones.

Modelo:

```
destino → siguiente salto/interfaz
```

El kernel consulta su tabla de rutas para decidir por dónde enviar tráfico.

17. ip route

Consulta:

```
ip route show
```

o:

```
ip route
```

Muestra rutas IPv4 conocidas por el kernel en la tabla principal según el contexto.

No modifica nada.

18. Ruta por defecto

Puedes encontrar una línea conceptual como:

```
default via 192.0.2.1 dev eth0
```

Significa, de forma simplificada:

- default: destinos sin una ruta más específica;
- via: siguiente salto o gateway;
- dev: interfaz utilizada.

La dirección es solo un ejemplo documental.

19. Gateway

Un **gateway** es un dispositivo o punto de siguiente salto que puede reenviar tráfico hacia otras redes.

En una red doméstica, el router suele cumplir ese papel para la ruta por defecto.

Pero no todo tráfico usa necesariamente el mismo gateway: las rutas más específicas pueden elegir otros caminos.

20. Regla de ruta más específica

En términos generales, el sistema selecciona rutas considerando la coincidencia del destino, y una ruta más específica puede prevalecer sobre una ruta por defecto.

No estudiaremos todavía:

- múltiples tablas;
- policy routing;
- métricas complejas;
- ECMP;
- VRF.

21. No modificar rutas

Comandos como:

```
ip route add ...
ip route del ...
ip route replace ...
```

modifican enrutamiento.

Una ruta equivocada puede cortar conectividad.

Por eso no se practican aquí.

22. DNS

DNS significa:

```
Domain Name System
```

Permite resolver nombres a información como direcciones IP.

Modelo:

```
nombre → resolución DNS → dirección
```

Ejemplo conceptual:

```
servidor.ejemplo → 192.0.2.20
```

DNS no es lo mismo que conectividad IP.

Puedes tener IP funcional y DNS roto, o resolver un nombre y aun así no poder conectarte al servicio.

23. Resolución de nombres y NSS

En Linux, una aplicación puede resolver nombres mediante mecanismos configurados por Name Service Switch (NSS), no necesariamente consultando DNS de forma directa.

Una herramienta útil es:

```
getent hosts localhost
```

Esto consulta la base de hosts mediante NSS.

No modifica nada.

24. getent ahosts

Si quieres consultar una resolución de nombre mediante NSS:

```
getent ahosts localhost
```

Para una práctica completamente local usamos localhost.

Resolver nombres públicos puede generar consultas de red, por lo que no es necesario para aprender el concepto inicial.

25. /etc/hosts

Este archivo puede proporcionar asociaciones locales de nombres:

```
/etc/hosts
```

Consulta limitada:

```
head -n 10 /etc/hosts
```

No lo edites en este módulo.

No publiques su contenido completo si contiene nombres internos o información privada.

26. /etc/resolv.conf

Tradicionalmente:

```
/etc/resolv.conf
```

contiene o apunta a información relacionada con resolución DNS.

En sistemas modernos puede ser:

- un archivo administrado;
- un symlink;
- parte de una integración con systemd-resolved;
- administrado por NetworkManager u otra herramienta.

No asumas una arquitectura universal.

Consulta segura:

```
ls -l /etc/resolv.conf
```

No lo modifiques manualmente.

27. resolvectl

Si tu sistema usa systemd-resolved y la herramienta existe:

```
command -v resolvectl
```

puedes consultar:

```
resolvectl status
```

La salida puede revelar:

- servidores DNS;
- dominios de búsqueda;
- interfaces.

No publiques esos datos completos.

Si `resolvectl` no existe, no instales nada para esta práctica.

28. Nombre DNS no equivale a puerto

DNS responde a preguntas sobre nombres.

Un servicio como web o SSH usa además protocolos y puertos.

Ejemplo conceptual:

```
DNS → ¿a qué dirección corresponde el nombre?  
TCP/UDP + puerto → ¿qué servicio/comunicación usa ese endpoint?
```

No mezcles ambos conceptos.

29. Socket

Un **socket** es un endpoint de comunicación.

En red puede asociarse a:

- protocolo;
- dirección local;
- puerto local;
- dirección remota;
- puerto remoto;
- estado.

No todos los sockets son de red IP; también existen sockets Unix.

En este módulo nos centraremos en TCP y UDP.

30. Puerto

TCP y UDP utilizan números de puerto de 16 bits.

Rango:

```
0-65535
```

Un puerto no identifica por sí solo una aplicación de manera universal.

Por ejemplo, aunque ciertos servicios tienen puertos convencionales, una aplicación puede configurarse en otro puerto.

31. TCP y UDP

Resumen inicial:

```
TCP → orientado a conexión  
UDP → datagramas, sin conexión en el mismo sentido que TCP
```

No estudiaremos todavía handshake, ventanas, retransmisión, MTU o control de congestión.

32. ss

`ss` forma parte de `iproute2` y muestra información de sockets.

Sustituye gran parte de los usos modernos que antiguamente se hacían con `netstat`.

Consulta:

```
ss
```

puede mostrar muchas conexiones/sockets.

Usaremos filtros.

33. ss -lnt

Consulta:

```
ss -lnt
```

Opciones:

```
-l → listening  
-n → mostrar valores numéricos, sin resolver nombres  
-t → TCP
```

Esto muestra sockets TCP en escucha.

Es una consulta local, no un escaneo de otras máquinas.

34. ss -lnu

Para UDP:

```
ss -lnu
```

Opciones:

```
-l → listening/unconnected según el contexto  
-n → numérico  
-u → UDP
```

La semántica de “listen” no es idéntica entre TCP y UDP porque UDP no establece conexiones como TCP.

35. ss -tn

Consulta:

```
ss -tn
```

muestra sockets TCP con valores numéricos.

Puede revelar direcciones remotas y conexiones activas.

No publiques la salida completa.

Úsalo solo para observar tu propio equipo.

36. Estado TCP

Puedes ver estados como:

```
LISTEN  
ESTAB  
TIME-WAIT  
CLOSE-WAIT
```

No necesitas memorizar todos ahora.

Dos básicos:

```
LISTEN → espera conexiones entrantes  
ESTAB → conexión establecida
```

Un socket LISTEN no significa automáticamente que esté accesible desde Internet. Influyen:

- dirección de bind;
- firewall;
- routing;
- NAT;
- red.

37. 0.0.0.0 y ::

En una escucha puedes encontrar:

```
0.0.0.0:PUERTO
```

o:

```
:::PUERTO
```

De forma general indican binding a direcciones wildcard de IPv4 o IPv6.

Pero la exposición real depende de firewall, namespaces, configuración del socket y red.

No concluyas “está público en Internet” solo por ver wildcard.

38. 127.0.0.1 y ::1 en escucha

Si un servicio escucha únicamente en:

```
127.0.0.1
::1
```

está ligado al loopback correspondiente.

Eso normalmente limita el acceso a la propia máquina desde ese namespace de red.

No cambies el bind para “hacerlo accesible” en esta lección.

39. ss y procesos

Una opción frecuente de ss permite mostrar procesos asociados, pero la visibilidad depende de permisos.

No usaremos esa opción como requisito porque puede revelar información de procesos y algunas entradas requieren privilegios.

En este nivel basta con aprender:

- dirección;
- puerto;
- protocolo;
- estado.

40. netstat e ifconfig en contexto

Herramientas históricas como:

```
ifconfig
route
netstat
```

pertenecen al conjunto net-tools y todavía pueden existir.

Este manual prioriza:

```
ip
ss
```

porque forman parte de iproute2 y cubren la administración/consulta moderna de red.

No afirmamos que net-tools haya “desaparecido”; simplemente no será la herramienta principal.

41. Diagnóstico por capas

Si “Internet no funciona”, no empieces cambiando DNS o firewall.

Sigue una secuencia:

1. interfaz
2. dirección
3. ruta
4. resolución de nombres
5. sockets/servicio
6. logs

Esto evita cambiar varias cosas a la vez.

42. Paso 1 — interfaz

Consulta:

```
ip -br link
```

Pregunta:

- ¿la interfaz esperada existe?;
- ¿está UP?;
- ¿es loopback, física o virtual?.

No la subas/bajes.

43. Paso 2 — dirección

```
ip -br address
```

Pregunta:

- ¿tiene IPv4?;
- ¿tiene IPv6?;
- ¿qué prefijo usa?;
- ¿la dirección parece local/privada?.

No compartas direcciones completas si no son necesarias.

44. Paso 3 — ruta

```
ip route
```

Pregunta:

- ¿existe una ruta por defecto?;
- ¿qué interfaz usa?;
- ¿existen rutas más específicas?.

No añadas ni borres rutas.

45. Paso 4 — nombres

Prueba local:

```
getent hosts localhost
```

Si necesitas revisar configuración:

```
ls -l /etc/resolv.conf
```

y, si existe:

```
resolvectl status
```

No edites nada.

46. Paso 5 — sockets

Consulta:

```
ss -lnt
```

Si investigas UDP:

```
ss -lnu
```

Pregunta:

- ¿el servicio parece estar escuchando?;
- ¿en qué dirección?;
- ¿en qué puerto?.

No escanees otro equipo para responderlo.

47. Paso 6 — logs

Si el problema corresponde a una unidad systemd:

```
systemctl status NOMBRE.service
journalctl -u NOMBRE.service -b -n 30
```

Observa antes de reiniciar.

48. Privacidad de información de red

No publiques sin necesidad:

- dirección IP pública;
- IPv6 global;
- MAC;
- gateway;
- servidores DNS internos;
- dominios de búsqueda internos;
- lista completa de sockets;
- IP remotas de conexiones;
- hostnames privados.

Para pedir ayuda, comparte solo los campos imprescindibles y sustituye identificadores privados cuando se pueda conservar el significado técnico.

49. Qué no haremos

No ejecutaremos:

```
ip link set ... up/down
ip addr add ...
ip addr del ...
ip route add ...
ip route del ...
edición de /etc/resolv.conf
cambios con nmcli
cambios con networkctl
escaneo de puertos
escaneo de subredes
```

Esas acciones requieren contexto adicional o pertenecen a administración/ciberseguridad posterior.

50. Práctica A — interfaces

```
ip -br link
```

Identifica:

- loopback;
- una interfaz adicional, si existe;
- estado.

No compartas MAC.

51. Práctica B — direcciones

```
ip -br address
```

Para ti, identifica:

- IPv4;
- IPv6;
- prefijo.

No necesitas pegar la salida.

52. Práctica C — rutas

```
ip route
```

Busca la palabra:

```
default
```

si existe.

Explica qué interfaz usaría esa ruta.

53. Práctica D — localhost

```
getent hosts localhost
```

Explica:

- nombre consultado;
- dirección devuelta;
- por qué no necesita representar Internet.

54. Práctica E — DNS del sistema

```
command -v resolvectl
```

Si existe:

```
resolvectl status
```

Solo observa.

Si no existe:

```
resolvectl no disponible; práctica omitida
```

55. Práctica F — TCP escuchando

```
ss -lnt
```

Identifica:

- columna de estado;
- dirección local;
- puerto.

No publiques la lista completa.

56. Práctica G — UDP local

```
ss -lnu
```

Compara con TCP.

No interpretes ausencia de estado ESTAB como error: UDP funciona de forma diferente.

57. Práctica H — clasificar datos

Clasifica:

```
192.168.1.20/24
default via ...
127.0.0.1
::1
LISTEN
ESTAB
53
443
```

Categorías:

- dirección/prefijo;
- ruta;
- loopback;
- estado TCP;
- puerto.

Los puertos 53 y 443 son ejemplos convencionales, no una garantía del servicio real.

58. Errores frecuentes

Error	Problema	Corrección
Confundes IP con DNS	Dirección y resolución son conceptos distintos	Separa capa de nombres de capa IP
"UP = Internet funciona"	Faltan ruta/DNS/otros factores	Diagnostica por pasos
"default = única ruta"	Puede haber rutas más específicas	Lee tabla completa
"LISTEN = abierto a Internet"	Depende de bind, firewall y red	No concluyas exposición solo con ss
"puerto 443 = siempre HTTPS"	Es convención, no garantía absoluta	Confirma aplicación/protocolo
Publicas ss -tn completo	Puede revelar IP remotas	Comparte solo fragmentos
Editas resolv.conf directamente	Puede ser administrado por otra herramienta	Identifica quién gestiona DNS
Usas ifconfig/netstat como única referencia	Son herramientas históricas	Aprende primero ip/ss
Cambias ruta por prueba	Puedes cortar conectividad	Solo consulta
Escaneas otra red para aprender	No es necesario	Observa tu propio host

59. Método de diagnóstico inicial de red

Cuando hay un problema:

1. identifica la interfaz con `ip -br link`;
2. revisa dirección con `ip -br address`;
3. revisa rutas con `ip route`;
4. prueba resolución local con `getent`;
5. revisa DNS del sistema si procede;
6. revisa sockets del servicio con `ss`;
7. revisa status/logs;
8. cambia una sola cosa cuando tengas una hipótesis;
9. documenta el resultado.

60. Práctica independiente

Sin cambiar configuración:

1. identifica la interfaz loopback;
2. identifica una interfaz de red adicional;
3. consulta sus direcciones;
4. localiza el prefijo;
5. identifica la ruta por defecto si existe;
6. resuelve localhost;
7. comprueba si `resolvectl` existe;
8. lista TCP en escucha;
9. lista UDP;
10. explica por qué ninguna de estas consultas es un escaneo de otra máquina.

61. Mini evaluación

1. ¿Qué muestra `ip -br link`?
 - A. Interfaces de forma breve.
 - B. Paquetes instalados.
 - C. Usuarios.
2. ¿Qué muestra `ip -br address`?
 - A. Direcciones asociadas a interfaces.
 - B. Logs.
 - C. Procesos.
3. ¿Qué muestra `ip route`?
 - A. Tabla de rutas.
 - B. DNS cache exclusivamente.
 - C. Contraseñas.
4. ¿127.0.0.1 es loopback IPv4?
 - A. Sí.
 - B. No.
5. ¿::1 es loopback IPv6?
 - A. Sí.
 - B. No.
6. ¿DNS e IP son el mismo concepto?
 - A. Sí.
 - B. No.
7. ¿`ss -lnt` muestra sockets TCP en escucha con valores numéricos?

- A. Sí.
 - B. No.
8. ¿un socket LISTEN demuestra por sí solo exposición a Internet?
- A. Sí.
 - B. No.
9. ¿debemos modificar rutas en este módulo?
- A. Sí.
 - B. No.
10. ¿debemos escanear otra red para aprender ss?
- A. Sí.
 - B. No.

62. Registro de aprendizaje

Puedes responder:

```
Una interfaz es:
IPv4 es:
IPv6 es:
Un prefijo /24 representa:
Una ruta sirve para:
Una ruta default sirve para:
DNS sirve para:
Loopback IPv4 es:
Loopback IPv6 es:
ss sirve para:
LISTEN significa:
ESTAB significa:
¿Por qué no comparto toda mi salida de red?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

63. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales verificadas:

- iproute2 upstream (repositorio original de ip y ss): <https://git.kernel.org/pub/scm/network/iproute2/iproute2.git/>
- iproute2 (espejo de publicación, no repositorio principal): <https://github.com/iproute2/iproute2>
- Las páginas HTML de man7 siguientes se conservan como referencias de consulta, no como repositorio upstream.
- RFC Editor / IETF — RFC 1918, *Address Allocation for Private Internets*: <https://www.rfc-editor.org/rfc/rfc1918.html>
- iproute2 — ip(8): <https://man7.org/linux/man-pages/man8/ip.8.html>
- iproute2 — ip-address(8): <https://man7.org/linux/man-pages/man8/ip-address.8.html>
- iproute2 — ip-route(8): <https://man7.org/linux/man-pages/man8/ip-route.8.html>
- iproute2 — ss(8): <https://man7.org/linux/man-pages/man8/ss.8.html>
- Linux man-pages / glibc — getent(1): <https://man7.org/linux/man-pages/man1/getent.1.html>
- systemd — resolvectl: <https://www.freedesktop.org/software/systemd/man/latest/resolvectl.html>

Se posponen:

- subnetting detallado;
- ARP/NDP;
- DHCP;
- configuración con NetworkManager;
- netplan;
- systemd-networkd;
- Wi-Fi administrativo;
- modificación de rutas;
- firewall;
- NAT;
- packet capture;
- escaneo;
- namespaces de red;
- troubleshooting avanzado TCP.

Estado de la lección: redactada y revisada documentalmente. Las prácticas son locales y de consulta; no modifican red ni examinan sistemas ajenos.

Módulo 20 — SSH en sistemas propios o expresamente autorizados

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Vigésima entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué es SSH y para qué sirve;
- distinguir cliente SSH y servidor SSH;
- reconocer host, usuario y puerto;
- comprender el papel de la host key;
- explicar para qué sirve `~/.ssh/known_hosts`;
- interpretar una primera conexión sin aceptar claves a ciegas;
- distinguir contraseña de autenticación por clave pública;
- distinguir clave pública y clave privada;
- consultar la configuración efectiva del cliente sin conectarte;
- generar, de forma opcional, un par de claves de laboratorio y proteger la clave privada;
- reconocer qué prácticas no deben usarse en sistemas ajenos o sin autorización.

Conocimientos previos: - IP, DNS, rutas y puertos; - usuarios y permisos; - archivos y rutas; - procesos; - conceptos básicos de seguridad.

Alcance autorizado: SSH se practica únicamente en equipos propios, máquinas virtuales, laboratorios, CTF autorizados o sistemas para los que tengas permiso expreso. El manual sí estudiará, en módulos avanzados, cómo pueden fallar los mecanismos de autenticación, qué configuraciones los debilitan, qué indicadores dejan esos intentos y cómo se detectan y corrigen. Las prácticas ofensivas se limitarán a entornos controlados y autorizados; no se usarán para intentar entrar a equipos ajenos ni para evadir controles en sistemas reales sin permiso.

2. Qué es SSH

SSH significa:

```
Secure Shell
```

Es un protocolo y conjunto de herramientas para comunicación remota cifrada.

Un uso típico es iniciar una sesión de terminal en otro sistema:

```
cliente SSH — conexión cifrada — servidor SSH
```

También puede utilizarse como base para transferencias seguras y otros mecanismos, pero este módulo se limita al acceso remoto básico.

3. Cliente y servidor

Dos componentes distintos:

```
ssh → cliente  
sshd → servidor/daemon
```

El cliente inicia una conexión.

El servidor escucha y decide si permite autenticación según su configuración.

Tener el comando `ssh` instalado **no significa** que tu equipo esté ejecutando un servidor SSH.

4. Comprobar el cliente

Consulta:

```
command -v ssh
```

Después:

```
ssh -V
```

`ssh -V` muestra información de versión del cliente.

No realiza una conexión remota.

5. Sintaxis básica

La forma general:

```
ssh usuario@host
```

Ejemplo conceptual, usando un nombre de laboratorio:

```
ssh alumno@servidor-lab
```

No copies ese nombre literalmente si no existe.

Tres datos importantes:

- **usuario**: cuenta remota autorizada;
- **host**: nombre DNS o dirección del sistema autorizado;
- **puerto**: por defecto SSH suele usar TCP 22, aunque puede configurarse otro.

6. Puerto

Para un servidor configurado en un puerto distinto:

```
ssh -p PUERTO usuario@host
```

-p del cliente SSH usa **p minúscula**.

No pruebes puertos al azar en sistemas externos.

En este curso el puerto debe provenir de la configuración de tu propio laboratorio o de la información proporcionada por el administrador autorizado.

7. Qué ocurre antes de autenticarte

Antes de aceptar tu usuario, el cliente debe identificar al servidor.

El servidor presenta una **host key**.

El cliente puede compararla con una identificación conocida previamente.

Esto ayuda a detectar:

- servidor equivocado;
- suplantación;
- cambios inesperados de clave;
- ciertos ataques de intermediario.

La identidad del servidor y la identidad del usuario son problemas distintos.

8. Host key

Una host key identifica criptográficamente al servidor SSH.

No es:

- tu contraseña;
- tu clave privada de usuario;
- la dirección IP;
- un certificado TLS de navegador.

Un servidor puede tener diferentes tipos de host key.

La verificación correcta debe hacerse mediante una huella (*fingerprint*) obtenida por un canal confiable.

9. Primera conexión

En una primera conexión a un host no conocido, SSH puede mostrar:

- tipo de clave;
- fingerprint;
- pregunta para confiar en el host.

No respondas yes automáticamente.

Primero compara la fingerprint con una fuente confiable, por ejemplo:

- consola local del servidor propio;
- documentación del laboratorio;
- administrador autorizado;
- inventario de infraestructura confiable.

Procedimiento en un servidor propio o autorizado: desde su consola confiable, el administrador puede consultar la huella de la clave pública del host con `ssh-keygen -l -f /etc/ssh/ssh_host_ed25519_key.pub`. En el cliente, compara visualmente la huella mostrada por SSH con la obtenida desde esa consola o canal independiente. No copies claves privadas ni aceptes huellas sin verificar.

Una fingerprint desconocida no debe aceptarse solo porque “es la primera vez”.

10. known_hosts

OpenSSH mantiene identificaciones de hosts conocidos normalmente en:

```
~/.ssh/known_hosts
```

Cuando un host ya conocido presenta una clave distinta, el cliente lo advierte. Con la configuración predeterminada (`StrictHostKeyChecking ask`), y también con `yes` o `accept-new`, OpenSSH **se niega a conectar**. Solo si alguien configuró `StrictHostKeyChecking no`, la conexión puede continuar con restricciones, entre ellas la desactivación de la autenticación por contraseña (fuentes: `ssh_config(5)` y `ssh(1)`).

Eso no significa automáticamente un ataque; también puede ocurrir tras:

- reinstalación;
- regeneración de claves;
- cambio de servidor;
- cambio de IP reutilizada.

Pero la advertencia debe investigarse antes de continuar.

11. No desactivar StrictHostKeyChecking para “quitar el error”

Opciones que eliminan o debilitan la verificación de host pueden ocultar una advertencia de seguridad.

En este manual no enseñaremos como receta:

```
StrictHostKeyChecking=no
```

ni prácticas equivalentes para ignorar cambios de host key.

La solución correcta es verificar la identidad del servidor.

12. Buscar una entrada conocida

Si ya tienes un host autorizado registrado, puedes consultar:

```
ssh-keygen -F NOMBRE_HOST
```

Esto busca el host en `known_hosts`.

No modifica el archivo.

Si no existe una entrada, simplemente puede significar que aún no te has conectado o que el nombre está almacenado de otra forma.

13. Configuración efectiva sin conectar

Una práctica especialmente segura:

```
ssh -G localhost
```

`-G` hace que el cliente evalúe la configuración efectiva para ese destino y la imprima, sin establecer una sesión remota.

Para limitar la salida:

```
ssh -G localhost | head -n 20
```

Puedes observar valores como:

- hostname;
- user;
- port;
- opciones de autenticación.

No modifica el sistema.

14. Archivo de configuración del cliente

La configuración del usuario suele estar en:

```
~/.ssh/config
```

y la configuración general del cliente en:

```
/etc/ssh/ssh_config
```

No todos los usuarios tienen un `~/.ssh/config`.

En esta lección solo consultamos si existen.

No publiques el archivo completo: puede contener hostnames, usuarios, rutas y nombres internos.

15. Autenticación

Después de verificar al servidor, SSH debe autenticar al usuario.

Métodos que OpenSSH puede soportar incluyen, según configuración:

- clave pública;
- password;
- keyboard-interactive;
- otros mecanismos.

Que un método exista en el cliente no significa que el servidor lo permita.

16. Autenticación por contraseña

En un entorno autorizado, el servidor puede solicitar una contraseña.

Reglas:

- no escribas la contraseña en el chat;
- no la guardes en scripts;
- no la pases como argumento visible;
- no reutilices credenciales sin autorización;
- no pruebes contraseñas repetidamente.

SSH no convierte una mala práctica de contraseñas en una práctica segura.

17. Autenticación por clave pública

Modelo:

```
usuario conserva clave privada
servidor conoce clave pública autorizada
```

Durante la autenticación, el cliente demuestra que posee la clave privada sin enviarla al servidor como archivo.

La clave privada nunca debe compartirse.

18. Clave pública y clave privada

Un par de claves contiene:

```
clave privada → SECRETA
clave pública → puede distribuirse al sistema autorizado
```

Ejemplos de nombres tradicionales:

```
id_ed25519      → privada
id_ed25519.pub  → pública
```

No dependas solo del nombre: entiende qué archivo es cuál.

19. Regla absoluta: la privada no va a GitHub

Nunca subas a GitHub:

- claves SSH privadas;
- passphrases;
- tokens;
- contraseñas;
- archivos `.env`;
- credenciales.

Si detectamos una clave privada antes de un commit, la subida debe detenerse.

Una clave privada expuesta debe considerarse comprometida y rotarse según el sistema afectado.

20. Permisos de `~/.ssh`

OpenSSH aplica controles sobre archivos sensibles.

Una clave privada debe ser accesible únicamente según permisos apropiados para el usuario.

No uses:

```
chmod 777
```

sobre `~/.ssh` o claves.

Como referencia habitual de OpenSSH: `~/.ssh` debe estar restringido al propietario (modo `700`); las claves privadas y `~/.ssh/config` pueden mantenerse en modo `600`. `authorized_keys` también debe estar protegido frente a escritura ajena. Comprueba los permisos con `ls -ld ~/.ssh` y `ls -l ~/.ssh/config` cuando existan. No cambies permisos de archivos reales sin comprender primero M12 y las políticas del equipo.

21. Generar una clave de laboratorio — opcional

Esta sección explica el procedimiento; **no lo ejecute aquí**. La práctica correspondiente es la sección 41 (Práctica E). Si lo ejecutaras dos veces, el segundo `mkdir` fallaría y `ssh-keygen` preguntaría si quieres sobrescribir la clave.

Solo para un ejercicio local, puedes generar un par nuevo **sin reutilizar una clave real**.

Dentro de:

```
~/linux-lab/modulo-20-ssh/
```

crea una carpeta privada:

```
mkdir -m 700 claves-lab
```

Después, de forma opcional:

```
ssh-keygen -t ed25519 -f ./claves-lab/lab_ed25519 -C "linux-lab"
```

Cuando pregunte una passphrase, elige una solo para esta clave y **no la compartas**.

Esto crea:

```
lab_ed25519      → privada
lab_ed25519.pub  → pública
```

No subas ninguno de esos archivos al repositorio de aprendizaje. La clave privada está explícitamente prohibida y la pública de laboratorio tampoco aporta valor al repositorio.

22. Por qué Ed25519

OpenSSH actual soporta Ed25519 como tipo de clave.

Para este curso es una opción moderna y sencilla para una clave de laboratorio.

No necesitas comparar algoritmos criptográficos en profundidad todavía.

No uses tipos obsoletos solo porque aparezcan en tutoriales antiguos.

23. Passphrase de la clave

La passphrase protege el archivo de clave privada si alguien obtiene una copia del archivo.

No es enviada como contraseña al servidor.

Modelo:

```
passphrase → protege la privada local
clave privada → demuestra identidad
clave pública → registrada en servidor autorizado
```

No confundas estos tres elementos.

24. Fingerprint de una clave pública

Sobre la pública de laboratorio:

```
ssh-keygen -lf ./claves-lab/lab_ed25519.pub
```

Esto muestra una fingerprint.

La fingerprint es una representación compacta útil para comparar claves.

No revela la clave privada.

25. authorized_keys — concepto

En una cuenta remota configurada para autenticación por clave pública, las claves autorizadas suelen almacenarse en un archivo como:

```
~/.ssh/authorized_keys
```

del usuario remoto.

Agregar una clave a ese archivo **otorga capacidad de autenticación** según la configuración.

Por eso no modificaremos `authorized_keys` en esta lección sin un servidor de laboratorio definido y autorización explícita.

26. ssh-copy-id — existe, pero se pospone

`ssh-copy-id` puede facilitar instalar una clave pública en una cuenta remota autorizada.

No se usa en esta lección porque:

- modifica la cuenta remota;
- requiere verificar primero el servidor;
- requiere confirmar qué clave se copia;
- requiere un objetivo autorizado.

Se introducirá solo cuando trabajemos con una VM o servidor propio claramente identificado.

27. Conexión a localhost — solo si ya existe sshd

Comprueba:

```
ss -lnt
```

y, si usas `systemd`:

```
systemctl status ssh.service
systemctl status ssh.socket
systemctl status sshd.service
```

Comprueba **solo la unidad que exista**: Ubuntu reciente puede activar OpenSSH mediante `ssh.socket`, mientras que RHEL suele utilizar `sshd.service`. Una unidad `ssh.service` inactiva no demuestra por sí sola que el servidor esté apagado. Contrasta con la escucha de puertos mediante `ss -lnt`.

No instales ni habilites un servidor SSH solo para completar este módulo.

Si ya tienes un `sshd` propio configurado y sabes que `localhost` es tu equipo, una conexión local puede ser válida.

Si no, se omite.

28. sshd no debe exponerse por accidente

Instalar o habilitar un servidor SSH puede hacer que escuche en una interfaz de red según configuración.

Eso implica decisiones de:

- firewall;
- usuarios;
- autenticación;
- exposición;
- actualizaciones;
- logs;
- políticas de acceso.

Por eso la instalación/configuración del servidor se pospone.

29. Root login

El acceso SSH directo como root aumenta el impacto de una credencial comprometida y depende de la política del sistema.

No configuraremos ni recomendaremos habilitar root login en este módulo.

Trabajaremos con cuentas normales y mínimo privilegio.

30. Ataques contra autenticación: qué sí estudiaremos

Para aprender ciberseguridad de forma completa, el manual sí abordará más adelante, dentro de laboratorios propios o CTF autorizados:

- qué es fuerza bruta y por qué funciona cuando existen credenciales débiles;
- qué es password spraying a nivel conceptual;
- qué configuraciones de SSH aumentan el riesgo;
- qué registros e indicadores dejan los intentos de autenticación fallidos;
- cómo detectar patrones anómalos;
- cómo aplicar bloqueo, MFA, claves, políticas de contraseña y mínimo privilegio;
- cómo revisar y corregir una configuración vulnerable.

La finalidad será comprender **el mecanismo de fallo, la detección y la mitigación**.

No se incluirán procedimientos operativos destinados a vulnerar sistemas reales sin autorización ni automatizaciones reutilizables contra terceros.

31. Ejecutar un comando remoto

SSH puede ejecutar un comando en un host autorizado:

```
ssh usuario@host comando
```

Pero todavía no lo practicaremos.

Primero debes dominar:

- identidad del servidor;
- autenticación;
- permisos;
- salida remota;
- diferencias entre shell local y remota.

32. SCP y SFTP

OpenSSH incluye herramientas como:

```
scp  
sftp
```

para transferencia de archivos.

No se desarrollan en este módulo para no mezclar acceso remoto con transferencia.

Se introducirán después de que puedas identificar claramente origen, destino y permisos.

33. Agente SSH

ssh-agent puede mantener claves cargadas en memoria para evitar reintroducir passphrases repetidamente.

Eso implica decisiones adicionales de seguridad de sesión.

No lo configuraremos aún.

Nunca copies variables, sockets o material sensible a repositorios.

34. Archivo known_hosts cambiado

Si SSH avisa que la host key cambió:

1. detén la conexión;
2. identifica el host;
3. confirma si hubo reinstalación/cambio legítimo;
4. compara la nueva fingerprint por un canal confiable;
5. solo entonces corrige la entrada antigua si corresponde.

No elimines la advertencia a ciegas.

35. ssh-keygen -R — solo tras verificar

Existe:

```
ssh-keygen -R NOMBRE_HOST
```

para retirar entradas de known_hosts.

Es una operación que modifica tu archivo.

No se practica aquí.

Primero debes demostrar que el cambio de host key es legítimo.

36. Preparar el laboratorio

```
cd ~/linux-lab  
pwd
```

Comprueba:

```
ls -ld ./modulo-20-ssh
```

Si no existe:

```
mkdir modulo-20-ssh
```

Después:

```
cd modulo-20-ssh  
pwd
```

No copies archivos reales de ~/ssh al laboratorio.

37. Práctica A — comprobar cliente

```
command -v ssh  
ssh -V
```

Responde para ti:

- ¿está instalado el cliente?;
- ¿qué implementación/versión reporta?.

No necesitas actualizarlo por esta práctica.

38. Práctica B — configuración efectiva

```
ssh -G localhost | head -n 20
```

Identifica:

- hostname;
- user;
- port.

No se realiza una conexión.

39. Práctica C — archivos de configuración

Consulta:

```
ls -ld ~/.ssh
ls -l ~/.ssh/config
```

Si no existen, no los crees solo por la práctica.

No publiques su contenido.

40. Práctica D — known_hosts

Consulta:

```
ls -l ~/.ssh/known_hosts
```

Si existe, no lo abras completo ni lo subas.

Si ya conoces un hostname autorizado:

```
ssh-keygen -F NOMBRE_HOST
```

Si no tienes uno, omite esa parte.

41. Práctica E — clave de laboratorio opcional

Solo si quieres practicar generación:

```
mkdir -m 700 claves-lab
ssh-keygen -t ed25519 -f ./claves-lab/lab_ed25519 -C "linux-lab"
```

Después:

```
ls -l ./claves-lab
ssh-keygen -lf ./claves-lab/lab_ed25519.pub
```

No abras ni copies la clave privada.

No hagas commit de claves-lab.

42. Práctica F — clasificar archivos

Clasifica:

```
~/.ssh/config
~/.ssh/known_hosts
id_ed25519
id_ed25519.pub
authorized_keys
/etc/ssh/ssh_config
```

Responde:

- configuración de cliente;
- hosts conocidos;
- privada;
- pública;
- claves autorizadas en cuenta remota;
- configuración global del cliente.

43. Práctica G — decidir si conectarse

Caso:

```
Servidor propio: sí
Fingerprint verificada: no
```

Decisión:

no aceptar todavía; verificar primero.

Caso:

Servidor desconocido encontrado en Internet
Autorización: no

Decisión:

no intentar conexión ni autenticación.

44. Errores frecuentes

Error	Problema	Corrección
Aceptas cualquier host key	Puedes confiar en el servidor equivocado	Verifica fingerprint
Confundes host key con user key	Identifican entidades distintas	Separa servidor y usuario
Subes la clave privada a GitHub	Comprometes identidad	Nunca la publiques
Usas <code>chmod 777 ~/.ssh</code>	Permisos inseguros	Usa mínimo privilegio
Ignoras aviso de host key cambiada	Puedes ocultar un incidente	Investiga
Desactivas <code>StrictHostKeyChecking</code>	Debilitas verificación	Mantén verificación
Instalas <code>sshd</code> sin pensar	Puedes exponer un servicio	Posponer hasta módulo de servidor
Pruebas contraseñas ajenas	No autorizado	Solo cuentas propias/autorizadas
Confundes cliente con servidor	ssh no implica sshd activo	Comprueba por separado
Publicas <code>known_hosts/config</code>	Revelas infraestructura	Mantén privados los detalles

45. Método seguro antes de una conexión SSH

1. confirma que el sistema es propio o autorizado;
2. identifica hostname/IP correcto;
3. confirma puerto;
4. confirma usuario;
5. obtén fingerprint por un canal confiable;
6. conecta;
7. verifica la host key;
8. autentícate con un método autorizado;
9. usa mínimo privilegio;
10. revisa logs ante anomalías.

46. Práctica independiente

Sin conectarte a sistemas externos:

1. confirma que existe el cliente SSH;
2. consulta su versión;
3. muestra configuración efectiva para `localhost`;
4. identifica puerto y usuario resultantes;
5. comprueba si existe `~/.ssh`;
6. explica para qué sirve `known_hosts`;
7. explica diferencia entre pública y privada;
8. explica por qué la privada nunca va a GitHub;
9. explica qué harías ante una host key cambiada;
10. explica qué autorización necesitarías antes de usar SSH contra otro equipo;
11. explica qué parte ofensiva de SSH estudiarías únicamente en un laboratorio autorizado;
12. indica qué evidencia revisarías para detectar intentos anómalos de autenticación.

47. Mini evaluación

1. ¿qué comando es el cliente OpenSSH?
 - A. ssh
 - B. sshd
 - C. ss
2. ¿sshd es el servidor?
 - A. Sí.
 - B. No.
3. ¿la host key identifica al servidor?
 - A. Sí.
 - B. No.
4. ¿known_hosts ayuda a recordar identidades de servidores?
 - A. Sí.

- B. No.
- 5. ¿debes aceptar una fingerprint desconocida sin verificar?
 - A. Sí.
 - B. No.
- 6. ¿la clave privada puede subirse a GitHub?
 - A. Sí.
 - B. No.
- 7. ¿la clave pública y la privada son el mismo archivo?
 - A. Sí.
 - B. No.
- 8. ¿ssh -G localhost establece una sesión remota?
 - A. Sí.
 - B. No.
- 9. ¿debes desactivar StrictHostKeyChecking para eliminar advertencias?
 - A. Sí.
 - B. No.
- 10. ¿puedes practicar SSH contra un sistema ajeno sin permiso?
 - A. Sí.
 - B. No.

48. Registro de aprendizaje

Puedes responder:

```
SSH sirve para:
ssh es:
sshd es:
Una host key identifica:
known_hosts sirve para:
Una clave privada:
Una clave pública:
authorized_keys sirve para:
¿Por qué verifico la fingerprint?:
¿Por qué no desactivo StrictHostKeyChecking?:
¿Qué sistemas puedo usar para practicar?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

49. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales verificadas:

- OpenSSH ssh(1): <https://man.openbsd.org/ssh>
- OpenSSH ssh-keygen(1): <https://man.openbsd.org/ssh-keygen>
- OpenSSH ssh_config(5): https://man.openbsd.org/ssh_config
- OpenSSH sshd(8): <https://man.openbsd.org/sshd>
- OpenSSH sshd_config(5): https://man.openbsd.org/sshd_config

Se posponen:

- instalación y exposición de sshd;
- configuración de sshd_config;
- firewall para SSH;
- authorized_keys práctico en un servidor;
- ssh-copy-id;
- ssh-agent;
- SCP/SFTP;
- forwarding de puertos;
- ProxyJump;
- certificados SSH;
- bastion hosts;
- hardening avanzado;
- análisis de ataques contra autenticación en laboratorio;
- detección de fuerza bruta/password spraying en logs;
- ejercicios ofensivos controlados en CTF o máquinas propias;
- mitigaciones avanzadas como MFA, rate limiting y políticas de acceso.

Estado de la lección: redactada y revisada documentalmente. Las prácticas son locales o requieren sistema propio/autorizado; no se realizan intentos contra terceros.

Módulo 21 — Firewall: nftables, UFW y firewalld según el entorno

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Vigésimoprimera entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué hace un firewall de host;
- distinguir filtrado de paquetes de autenticación, antivirus y cifrado;
- reconocer Netfilter como infraestructura del kernel Linux;
- explicar qué es nftables y qué hace la utilidad nft;
- reconocer UFW como interfaz simplificada documentada por Ubuntu;
- reconocer firewalld como gestor dinámico documentado por RHEL;
- distinguir tráfico de entrada, salida y reenvío;
- comprender reglas, políticas, estados de conexión y orden de evaluación;
- consultar qué herramientas existen sin modificar reglas;
- explicar por qué no debes mezclar varios gestores sin comprender cómo interactúan;
- elaborar un plan seguro antes de cambiar un firewall en una máquina local o remota.

Conocimientos previos: - interfaces, direcciones IP y rutas; - TCP/UDP y puertos; - ss; - systemd; - SSH; - permisos y mínimo privilegio.

Seguridad: las prácticas obligatorias son de identificación y lectura. No abriremos ni cerraremos puertos, no cambiaremos políticas, no habilitaremos/deshabilitaremos el firewall, no recargaremos reglas y no vaciaremos rulesets. Toda práctica futura de modificación se realizará únicamente en equipos propios, VMs, laboratorios, CTF o sistemas expresamente autorizados.

2. Qué es un firewall

Un firewall aplica reglas al tráfico de red.

Modelo simplificado:

```
paquete
↓
reglas del firewall
↓
permitir / rechazar / descartar / continuar evaluación
```

Un firewall puede decidir según información como:

- dirección origen;
- dirección destino;
- protocolo;
- puerto;
- interfaz;
- estado de conexión;
- zona;
- otras propiedades.

3. Lo que un firewall NO es

Un firewall no sustituye:

- autenticación;
- parches;
- cifrado;
- permisos;
- configuración segura;
- antivirus/antimalware;
- monitoreo;
- copias de seguridad.

Ejemplo:

■ permitir TCP 22 no autentica al usuario SSH.

Solo permite que el tráfico alcance el servicio si las demás condiciones también lo permiten.

4. Netfilter

Linux incluye el subsistema **Netfilter** en el kernel.

Las soluciones modernas de firewall en Linux utilizan esta infraestructura para clasificar y decidir el destino del tráfico.

Herramientas de espacio de usuario pueden configurar reglas que el kernel aplica.

Modelo:

```
herramienta de usuario
  ↓
  Netfilter
  ↓
  kernel
```

No confundas Netfilter con una única utilidad de línea de comandos.

5. nftables

nftables es el framework moderno de filtrado de paquetes que sucede a los conjuntos históricos de herramientas como:

```
iptables
ip6tables
arptables
ebtables
```

La utilidad principal es:

```
nft
```

RHEL 10 documenta nftables como opción para escenarios donde se necesita control detallado del ruleset.

6. Elementos de nftables

Modelo inicial:

```
tabla
├── cadena
│   └── regla
```

Tabla

Agrupar cadenas y reglas dentro de una familia/contexto.

Cadena

Contiene reglas y puede asociarse a puntos de procesamiento del tráfico.

Regla

Describe coincidencias y una acción.

No vamos a crear ninguna todavía.

7. Familias de nftables

nftables admite familias como:

```
ip
ip6
inet
arp
bridge
netdev
```

Para principiantes, `inet` es importante porque permite trabajar con IPv4 e IPv6 dentro de la misma familia de reglas.

No crearemos tablas ni cadenas en este módulo.

8. Tráfico de entrada, salida y reenvío

Tres conceptos fundamentales:

INPUT/FILTRO DE ENTRADA	→ tráfico dirigido al propio host
OUTPUT/FILTRO DE SALIDA	→ tráfico originado por el propio host
FORWARD/REENVÍO	→ tráfico que atraviesa el host hacia otro destino

Los nombres exactos de cadenas dependen de la herramienta y configuración.

Un equipo de escritorio normalmente no necesita comportarse como router solo porque tenga varias interfaces.

9. Aceptar, descartar y rechazar

Acciones conceptuales:

```
accept → permitir
drop   → descartar silenciosamente
reject → rechazar y, según configuración, responder con un error
```

No existe una única elección correcta para todos los contextos.

La política debe diseñarse según el servicio, red y amenaza.

10. Stateful firewall

Un firewall **stateful** mantiene información sobre el estado de las conexiones.

Eso permite distinguir, por ejemplo:

```
conexión nueva
conexión ya establecida
tráfico relacionado
```

Netfilter proporciona connection tracking para estos escenarios.

UFW es stateful por diseño según la documentación de Ubuntu.

11. Regla y política por defecto

Una cadena puede tener reglas específicas y una política aplicable cuando ninguna regla coincide, según el sistema utilizado.

Conceptualmente:

```
reglas específicas
    ↓
si ninguna coincide
    ↓
política por defecto
```

La idea de “denegar por defecto” es importante en seguridad, pero **no debe aplicarse a ciegas** en una máquina remota porque podrías bloquear tu propio acceso.

12. Orden de evaluación

En muchos diseños de firewall, el orden de reglas importa.

Una regla anterior puede decidir el tráfico antes de que se evalúe otra posterior.

Por eso no basta con preguntar:

■ “¿existe una regla para el puerto?”

También debes preguntar:

- ¿en qué cadena?;
- ¿en qué orden?;
- ¿qué tráfico coincide?;
- ¿qué política existe?;
- ¿qué gestor creó esa regla?.

13. UFW en Ubuntu

Ubuntu documenta:

```
ufw
```

como una herramienta simplificada para crear un firewall de host.

UFW significa:

La documentación de Ubuntu señala que está disponible y **deshabilitado inicialmente por defecto** (comprobado en la documentación de seguridad de Ubuntu el 9 de octubre de 2026).

No generalizamos ese comportamiento a Debian.

14. UFW no es “el firewall del kernel”

UFW es una interfaz de administración.

Por debajo utiliza capacidades de Netfilter mediante backends apropiados.

Modelo:

```
UFW
↓
reglas de filtrado
↓
Netfilter
```

Por eso no debes pensar que UFW, nftables y Netfilter son tres firewalls completamente independientes.

15. Consulta de disponibilidad de UFW

Consulta segura:

```
command -v ufw
```

Si existe, puedes saber que la herramienta está instalada.

Una consulta como:

```
ufw status
```

es de lectura, pero algunas configuraciones pueden requerir privilegios.

Si obtienes permiso denegado, **no añadas sudo solo para completar la práctica**.

16. Comandos UFW que modifican

Ejemplos administrativos:

```
ufw enable
ufw disable
ufw allow ...
ufw deny ...
ufw delete ...
```

Estos comandos modifican la política o las reglas.

No se ejecutan en esta lección.

Riesgo especial

Si administras un equipo por SSH y habilitas un firewall sin haber permitido correctamente el acceso necesario, puedes bloquear tu propia sesión o conexiones futuras.

17. firewalld

RHEL 10 documenta:

```
firewalld
```

como firewall dinámico para casos comunes.

Conceptos importantes:

- zonas;
- servicios;
- puertos;
- políticas;
- reglas.

La interfaz principal de línea de comandos es:

```
firewall-cmd
```

18. Zonas de firewalld

Una **zona** representa un nivel/contexto de confianza para tráfico asociado a interfaces o fuentes.

Ejemplos de nombres que pueden existir:

```
public
home
internal
trusted
drop
```

No memorices sus políticas solo por el nombre.

Consulta la configuración efectiva del sistema.

19. Servicio en firewalld

Un “service” de firewalld agrupa información necesaria para permitir un servicio de red.

Puede incluir:

- puertos;
- protocolos;
- helpers/módulos según configuración.

Ejemplo conceptual:

```
servicio SSH
→ TCP 22 de forma convencional
```

Pero SSH puede usar otro puerto, y la definición efectiva debe comprobarse.

20. runtime y permanent en firewalld

firewalld distingue entre:

```
runtime    → estado efectivo actual
permanent → configuración persistente
```

Un cambio runtime puede perderse al recargar/reiniciar.

Un cambio permanent no necesariamente se aplica inmediatamente al runtime hasta realizar la acción correspondiente.

Esta diferencia es crítica.

No realizaremos cambios en ninguno de los dos.

21. Consultas firewalld

Si existe:

```
command -v firewall-cmd
```

puedes intentar una consulta no modificadora:

```
firewall-cmd --state
```

Si responde:

```
running
```

el servicio está activo.

También existen consultas como:

```
firewall-cmd --get-active-zones
firewall-cmd --list-all
```

Si requieren autorización en tu entorno, no eleves privilegios solo por la práctica.

22. nft y lectura del ruleset

Consulta de disponibilidad:

```
command -v nft
```

La orden conceptual para listar el ruleset es:

```
nft list ruleset
```

A menudo la lectura completa requiere privilegios.

Si tu usuario no puede verla, registra:

```
ruleset no accesible sin privilegios
```

y continúa.

No uses sudo solo para terminar el ejercicio.

23. nft flush ruleset — comando de alto riesgo

Existe una operación que puede vaciar el conjunto de reglas:

```
nft flush ruleset
```

No la ejecutes.

Puede eliminar protecciones de red y afectar reglas instaladas por otras herramientas que compartan infraestructura.

Se incluye solo para que puedas reconocerla como una orden peligrosa.

24. No mezclar gestores a ciegas

RHEL 10 advierte que no deben operar simultáneamente servicios de firewall que puedan interferirse entre sí.

Problema conceptual:

```
firewalld administra reglas
+
administrador modifica nftables manualmente sin coordinación
=
estado difícil de razonar
```

Antes de modificar, identifica quién administra el firewall.

25. iptables en contexto

iptables sigue apareciendo en documentación, sistemas heredados y capas de compatibilidad.

Pero nftables es el framework moderno sucesor.

No significa que todo comando iptables haya dejado de existir.

Tampoco significa que debas convertir reglas automáticamente sin comprender la implementación.

26. iptables-nft

Algunos sistemas ofrecen herramientas iptables que traducen o interactúan con infraestructura nftables.

Esto hace todavía más importante no asumir que:

```
“veo iptables” = “estoy usando exclusivamente el framework histórico”
```

La implementación concreta debe verificarse.

27. Firewall y SSH

Antes de modificar un firewall en un servidor remoto:

1. confirma el puerto real de SSH;

2. confirma la interfaz/red desde la que administras;
3. conserva una sesión existente;
4. asegúrate de tener acceso alternativo o consola;
5. aplica el cambio mínimo;
6. verifica antes de cerrar la sesión.

En este módulo **no hacemos ese cambio**; solo aprendemos el procedimiento mental.

28. Firewall y servicios escuchando

Compara:

```
ss -lnt
```

con el firewall.

ss responde:

¿qué sockets están escuchando localmente?

El firewall responde:

¿qué tráfico puede pasar según las reglas?

Un servicio puede escuchar pero estar bloqueado por firewall.

También puede existir una regla permitiendo un puerto sin que ningún servicio esté escuchando.

29. Firewall no “abre un servicio”

Si permites un puerto en un firewall:

- no instala software;
- no inicia el servicio;
- no configura autenticación;
- no crea automáticamente una aplicación que escuche.

Por eso:

```
servicio
+
socket escuchando
+
ruta
+
firewall
+
autenticación
```

son piezas distintas.

30. IPv4 e IPv6

Una política que protege solo IPv4 puede dejar IPv6 con un comportamiento distinto si el host lo utiliza.

Al diseñar firewalls debes revisar ambos protocolos cuando sean relevantes.

nftables con familia inet puede tratar IPv4 e IPv6 en un mismo ruleset.

No deshabilites IPv6 simplemente para evitar aprender sus reglas.

31. Entrada y salida

Muchos tutoriales se concentran en tráfico entrante.

Pero un firewall también puede controlar:

- tráfico saliente;
- tráfico reenviado.

Bloquear salida sin planificación puede romper:

- DNS;
- actualizaciones;
- sincronización;
- APIs;

- repositorios;
- servicios.

No establezcas políticas de salida restrictivas a ciegas.

32. Forwarding

El tráfico **forwarded** atraviesa el equipo.

Ejemplos:

- router;
- gateway;
- host con redes virtuales;
- host de contenedores.

Un equipo con Docker, Podman, Kubernetes u otras tecnologías puede tener reglas adicionales.

No modifiques rulesets de un host con contenedores sin comprender qué componente los administra.

33. NAT no es igual a firewall

NAT modifica información de direccionamiento/puertos para determinados flujos.

Firewall filtra tráfico.

Pueden coexistir en Netfilter, pero son conceptos distintos.

No estudiaremos NAT en profundidad todavía.

34. REJECT frente a DROP

Conceptualmente:

```
DROP → descarta
REJECT → rechaza de forma explícita según el protocolo/regla
```

No hay una regla universal de “siempre usa DROP”.

La decisión depende del entorno, diagnóstico y política.

35. Logs del firewall

Los firewalls pueden registrar determinados eventos.

Pero habilitar logging excesivo puede:

- llenar logs;
- consumir recursos;
- crear ruido;
- registrar datos sensibles.

No activaremos logging nuevo en esta lección.

Usaremos los logs existentes cuando lleguemos a laboratorios defensivos.

36. Preparar la práctica

No necesitas crear reglas.

Primero identifica el sistema:

```
cat /etc/os-release
```

Después:

```
command -v nft
command -v ufw
command -v firewall-cmd
command -v iptables
```

Anota solo qué herramientas existen.

37. Práctica A — identificar gestor probable

Completa:

```
Distribución:
nft disponible:
ufw disponible:
firewall-cmd disponible:
iptables disponible:
```

No concluyas todavía cuál administra realmente el firewall solo porque un binario exista.

38. Práctica B — servicios systemd relacionados

Consulta:

```
systemctl list-unit-files | grep -E 'firewalld|nftables|ufw'
```

Esto solo busca nombres conocidos entre unidades.

No habilites ni inicies ninguno.

39. Práctica C — firewalld si existe

Si firewall-cmd existe:

```
firewall-cmd --state
```

Si funciona sin privilegios, registra el estado.

Si no funciona por permisos o porque firewalld no está activo, no fuerces la práctica.

40. Práctica D — UFW si existe

Si ufw existe, primero:

```
command -v ufw
```

La consulta conceptual de estado es:

```
ufw status
```

Si tu sistema requiere privilegios para verla, no añadas sudo solo por este ejercicio.

41. Práctica E — nftables si existe

Si nft existe, reconoce:

```
nft list ruleset
```

como la consulta del conjunto de reglas.

Si no tienes permisos para verla, no eleves privilegios.

Nunca ejecutes `nft flush ruleset`.

42. Práctica F — comparar servicio y firewall

Ejecuta:

```
ss -lnt
```

Elige un socket local que reconozcas.

Responde conceptualmente:

- ¿qué dirección escucha?
- ¿qué puerto?
- ¿el hecho de escuchar demuestra que el firewall lo permite externamente?

Respuesta esperada a la última: no.

43. Práctica G — clasificar acciones

Clasifica como:

CONSULTA

MODIFICA

ALTO RIESGO

Ejemplos:

```
command -v nft
firewall-cmd --state
ufw status
nft list ruleset
ufw enable
ufw allow 22
firewall-cmd --add-service=ssh
nft add rule ...
nft flush ruleset
```

No ejecutes las acciones que modifican.

44. Práctica H — escenario SSH

Escenario:

Adminstras un servidor propio solo por SSH.
Quieres activar un firewall.
No tienes consola alternativa.
No has confirmado el puerto SSH.

Decisión correcta:

☐ no activar todavía.

Primero debes diseñar y verificar el acceso de administración y un plan de recuperación.

45. Método seguro antes de cambiar un firewall

1. identifica distribución;
2. identifica gestor real;
3. guarda/consulta la configuración actual;
4. identifica interfaces;
5. identifica servicios escuchando;
6. identifica qué tráfico necesitas;
7. considera IPv4 e IPv6;
8. prepara acceso alternativo;
9. aplica el cambio mínimo;
10. verifica conectividad;
11. revisa logs;
12. documenta el cambio.

46. Método de diagnóstico

Si un servicio “no responde”:

1. ¿el proceso está activo?;
2. ¿escucha con ss?;
3. ¿en qué dirección/puerto?;
4. ¿existe ruta?;
5. ¿el firewall permite el flujo?;
6. ¿la aplicación exige autenticación?;
7. ¿los logs muestran errores?.

No empieces abriendo puertos al azar.

47. Errores frecuentes

Error	Problema	Corrección
“Firewall = antivirus”	Funciones distintas	Separa controles
“Puerto permitido = servicio activo”	La app puede no estar escuchando	Comprueba con ss

Error	Problema	Corrección
“LISTEN = accesible desde Internet”	Faltan firewall, ruta y NAT	Evalúa todo el camino
Habilitas UFW remotamente sin plan	Puedes bloquear SSH	Prepara acceso antes
Mezclas firewalld y nft manual	Estado difícil de predecir	Identifica un gestor
Ejecutas nft flush ruleset	Puedes eliminar protecciones	No usar en práctica
Solo proteges IPv4	IPv6 puede seguir activo	Evalúa ambos
Abres un puerto para “probar”	Amplías superficie	Diagnostica primero
Usas sudo para cualquier consulta	Privilegios innecesarios	Mantén mínimo privilegio
Copias reglas de otra distro	Modelo/gestor puede diferir	Sigue documentación del entorno

48. Práctica independiente

Sin modificar el firewall:

1. identifica distribución;
2. detecta nft, ufw, firewall-cmd e iptables;
3. identifica unidades relacionadas mediante systemd;
4. consulta firewalld solo si está disponible y accesible sin elevar privilegios;
5. reconoce cómo consultar UFW sin ejecutarlo si requiere privilegios;
6. reconoce nft list ruleset como consulta;
7. revisa tus sockets locales con ss -lnt;
8. explica por qué un socket en LISTEN no demuestra exposición;
9. explica por qué no mezclarías gestores;
10. explica el riesgo de cambiar un firewall por SSH.

49. Mini evaluación

1. ¿Netfilter pertenece al kernel Linux?
 - A. Sí.
 - B. No.
2. ¿nftables es el sucesor moderno de varias herramientas históricas de filtrado?
 - A. Sí.
 - B. No.
3. ¿UFW es documentado por Ubuntu como interfaz simplificada?
 - A. Sí.
 - B. No.
4. ¿Ubuntu documenta UFW inicialmente deshabilitado por defecto?
 - A. Sí.
 - B. No.
5. ¿firewalld utiliza el concepto de zonas?
 - A. Sí.
 - B. No.
6. ¿runtime y permanent significan lo mismo en firewalld?
 - A. Sí.
 - B. No.
7. ¿un puerto permitido significa que un servicio está ejecutándose?
 - A. Sí.
 - B. No.
8. ¿un socket LISTEN garantiza exposición pública?
 - A. Sí.
 - B. No.
9. ¿debes ejecutar nft flush ruleset para aprender?
 - A. Sí.
 - B. No.
10. ¿debes planificar acceso alternativo antes de cambiar el firewall de un servidor remoto?
 - A. Sí.
 - B. No.

50. Registro de aprendizaje

Puedes responder:

```
Un firewall sirve para:
Netfilter es:
nftables es:
nft es:
UFW es:
firewalld es:
Una zona es:
runtime significa:
permanent significa:
LISTEN no significa exposición porque:
¿Por qué no mezclo gestores?:
¿Por qué no ejecuto nft flush ruleset?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

51. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales verificadas para esta edición:

- Netfilter/nftables (proyecto original): <https://www.netfilter.org/projects/nftables/index.html>
- nftables (documentación del proyecto): <https://wiki.nftables.org/>
- firewalld (documentación original): <https://firewalld.org/documentation/>
- UFW se documenta con las fuentes oficiales de Ubuntu que siguen.
- Ubuntu Security — Firewall: <https://documentation.ubuntu.com/security/security-features/network/firewall/>
- Ubuntu Server — Firewall / UFW: <https://ubuntu.com/server/docs/firewalls/>
- RHEL 10 — Configuring firewalls and packet filters:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/configuring_firewalls_and_packet_filters/
- RHEL 10 — Getting started with nftables:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/configuring_firewalls_and_packet_filters/getting-started-with-nftables

Se posponen:

- creación real de reglas nftables;
- habilitación real de UFW;
- zonas y servicios firewalld prácticos;
- cambios runtime/permanent;
- reglas IPv4/IPv6;
- forwarding;
- NAT;
- logging de firewall;
- sets/maps avanzados;
- rate limiting;
- políticas por interfaz/origen;
- hardening práctico de SSH con firewall;
- laboratorios ofensivos/defensivos controlados.

Estado de la lección: redactada y revisada documentalente. Las prácticas obligatorias son de consulta; no modifican el firewall.

Módulo 22 — vi/Vim: edición segura de archivos de texto

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Vigésimosegunda entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué son `vi` y `Vim`;
- distinguir modo normal, inserción y línea de comandos;
- abrir un archivo de práctica;
- desplazarte sin modificar contenido;
- insertar texto;
- guardar cambios;
- salir con y sin guardar;
- deshacer y rehacer;
- buscar texto;
- copiar, pegar y borrar líneas de forma controlada;
- activar números de línea;
- reconocer comandos que descartan cambios;
- evitar editar archivos del sistema con privilegios innecesarios.

Conocimientos previos: - terminal; - rutas; - archivos; - permisos; - `cat`, `less`, `head`, `tail`; - redirecciones.

Seguridad: toda la práctica se realiza dentro de `~/linux-lab/modulo-22-vim`. No editaremos `/etc`, `/usr`, `/var` ni otros archivos del sistema. No usaremos `sudo vi` o `sudo vim`. Algunos comandos de Vim pueden borrar texto o descartar cambios; se explican antes de usarlos.

2. vi y Vim

`vi` es un editor clásico de Unix.

Vim significa:

```
Vi IMproved
```

Vim amplía el comportamiento tradicional de `vi` con muchas funciones adicionales.

En algunos sistemas:

```
vi
```

puede ser Vim ejecutándose en modo compatible, otro editor compatible con `vi` o una implementación distinta.

Por eso primero consulta:

```
command -v vi
command -v vim
```

Si `vim` no existe pero `vi` sí, puedes practicar con `vi`.

Nota para instalaciones mínimas (`vim.tiny`): en Debian y derivados, `vi` puede apuntar a una compilación reducida de Vim. Consulta `readlink -f "$(command -v vi)"` si `readlink` está disponible, y `vi --version` si la implementación lo admite. Algunas funciones avanzadas de Vim pueden faltar. Para desplazarte de forma portable en modo normal usa `h`, `j`, `k`, `l`; no asumas que las flechas se comportan igual en todos los modos.

3. Un editor modal

Vim es un editor **modal**.

Eso significa que las teclas hacen cosas distintas según el modo actual.

Tres modos fundamentales:

```
NORMAL
INSERT
COMMAND-LINE
```

No escribas texto hasta entender en qué modo estás.

4. Modo normal

Es el modo principal para:

- moverte;
- borrar;
- copiar;
- pegar;
- deshacer;
- buscar;
- ejecutar comandos de edición.

Cuando abres Vim normalmente comienzas en modo normal.

Para volver a modo normal desde inserción:

```
Esc
```

Esta es una de las teclas más importantes del módulo.

5. Modo inserción

En modo inserción, lo que escribes se introduce como texto.

Una forma básica de entrar:

```
i
```

`i` significa insertar antes de la posición actual.

Para salir del modo inserción:

```
Esc
```

Secuencia básica:

```
i
escribir texto
Esc
```

6. Línea de comandos

Desde modo normal, pulsa:

```
:
```

para entrar en la línea de comandos de Vim.

Ahí se utilizan órdenes como:

```
:w
:q
:wq
:q!
```

Después pulsas Enter.

7. Preparar el laboratorio

Entra a:

```
cd ~/linux-lab
pwd
```

Comprueba:

```
ls -ld ./modulo-22-vim
```

Si no existe:

```
mkdir modulo-22-vim
```

Después:

```
cd modulo-22-vim
pwd
ls -la
```

8. Crear un archivo de práctica

Comprueba primero que el nombre no exista:

```
ls -l practica.txt
```

Si no existe, créalo:

```
touch practica.txt
```

Después abre:

```
vim practica.txt
```

o, si Vim no está disponible:

```
vi practica.txt
```

9. Primera inserción

Dentro de Vim:

1. pulsa i;
2. escribe:

```
Primera línea de práctica.
Segunda línea de práctica.
```

3. pulsa Esc.

Ahora estás de nuevo en modo normal.

10. Guardar con :w

Desde modo normal:

```
:w
```

y Enter.

w significa **w**rite.

Guarda los cambios en el archivo actual.

Importante: guardar modifica el archivo en disco.

Por eso solo practicamos con archivos creados expresamente para el laboratorio.

11. Salir con :q

Si no tienes cambios pendientes:

```
:q
```

y Enter.

q significa **q**uit.

Si existen cambios sin guardar, Vim normalmente impedirá salir y mostrará una advertencia.

Eso es una protección útil.

12. Guardar y salir con :wq

```
:wq
```

guarda y sale.

Equivale conceptualmente a:

```
:w  
:q
```

en una sola orden.

13. :q! descarta cambios

Este comando necesita una advertencia explícita:

```
:q!
```

sale descartando cambios no guardados.

El:

```
!
```

fuerza la acción en este contexto.

Riesgo: puedes perder trabajo no guardado.

No uses :q! por costumbre.

Úsalo solo cuando estés seguro de que deseas abandonar los cambios.

14. :x

```
:x
```

guarda si existen cambios y luego sale.

No es exactamente idéntico internamente a :wq en todas las situaciones, aunque para un principiante suelen parecer equivalentes.

Por claridad inicial usaremos principalmente:

```
:w  
:q  
:wq
```

15. Movimiento básico

En modo normal:

```
h → izquierda  
j → abajo  
k → arriba  
l → derecha
```

También pueden funcionar las flechas según la terminal/editor.

Aprender h j k l es útil porque funciona dentro del modelo clásico de vi.

16. Movimiento por palabras

En modo normal:

```
w → siguiente palabra  
b → palabra anterior  
e → final de palabra
```

No necesitas memorizar todos los movimientos hoy.

Primero domina:

```
h j k l  
w b
```

17. Inicio y final de línea

```
0 → inicio físico de línea  
^ → primer carácter no blanco  
$ → final de línea
```

No confundas:

```
0
```

con la letra O mayúscula.

18. Ir a una línea

Ejemplo:

```
10G
```

va a la línea 10.

```
G
```

va al final del archivo.

```
gg
```

en Vim suele ir al principio.

19. Añadir texto con a

En modo normal:

```
a
```

entra en inserción **después** del carácter actual.

Comparación:

```
i → insertar antes  
a → insertar después
```

Después recuerda:

```
Esc
```

20. Nueva línea con o

En modo normal:

```
o
```

crea una nueva línea debajo y entra en inserción.

```
O
```

crea una nueva línea arriba.

Distingue mayúsculas y minúsculas.

21. Deshacer con u

En modo normal:

```
u
```

deshace el último cambio.

Si borraste o modificaste algo accidentalmente, detente y prueba:

```
u
```

antes de seguir haciendo cambios.

22. Rehacer con Ctrl+r

En modo normal:

```
Ctrl+r
```

rehace un cambio que habías deshecho.

Práctica:

1. modifica una palabra;
2. pulsa `Esc`;
3. pulsa `u`;
4. pulsa `Ctrl+r`.

Observa el cambio.

Compatibilidad de vi: en sistemas mínimos, `vi` puede ser una variante reducida o funcionar en modo compatible. Si las flechas insertan caracteres inesperados, vuelve al modo normal con `Esc` y utiliza `h`, `j`, `k`, `l`. Consulta `vi --version` cuando esté disponible; si existe `vim`, puede ofrecer un comportamiento más predecible para las funciones avanzadas. En algunos modos compatibles, pulsar `u` repetidamente alterna deshacer y rehacer, y `Ctrl+r` puede no comportarse como en Vim. Verifica el editor instalado antes de seguir el ejercicio.

23. Borrar un carácter con x

En modo normal:

```
x
```

borra el carácter bajo el cursor.

Riesgo: modifica el contenido.

Si lo haces por accidente:

```
u
```

para deshacer.

24. Borrar una línea con dd

```
dd
```

borra la línea actual.

Esto es una operación destructiva sobre el contenido del buffer.

Antes de practicarla:

- usa solo `practica.txt`;
- asegúrate de poder deshacer;
- no guardes si borraste algo que querías conservar.

Puedes recuperar inmediatamente con:

```
u
```

25. Copiar una línea con yy

En modo normal:

```
yy
```

copia o “yank” la línea actual al registro correspondiente.

No modifica el texto por sí mismo.

26. Pegar con p

Después de `yy`:


```
p
```

pega después de la posición/línea actual según el contenido copiado.

Secuencia:

```
yy  
p
```

duplica una línea en un caso sencillo.

27. dd también coloca texto en un registro

Cuando haces:

```
dd
```

la línea borrada normalmente queda disponible para pegar.

Por eso:

```
dd  
p
```

puede mover una línea.

No necesitas estudiar registros de Vim en profundidad todavía.

28. Buscar con /

En modo normal:

```
/palabra
```

y Enter.

Busca hacia adelante.

Para repetir la búsqueda:

```
n
```

Para repetir en dirección contraria:

```
N
```

La búsqueda no modifica el archivo.

29. Buscar hacia atrás con ?

```
?palabra
```

busca hacia atrás.

No necesitas dominar ambas direcciones inmediatamente; / y n son suficientes al principio.

30. Números de línea

Desde modo normal:

```
:set number
```

activa números de línea.

Para desactivarlos:

```
:set nonumber
```

Esto cambia la visualización de la sesión actual, no el contenido del archivo.

31. Mostrar el número actual

```
Ctrl+g
```

puede mostrar información como nombre de archivo y posición.

También puedes usar:

```
:set ruler
```

según configuración.

32. Reemplazar un carácter con r

En modo normal:

```
rX
```

reemplaza el carácter actual por X.

No entra permanentemente en modo inserción.

Práctica solo sobre texto desechable.

33. Cambiar una palabra con cw

```
cw
```

cambia desde la posición actual hasta el final de la palabra según las reglas de movimiento.

Después entras en inserción.

Secuencia:

```
cw  
nuevo texto  
Esc
```

Si todavía resulta confuso, usa i y edición simple.

34. Visual mode

Vim tiene modo visual.

Una forma de entrar:

```
v
```

Permite seleccionar texto visualmente.

No es esencial para dominar vi básico, así que solo lo introducimos.

Pulsa:

```
Esc
```

para salir de la selección.

35. Guardar con otro nombre

Desde Vim:

```
:w copia.txt
```

guarda el buffer en otro archivo.

Riesgo: puede crear o sobrescribir un archivo según el contexto y opciones.

En este módulo solo úsalo con un nombre nuevo dentro de ~/linux-lab/modulo-22-vim.

Antes verifica desde otra terminal si el nombre está libre, o usa un nombre claramente nuevo.

36. Abrir archivo de solo lectura

Si solo quieres inspeccionar:

```
view archivo.txt
```

cuando `view` está disponible, suele abrir Vim en modo de solo lectura.

Otra posibilidad:

```
vim -R archivo.txt
```

No garantiza protección absoluta contra todas las formas de forzar escritura, pero ayuda a evitar modificaciones accidentales.

Para lectura simple sigue siendo válido usar:

```
less archivo.txt
```

37. No usar Vim para todo

Herramienta según tarea:

```
cat   → archivo pequeño, salida completa
less  → lectura navegable
head  → principio
tail  → final
vim   → edición interactiva
```

No abras un archivo enorme en un editor solo para consultar dos líneas si otra herramienta es más apropiada.

38. Archivos swap

Vim puede crear archivos temporales/swap para ayudar a recuperar sesiones.

Si al abrir un archivo aparece una advertencia de swap, no elijas opciones al azar.

Puede significar:

- otra sesión está editando el archivo;
- la sesión anterior terminó de forma inesperada;
- existe una recuperación pendiente;
- el swap es antiguo.

Primero lee la advertencia y verifica si otra sesión está abierta.

39. No borrar un swap sin revisar

Eliminar un archivo swap a ciegas puede hacerte perder una oportunidad de recuperación.

Primero identifica:

- archivo original;
- fecha;
- proceso;
- otra sesión;
- si existen cambios recuperables.

La recuperación avanzada se pospone.

40. vimtutor

Si está instalado:

```
command -v vimtutor
```

puedes ejecutar:

```
vimtutor
```

Es un tutorial interactivo de Vim.

No es obligatorio.

Si no existe, no instales paquetes solo para esta práctica.

41. Editar archivos del sistema

Una instrucción como:

```
sudo vim /etc/archivo
```

combina:

- editor poderoso;
- privilegios elevados;
- archivo de configuración del sistema.

Un error puede afectar servicios o arranque.

Por eso no se practica en este módulo.

Primero aprenderás:

- copia de seguridad;
- validación de sintaxis;
- overrides;
- cambios mínimos;
- rollback.

42. No usar :w! a ciegas

```
:w!
```

intenta forzar una escritura en determinados contextos.

No debe convertirse en respuesta automática a:

```
E45
E212
readonly
permission denied
```

Primero pregunta:

¿debo realmente escribir en este archivo?

Un error de permisos puede ser una protección correcta.

43. No usar :q! por frustración

Si Vim no te deja salir porque hay cambios:

1. pulsa Esc;
2. decide si quieres conservarlos;
3. si sí: :wq;
4. si no: :q!.

No memorices solo :q!.

44. Práctica A — abrir, insertar y guardar

```
vim practica.txt
```

Dentro:

```
i
Linux y Vim.
Esc
:w
:q
```

Después verifica:

```
cat practica.txt
```

45. Práctica B — movimiento

Abre de nuevo:

```
vim practica.txt
```

Practica únicamente en modo normal:

```
h j k l  
w b  
0 $
```

No cambies texto.

Sal:

```
:q
```

46. Práctica C — deshacer

Dentro del archivo de práctica:

1. pulsa i;
2. escribe una palabra de prueba;
3. pulsa Esc;
4. pulsa u.

Comprueba que el cambio se deshace.

No guardes hasta verificar.

47. Práctica D — rehacer

Después de deshacer:

```
Ctrl+r
```

Comprueba que el cambio reaparece.

Después decide si deseas conservarlo.

48. Práctica E — copiar y pegar

Colócate sobre una línea.

Ejecuta:

```
yy  
p
```

Deberías duplicar la línea.

Después:

```
u
```

si no quieres conservar el duplicado.

49. Práctica F — borrar y recuperar

Sobre una línea desechable:

```
dd
```

Observa que desaparece.

Inmediatamente:

```
u
```

para recuperarla.

No guardes entre ambos pasos.

50. Práctica G — búsqueda

Añade varias líneas y guarda.

Después:

```
/Linux
```

Enter.

Repite:

```
n
```

La búsqueda no modifica el archivo.

51. Práctica H — números de línea

```
:set number
```

Observa.

Después:

```
:set nonumber
```

No cambia el archivo.

52. Práctica I — salir correctamente

Escenario 1: no hiciste cambios.

```
:q
```

Escenario 2: quieres guardar.

```
:wq
```

Escenario 3: hiciste cambios que deliberadamente quieres descartar.

```
:q!
```

Explica antes de usar el tercero qué perderás.

53. Errores frecuentes

Error	Qué ocurrió	Corrección
Escribes y aparecen comandos extraños	Estás en modo normal	Pulsa <code>i</code> para insertar
Teclas <code>hjk</code> aparecen como texto	Estás en inserción	Pulsa <code>Esc</code>
No puedes salir	Hay cambios pendientes o sigues en otro modo	<code>Esc</code> , luego decide <code>:wq</code> o <code>:q!</code>
Usas <code>:q!</code> por costumbre	Pierdes cambios	Decide antes
Borras línea con <code>dd</code>	Comando normal de borrado	Usa <code>u</code> si fue accidental
Guardas archivo incorrecto	No verificaste nombre/ruta	Consulta nombre y <code>pwd</code>
Abres archivo del sistema con <code>sudo</code>	Riesgo elevado	Practica en <code>linux-lab</code>
Usas <code>:w!</code> ante error de permisos	Intentas forzar sin diagnosticar	Revisa propiedad/ruta/necesidad
Borras <code>swap</code> sin leer aviso	Puedes perder recuperación	Investiga primero
Editas cuando solo querías leer	Riesgo innecesario	Usa <code>less</code> o <code>vim -R</code>

54. Método seguro antes de editar

1. ejecuta `pwd`;
2. confirma el nombre del archivo;
3. comprueba permisos con `ls -l`;
4. decide si necesitas editar o solo leer;

5. si es importante, asegúrate de tener respaldo o control de versiones;
6. abre sin sudo;
7. realiza un cambio pequeño;
8. revisa;
9. guarda;
10. valida el resultado.

55. Práctica independiente

Dentro de una nueva carpeta del laboratorio:

1. crea un archivo vacío;
2. ábrelo con Vim/vi;
3. inserta tres líneas;
4. guarda;
5. busca una palabra;
6. activa números de línea;
7. duplica una línea con yy y p;
8. deshaz el duplicado;
9. borra una línea y recupérala;
10. guarda y sal;
11. verifica con cat.

No uses sudo.

56. Mini evaluación

1. ¿en qué modo puedes usar dd?
 - A. Normal.
 - B. Inserción.
2. ¿qué tecla suele volver a modo normal?
 - A. Esc.
 - B. Enter.
3. ¿qué hace :w?
 - A. Guarda.
 - B. Borra.
4. ¿qué hace :q?
 - A. Sale si es posible.
 - B. Guarda siempre.
5. ¿qué hace :q!?
 - A. Sale descartando cambios no guardados.
 - B. Crea una copia.
6. ¿qué hace u en modo normal?
 - A. Deshacer.
 - B. Guardar.
7. ¿qué hace yy?
 - A. Copia una línea.
 - B. La elimina permanentemente.
8. ¿qué hace dd?
 - A. Borra/corta una línea.
 - B. Activa números.
9. ¿:set number modifica el contenido del archivo?
 - A. Sí.
 - B. No.
10. ¿debes practicar con sudo vim /etc/...?
 - A. Sí.
 - B. No.

57. Registro de aprendizaje

Puedes responder:

```
Modo normal sirve para:
Modo inserción sirve para:
Esc sirve para:
i hace:
:w hace:
:q hace:
:wq hace:
:q! hace:
u hace:
Ctrl+r hace:
yy hace:
dd hace:
p hace:
/texto sirve para:
¿Por qué no uso sudo vim para practicar?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

58. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- Vim documentation: <https://vimhelp.org/>
- Vim user manual: https://vimhelp.org/usr_toc.txt.html
- Vim reference manual: https://vimhelp.org/#reference_toc
- Vim project: <https://www.vim.org/>

Se posponen:

- macros;
- registros avanzados;
- sustituciones complejas;
- regex de Vim;
- buffers;
- ventanas;
- tabs;
- plugins;
- configuración avanzada de `.vimrc`;
- edición de archivos del sistema;
- recuperación avanzada de swap;
- diff;
- scripting de Vim.

Estado de la lección: redactada y revisada documentalmente. Las prácticas se limitan a archivos propios del laboratorio.

Módulo 23 — Primer script Bash y shebang

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Vigésimotercera entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué es un script;
- crear un archivo `.sh` de práctica;
- escribir un primer script Bash;
- explicar qué es un shebang;
- distinguir `bash script.sh` de `./script.sh`;
- comprender por qué un archivo necesita permiso de ejecución para usarse como `./script.sh`;
- aplicar `chmod +x` sobre un archivo propio del laboratorio;
- interpretar errores básicos como `Permission denied` y `command not found`;
- comprobar qué intérprete ejecutará el script;
- mantener el script dentro de un entorno seguro y versionable.

Conocimientos previos: - terminal; - rutas y archivos; - permisos básicos; - vi/Vim; - `chmod`; - ejecución de comandos.

Seguridad: todas las prácticas se realizan en `~/linux-lab/modulo-23-bash`. No ejecutaremos scripts descargados de Internet, no usaremos `sudo`, no modificaremos archivos del sistema y no añadiremos comandos destructivos. Antes de ejecutar un script, debes poder leerlo y explicar qué hace.

2. Qué es un script

Un **script** es un archivo de texto que contiene instrucciones que un intérprete puede ejecutar.

Ejemplo conceptual:

```
script.sh
├─ línea 1
├─ línea 2
└─ línea 3
```

En este módulo, el intérprete será Bash.

Un script no es automáticamente “un programa compilado”. Bash lee e interpreta sus instrucciones.

3. Extensión `.sh`: útil, pero no obligatoria

Un archivo Bash puede llamarse:

```
hola.sh
```

La extensión `.sh` ayuda a reconocer su propósito.

Pero Bash no exige obligatoriamente esa extensión.

Lo importante es:

- contenido válido;
- intérprete correcto;
- permisos apropiados;
- forma de ejecución.

Para aprender, usaremos `.sh` por claridad.

4. Primer archivo

Entra al laboratorio:

```
cd ~/linux-lab
pwd
```

Comprueba:

```
ls -ld ./modulo-23-bash
```

Si no existe:

```
mkdir modulo-23-bash
```

Después:

```
cd modulo-23-bash
pwd
```

Crea:

```
touch hola.sh
```

Comprueba:

```
ls -l hola.sh
```

5. Editar el script

Abre:

```
vim hola.sh
```

o:

```
vi hola.sh
```

Escribe:

```
#!/usr/bin/env bash

printf 'Hola desde Bash\n'
```

Guarda y sal.

6. Línea 1 — shebang

```
#!/usr/bin/env bash
```

Esta primera línea se llama **shebang**.

Comienza con:

```
#!
```

Cuando el sistema ejecuta el archivo directamente como programa, esa línea indica qué intérprete debe utilizar.

En este caso:

```
/usr/bin/env
```

busca bash según el entorno y PATH.

7. Por qué usamos /usr/bin/env bash en el laboratorio

Ventaja:

- no depende de que Bash esté exactamente en /bin/bash.

Ejemplo:

```
#!/usr/bin/env bash
```

puede localizar Bash mediante PATH.

Pero esto tiene una consecuencia:

el Bash elegido depende del entorno.

En entornos controlados, scripts administrativos o contextos de seguridad, un intérprete de ruta fija puede ser preferible.

8. Alternativa: /bin/bash

También verás:

```
#!/bin/bash
```

Esto solicita específicamente:

```
/bin/bash
```

Ventaja:

- intérprete determinado por una ruta concreta.

Desventaja:

- esa ruta no es universal en todos los sistemas tipo Unix.

En muchas distribuciones Linux existe, pero no debes asumirlo fuera de tu entorno.

9. Qué opción usaremos

Para scripts de aprendizaje del usuario:

```
#!/usr/bin/env bash
```

Para scripts de sistema o entornos donde necesitas un intérprete exacto:

verifica primero la ruta y política del entorno.

No existe una única elección perfecta para todos los contextos.

10. Línea vacía

Después del shebang usamos una línea vacía:

```
#!/usr/bin/env bash

printf ...
```

No es obligatoria para ejecutar.

Se utiliza por legibilidad.

Ayuda a separar:

- cabecera;
- cuerpo del script.

11. printf

La segunda instrucción real:

```
printf 'Hola desde Bash\n'
```

printf imprime texto con formato.

Aquí:

```
'Hola desde Bash\n'
```

contiene:

- texto;
- \n = salto de línea.

Salida:

```
Hola desde Bash
```

12. Por qué printf y no echo

echo es muy común:

```
echo "Hola"
```

Pero su comportamiento con algunas opciones y secuencias puede variar entre implementaciones.

Para scripting didáctico y salida controlada, `printf` suele ser más predecible.

Esto no significa que `echo` esté prohibido.

13. Ejecutar mediante Bash

Sin cambiar permisos:

```
bash hola.sh
```

Aquí ejecutas:

```
bash
```

y le entregas:

```
hola.sh
```

como archivo de instrucciones.

Modelo:

```
bash → abre hola.sh → interpreta contenido
```

En esta forma, el permiso de ejecución del archivo no es necesario si Bash puede leerlo.

14. El shebang no decide esta ejecución

Si escribes:

```
bash hola.sh
```

tú ya elegiste Bash explícitamente.

El shebang no es lo que seleccionó al intérprete en esa orden.

Esto es importante.

15. Ejecutar directamente

Otra forma:

```
./hola.sh
```

Aquí pides al sistema ejecutar el archivo directamente.

Para eso necesitas:

- shebang válido;
- permiso de ejecución;
- permisos sobre el camino/directorio;
- intérprete disponible.

16. Por qué usamos ./

Si escribes:

```
hola.sh
```

la shell normalmente busca el comando en los directorios de `PATH`.

El directorio actual:

```
.
```

no suele estar en `PATH` por razones prácticas y de seguridad.

Por eso:

```
./hola.sh
```

significa:

| ejecuta `hola.sh` desde el directorio actual.

17. No añadas `.` a PATH por comodidad

Agregar el directorio actual globalmente a PATH puede hacer que ejecutes accidentalmente un archivo local con el mismo nombre que una herramienta esperada.

Para aprender, usa:

```
./hola.sh
```

No cambies PATH para evitar escribir `./`.

18. Permission denied

Antes de dar permiso de ejecución, prueba:

```
./hola.sh
```

Es posible que recibas:

```
Permission denied
```

Eso no significa que debas usar `sudo`.

Consulta:

```
ls -l hola.sh
```

Si no tiene `x`, falta permiso de ejecución.

19. chmod +x

Sobre tu archivo de laboratorio:

```
chmod u+x hola.sh
```

Esto añade ejecución al propietario.

Comprueba:

```
ls -l hola.sh
```

Podrías ver algo como:

```
-rwxr--r--
```

según permisos previos.

No necesitamos dar ejecución a grupo y otros para esta práctica.

20. Ejecutar después del permiso

Ahora:

```
./hola.sh
```

Salida esperada:

```
Hola desde Bash
```

21. chmod +x frente a chmod 777

No uses:

```
chmod 777 hola.sh
```

para “hacerlo ejecutable”.

Eso también concede escritura/lectura/ejecución a clases que probablemente no lo necesitan.

Usamos:

```
chmod u+x hola.sh
```

porque modifica solo el permiso necesario para el propietario.

22. Verificar el shebang

Puedes consultar la primera línea:

```
head -n 1 hola.sh
```

Debe mostrar:

```
#!/usr/bin/env bash
```

Comprueba también:

```
command -v bash
```

Esto indica qué Bash encontraría tu shell actual mediante PATH.

23. env y PATH

/usr/bin/env bash utiliza el entorno para localizar bash.

Por eso:

```
PATH
```

influye.

En un laboratorio de usuario es práctico.

En contextos privilegiados no debes depender de un PATH no confiable.

No ejecutaremos scripts Bash con sudo en este módulo.

24. Comentarios

En Bash:

```
# Esto es un comentario
```

Bash ignora el comentario durante la ejecución.

Ejemplo:

```
#!/usr/bin/env bash

# Primer script del laboratorio.
printf 'Hola desde Bash\n'
```

25. ¿El shebang es un comentario?

Para Bash, una línea que comienza con # normalmente tiene comportamiento de comentario.

Pero:

```
#!
```

al inicio de un archivo ejecutado directamente tiene significado especial para el mecanismo de ejecución del sistema.

Por eso el shebang debe estar en la primera línea.

26. No poner espacios antes del shebang

Correcto:

```
#!/usr/bin/env bash
```

Evita:

```
#!/usr/bin/env bash
```

El reconocimiento del shebang depende de que aparezca al inicio del archivo.

27. Saltos de línea Windows

Un script creado en Windows puede usar terminaciones CRLF.

Eso puede producir errores como:

```
/usr/bin/env: 'bash\r': No such file or directory
```

o mensajes parecidos.

La causa no es necesariamente que Bash falte.

Puede existir un carácter `\r` oculto.

28. Consultar el tipo de archivo

Puedes usar:

```
file hola.sh
```

La salida puede ayudar a detectar:

- texto;
- codificación;
- terminaciones CRLF en algunos casos.

No cambia el archivo.

29. Bash -n

Antes de ejecutar un script Bash puedes revisar su sintaxis:

```
bash -n hola.sh
```

`-n` hace que Bash lea las órdenes sin ejecutarlas, comprobando sintaxis.

Si no imprime errores:

no garantiza que el programa haga lo correcto.

Solo significa que no detectó determinados errores sintácticos.

30. Bash -x

Existe:

```
bash -x script.sh
```

que muestra trazas de órdenes durante la ejecución.

No lo usaremos todavía porque en scripts reales podría mostrar datos sensibles como argumentos o variables.

Se estudiará cuando veamos depuración.

31. Error command not found dentro del script

Ejemplo:

```
#!/usr/bin/env bash

printf 'Hola\n'
```

Tiene un error:

```
printf
```

en lugar de:

```
printf
```

Bash puede responder:

```
command not found
```

La corrección no es instalar un paquete.

Primero revisa la ortografía.

32. Error de sintaxis

Ejemplo conceptual:

```
printf 'Hola\n
```

falta cerrar la comilla.

bash -n puede detectar este tipo de problema.

Cuando llegue el momento de corregir scripts:

1. localiza línea;
2. identifica la estructura abierta;
3. haz la corrección mínima;
4. vuelve a validar.

33. No ejecutar scripts descargados sin revisar

No hagas:

```
bash archivo-descargado.sh
```

solo porque una guía diga “ejecútalo”.

Primero:

1. identifica procedencia;
2. abre el archivo;
3. comprende comandos;
4. busca cambios destructivos;
5. busca credenciales;
6. revisa red;
7. comprueba si solicita privilegios.

En este curso los scripts iniciales los escribimos nosotros.

34. curl | bash

No utilizaremos:

```
curl ... | bash
```

como método de aprendizaje.

Ese patrón descarga contenido y lo ejecuta sin revisión previa.

Primero descarga, inspecciona y comprende si alguna vez un entorno autorizado lo requiere.

35. No ejecutar con sudo

Evita:

```
sudo ./hola.sh
```

No hace falta.

Nuestro script solo imprime texto.

Ejecutar un script con privilegios amplía el impacto de cualquier error.

36. Primera estructura recomendada

Para los primeros scripts:

```
#!/usr/bin/env bash

# Descripción breve.
printf 'Mensaje\n'
```

Mantén el script:

- corto;
- legible;
- sin privilegios;
- sin operaciones destructivas.

37. Práctica A — crear el script

Dentro del laboratorio:

```
touch saludo.sh
vim saludo.sh
```

Contenido:

```
#!/usr/bin/env bash

printf 'Bienvenido al laboratorio Bash\n'
```

Guarda.

38. Práctica B — validación

```
bash -n saludo.sh
```

Si no aparece error:

```
bash saludo.sh
```

Debes ver:

```
Bienvenido al laboratorio Bash
```

39. Práctica C — permiso

Consulta:

```
ls -l saludo.sh
```

Después:

```
chmod u+x saludo.sh
```

Comprueba otra vez:

```
ls -l saludo.sh
```

Explica qué permiso cambió.

40. Práctica D — ejecución directa

```
./saludo.sh
```

Explica:

- qué significa ./;
- qué función cumple el shebang;
- por qué ahora puede ejecutarse.

41. Práctica E — comentario

Edita:

```
#!/usr/bin/env bash

# Muestra un saludo.
printf 'Bienvenido al laboratorio Bash\n'
```

Ejecuta otra vez.

El comentario no debe imprimirse.

42. Práctica F — error intencional y corrección

En una copia del script:

```
cp saludo.sh saludo-error.sh
```

Edita la copia y cambia temporalmente:

```
printf
```

por:

```
prinf
```

Ejecuta:

```
bash saludo-error.sh
```

Observa el error.

Después corrige **solo** esa palabra.

No borres el script original.

43. Práctica G — sintaxis rota

En otra copia:

```
cp saludo.sh sintaxis-error.sh
```

Quita deliberadamente la comilla final del texto.

Después:

```
bash -n sintaxis-error.sh
```

Lee el error.

Corrige la comilla y vuelve a validar.

44. GitHub y scripts de aprendizaje

Los scripts de esta lección sí tienen valor educativo y deben conservarse en GitHub.

Antes de subir:

- revisa que no haya contraseñas;
- revisa que no haya tokens;
- revisa rutas privadas;
- revisa claves;
- revisa datos personales.

Ejemplo de ruta:

```
linux/bash/
```

Ejemplo de commit:

```
linux: añadir primer script Bash con shebang
```

No se hará commit de archivos con secretos.

45. Errores frecuentes

Error	Problema	Corrección
<code>./script.sh: Permission denied</code>	Puede faltar permiso de ejecución; también pueden intervenir permisos de directorios, un montaje <code>noexec</code> u otros controles	Revisar ruta y permisos; añadir <code>chmod u+x</code> solo si falta ese permiso en el archivo propio
<code>script.sh: command not found</code>	Directorio actual no está en PATH	Usa <code>./script.sh</code>
Shebang no funciona	No está en primera línea o intérprete no existe	Revisa primera línea
<code>bash\r</code> no encontrado	Posible CRLF	Revisa formato del archivo
Usas <code>chmod 777</code>	Das permisos excesivos	Añade solo <code>u+x</code>
Usas <code>sudo</code>	Privilegios innecesarios	Ejecuta como usuario normal
Script descargado se ejecuta sin leer	Riesgo de cambios no entendidos	Inspecciona antes
<code>bash -n</code> no da error y asumes que todo está bien	Solo verifica sintaxis	Prueba lógica por separado
Escribes mal un comando	Bash no lo encuentra	Revisa ortografía antes de instalar nada

46. Método antes de ejecutar un script

1. identifica quién lo creó;
2. abre el archivo;
3. lee cada línea;
4. comprueba rutas;
5. identifica operaciones que modifican/borran;
6. comprueba llamadas de red;
7. identifica privilegios;
8. valida sintaxis;
9. ejecuta en entorno adecuado;
10. revisa resultado.

47. Práctica independiente

Crea:

```
mi-primer-script.sh
```

Debe contener:

- shebang;
- un comentario;
- dos líneas `printf`.

Después:

1. valida con `bash -n`;
2. ejecuta con `bash archivo`;
3. consulta permisos;
4. añade `u+x`;
5. ejecuta con `./archivo`;
6. explica la diferencia entre ambas formas.

No copies el ejemplo exacto.

48. Mini evaluación

1. ¿qué es un script?
 - A. Archivo de instrucciones interpretables.
 - B. Solo un binario compilado.
2. ¿qué es un shebang?
 - A. Línea que indica intérprete al ejecutar directamente.
 - B. Un permiso.
3. ¿debe estar en la primera línea?
 - A. Sí.
 - B. No.
4. ¿`bash script.sh` necesita obligatoriamente permiso `x` en el archivo?
 - A. Sí.
 - B. No, si Bash puede leerlo.
5. ¿`./script.sh` normalmente requiere permiso de ejecución?

- A. Sí.
 - B. No.
6. ¿qué significa ./?
- A. Directorio actual.
 - B. Directorio raíz.
7. ¿es recomendable chmod 777 para ejecutar un script?
- A. Sí.
 - B. No.
8. ¿bash -n ejecuta normalmente las órdenes?
- A. Sí.
 - B. No.
9. ¿una validación sintáctica garantiza lógica correcta?
- A. Sí.
 - B. No.
10. ¿debes ejecutar scripts descargados sin revisarlos?
- A. Sí.
 - B. No.

49. Registro de aprendizaje

Puedes responder:

```
Un script es:
Un shebang sirve para:
#!/usr/bin/env bash significa:
bash script.sh hace:
./script.sh hace:
./ significa:
chmod u+x hace:
bash -n sirve para:
¿Por qué no uso chmod 777?:
¿Por qué no ejecuto scripts sin leerlos?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

50. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Bash Reference Manual: <https://www.gnu.org/software/bash/manual/>
- GNU Bash — Shell Scripts: https://www.gnu.org/software/bash/manual/html_node/Shell-Scripts.html
- GNU Bash — Invoking Bash: https://www.gnu.org/software/bash/manual/html_node/Invoking-Bash.html
- Linux man-pages — execve(2): <https://man7.org/linux/man-pages/man2/execve.2.html>
- GNU Coreutils — chmod: https://www.gnu.org/software/coreutils/manual/html_node/chmod-invocation.html

Se posponen:

- variables;
- read;
- parámetros;
- \$@;
- expansión;
- quoting avanzado;
- condicionales;
- bucles;
- funciones;
- exit codes avanzados;
- set -e;
- traps;
- ShellCheck;
- debugging con set -x.

Estado de la lección: redactada y revisada documentalmente. Las prácticas usan scripts propios, cortos y sin privilegios.

Módulo 24 — Variables, entrada, argumentos, expansiones y quoting en Bash

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Vigésimocuarta entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- crear variables en Bash;
- diferenciar asignación y expansión;
- usar `${variable}`;
- entender por qué una asignación no lleva espacios alrededor de `=`;
- leer entrada con `read -r`;
- recibir argumentos posicionales;
- reconocer `$0`, `$1`, `$2`, `$#`, `"$@"` y `$?`;
- explicar la diferencia entre comillas simples, dobles y ausencia de comillas;
- evitar errores por espacios y expansión de nombres de archivo;
- distinguir `"$@"` de `"$*"`;
- usar sustitución de comandos `$(...)` de forma básica;
- mantener datos separados de código.

Conocimientos previos: - primer script Bash; - shebang; - `printf`; - archivos; - permisos; - pipes y redirecciones.

Seguridad: las prácticas son locales y no usan privilegios. No colocaremos contraseñas, tokens, claves ni otros secretos en variables, argumentos o scripts. Los argumentos de línea de comandos pueden quedar visibles en procesos, historial o logs según el entorno.

2. Qué es una variable

Una variable asocia un nombre con un valor.

Ejemplo:

```
nombre='Ana'
```

Aquí:

- `nombre` = nombre de la variable;
- `=` = asignación;
- `'Ana'` = valor.

Después puedes consultar su valor mediante expansión:

```
printf '%s\n' "$nombre"
```

3. Asignar no es expandir

Estas dos operaciones son distintas:

```
nombre='Ana'
```

asigna.

En cambio:

```
printf '%s\n' "$nombre"
```

expande el valor.

Modelo:

```
asignación → guardar
expansión  → obtener/usar
```

4. No hay espacios alrededor de =

Correcto:

```
edad=15
```

Incorrecto:

```
edad = 15
```

Bash interpretaría eso como palabras de una orden, no como una asignación.

Regla inicial:

```
nombre=valor
```

sin espacios alrededor de =.

5. Nombres de variables

Un nombre sencillo puede contener:

- letras;
- números;
- guion bajo.

Pero no debe comenzar con número.

Ejemplos válidos:

```
nombre
edad2
nombre_usuario
```

Ejemplos no válidos como nombres simples de variables:

```
2edad
nombre-usuario
```

Para aprender, usa nombres descriptivos.

6. Convención de nombres

En scripts educativos usaremos:

```
nombre_usuario
archivo_salida
contador
```

y reservaremos MAYÚSCULAS principalmente para variables de entorno o constantes convencionales:

```
HOME
PATH
USER
```

No es una regla sintáctica absoluta, sino una convención útil.

7. Expansión con \$

Si:

```
nombre='Ana '
```

entonces:

```
printf '%s\n' "$nombre"
```

expande a:

```
Ana
```

El símbolo \$ indica que Bash debe realizar una expansión de parámetro en este contexto.

8. Forma con llaves

También puedes escribir:

```
printf '%s\n' "${nombre}"
```

Las llaves ayudan a delimitar el nombre.

Ejemplo:

```
archivo='reporte'
printf '%s\n' "${archivo}.txt"
```

Salida:

```
reporte.txt
```

9. Comillas dobles

Las comillas dobles:

```
" "
```

permiten varias expansiones de Bash dentro del texto.

Ejemplo:

```
nombre='Ana'
printf '%s\n' "Hola, $nombre"
```

Salida:

```
Hola, Ana
```

10. Comillas simples

Las comillas simples:

```
' '
```

preservan el contenido de forma literal dentro de ellas.

Ejemplo:

```
nombre='Ana'
printf '%s\n' 'Hola, $nombre'
```

Salida:

```
Hola, $nombre
```

La variable no se expande.

11. Sin comillas

Ejemplo:

```
texto='uno dos'
printf '%s\n' $texto
```

La expansión sin comillas puede sufrir:

- división en palabras;
- expansión de nombres de archivo (*globbing*).

Eso puede cambiar el número de argumentos que recibe el comando.

Por eso la regla inicial será:

■ Si una expansión representa texto o una ruta, normalmente protégela con comillas dobles.

12. Quoting contextual, no regla ciega

No diremos:

■ “toda expansión siempre debe llevar comillas dobles”.

Hay contextos donde las reglas son distintas, por ejemplo:

- dentro de `[ [ ... ] ]`;
- en contextos aritméticos `(( ... ))`;
- determinadas asignaciones.

Pero para comandos normales y rutas, `"$variable"` es la opción inicial segura.

13. Ejemplo con espacios

```
archivo='mi documento.txt'
printf '%s\n' "$archivo"
```

se trata como un solo argumento.

Sin comillas:

```
printf '%s\n' $archivo
```

Bash puede dividirlo en:

```
mi
documento.txt
```

Eso es un error frecuente.

14. Globbing accidental

Supón:

```
patron='*.txt'
```

Si usas:

```
printf '%s\n' $patron
```

Bash puede expandir `*.txt` a nombres reales del directorio.

Si quieres imprimir literalmente el valor:

```
printf '%s\n' "$patron"
```

15. Unquoted expansion no “ejecuta texto como código” por sí sola

Este punto es importante.

Si una variable contiene:

```
; algo
```

una expansión normal no vuelve a analizar automáticamente ese texto como nueva sintaxis de shell.

Los riesgos típicos de expansión sin comillas son principalmente:

- word splitting;
- pathname expansion;
- opciones inesperadas.

Construcciones como `eval` sí pueden introducir una nueva interpretación y se posponen.

16. Variable vacía

```
dato=' '
```

Con:

```
printf '<%s>\n' "$dato"
```

obienes:

```
<>
```


La variable existe y su valor es vacío.

No confundas:

- variable vacía;
- variable no definida.

Esa diferencia será importante al estudiar validaciones.

17. Variables de entorno

Bash hereda variables del entorno.

Ejemplos comunes:

```
printf '%s\n' "$HOME"
printf '%s\n' "$PATH"
```

No compartas PATH, HOME u otras variables completas si revelan rutas privadas innecesarias.

18. export

Una variable normal de shell no se exporta automáticamente a procesos hijos.

Ejemplo conceptual:

```
nombre='Ana '
```

Para marcar una variable para exportación:

```
export nombre
```

También puede escribirse:

```
export nombre='Ana '
```

No exportes secretos como práctica.

19. Variables no son tipos estrictos

Bash trata gran parte de los valores como cadenas de caracteres.

Ejemplo:

```
numero='12 '
```

No es equivalente al sistema de tipos de Python o Java.

Más adelante veremos contextos aritméticos.

20. read — recibir entrada

Un script puede leer entrada del usuario.

Ejemplo:

```
read -r nombre
printf 'Hola, %s\n' "$nombre"
```

read lee una línea, pero normalmente la divide según IFS. Con un solo nombre de variable puede eliminar espacios iniciales y finales.

-r evita que las barras invertidas se interpreten como escapes por read, **pero no desactiva la separación por IFS**. Para conservar los espacios de una línea completa, utiliza:

```
IFS= read -r linea
printf '%s\n' "$linea"
```

IFS= aplica el valor vacío solo a esa invocación de read.

Para aprendizaje general preferiremos:

```
read -r
```

21. Prompt con read -p

En Bash:

```
read -r -p 'Escribe tu nombre: ' nombre
printf 'Hola, %s\n' "$nombre"
```

-p muestra el mensaje **solo cuando la entrada procede de una terminal**; si se redirige la entrada desde un archivo o una tubería, no debe esperarse ese prompt.

Es una opción de Bash; no debes asumir que todos los shells POSIX tienen exactamente la misma opción.

22. No leer contraseñas como ejercicio

No pediremos contraseñas ni tokens.

Incluso cuando herramientas permiten entrada oculta, el manejo de secretos exige más cuidado.

Para practicar read, usa datos no sensibles:

- nombre ficticio;
- color;
- ciudad ficticia;
- número de laboratorio.

23. Argumentos posicionales

Si ejecutas:

```
./saludo.sh Ana
```

el script puede acceder al primer argumento mediante:

```
$1
```

Ejemplo:

```
#!/usr/bin/env bash
printf 'Hola, %s\n' "$1"
```

24. \$0

Dentro del script:

```
$0
```

representa cómo fue invocado el script/comando.

Ejemplo:

```
printf 'Programa: %s\n' "$0"
```

No asumas que siempre contiene una ruta absoluta.

25. \$1, \$2, ...

Los primeros parámetros posicionales:

```
$1
$2
$3
...
```

corresponden a argumentos.

Ejemplo:

```
./datos.sh Ana 15
```

Dentro:

```
$1 → Ana  
$2 → 15
```

Cítalos normalmente:

```
printf '%s\n' "$1"
```

26. \$

```
$#
```

indica cuántos argumentos posicionales recibió el script.

Ejemplo:

```
printf 'Recibí %s argumentos\n' "$#"
```

Si ejecutas:

```
./script.sh uno dos tres
```

el valor será:

```
3
```

27. "\$@"

```
"$@"
```

expande todos los argumentos posicionales preservándolos como argumentos separados.

Ejemplo:

```
./script.sh 'uno dos' tres
```

Conceptualmente:

```
"$@" →  
argumento 1: "uno dos"  
argumento 2: "tres"
```

Esto es fundamental para scripts robustos.

28. "\$*" no es igual a "\$@"

Dentro de comillas dobles:

```
"$*"
```

combina los argumentos en una sola cadena, separada normalmente por el primer carácter de IFS.

Mientras:

```
"$@"
```

mantiene cada argumento separado.

Por eso, cuando quieres reenviar argumentos preservando sus límites, normalmente necesitas:

```
"$@"
```

29. Ejemplo seguro para ver argumentos

Script:

```
#!/usr/bin/env bash

printf 'Cantidad: %s\n' "$#"

for argumento in "$@"; do
    printf '<%s>\n' "$argumento"
done
```

El bucle `for` se explicará formalmente más adelante.

Aquí solo sirve para visualizar que cada argumento permanece separado.

30. `$*` sin comillas

Evita aprender patrones como:

```
$*
```

sin comillas para reenviar argumentos.

Puede sufrir división en palabras y globbing.

La forma recomendada para preservar argumentos es:

```
"$@"
```

31. `$?` — estado del último comando

```
$?
```

contiene el estado de salida del último comando ejecutado.

Ejemplo:

```
printf 'Hola\n'
printf 'Estado: %s\n' "$?"
```

El valor `0` normalmente indica éxito.

Los estados de salida se estudiarán en profundidad en el Módulo 25.

32. No sobrescribas `$?` accidentalmente antes de consultarlo

`$?` cambia después de cada comando.

Ejemplo:

```
comando
estado=$?
```

es una forma de guardar el valor.

Si ejecutas otro comando antes, `$?` ya corresponderá a ese comando posterior.

33. Sustitución de comandos

```
$(comando)
```

ejecuta un comando y sustituye por su salida estándar, eliminando saltos finales según las reglas de Bash.

Ejemplo:

```
directorio=$(pwd)
printf 'Estoy en: %s\n' "$directorio"
```

No uses sustitución de comandos con operaciones que no comprendes.

34. Preferir `$(...)` frente a backticks

También existe la sintaxis histórica:

```
`comando`
```

Pero este manual prioriza:

```
$(comando)
```

porque es más legible y se anida mejor.

35. Expansión de parámetro con valor predeterminado

Una forma útil:

```
nombre=${1:-Invitado}
```

Significa conceptualmente:

- usa \$1 si tiene un valor adecuado;
- en caso contrario usa Invitado.

Ejemplo:

```
printf 'Hola, %s\n' "$nombre"
```

No estudiaremos todavía todas las formas de expansión de parámetros.

36. Diferencia entre unset y empty con :-

La forma:

```
${variable:-valor}
```

usa el valor alternativo cuando la variable está no definida o vacía.

Existen variantes con semánticas diferentes.

No las memorices todavía.

37. IFS — introducción

IFS significa **I**nternal **F**ield **S**eparator.

Influye en determinadas operaciones de separación de palabras y en "\$*".

No modificaremos IFS en este módulo.

Solo debes saber que existe para entender por qué "\$*" y "\$@" no son equivalentes.

38. Datos no son código

Una regla central:

| trata la entrada del usuario como datos.

No construyas nuevas órdenes concatenando texto recibido para volver a interpretarlo.

Por eso no utilizaremos eval para procesar entrada.

eval se pospone porque puede reinterpretar texto como código shell.

39. Opciones que empiezan con -

Otro riesgo no relacionado con “ejecutar código”:

si un valor empieza por:

```
-
```

algunos comandos pueden tratarlo como una opción.

Muchas herramientas permiten usar:

```
--
```

para marcar el fin de opciones.

Ejemplo conceptual:

```
comando -- "$archivo"
```

No todas las herramientas usan --, así que debes consultar documentación.

40. Preparar el laboratorio

```
cd ~/linux-lab
pwd
```

Si **cd falla, detente**. Comprueba que la ruta mostrada por **pwd** sea tu laboratorio antes de crear nada.

Primero comprueba si el nombre está ocupado:

```
ls -ld ./modulo-24-bash-datos
```

Si el resultado indica que no existe, crea la carpeta desde ~/linux-lab y entra en ella:

```
mkdir ./modulo-24-bash-datos
cd ./modulo-24-bash-datos
pwd
```

Si la carpeta ya existe, **no la sobrescribas ni la recrees**; inspecciona su contenido antes de continuar. Detente ante cualquier error.

41. Práctica A — variable simple

Crea:

```
variables.sh
```

Contenido:

```
#!/usr/bin/env bash

nombre='Ana'
printf 'Hola, %s\n' "$nombre"
```

Valida:

```
bash -n variables.sh
```

Ejecuta:

```
bash variables.sh
```

42. Explicación línea por línea

```
#!/usr/bin/env bash
```

selecciona Bash al ejecutar directamente.

```
nombre='Ana'
```

asigna texto.

```
printf 'Hola, %s\n' "$nombre"
```

expande la variable dentro de comillas dobles y la pasa como argumento a **printf**.

43. Práctica B — espacios

Modifica:

```
nombre='Ana María'
```

Mantén:

```
printf '<%s>\n' "$nombre"
```

Debe aparecer como un solo valor.

Después compara conceptualmente con la expansión sin comillas.

No necesitas adoptar la forma insegura en scripts reales.

44. Práctica C — read

Crea leer_nombre.sh:

```
#!/usr/bin/env bash

read -r -p 'Escribe un nombre ficticio: ' nombre
printf 'Recibí: %s\n' "$nombre"
```

Usa únicamente datos no sensibles.

45. Práctica D — primer argumento

Crea argumento.sh:

```
#!/usr/bin/env bash

printf 'Primer argumento: %s\n' "$1"
```

Ejecuta:

```
bash argumento.sh prueba
```

Después:

```
bash argumento.sh 'dos palabras'
```

Observa que las comillas de la shell hacen que dos palabras sea un solo argumento.

46. Práctica E — contar argumentos

Crea contar.sh:

```
#!/usr/bin/env bash

printf 'Cantidad: %s\n' "$#"
```

Ejecuta con:

- cero argumentos;
- uno;
- tres.

Predice antes el resultado.

47. Práctica F — "\$@"

Crea todos.sh:

```
#!/usr/bin/env bash

printf 'Cantidad: %s\n' "$#"

for argumento in "$@"; do
    printf '<%s>\n' "$argumento"
done
```

Ejecuta:

```
bash todos.sh uno 'dos palabras' tres
```

Debe conservar tres argumentos.

No necesitas dominar aún la sintaxis del bucle; el Módulo 27 la enseñará formalmente.

48. Práctica G — sustitución de comandos

Crea directorio.sh:

```
#!/usr/bin/env bash

directorio=$(pwd)
printf 'Directorio: %s\n' "$directorio"
```

Ejecuta dentro del laboratorio.

No guardes rutas privadas en GitHub si contienen información personal innecesaria.

49. Práctica H — valor predeterminado

Crea predeterminado.sh:

```
#!/usr/bin/env bash

nombre=${1:-Invitado}
printf 'Hola, %s\n' "$nombre"
```

Ejecuta:

```
bash predeterminado.sh
```

y después:

```
bash predeterminado.sh Ana
```

Compara.

50. Errores frecuentes

Error	Problema	Corrección
nombre = valor	Bash intenta ejecutar palabras como comando	Usa nombre=valor
\$variable sin comillas en una ruta/texto	Puede dividirse o expandir globbing	Usa "\$variable"
Comillas simples y esperas expansión	Preservan texto literal	Usa dobles si necesitas expansión
Confundes "\$*" con "\$@"	Uno agrupa, otro preserva argumentos	Usa "\$@" al reenviar
Usas argumentos para secretos	Pueden quedar expuestos	No pases secretos por prácticas
Consultas \$? después de otro comando	Ya cambió	Guárdalo inmediatamente
Usas eval con entrada	Reinterpreta texto como código	No usar en este nivel
Variable empieza con número	Nombre inválido	Empieza con letra o guion bajo
Modificas IFS sin comprender	Cambia separación	Posponer

51. Método seguro con datos de entrada

1. identifica qué dato recibes;
2. no lo trates como código;
3. conserva sus límites con quoting;
4. valida cuando sea necesario;
5. no almacenes secretos en scripts;
6. no uses eval;
7. prueba valores con espacios;
8. prueba valores vacíos;
9. revisa resultados antes de automatizar.

52. Práctica independiente

Crea un script que:

1. reciba un nombre ficticio como \$1;
2. use \${1:-Invitado};
3. muestre cuántos argumentos recibió;
4. muestre el directorio actual mediante \$(pwd);
5. use comillas dobles correctamente;
6. valide con bash -n.

Después explica cada línea sin copiar la explicación del manual.

53. Mini evaluación

1. ¿una asignación Bash lleva espacios alrededor de =?
 - A. Sí.
 - B. No.
2. ¿qué hace "\$variable"?
 - A. Expande preservando el valor como una palabra en un contexto normal de comando.
 - B. Convierte el valor en código.
3. ¿las comillas simples expanden \$variable?
 - A. Sí.
 - B. No.
4. ¿\$1 representa el primer argumento?
 - A. Sí.
 - B. No.
5. ¿\$# representa la cantidad de argumentos?
 - A. Sí.
 - B. No.
6. ¿"\$@" preserva argumentos separados?
 - A. Sí.
 - B. No.
7. ¿"\$*" es idéntico a "\$@"?
 - A. Sí.
 - B. No.
8. ¿\$(pwd) es sustitución de comandos?
 - A. Sí.
 - B. No.
9. ¿debes usar eval para procesar entrada de usuario en este nivel?
 - A. Sí.
 - B. No.
10. ¿es buena práctica pasar contraseñas como argumentos de línea de comandos?
 - A. Sí.
 - B. No.

54. Registro de aprendizaje

```
Una variable es:  
Asignar significa:  
Expandir significa:  
{nombre} sirve para:  
Comillas dobles:  
Comillas simples:  
$1 representa:  
$# representa:  
"$@" representa:  
"$*" se diferencia porque:  
$(...) sirve para:  
read -r sirve para:  
¿Por qué trato entrada como datos y no como código?:  
Algo que todavía confundo:  
Estado: EN APRENDIZAJE / PRACTICADO
```

55. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Bash Reference Manual — Shell Parameters: https://www.gnu.org/software/bash/manual/html_node/Shell-Parameters.html
- GNU Bash — Shell Parameter Expansion: https://www.gnu.org/software/bash/manual/html_node/Shell-Parameter-Expansion.html
- GNU Bash — Quoting: https://www.gnu.org/software/bash/manual/html_node/Quoting.html
- GNU Bash — Command Substitution: https://www.gnu.org/software/bash/manual/html_node/Command-Substitution.html
- GNU Bash — Bourne Shell Builtins / read: https://www.gnu.org/software/bash/manual/html_node/Bourne-Shell-Builtins.html

Se posponen:

- aritmética;
- arrays;
- [[...]];
- condicionales;

- case;
- bucles formales;
- funciones;
- shift;
- getopt/getopts;
- expansiones avanzadas;
- namerefs;
- indirect expansion;
- eval;
- set -u.

Estado de la lección: redactada y revisada documentalmente. Las prácticas usan datos no sensibles y scripts locales.

Módulo 25 — Códigos de salida y composición de órdenes en Bash

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Vigésimoquinta entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué es un código o estado de salida;
- interpretar 0 como éxito y un valor distinto de 0 como condición no exitosa para la lógica de Bash, sin asumir que siempre indica un error operativo;
- consultar inmediatamente el último estado con `$?`;
- guardar un estado antes de ejecutar otra orden;
- diferenciar `;`, `&&` y `||`;
- predecir qué órdenes se ejecutarán en listas AND y OR;
- usar `true` y `false` para practicar estados de forma segura;
- terminar un script de forma explícita con `exit`;
- reconocer los estados especiales 126 y 127 usados por Bash en determinadas situaciones;
- entender por qué `A && B || C` no debe memorizarse como sustituto general de `if/else`;
- reconocer cómo se determina por defecto el estado de una tubería.

Conocimientos previos: scripts Bash, `printf`, variables, `$?`, redirecciones, pipes y navegación básica.

Seguridad: todas las prácticas se realizan en `~/linux-lab`, sin `sudo` y sin operaciones destructivas. `&&` y `||` automatizan decisiones, por lo que no los combinaremos aquí con borrados, cambios de permisos, servicios, discos o cortafuegos.

2. Qué es un código de salida

Cuando una orden termina comunica a la shell un número llamado **estado de salida** o **código de salida**.

```
0      → éxito
1-255  → estado no exitoso para las decisiones de Bash; significado específico según el programa
```

Para empezar no necesitas memorizar todos los números. La pregunta principal es: **¿la orden terminó con éxito o no?**

3. Por qué 0 significa éxito

Bash dispone de una forma general de representar éxito: 0. Los valores distintos de cero hacen que Bash tome la rama «no exitosa», pero pueden representar errores, ausencia de coincidencias, resultados especiales u otras condiciones documentadas por cada herramienta.

No interpretes automáticamente 1, 2, 3, etc. con un significado universal. Muchos programas documentan sus propios códigos.

4. Salida visible y estado de salida son cosas distintas

Ejemplo:

```
pwd
```

Si `pwd` funciona, normalmente termina con estado 0. La ruta que ves en pantalla y el código de salida son conceptos diferentes.

Un comando puede mostrar texto y terminar bien, no mostrar nada y terminar bien, o mostrar un mensaje de error y terminar con un estado distinto de cero.

5. `$?` contiene el estado de la última orden

```
pwd
printf 'Estado: %s\n' "$?"
```

Si `pwd` tuvo éxito, el valor esperado es 0.

6. `$?` cambia después de cada orden

Observa:

```
pwd
printf 'Hola\n'
printf 'Estado: %s\n' "$?"
```

El último `printf` ya no consulta el estado de `pwd`; consulta el estado del `printf` anterior.

Regla: **si necesitas conservar un estado, guárdalo inmediatamente.**

7. Guardar el estado

```
pwd
estado=$?
printf 'Estado de pwd: %s\n' "$estado"
```

estado=\$? copia el resultado antes de que otro comando lo sustituya.

8. true y false para practicar

true termina correctamente y normalmente devuelve 0.

false termina de forma no exitosa y normalmente devuelve 1.

```
true
estado=$?
printf 'true devolvió: %s\n' "$estado"
```

```
false
estado=$?
printf 'false devolvió: %s\n' "$estado"
```

El 1 de false sirve para practicar; no significa que todos los errores de Linux devuelvan 1.

9. El operador ;

; separa órdenes secuenciales.

```
printf 'uno\n'; printf 'dos\n'
```

La segunda orden se ejecuta después de la primera independientemente de que la primera haya tenido éxito.

Ejemplo:

```
false; printf 'Esta línea sí se ejecuta\n'
```

10. El operador &&

Una lista AND tiene esta forma:

```
orden1 && orden2
```

orden2 se ejecuta **solo si orden1 devuelve estado 0.**

```
true && printf 'La primera orden tuvo éxito\n'
```

En cambio:

```
false && printf 'Esta línea no debe aparecer\n'
```

la segunda orden no se ejecuta.

11. Uso práctico de &&

```
mkdir -p ~/linux-lab/modulo-25-status && cd ~/linux-lab/modulo-25-status
```

Primero se intenta crear la carpeta. Solo si esa operación termina con éxito se ejecuta cd.

Esto evita continuar ciegamente con una acción que depende de una preparación anterior.

12. El operador ||

Una lista OR tiene esta forma:

```
orden1 || orden2
```

orden2 se ejecuta **solo si orden1 devuelve un estado distinto de 0.**

```
false || printf 'La primera orden falló\n'
```

Con:

```
true || printf 'Esta línea no debe aparecer\n'
```

la segunda orden no se ejecuta.

13. || para reaccionar ante un fallo

Ejemplo local:

```
cd ~/linux-lab || printf 'No pude entrar al laboratorio\n'
```

Si `cd` falla, se muestra el aviso. Sin embargo, el script puede continuar después de esa lista.

`||` por sí solo no significa “termina el script”.

14. Combinar || con exit

Si el script no puede continuar cuando falla una preparación:

```
cd ~/linux-lab || exit 1
```

Interpretación:

1. intenta entrar a `~/linux-lab`;
2. si `cd` tiene éxito, `exit 1` no se ejecuta;
3. si `cd` falla, se ejecuta `exit 1`;
4. el script termina con estado 1.

15. exit

`exit` termina la shell o el script actual.

```
exit 0
```

indica terminación exitosa.

```
exit 1
```

termina con un estado de fallo.

Si escribes `exit` sin número, Bash utiliza el estado de la última orden ejecutada.

16. No inventes significados universales

Un programa puede documentar códigos diferentes a otro. Para códigos específicos, consulta la documentación del programa que los genera.

Dos niveles de interpretación:

1. **Bash:** `0` permite continuar por `&&`; un estado no cero permite continuar por `||`.
2. **Programa:** el significado real del número depende de su contrato documentado.

Por ejemplo, `dnf check-update` documenta `0` si no hay actualizaciones, `100` si las hay y `1` cuando ocurre un error. Así, `100` activa la rama de `||` aunque no signifique un error operativo. Esta distinción se desarrolla también en los Módulos 15 y 35.

Ejercicio de interpretación: si `dnf check-update` devuelve `100`, responde por separado: ¿qué rama elegiría Bash en orden `&& A || B`? ¿Qué significa `100` para DNF? No ejecute DNF para contestar.

17. Estados especiales 126 y 127

En Bash:

- `127` se usa cuando no se encuentra una orden;
- `126` se usa cuando se encuentra la orden pero no puede ejecutarse de la forma solicitada.

No memorices `126` como explicación universal de todos los problemas de permisos.

18. Señales y 128 + N

Si una orden termina por una señal fatal número `N`, Bash representa ese caso mediante `128 + N`.

En este módulo solo conectamos el concepto con lo ya estudiado sobre señales; no realizaremos prácticas para provocar terminaciones de procesos.

19. Estado final de listas AND y OR

El estado de una lista AND u OR es el estado de la última orden que realmente se ejecutó.

Ejemplo:

```
false || printf 'recuperado\n'
```

Si printf termina correctamente, la lista completa puede terminar con estado 0, aunque la primera orden haya fallado.

20. Cortocircuito

&& y || aplican evaluación con cortocircuito.

Con &&, si la izquierda falla, la derecha no se ejecuta.

Con ||, si la izquierda tiene éxito, la derecha no se ejecuta.

Esta idea será la base del siguiente módulo sobre if.

21. No memorices A && B || C como if/else

Este patrón:

```
A && B || C
```

parece significar “si A funciona ejecuta B; si no, C”, pero no siempre es equivalente.

Si A tiene éxito pero B falla, entonces C puede ejecutarse.

Por eso no lo usaremos como sustituto general de una estructura if/else.

22. Asociatividad

Las listas && y || se evalúan con asociatividad hacia la izquierda. Las cadenas mixtas pueden resultar menos intuitivas de lo que parecen.

Regla inicial: usa cadenas cortas y claras.

23. Diferencia esencial

Operador	¿Cuándo se ejecuta la orden de la derecha?
;	después de la izquierda, independientemente del resultado
&&	solo si la izquierda devuelve 0
	solo si la izquierda devuelve un valor distinto de 0

24. Varias órdenes con &&

```
mkdir -p ~/linux-lab/modulo-25-status \  
&& cd ~/linux-lab/modulo-25-status \  
&& pwd
```

Cada orden depende del éxito de la anterior. Si alguna falla, las posteriores de esa cadena AND ya no se ejecutan.

25. Tuberías y estado de salida

Ya conoces:

```
orden1 | orden2
```

En Bash, por defecto, el estado de una tubería es el estado de la **última orden** de la tubería.

Eso significa que un fallo anterior puede no quedar reflejado en el estado final.

26. pipefail

Bash dispone de la opción pipefail, que cambia cómo se calcula el estado de una tubería.

No la activaremos todavía como política del manual. Se estudiará junto con manejo de errores en el Módulo 29.

Por ahora recuerda: **sin pipefail, normalmente manda el estado de la última orden de la tubería.**

27. La negación !

Bash puede negar lógicamente el estado de una tubería:

```
! true
```

No la necesitamos todavía para nuestras prácticas principales; se incluye para reconocerla cuando aparezca en ejemplos posteriores.

28. Preparar el laboratorio

```
test -d ~/linux-lab && \  
cd ~/linux-lab && \  
mkdir -p modulo-25-status && \  
cd modulo-25-status && pwd
```

Etiqueta de práctica: **creación en laboratorio**. No uses sudo.

29. Práctica A — observar éxito

```
true  
estado=$?  
printf 'true devolvió: %s\n' "$estado"
```

Predice el resultado antes de ejecutar.

30. Práctica B — observar fallo

```
false  
estado=$?  
printf 'false devolvió: %s\n' "$estado"
```

Resultado esperado: 1.

31. Práctica C — comprobar que \$? cambia

```
false  
printf 'Este printf se ejecutó\n'  
estado=$?  
printf 'Estado guardado: %s\n' "$estado"
```

Pregunta: ¿ese estado corresponde a false o al primer printf?

Respuesta: corresponde al printf, porque fue la orden más reciente.

32. Práctica D — conservar correctamente el estado

```
false  
estado=$?  
printf 'Estado de false: %s\n' "$estado"
```

33. Práctica E — &&

```
true && printf 'segunda orden ejecutada\n'
```

```
false && printf 'esta línea no debe aparecer\n'
```

Explica con tus palabras por qué cambia el resultado.

34. Práctica F — ||

```
false || printf 'se ejecutó la alternativa\n'
```

```
true || printf 'esta línea no debe aparecer\n'
```

35. Práctica G — comparar operadores

Predice antes de ejecutar:

```
false; printf 'A\n'
false && printf 'B\n'
false || printf 'C\n'
```

Debes poder explicar por qué aparecen unas letras y otras no.

36. Práctica H — script con salida explícita

Crea estado.sh:

```
#!/usr/bin/env bash

printf 'El script terminó correctamente\n'
exit 0
```

Valida:

```
bash -n estado.sh
```

Ejecuta:

```
bash estado.sh
printf 'Estado del script: %s\n' "$?"
```

37. Práctica I — dependencia obligatoria

Crea entrar_laboratorio.sh:

```
#!/usr/bin/env bash

cd ~/linux-lab || exit 1
printf 'Estoy dentro del laboratorio\n'
pwd
```

Si cd falla, el script termina. Si tiene éxito, continúa.

38. Errores frecuentes

Error	Problema	Corrección
Consultar \$? después de otra orden	El estado anterior ya se perdió	Guárdalo inmediatamente
Usar ; esperando dependencia	La derecha se ejecuta aunque falle la izquierda	Usa && si depende del éxito
Usar && esperando reacción a fallo	La derecha solo corre con 0	Usa para una alternativa por fallo
Usar como si terminara el script	Puede ejecutar una alternativa y continuar	Usa exit cuando corresponda
Memorizar A && B C como if/else	C puede ejecutarse si B falla	Usa if cuando lo estudies
Interpretar texto visible como estado	Son conceptos separados	Consulta el código de salida
Tratar todo estado no cero como error operativo	Algunos programas usan estados especiales esperados	Consulta la documentación del comando y separa semántica de Bash de semántica del programa
Encadenar cambios de alto impacto	Reduce oportunidades de revisar	Mantén pasos controlados
Asumir que una tubería refleja todos los fallos	Por defecto usa la última orden	Estudiaremos pipefail en el Módulo 29

39. Método seguro para componer órdenes

1. comprende cada orden por separado;
2. identifica cuál depende de cuál;
3. decide si la derecha debe ejecutarse siempre, solo con éxito o solo con fallo;
4. elige ;, && o ||;
5. evita cadenas largas;
6. verifica ruta y entorno;
7. no introduces operaciones destructivas para practicar;
8. conserva estados importantes inmediatamente;
9. prueba primero con true, false y printf.

40. Práctica independiente

Crea `comprobar_laboratorio.sh` que:

1. use Bash;
2. intente entrar a `~/linux-lab`;
3. termine con fallo si no puede entrar;
4. muestre el directorio actual si tuvo éxito;
5. cree o entre en `modulo-25-status` usando una dependencia con `&&`;
6. termine explícitamente con `exit 0`;
7. pase `bash -n`;
8. no use `sudo` ni borre archivos;
9. pueda explicarse línea por línea.

No se considerará dominado solo porque funcione: también debes explicar por qué elegiste `&&` o `||`.

41. Mini evaluación

1. ¿qué valor representa éxito para Bash? A) 0 B) 1
2. ¿todo valor distinto de 0 tiene exactamente el mismo significado? A) Sí B) No
3. ¿qué contiene `$_`? A) Estado de la última orden B) PID del script
4. ¿qué operador ejecuta la derecha solo si la izquierda tuvo éxito? A) `&&` B) `||`
5. ¿qué operador ejecuta la derecha solo si la izquierda falló? A) `&&` B) `||`
6. `;` impide ejecutar la segunda orden si falla la primera? A) Sí B) No
7. `false` devuelve normalmente un estado distinto de cero? A) Sí B) No
8. `exit 0` indica éxito? A) Sí B) No
9. ¿Bash usa 127 cuando no encuentra una orden? A) Sí B) No
10. ¿A `&&` B `||` C es siempre equivalente a `if/else`? A) Sí B) No
11. Por defecto, ¿qué orden determina el estado de una tubería? A) Primera B) Última
12. ¿debes consultar la documentación para interpretar códigos específicos? A) Sí B) No
13. Si `dnf check-update` devuelve 100, ¿significa necesariamente un error? A) Sí B) No
14. ¿un estado 100 hace que Bash considere exitosa la condición de `&&`? A) Sí B) No

42. Registro de aprendizaje

```
Un código de salida es:
0 significa:
Un valor no-cero significa para el control de flujo de Bash:
¿Por qué no siempre equivale a un error operativo?:
¿Qué significa 100 para dnf check-update?:
$_ contiene:
¿Por qué debo guardar $_ inmediatamente?:
; significa:
&& significa:
|| significa:
exit sirve para:
127 en Bash puede indicar:
126 en Bash puede indicar:
¿Por qué A && B || C no es un if/else general?:
Estado por defecto de una tubería:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

43. Conexión con el siguiente módulo

El modelo aprendido es:

```
orden → estado 0/no-cero → decisión
```

El Módulo 26 añadirá `if`, `test`, `[ ]`, `[[ ]]`, `case` y aritmética.

44. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- DNF Project — `check-update`: https://dnf.readthedocs.io/en/latest/command_ref.html#check-update-command
- GNU Bash Reference Manual — Exit Status: https://www.gnu.org/software/bash/manual/html_node/Exit-Status.html
- GNU Bash Reference Manual — Lists of Commands: https://www.gnu.org/software/bash/manual/html_node/Lists.html
- GNU Bash Reference Manual — Pipelines: https://www.gnu.org/software/bash/manual/html_node/Pipelines.html
- GNU Bash Reference Manual — Bourne Shell Builtins: https://www.gnu.org/software/bash/manual/html_node/Bourne-Shell-Builtins.html

Puntos verificados documentalmente: 0 representa éxito; \$? contiene el estado de la última orden; && depende de éxito; || depende de fallo; 126 y 127 tienen usos especiales; una señal fatal N se representa como 128 + N; y por defecto una tubería toma el estado de su última orden.

Se posponen: if, test, [], [[]], case, comparaciones, set -e, set -u, pipefail como política, trap, funciones y return.

Estado de la lección: redactada y revisada documentalmente contra GNU Bash 5.3 (edición del manual del 18 de mayo de 2025; vigente al consultarla el 9 de octubre de 2026). Las prácticas son locales, no destructivas y no requieren privilegios.

Módulo 26 — Decisiones con `if`, `test`, `[ ]`, `[[ ]]`, `case` y aritmética

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Vigésimosexta entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar cómo Bash toma decisiones usando estados de salida;
- escribir un `if` mínimo y explicar cada palabra;
- añadir `else` y `elif`;
- entender que `if` no exige obligatoriamente `[ ... ]`;
- usar `test` y `[ ... ]` con cadenas, números y archivos;
- reconocer que `[` es una orden y que `]` debe ser un argumento separado;
- usar `[[ ... ]]` cuando el script depende de Bash;
- distinguir portabilidad POSIX de características específicas de Bash;
- comparar cadenas sin confundirlas con números;
- comparar enteros con `-eq`, `-ne`, `-lt`, `-le`, `-gt` y `-ge`;
- comprobar archivos con pruebas como `-e`, `-f` y `-d`;
- usar `case` para elegir entre varios patrones;
- diferenciar `(( ... ))` de `$( ... )`;
- evitar errores frecuentes de espacios, quoting y comparaciones.

Conocimientos previos:

- scripts Bash;
- variables y expansión;
- quoting;
- parámetros posicionales;
- códigos de salida;
- `&&`, `||` y `exit`.

Seguridad: todas las prácticas se realizan dentro de `~/linux-lab`. No se usa `sudo`, no se modifican servicios, permisos del sistema, discos, red ni cortafuegos. Las pruebas de archivo son de consulta o sobre archivos creados por el estudiante.

2. La idea central: una condición en Bash es una orden

En Bash, `if` no pregunta necesariamente por un valor booleano como en otros lenguajes.

Su modelo básico es:

```
ejecutar una orden
    ↓
leer su estado de salida
    ↓
0      → condición verdadera
no-cero → condición falsa
```

Esto conecta directamente con el Módulo 25.

3. Primer `if`

```
if true; then
    printf 'La condición tuvo éxito\n'
fi
```

Línea por línea:

- `if` inicia la decisión;
- `true` es la orden cuya salida se evalúa;
- `;` termina esa lista en la misma línea;
- `then` inicia el bloque que se ejecuta si el estado fue `0`;
- `printf` es la acción;
- `fi` cierra el `if`.

`fi` es `if` escrito al revés; es la palabra de cierre de esta estructura.

4. Forma en varias líneas

También puedes escribir:

```
if true
then
    printf 'Éxito\n'
fi
```

Ambas formas son válidas. En este manual usaremos normalmente:

```
if condición; then
    acción
fi
```

5. if evalúa el estado, no el texto mostrado

```
if false; then
    printf 'No debe aparecer\n'
fi
```

false no necesita imprimir la palabra false. Su estado no-cero basta para que Bash omita el bloque.

6. if puede evaluar una orden normal

No necesitas escribir siempre [...].

Ejemplo:

```
if cd ~/linux-lab; then
    printf 'Pude entrar al laboratorio\n'
fi
```

if evalúa directamente el estado de cd.

Esto es una idea esencial: **if decide según el resultado de órdenes.**

7. Añadir else

```
if true; then
    printf 'Éxito\n'
else
    printf 'Fallo\n'
fi
```

else se ejecuta cuando la condición anterior produce un estado distinto de cero.

8. Añadir elif

elif significa conceptualmente “si no, prueba esta otra condición”.

```
numero=2

if (( numero == 1 )); then
    printf 'uno\n'
elif (( numero == 2 )); then
    printf 'dos\n'
else
    printf 'otro valor\n'
fi
```

Las comparaciones aritméticas se explicarán más adelante en este mismo módulo.

9. Qué es test

test es un builtin de Bash que evalúa una expresión condicional y devuelve:

```
0 → verdadera
1 → falsa
```

Ejemplo:

```
test 'Ana' = 'Ana'
printf 'Estado: %s\n' "$?"
```

10. Qué es [...]

En Bash, [es también un builtin.

Esta forma:

```
[ 'Ana' = 'Ana' ]
```

es una forma alternativa de utilizar una prueba condicional.

El] final es obligatorio cuando usas la forma [. No es decoración visual.

11. Los espacios en [...] son parte de la sintaxis práctica

Correcto:

```
[ "$nombre" = 'Ana' ]
```

Incorrecto:

```
[ "$nombre" = 'Ana' ]
```

Como [recibe argumentos, necesitas separar cada operador y operando.

Regla inicial:

```
[ ESPACIO expresión ESPACIO ]
```

12. Comparar cadenas con test o []

```
nombre='Ana'

if [ "$nombre" = 'Ana' ]; then
    printf 'Coincide\n'
fi
```

Para igualdad de cadenas con [...], usa aquí =.

Para desigualdad:

```
[ "$nombre" != 'Ana' ]
```

13. Por qué citamos variables en [...]

Con la forma clásica [...], una expansión sin comillas puede cambiar el número de argumentos.

Por eso preferimos:

```
[ "$nombre" = 'Ana' ]
```

y no:

```
[ $nombre = 'Ana' ]
```

Especialmente mientras estás aprendiendo, cita las expansiones de texto dentro de [...].

14. Cadena vacía y no vacía

Prueba de cadena no vacía:

```
[ -n "$texto" ]
```

Prueba de cadena vacía:

```
[ -z "$texto" ]
```

Ejemplo:

```

texto=''

if [ -z "$texto" ]; then
    printf 'La cadena está vacía\n'
fi

```

15. Comparaciones numéricas con [...]

Para enteros:

Operador	Significado
-eq	igual
-ne	distinto
-lt	menor que
-le	menor o igual
-gt	mayor que
-ge	mayor o igual

Ejemplo:

```

numero=8

if [ "$numero" -gt 5 ]; then
    printf 'El número es mayor que 5\n'
fi

```

16. No uses comparación de texto como si fuera comparación numérica

Esto:

```
[ "$a" = "$b" ]
```

compara cadenas.

Esto:

```
[ "$a" -eq "$b" ]
```

compara enteros.

Son operaciones distintas.

17. Pruebas básicas de archivos

Algunas pruebas útiles:

Prueba	Pregunta
-e ruta	¿existe?
-f ruta	¿es archivo regular?
-d ruta	¿es directorio?
-r ruta	¿es legible para el proceso actual?
-w ruta	¿es escribible para el proceso actual?
-x ruta	¿es ejecutable/buscable según el tipo y permisos?

Ejemplo dentro del laboratorio:

```

archivo=~/.linux-lab/modulo-26-condiciones/nota.txt

if [ -f "$archivo" ]; then
    printf 'Existe como archivo regular\n'
fi

```

18. -e y -f no preguntan exactamente lo mismo

-e pregunta si la ruta existe.

-f pregunta si existe y es un archivo regular.

Un directorio puede cumplir -e y no cumplir -f.

19. Introducción a `[[ ... ]]`

Bash ofrece el comando compuesto:

```
[[ expresión ]]
```

Ejemplo:

```
nombre='Ana María'

if [[ $nombre == 'Ana María' ]]; then
    printf 'Coincide\n'
fi
```

Dentro de `[[ ... ]]`, las palabras no sufren *word splitting* ni expansión de nombres de archivo como en una expansión normal de comando.

20. `[[ ... ]]` no es POSIX `sh`

`[[ ... ]]` es una característica de Bash y otros shells compatibles, pero no pertenece a la sintaxis POSIX básica de `sh`.

Si el script declara:

```
#!/usr/bin/env bash
```

puedes usar características de Bash conscientemente.

Si necesitas portabilidad estricta a `/bin/sh`, debes diseñar con otras construcciones.

21. `[ ... ]` frente a `[[ ... ]]`

Característica	<code>[...]</code>	<code>[[...]]</code>
Forma	builtin test	comando compuesto de Bash
Requiere <code>]</code> separado	sí	usa <code>]]</code> como sintaxis
Word splitting de expansiones	requiere cuidado/quoting	no dentro de la expresión
Globbering accidental de expansiones	requiere cuidado/quoting	no
Portabilidad POSIX	mucho mayor	no POSIX
Patrones con <code>==</code>	no es la razón principal para usarlo	sí, RHS sin citar puede ser patrón

Recomendación del manual:

- si escribes específicamente para Bash, `[[ ... ]]` suele ofrecer semántica más segura y clara para cadenas;
- si el objetivo es portabilidad POSIX, aprende `test/[ ... ]` correctamente.

22. Igualdad literal dentro de `[[ ... ]]`

```
nombre='Ana'

if [[ $nombre == 'Ana' ]]; then
    printf 'Coincide literalmente\n'
fi
```

Cuando el lado derecho se cita, se trata literalmente.

23. Patrones dentro de `[[ ... ]]`

Cuando el lado derecho de `==` no está citado, Bash puede tratarlo como patrón.

```
archivo='reporte.txt'

if [[ $archivo == *.txt ]]; then
    printf 'Termina en .txt\n'
fi
```

Aquí `*.txt` es un patrón, no el nombre literal de un archivo.

No confundas este patrón con una expansión de nombres de archivo del directorio: dentro de `[[ ... ]]` tiene semántica de comparación de patrón.

24. Operadores lógicos dentro de `[[ ... ]]`

Puedes combinar condiciones:

```
numero=8

if [[ $numero -ge 1 && $numero -le 10 ]]; then
    printf 'Está entre 1 y 10\n'
fi
```

También existe `||` para OR y `!` para negación.

Estas operaciones están dentro de la expresión condicional de `[[ ... ]]`, no son exactamente la misma lista de comandos estudiada en el módulo anterior, aunque el significado lógico resulte relacionado.

25. Evitar `-a` y `-o` en expresiones complejas de `test`

`test` y `[` admiten formas históricas con `-a` y `-o`, pero su análisis depende del número de argumentos y puede ser confuso.

Para código Bash nuevo, este manual prefiere:

```
[[ condición1 && condición2 ]]
```

o varias pruebas combinadas a nivel de comandos cuando corresponda.

26. Qué es `case`

`case` selecciona una acción según el valor de una palabra y patrones.

Estructura:

```
case "$valor" in
    patron1)
        acciones
        ;;
    patron2)
        acciones
        ;;
    *)
        acciones_por_defecto
        ;;
esac
```

`esac` es `case` escrito al revés.

27. Primer `case`

```
opcion='ver'

case "$opcion" in
    ver)
        printf 'Elegiste ver\n'
        ;;
    salir)
        printf 'Elegiste salir\n'
        ;;
    *)
        printf 'Opción desconocida\n'
        ;;
esac
```

28. Qué significa `*` en `case`

El patrón:

```
*
```

coincide con cualquier valor que no haya coincidido antes.

Funciona como caso predeterminado.

29. Varias alternativas en un patrón

Puedes usar `|` para varias alternativas:

```
respuesta='s'

case "$respuesta" in
  s|S)
    printf 'Respuesta afirmativa\n'
    ;;
  n|N)
    printf 'Respuesta negativa\n'
    ;;
  *)
    printf 'Respuesta no reconocida\n'
    ;;
esac
```

En este ejemplo solo clasificamos texto; no ejecutamos acciones de impacto.

30. `case` frente a muchos `elif`

Cuando comparas una misma variable contra varias opciones o patrones, `case` suele resultar más legible.

Cuando las condiciones son heterogéneas —por ejemplo, números, archivos y varias pruebas diferentes— `if` puede ser más natural.

No existe una regla de que uno sea siempre mejor que el otro.

31. Introducción a aritmética con `(( ... ))`

Bash dispone del comando compuesto aritmético:

```
(( expresión ))
```

Ejemplo:

```
numero=8

if (( numero > 5 )); then
  printf 'Mayor que 5\n'
fi
```

Dentro de este contexto se utilizan operadores aritméticos como `>`, `<`, `==`, `!=`, `+`, `-`, `*`, `/` y `%`.

32. Estado de `(( ... ))`

Este punto puede parecer invertido al principio.

`(( expresión ))` devuelve:

```
estado 0 → si el valor aritmético de la expresión es distinto de 0
estado 1 → si el valor aritmético de la expresión es 0
```

Ejemplo:

```
(( 5 > 2 ))
```

La expresión vale 1, por lo que el comando termina con éxito.

En cambio:

```
(( 2 > 5 ))
```

la expresión vale 0, por lo que el comando devuelve estado 1.

33. `(( ... ))` frente a `$( ... )`

No son lo mismo.

`(( ... ))` evalúa aritmética como comando y produce un estado:

```
if (( 4 < 9 )); then
    printf 'verdadero\n'
fi
```

`$( ( ... ) )` es **expansión aritmética** y produce un valor que puede usarse como texto/argumento:

```
resultado=$(( 4 + 9 ))
printf 'Resultado: %s\n' "$resultado"
```

Resultado:

Resultado: 13

34. Aritmética entera

La aritmética de Bash trabaja con enteros de ancho fijo.

Ejemplo:

```
resultado=$(( 7 / 2 ))
printf '%s\n' "$resultado"
```

El resultado aritmético entero es 3, no 3.5.

No confundas este comportamiento con Python, donde `/` produce división de punto flotante.

35. División entre cero

Una división aritmética entre cero es un error.

No la uses como ejercicio.

Valida el divisor antes de dividir cuando provenga de datos variables.

36. Comparación recomendada según el dato

Modelo inicial:

texto portátil	→ ["\$a" = "\$b"]
texto en script Bash	→ [[\$a == "\$b"]]
entero con test	→ ["\$a" -gt "\$b"]
entero en contexto Bash	→ ((a > b))
archivo/directorio	→ [-f "\$ruta"] o [[-f \$ruta]]
varias opciones/patrones	→ case

No es una tabla absoluta; es una guía pedagógica inicial.

37. Preparar el laboratorio

```
test -d ~/linux-lab && \
cd ~/linux-lab && \
mkdir -p modulo-26-condiciones && \
cd modulo-26-condiciones && pwd
```

Etiqueta de práctica: **creación en laboratorio**.

Si alguna orden falla, la cadena se detiene: no continúes hasta revisar el mensaje. `test -d` comprueba primero que el laboratorio exista.

38. Práctica A — primer if

Crea `condicion_true.sh`:

```
#!/usr/bin/env bash

if true; then
    printf 'La condición fue verdadera\n'
fi
```

Valida:

```
bash -n condicion_true.sh
```

Ejecuta:

```
bash condicion_true.sh
```

39. Práctica B — if y else

Crea condicion_else.sh:

```
#!/usr/bin/env bash

if false; then
    printf 'Éxito\n'
else
    printf 'La condición devolvió un estado no-cero\n'
fi
```

Antes de ejecutar, predice qué línea aparecerá.

40. Práctica C — comparación de texto

Crea comparar_texto.sh:

```
#!/usr/bin/env bash

nombre='Ana María'

if [[ $nombre == 'Ana María' ]]; then
    printf 'El nombre coincide\n'
else
    printf 'No coincide\n'
fi
```

Explica por qué esta práctica usa [[...]] y por qué el script declara Bash.

41. Práctica D — comparación numérica

Crea comparar_numero.sh:

```
#!/usr/bin/env bash

numero=8

if (( numero > 5 )); then
    printf 'El número es mayor que 5\n'
else
    printf 'El número no es mayor que 5\n'
fi
```

Cambia únicamente numero a 3 y predice el resultado.

42. Práctica E — probar archivo y directorio

Prepara datos locales:

```
printf 'nota de práctica\n' > nota.txt
mkdir -p carpeta_prueba
```

Después:

```
if [[ -f nota.txt ]]; then
    printf 'nota.txt es un archivo regular\n'
fi

if [[ -d carpeta_prueba ]]; then
    printf 'carpeta_prueba es un directorio\n'
fi
```

> sobrescribe nota.txt si ya existe. Por eso esta práctica se realiza únicamente sobre el archivo de laboratorio creado para este módulo.

43. Práctica F — case

Crea opciones.sh:

```
#!/usr/bin/env bash

opcion=${1:-ayuda}

case "$opcion" in
    ver)
        printf 'Modo: ver\n'
        ;;
    ayuda)
        printf 'Modo: ayuda\n'
        ;;
    *)
        printf 'Opción no reconocida: %s\n' "$opcion"
        ;;
esac
```

Prueba:

```
bash opciones.sh
bash opciones.sh ver
bash opciones.sh otra
```

44. Práctica G — elif

Crea signo.sh:

```
#!/usr/bin/env bash

numero=${1:-0}

if (( numero > 0 )); then
    printf 'positivo\n'
elif (( numero < 0 )); then
    printf 'negativo\n'
else
    printf 'cero\n'
fi
```

Para esta práctica usa argumentos enteros sencillos: 5, -2 y 0.

La validación de entradas no numéricas se estudiará con mayor profundidad posteriormente.

45. Práctica H — usar una orden directamente

Crea entrar_lab.sh:

```
#!/usr/bin/env bash

if cd ~/linux-lab; then
    printf 'Entrada correcta\n'
    pwd
else
    printf 'No se pudo entrar al laboratorio\n' >&2
    exit 1
fi
```

Esta práctica demuestra que if no necesita obligatoriamente test, [] o [[]].

46. Errores frecuentes

Error	Por qué ocurre	Corrección mínima
["\$a" = "\$b"]	faltan argumentos separados	["\$a" = "\$b"]
olvidar]	[necesita el cierre como argumento	añade]
usar = pensando en números	compara cadenas	usa -eq o ((...))
usar -gt para texto	operador numérico	usa comparación de cadenas
olvidar fi	el if queda sin cerrar	añade fi
olvidar ;; en un case básico	la rama no termina como se espera	añade ;;
olvidar esac	case queda sin cerrar	añade esac
ejecutar script con sh aunque usa [[]]	[[]] no es POSIX sh	ejecútalo con Bash
citar *.txt en RHS de [[==]] esperando patrón	citado se interpreta literalmente	deja el patrón sin citar si buscas patrón

Error	Por qué ocurre	Corrección mínima
dejar expansión sin comillas dentro de [...]	puede alterar argumentos	cita variables de texto
confundir ((...)) con \$(...)	uno es comando; otro expansión	elige según necesites estado o valor
encadenar una acción peligrosa dentro de una condición	automatiza un cambio sin revisión	practica con consultas/datos de laboratorio

47. Detección de error 1

Analiza:

```
nombre='Ana María'

if [ $nombre = 'Ana María' ]; then
    printf 'Coincide\n'
fi
```

Problema: \$nombre puede expandirse a más de un argumento.

Corrección mínima:

```
if [ "$nombre" = 'Ana María' ]; then
    printf 'Coincide\n'
fi
```

48. Detección de error 2

Analiza:

```
numero=10

if [ "$numero" > 5 ]; then
    printf 'Mayor\n'
fi
```

Problema: > aquí no es la forma numérica adecuada para test y, fuera de contextos protegidos, también tiene significado de redirección para la shell.

No ejecute el ejemplo incorrecto: Bash interpreta > 5 como una redirección y puede crear o vaciar un archivo llamado 5 en el directorio actual. Analízalo únicamente como ejercicio de detección de errores.

Corrección con [...]:

```
if [ "$numero" -gt 5 ]; then
    printf 'Mayor\n'
fi
```

Corrección Bash aritmética:

```
if (( numero > 5 )); then
    printf 'Mayor\n'
fi
```

49. Detección de error 3

Analiza:

```
if [[ $archivo == '*.txt' ]]; then
    printf 'TXT\n'
fi
```

Si querías un patrón para cualquier nombre terminado en .txt, las comillas lo convierten en texto literal.

Corrección:

```
if [[ $archivo == *.txt ]]; then
    printf 'TXT\n'
fi
```

50. Método para elegir una estructura

Antes de escribir la condición:

1. identifica qué dato estás evaluando;
2. decide si es texto, entero, archivo/directorio o estado de una orden;
3. decide si el script depende específicamente de Bash;
4. usa una sola condición pequeña al principio;
5. predice qué estado debería producir;
6. añade `else` solo si necesitas manejar el caso contrario;
7. usa `case` cuando una misma entrada tiene varias alternativas claras;
8. valida con `bash -n`;
9. prueba primero con datos del laboratorio;
10. explica la condición con tus propias palabras.

51. Práctica independiente

Crea `clasificar_entrada.sh`.

Debe:

1. usar Bash;
2. recibir un primer argumento;
3. usar `${1:-ayuda}` como valor predeterminado;
4. usar `case` para reconocer archivo, directorio y ayuda;
5. si recibe archivo, comprobar con `[[ -f ... ]]` un archivo de práctica dentro del directorio actual;
6. si recibe directorio, comprobar con `[[ -d ... ]]` una carpeta de práctica;
7. usar al menos un `if`;
8. no usar `sudo`;
9. no borrar ni modificar archivos ajenos al laboratorio;
10. pasar `bash -n`;
11. poder explicarse línea por línea.

Primero escribe tu versión. No copies directamente las prácticas anteriores.

52. Mini evaluación

1. ¿qué interpreta `if` como verdadero en Bash? A) estado 0 B) cualquier texto impreso
2. ¿`if` necesita obligatoriamente `[ ... ]`? A) Sí B) No
3. ¿`[` es una orden/builtin en Bash? A) Sí B) No
4. ¿el `]` final de `[ ... ]` puede omitirse? A) Sí B) No
5. ¿qué operador compara enteros por igualdad con `test`? A) `-eq` B) `=`
6. ¿qué comprueba `-d`? A) directorio B) cadena vacía
7. ¿qué comprueba `-f`? A) archivo regular B) número
8. ¿`[[ ... ]]` pertenece a la sintaxis POSIX básica de `sh`? A) Sí B) No
9. dentro de `[[ ... ]]`, ¿una expansión sufre word splitting normal? A) Sí B) No
10. en `[[ $archivo == *.txt ]]`, ¿`*.txt` actúa como patrón? A) Sí B) No
11. ¿`case` es útil para varias alternativas de una misma entrada? A) Sí B) No
12. ¿`(( 5 > 2 ))` termina normalmente con estado 0? A) Sí B) No
13. ¿`$( ( 5 + 2 ) )` devuelve el valor aritmético para expansión? A) Sí B) No
14. ¿Bash realiza división entera en su aritmética? A) Sí B) No
15. ¿`[[ "$a" = "$b" ]]` y `(( a == b ))` expresan exactamente el mismo tipo de comparación? A) Sí B) No

53. Registro de aprendizaje

```
`if` decide usando:
`then` significa:
`else` sirve para:
`elif` sirve para:
`fi` cierra:
`test` devuelve:
`[` es:
¿por qué necesito espacios en `[ ... ]`?:
`-eq` significa:
`-gt` significa:
`-f` comprueba:
`-d` comprueba:
`[[ ... ]] se diferencia de [ ... ] porque:
`case` sirve para:
`esac` cierra:
`(( ... ))` sirve para:
`$( ... )` sirve para:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

54. Puerta de dominio

Este módulo no se considera dominado por contestar una sola evaluación.

Para avanzar como **DOMINADO** deberás poder:

1. explicar con tus palabras por qué `if` trabaja con estados;
2. escribir un `if/else` sin copiar;
3. detectar un error de espacios en `[ ... ]`;
4. elegir correctamente entre comparación de texto y numérica;
5. escribir un `case` pequeño sin copiar;
6. explicar la diferencia entre `(( ... ))` y `$( ... )`;
7. resolver otra práctica en una sesión posterior.

55. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Bash Reference Manual — Conditional Constructs: https://www.gnu.org/software/bash/manual/html_node/Conditional-Constructs.html
- GNU Bash Reference Manual — Bash Conditional Expressions: https://www.gnu.org/software/bash/manual/html_node/Bash-Conditional-Expressions.html
- GNU Bash Reference Manual — Bourne Shell Builtins (`test` y `[]`): https://www.gnu.org/software/bash/manual/html_node/Bourne-Shell-Builtins.html
- GNU Bash Reference Manual — Shell Arithmetic: https://www.gnu.org/software/bash/manual/html_node/Shell-Arithmetic.html

Puntos verificados documentalmente:

- `test` y `[]` devuelven `0` para una expresión verdadera y `1` para una falsa;
- con `[]`, el último argumento debe ser `]`;
- `[[ ... ]]` es un comando compuesto de Bash;
- dentro de `[[ ... ]]` no se realizan word splitting ni filename expansion sobre sus palabras;
- con `[[ ... ]]`, el lado derecho sin citar de `==` puede actuar como patrón;
- `(( expresión ))` devuelve estado `0` cuando la expresión aritmética evalúa a un valor distinto de cero y estado `1` cuando evalúa a cero;
- la aritmética de Bash utiliza enteros de ancho fijo y detecta división entre cero.

Se posponen:

- expresiones regulares con `==` en profundidad;
- `BASH_REMATCH`;
- `select`;
- bucles formales;
- validación completa de entrada numérica;
- funciones y `return`;
- arrays;
- `set -e`, `set -u` y `pipefail` como políticas;
- `trap`.

Estado de la lección: redactada y revisada documentalmente contra el GNU Bash Reference Manual disponible en 2026. Las prácticas son locales, no destructivas y no requieren privilegios.

Módulo 27 — Bucles, lectura de líneas y nombres de archivo seguros

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Vigésimoséptima entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar para qué sirve un bucle;
- escribir bucles `for`, `while` y `until`;
- usar `break` y `continue` conscientemente;
- recorrer argumentos con `for argumento in "$@"`;
- recorrer nombres de archivo mediante globbing sin analizar la salida de `ls`;
- explicar por qué `for archivo in $(ls)` es un patrón frágil;
- leer un archivo línea por línea con `while IFS= read -r línea`;
- explicar qué hacen `IFS=` y `read -r`;
- evitar el patrón innecesario `cat archivo | while read ...`;
- reconocer el efecto de una tubería sobre el entorno de un bucle;
- usar un contador aritmético sencillo;
- reconocer cuándo un bucle puede convertirse accidentalmente en infinito;
- procesar datos de laboratorio con espacios sin romper sus límites;
- distinguir una práctica Bash de una solución estrictamente POSIX.

Conocimientos previos:

- variables y quoting;
- `"$@"`;
- códigos de salida;
- `if`, `[ [ ... ]` y aritmética `(( ... ))`;
- expansión de nombres de archivo;
- archivos, rutas y redirecciones.

Seguridad: todas las prácticas se limitan a `~/linux-lab`. Los bucles pueden repetir una acción muchas veces, así que aquí solo repetiremos operaciones de lectura, impresión y creación controlada de archivos de práctica. No se usan bucles para borrar, cambiar permisos, administrar servicios, discos, usuarios, red o cortafuegos.

2. Qué es un bucle

Un bucle permite repetir una acción.

Modelo:

```
datos o condición
    ↓
repetir una o más órdenes
    ↓
detenerse cuando corresponda
```

Los bucles principales de este módulo son:

- `for`: recorrer elementos;
- `while`: repetir mientras una condición tenga éxito;
- `until`: repetir mientras una condición no tenga éxito.

3. Primer `for`

```
for palabra in uno dos tres; do
    printf '%s\n' "$palabra"
done
```

Resultado:

```
uno
dos
tres
```

4. Explicación línea por línea

```
for palabra in uno dos tres; do
```


for inicia el bucle, palabra es la variable que recibe cada elemento. in introduce la lista. do inicia el cuerpo.

```
printf '%s\n' "$palabra"
```

usa el valor actual de la variable. Las comillas dobles preservan el elemento como un solo argumento.

```
done
```

cierra el bucle.

5. Una iteración

Cada repetición del cuerpo se llama **iteración**.

En:

```
for n in 1 2 3; do
    printf '%s\n' "$n"
done
```

hay tres iteraciones.

6. Elementos con espacios

```
for nombre in 'Ana María' 'Luis Pérez'; do
    printf '<%s>\n' "$nombre"
done
```

Cada texto citado es un elemento independiente, aunque contenga espacios.

7. Recorrer argumentos con "\$@"

Este patrón conecta con el Módulo 24:

```
for argumento in "$@"; do
    printf '<%s>\n' "$argumento"
done
```

"\$@" conserva cada argumento recibido por el script como un elemento separado.

Si ejecutas:

```
bash argumentos.sh uno 'dos palabras' tres
```

el bucle realiza tres iteraciones, no cuatro.

8. Forma abreviada de for con parámetros posicionales

En Bash, si omites in ..., for nombre; do recorre los parámetros posicionales como si usara "\$@".

```
for argumento; do
    printf '<%s>\n' "$argumento"
done
```

En este manual seguiremos mostrando in "\$@" durante el aprendizaje porque hace la intención más visible.

9. Globbing para recorrer archivos

Si el directorio contiene archivos .txt, Bash puede expandir:

```
./*.txt
```

Ejemplo:

```
for archivo in ./*.txt; do
    printf 'Archivo: %s\n' "$archivo"
done
```

Los nombres resultantes del glob se conservan como elementos separados, incluso si contienen espacios.

10. Por qué usamos `./*.txt`

El prefijo `./` tiene dos ventajas pedagógicas:

1. deja claro que buscamos en el directorio actual;
2. un nombre como `-opcion.txt` queda representado como `./-opcion.txt`, por lo que es menos probable que una herramienta lo confunda con una opción.

Aun así, cuando una orden admita `--`, conviene usarlo para marcar el fin de opciones.

11. No analices la salida de `ls`

Evita:

```
for archivo in $(ls)
```

Problemas:

- la sustitución de comandos produce texto, no una colección fiable de nombres de archivo;
- el resultado puede sufrir división en palabras;
- los espacios, tabulaciones y saltos de línea de nombres válidos pueden romper el recorrido;
- la presentación de `ls` fue diseñada principalmente para mostrar información, no como interfaz de datos para scripts.

Para archivos del directorio actual, prefiere globbing:

```
for archivo in ./*; do
    ...
done
```

12. Qué ocurre si un glob no encuentra coincidencias

Por defecto en Bash, si un patrón como:

```
./*.txt
```

no encuentra coincidencias y `nullglob` no está activado, el patrón puede permanecer sin expandir.

Por eso una práctica robusta debe comprobar que la ruta exista antes de procesarla.

Ejemplo:

```
for archivo in ./*.txt; do
    [[ -e $archivo ]] || continue
    printf '%s\n' "$archivo"
done
```

Precisión: enlaces simbólicos rotos

`[[ -e $archivo ]]` es falso para un enlace simbólico cuyo destino ya no existe. Si quieres **incluir los nombres de enlaces simbólicos rotos**, utiliza:

```
for archivo in ./*.txt; do
    [[ -e $archivo || -L $archivo ]] || continue
    printf '%s\n' "$archivo"
done
```

- `-e` comprueba que la ruta resuelva a un objeto existente.
- `-L` comprueba si la ruta es un enlace simbólico, aunque su destino no exista.
- `|| continue` evita procesar el patrón literal cuando no hubo coincidencias.

Si tu objetivo es trabajar **solo con archivos regulares accesibles**, usa `[[ -f $archivo ]]` en lugar de afirmar que `-e` incluye todos los nombres. El filtro adecuado depende de la tarea.

Alternativa avanzada: `nullglob`

En Bash, `shopt -s nullglob` hace que un patrón sin coincidencias se expanda a cero palabras, en lugar de quedar literal. Esta opción modifica el comportamiento de expansión en la shell actual; no la actives indiscriminadamente en scripts existentes. La dejamos como alternativa conceptual y conservamos el filtro explícito en las prácticas básicas.

13. `continue`

`continue` salta el resto de la iteración actual y pasa a la siguiente.

Ejemplo:

```
for n in 1 2 3; do
    if (( n == 2 )); then
        continue
    fi
    printf '%s\n' "$n"
done
```

Salida:

```
1
3
```

14. break

break termina el bucle actual.

```
for n in 1 2 3 4; do
    if (( n == 3 )); then
        break
    fi
    printf '%s\n' "$n"
done
```

Salida:

```
1
2
```

15. Primer while

while repite su cuerpo mientras la condición produzca estado 0.

```
contador=1

while (( contador <= 3 )); do
    printf '%s\n' "$contador"
    (( contador += 1 ))
done
```

16. Explicación del contador

```
contador=1
```

establece el valor inicial.

```
while (( contador <= 3 )); do
```

comprueba la condición antes de cada iteración.

```
(( contador += 1 ))
```

suma uno.

Sin una actualización que permita que la condición cambie, el bucle podría no terminar.

17. Riesgo de bucle infinito

Ejemplo conceptual que **no necesitas ejecutar**:

```
while true; do
    acción
done
```

Puede repetirse indefinidamente.

Antes de ejecutar un while, identifica:

- condición inicial;
- qué cambia en cada iteración;

- en qué momento la condición dejará de cumplirse;
- cómo detenerías manualmente la prueba si cometieras un error.

18. until

until repite mientras la condición tenga un estado no-cero y se detiene cuando la condición tiene éxito.

```
contador=1

until (( contador > 3 )); do
    printf '%s\n' "$contador"
    (( contador += 1 ))
done
```

Produce:

```
1
2
3
```

19. while y until expresan ideas opuestas

```
while condición → repite mientras sea verdadera
until condición → repite hasta que sea verdadera
```

Elige la forma que exprese con mayor claridad la intención.

20. Leer un archivo línea por línea

Patrón Bash recomendado para una lectura básica:

```
while IFS= read -r linea; do
    printf '<%s>\n' "$linea"
done < datos.txt
```

Este patrón conserva mucho mejor el contenido de cada línea.

21. Qué hace read

read lee datos de la entrada estándar y los asigna a variables.

```
read -r linea
```

lee una línea y la guarda en linea.

22. Por qué usamos -r

Sin -r, read puede tratar la barra invertida \ como carácter de escape.

Con:

```
read -r linea
```

la barra invertida se conserva como parte de los datos.

Para lectura general de texto, este manual prefiere read -r.

23. Qué hace IFS= delante de read

IFS es el separador interno de campos.

Al escribir:

```
IFS= read -r linea
```

asignamos un IFS vacío únicamente para esa invocación de read, evitando que se eliminen separadores de campo iniciales o finales de la línea de la forma habitual.

Modelo:

```
IFS= → no recortar por separadores de campo
read -r → no tratar \ como escape
```

24. Redirección del archivo hacia el bucle

```
done < datos.txt
```

conecta el archivo con la entrada estándar del bucle.

Así read toma una línea en cada iteración.

25. No necesitas cat para este caso

Evita aprender:

```
cat datos.txt | while read linea; do
    ...
done
```

Para leer directamente un archivo, es más claro:

```
while IFS= read -r linea; do
    ...
done < datos.txt
```

Además evita introducir una tubería innecesaria.

26. Bucles dentro de tuberías y subshells

En Bash, los comandos de una tubería normalmente se ejecutan en subshells, con excepciones configurables como lastpipe.

Por eso este patrón puede sorprender:

```
printf ... | while read ...; do
    contador=...
done
printf '%s\n' "$contador"
```

Los cambios de variables realizados dentro del bucle pueden no persistir en la shell exterior.

Para archivos locales sencillos, preferimos redirección:

```
while IFS= read -r linea; do
    ...
done < datos.txt
```

27. Última línea sin salto final

read devuelve un estado no-cero al encontrar EOF. Si necesitas procesar también una última línea que no termina en salto de línea, existe este patrón:

```
while IFS= read -r linea || [[ -n $linea ]]; do
    printf '<%s>\n' "$linea"
done < datos.txt
```

No necesitas memorizarlo todavía. Lo incluimos porque evita una pérdida silenciosa en archivos imperfectamente terminados.

28. Crear un archivo de práctica

Dentro del laboratorio:

```
printf '%s\n' \
'primera línea' \
'segunda línea con espacios' \
'tercera \\ línea' \
> datos.txt
```

> sobrescribe datos.txt si ya existe. Por eso se usa únicamente en la carpeta específica de práctica.

29. Procesar líneas con contador

```
contador=0

while IFS= read -r linea; do
    (( contador += 1 ))
    printf '%s: <%s>\n' "$contador" "$linea"
done < datos.txt
```

El contador queda disponible después del bucle porque usamos redirección y no una tubería para alimentar el while.

30. Filtrar dentro de un bucle

```
while IFS= read -r linea; do
    if [[ -z $linea ]]; then
        continue
    fi
    printf '<%s>\n' "$linea"
done < datos.txt
```

Las líneas vacías se omiten.

31. Recorrer archivos con espacios

Crea archivos de laboratorio:

```
printf 'A\n' > 'uno.txt'
printf 'B\n' > 'dos palabras.txt'
```

Después:

```
for archivo in ./*.txt; do
    [[ -e $archivo ]] || continue
    printf 'Procesando: <%s>\n' "$archivo"
done
```

dos palabras.txt sigue siendo un solo elemento.

32. Evitar expansiones sin comillas en el cuerpo

Incorrecto:

```
printf '%s\n' $archivo
```

Aunque el glob haya producido cada pathname correctamente, una expansión posterior sin comillas puede volver a introducir división en palabras y globbing.

Correcto:

```
printf '%s\n' "$archivo"
```

33. Nombres que empiezan con guion

Un nombre puede comenzar con -.

Por eso, cuando una herramienta acepta --, usa:

```
comando -- "$archivo"
```

En nuestras prácticas ./*.txt produce rutas que comienzan por ./, lo que también reduce este riesgo.

34. Nombres con saltos de línea

Linux permite nombres de archivo que contienen saltos de línea.

Esto explica por qué procesar nombres como texto separado por líneas no es una estrategia universalmente segura.

El globbing de Bash conserva cada pathname como una palabra independiente, aunque su representación visual pueda ser confusa al imprimirla.

35. Recorrido recursivo: no usar `ls -R` para parsear

Para recorrer subdirectorios no conviertas la presentación de `ls -R` en datos.

Las herramientas diseñadas para recorrer el árbol, como `find`, son más apropiadas.

El uso completo de `find` ya fue introducido en el Módulo 10; aquí conectamos ese conocimiento con bucles.

36. Técnica avanzada GNU: `find -print0`

Cuando sea necesario procesar recursivamente nombres arbitrarios en un entorno GNU, una técnica robusta es separar pathnames con NUL:

```
while IFS= read -r -d '' archivo; do
    printf '<%s>\n' "$archivo"
done < <(find . -type f -print0)
```

Esta construcción combina características de Bash (`<(...)`, `read -d`) con `find -print0`. No es una receta POSIX genérica.

No necesitas memorizarla todavía. Se incluye para mostrar cómo se evita usar saltos de línea como separador de nombres.

37. Sustitución de proceso `<(...)`

En el ejemplo anterior:

```
<(comando)
```

es **sustitución de proceso** de Bash.

Permite presentar la salida de un comando mediante un nombre de archivo especial al comando consumidor.

Se menciona aquí solo por su utilidad para un recorrido robusto; no forma parte del núcleo que debes dominar todavía.

38. `for (( ... ))` aritmético

Bash también permite un bucle aritmético estilo C:

```
for (( i = 1; i <= 3; i += 1 )); do
    printf '%s\n' "$i"
done
```

Es específico de Bash y no es la forma POSIX de `for`.

Para principiantes usaremos esta forma solo cuando el problema sea claramente numérico.

39. Elegir el bucle correcto

Guía inicial:

lista de elementos	→ <code>for</code>
argumentos del script	→ <code>for ... in "\$@"</code>
archivos por patrón	→ <code>for archivo in ./*.ext</code>
repetir según condición	→ <code>while</code>
repetir hasta condición	→ <code>until</code>
leer archivo línea por línea	→ <code>while IFS= read -r</code>
contador numérico Bash	→ <code>while ((...))</code> o <code>for ((...))</code>

40. Preparar el laboratorio

```
test -d ~/linux-lab && \
cd ~/linux-lab && \
mkdir -p modulo-27-bucles && \
cd modulo-27-bucles && pwd
```

Etiqueta: **creación en laboratorio**.

Si alguna orden falla, la cadena se detiene: no continúes hasta revisar el mensaje. `test -d` comprueba primero que el laboratorio exista.

41. Práctica A — `for` básico

Crea `for_basico.sh`:

```
#!/usr/bin/env bash

for palabra in Linux Bash Git; do
    printf '%s\n' "$palabra"
done
```

Valida:

```
bash -n for_basico.sh
```

Después ejecútalo y explica cuántas iteraciones realiza.

42. Práctica B — argumentos seguros

Crea argumentos.sh:

```
#!/usr/bin/env bash

for argumento in "$@"; do
    printf '<%s>\n' "$argumento"
done
```

Prueba:

```
bash argumentos.sh uno 'dos palabras' tres
```

Debes obtener tres elementos.

43. Práctica C — while con contador

```
#!/usr/bin/env bash

contador=1

while (( contador <= 5 )); do
    printf 'Iteración %s\n' "$contador"
    (( contador += 1 ))
done
```

Antes de ejecutar, identifica qué línea evita que el bucle sea infinito.

44. Práctica D — until

```
#!/usr/bin/env bash

contador=1

until (( contador > 3 )); do
    printf '%s\n' "$contador"
    (( contador += 1 ))
done
```

Explica por qué se detiene cuando el contador llega a 4.

45. Práctica E — leer líneas

Crea datos.txt dentro del laboratorio y después un script:

```
#!/usr/bin/env bash

while IFS= read -r linea; do
    printf '<%s>\n' "$linea"
done < datos.txt
```

Agrega una línea con espacios y otra con una barra invertida para comprobar qué conserva.

46. Práctica F — archivos con espacios

Crea únicamente dentro de la carpeta del módulo:


```
printf 'uno\n' > 'archivo uno.txt'
printf 'dos\n' > 'archivo dos.txt'
```

Después:

```
for archivo in /*.txt; do
    [[ -e $archivo ]] || continue
    printf '<%s>\n' "$archivo"
done
```

Explica por qué cada nombre continúa siendo una sola unidad.

47. Práctica G — break y continue

```
#!/usr/bin/env bash

for n in 1 2 3 4 5; do
    if (( n == 2 )); then
        continue
    fi

    if (( n == 5 )); then
        break
    fi

    printf '%s\n' "$n"
done
```

Predice la salida antes de ejecutar.

48. Errores frecuentes

Error	Problema	Corrección
for f in \$(ls)	convierte nombres en texto sujeto a división	usa globbing o una interfaz delimitada correctamente
for f in \$@	pierde límites de argumentos	usa for f in "\$@"
printf ... \$archivo	vuelve a exponer el nombre a word splitting/globbering	usa "\$archivo"
while read línea para texto arbitrario	puede interpretar \ y tratar IFS	usa while IFS= read -r línea
contador nunca cambia	el bucle puede no terminar	actualiza la condición
break donde querías omitir una sola vuelta	termina todo el bucle	usa continue
continue donde querías terminar	solo salta a la siguiente iteración	usa break
alimentar un while con tubería y esperar variables fuera	puede ejecutarse en subshell	usa redirección cuando aplique
asumir que *.txt sin coincidencias desaparece	por defecto puede quedar literal	filtra el patrón literal o estudia nullglob
suponer que -e incluye enlaces simbólicos rotos	-e es falso si el destino no existe	si necesitas incluirlos, combina -e con -L
usar un bucle para cambios destructivos masivos	amplifica errores	prueba solo con acciones seguras y revisables

49. Detección de error 1

Analiza:

```
for archivo in $(ls *.txt); do
    printf '%s\n' "$archivo"
done
```

Problema: \$(...) convierte la salida en texto y después Bash puede dividirlo en palabras.

Corrección mínima para el directorio actual:

```
for archivo in /*.txt; do
    [[ -e $archivo ]] || continue
    printf '%s\n' "$archivo"
done
```

50. Detección de error 2

Analiza:

```

contador=1

while (( contador <= 3 )); do
    printf '%s\n' "$contador"
done

```

Problema: contador nunca cambia.

Corrección:

```

contador=1

while (( contador <= 3 )); do
    printf '%s\n' "$contador"
    (( contador += 1 ))
done

```

51. Detección de error 3

Analiza:

```

cat datos.txt | while read linea; do
    printf '%s\n' "$linea"
done

```

Problemas pedagógicos:

- cat es innecesario para leer ese archivo;
- falta -r;
- no se controla IFS;
- la tubería añade semántica de subshell.

Corrección:

```

while IFS= read -r linea; do
    printf '%s\n' "$linea"
done < datos.txt

```

52. Práctica independiente

Crea inventario_txt.sh.

Debe:

1. usar Bash;
2. trabajar únicamente en el directorio actual del laboratorio;
3. recorrer ./*.txt sin usar ls ni sustitución de comandos;
4. ignorar correctamente el patrón si no hay coincidencias;
5. llevar un contador;
6. imprimir cada pathname entre < >;
7. conservar nombres con espacios;
8. terminar mostrando cuántos archivos encontró;
9. no borrar, renombrar ni modificar los archivos;
10. pasar bash -n;
11. poder explicarse línea por línea.

53. Mini evaluación

1. ¿qué bucle es natural para recorrer una lista? A) for B) if
2. ¿qué conserva los argumentos separados? A) \$@ sin comillas B) "\$@"
3. ¿es recomendable for f in \$(ls) para nombres arbitrarios? A) Sí B) No
4. ¿qué hace break? A) termina el bucle B) salta solo la iteración
5. ¿qué hace continue? A) termina el script B) pasa a la siguiente iteración
6. ¿while repite mientras su condición tenga estado 0? A) Sí B) No
7. ¿until se detiene cuando su condición tiene éxito? A) Sí B) No
8. ¿qué opción de read evita tratar \ como escape? A) -r B) -x
9. ¿por qué usamos IFS= con read? A) para evitar el tratamiento habitual de separadores B) para borrar el archivo
10. ¿es necesaria una tubería con cat para leer un archivo línea por línea? A) Sí B) No
11. ¿un glob de Bash conserva un pathname con espacios como un elemento? A) Sí B) No
12. ¿un bucle sin cambio de condición puede volverse infinito? A) Sí B) No
13. ¿./*.txt puede permanecer literal si no hay coincidencias con la configuración predeterminada? A) Sí B) No
14. ¿las variables cambiadas en un bucle alimentado por una tubería siempre persisten fuera? A) Sí B) No

15. ¿debes usar bucles destructivos para aprender esta parte? A) Sí B) No
16. ¿-e es verdadero para un enlace simbólico roto? A) Sí B) No
17. ¿-L puede reconocer un enlace simbólico roto? A) Sí B) No

54. Registro de aprendizaje

```
Un bucle sirve para:
`for` sirve para:
`while` repite mientras:
`until` repite hasta:
`break` hace:
`continue` hace:
`"${@}"` dentro de for conserva:
¿por qué no debo usar `for f in $(ls)`?:
`read -r` sirve para:
`IFS=` antes de read ayuda a:
¿por qué prefiero `done < archivo` a una tubería innecesaria?:
¿qué puede provocar un bucle infinito?:
Forma segura inicial de recorrer .txt:
¿Por qué `-e` puede omitir un enlace simbólico roto?:
¿Cuándo usaría `-L`?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

55. Puerta de dominio

Para considerar este módulo **DOMINADO** deberás poder, en más de una práctica:

1. escribir un for sin copiar;
2. recorrer "\${@}" conservando un argumento con espacios;
3. explicar por qué no se parsea ls;
4. crear un while que termine correctamente;
5. leer líneas con IFS= read -r;
6. detectar un bucle que no actualiza su condición;
7. distinguir break de continue;
8. resolver una nueva práctica con nombres de archivo con espacios.

56. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Bash Reference Manual — Looping Constructs: https://www.gnu.org/software/bash/manual/html_node/Looping-Constructs.html
- GNU Bash Reference Manual — Bourne Shell Builtins (read, break, continue): https://www.gnu.org/software/bash/manual/html_node/Bourne-Shell-Builtins.html
- GNU Bash Reference Manual — Filename Expansion: https://www.gnu.org/software/bash/manual/html_node/Filename-Expansion.html
- GNU Bash Reference Manual — Pipelines: https://www.gnu.org/software/bash/manual/html_node/Pipelines.html
- GNU Bash Reference Manual — Process Substitution: https://www.gnu.org/software/bash/manual/html_node/Process-Substitution.html

Puntos verificados documentalente:

- for, while y until son construcciones de bucle de Bash;
- break termina un bucle y continue avanza a la siguiente iteración;
- read -r evita que la barra invertida actúe como carácter de escape;
- la expansión de nombres de archivo reemplaza patrones por pathnames coincidentes;
- si nullglob está desactivado y no hay coincidencias, el patrón queda sin modificar;
- los componentes de una tubería normalmente se ejecutan en subshells en Bash, con excepciones configurables;
- la sustitución de proceso <(...) es una característica de Bash y no una construcción POSIX básica.

Se posponen:

- arrays y mapfile como técnica principal;
- select;
- bucles anidados complejos;
- control de múltiples niveles con break n / continue n;
- opciones nullglob, failglob, dotglob como política de scripts;
- find -exec en profundidad;
- funciones y ámbito;
- getopt;
- manejo avanzado de errores y trap.

Estado de la lección: redactada y revisada documentalmente contra GNU Bash. Las prácticas son locales, no destructivas y no requieren privilegios.

Módulo 28 — Funciones, parámetros y ámbito en Bash

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Vigésimoctava entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué es una función de shell y para qué sirve;
- definir y llamar funciones Bash;
- distinguir definir una función de ejecutarla;
- pasar argumentos a una función;
- usar `$1`, `$2`, `$#` y `"$@"` dentro de una función;
- reconocer que los parámetros posicionales de una función son temporales;
- usar `local` para reducir efectos laterales sobre variables externas;
- explicar el ámbito dinámico de Bash a nivel introductorio;
- diferenciar `return` de imprimir o producir un resultado;
- usar el estado de retorno de una función con `if`, `&&` y `||`;
- producir texto por salida estándar y capturarlo cuando sea apropiado;
- evitar depender innecesariamente de variables globales;
- validar argumentos básicos antes de trabajar con ellos;
- separar una tarea grande en funciones pequeñas y comprensibles.

Conocimientos previos:

- variables y quoting;
- argumentos posicionales;
- códigos de salida;
- condicionales;
- bucles;
- sustitución de comandos;
- redirecciones.

Seguridad: las prácticas son locales, sin `sudo` y dentro de `~/linux-lab`. Las funciones pueden concentrar automatización; por eso no se usarán para borrados masivos, cambios de permisos, administración de servicios, usuarios, discos, red o cortafuegos.

2. Qué es una función

Una función agrupa una serie de órdenes bajo un nombre.

Modelo:

```
nombre de función
  ↓
varias órdenes relacionadas
  ↓
una tarea reutilizable
```

Sirve para evitar repetir código y para dividir un script en partes pequeñas.

3. Primera función

```
saludar() {
    printf 'Hola desde una función\n'
}
```

Esto **define** la función. Todavía no la ejecuta.

Para llamarla:

```
saludar
```

4. Explicación línea por línea

```
saludar() {
```

`saludar` es el nombre. `()` forma parte de esta sintaxis de definición. `{` inicia el cuerpo.

```
    printf 'Hola desde una función\n'
```

es una orden perteneciente al cuerpo.

```
}
```

cierra la función.

Después:

```
saludar
```

ejecuta la función como una orden de shell.

5. El espacio y los separadores alrededor de { y }

Esta forma es correcta:

```
saludar() {  
    printf 'Hola\n'  
}
```

En una forma de una sola línea necesitas separar correctamente las órdenes:

```
saludar() { printf 'Hola\n'; }
```

El ; antes de } termina la orden printf.

6. Sintaxis que prioriza este manual

Bash admite más de una forma de declarar funciones.

Este manual prioriza:

```
nombre() {  
    órdenes  
}
```

porque es clara y ampliamente reconocible.

No necesitamos usar la palabra reservada `function` para aprender el concepto.

7. Define antes de llamar

En un script secuencial, la definición debe haberse ejecutado antes de llamar la función.

Correcto:

```
saludar() {  
    printf 'Hola\n'  
}  
  
saludar
```

No coloques una llamada inicial a una función cuya definición todavía no ha sido procesada por el script.

8. Las funciones reciben argumentos

Ejemplo:

```
saludar() {  
    printf 'Hola, %s\n' "$1"  
}  
  
saludar 'Ana María'
```

Dentro de la función:

```
$1 → Ana María
```

9. Parámetros posicionales temporales

Cuando una función se ejecuta, sus argumentos sustituyen temporalmente los parámetros posicionales \$1, \$2, etc. durante esa llamada.

Al terminar la función, los parámetros posicionales del contexto llamador vuelven a estar disponibles.

Esto permite que una función tenga sus propios argumentos sin destruir permanentemente los argumentos originales del script.

10. \$# dentro de una función

Dentro de una función:

```
$#
```

indica cuántos argumentos recibió **esa función**.

```
contar_argumentos() {  
    printf 'La función recibió %s argumentos\n' "$#"   
}  
  
contar_argumentos uno 'dos palabras' tres
```

El resultado esperado es 3.

11. "\$@" dentro de una función

```
mostrar_argumentos() {  
    for argumento in "$@"; do  
        printf '<%s>\n' "$argumento"  
    done  
}
```

"\$@" conserva cada argumento de la función como una unidad independiente.

12. Parámetros mayores que 9

Para el décimo parámetro y posteriores usa llaves:

```
${10}  
${11}
```

\$10 no debe interpretarse como una forma segura de referirse al décimo parámetro.

Para la mayoría de nuestras funciones iniciales no necesitaremos tantos argumentos.

13. Validar que llegó un argumento

Ejemplo:

```
saludar() {  
    if (( $# < 1 )); then  
        printf 'Error: falta el nombre\n' >&2  
        return 1  
    fi  
  
    printf 'Hola, %s\n' "$1"  
}
```

La función comprueba primero su cantidad de argumentos.

14. Qué significa `return`

`return` termina una función y establece su estado de salida.

```
return 0
```

indica éxito.

```
return 1
```

indica fallo.

`return` no es la forma general de devolver una cadena, una ruta o un número arbitrario como dato.

15. return y exit no son lo mismo

Dentro de una función:

```
return 1
```

termina la función y devuelve control al llamador.

En cambio:

```
exit 1
```

termina la shell/script actual.

Regla inicial:

```
quiero salir de la función → return  
quiero terminar el script → exit
```

16. Estado implícito de una función

Si no ejecutas return n explícitamente, el estado de la función es normalmente el estado de la última orden ejecutada en su cuerpo.

Ejemplo:

```
funcion_ok() {  
    printf 'Todo bien\n'  
}
```

Si printf termina con éxito, la función también tendrá estado 0.

Cuando el significado importa, un return explícito puede hacer la intención más clara.

17. Usar una función con if

```
es_directorio() {  
    [[ -d $1 ]]  
}  
  
if es_directorio ~/linux-lab; then  
    printf 'El laboratorio existe como directorio\n'  
fi
```

La última expresión [[-d \$1]] determina el estado de la función.

18. Usar una función con && y ||

```
comprobar_archivo() {  
    [[ -f $1 ]]  
}  
  
comprobar_archivo ./nota.txt && printf 'Existe\n'
```

o:

```
comprobar_archivo ./nota.txt || printf 'No existe como archivo regular\n'
```

Las funciones participan en la lógica de estados igual que otras órdenes.

19. Resultado como texto: salida estándar

Si una función necesita producir texto, puede escribirlo por salida estándar:

```
obtener_saludo() {  
    printf 'Hola, %s\n' "$1"  
}
```

Puedes llamarla directamente:

```
obtener_saludo 'Ana'
```


20. Capturar la salida de una función

```
obtener_saludo() {  
    printf 'Hola, %s\n' "$1"  
}  
  
mensaje=$(obtener_saludo 'Ana')  
printf '%s\n' "$mensaje"
```

`$(...)` captura la salida estándar.

Recuerda que la sustitución de comandos elimina los saltos de línea finales de la salida capturada.

21. No mezcles datos con mensajes de diagnóstico

Si una función produce un dato por salida estándar y además imprime mensajes informativos por el mismo canal, la captura puede mezclar ambos.

Ejemplo problemático:

```
funcion → imprime 'Procesando...' y también imprime el resultado
```

Si el texto es diagnóstico, puede enviarse a `stderr`:

```
printf 'Aviso: falta un argumento\n' >&2
```

Así la salida estándar puede reservarse para el dato cuando ese diseño sea apropiado.

22. Variables globales por defecto

En Bash, una variable creada fuera de una función puede ser visible dentro de ella.

Además, si asignas una variable dentro de una función sin declararla local, puedes modificar una variable del ámbito exterior.

Ejemplo:

```
nombre='global'  
  
cambiar() {  
    nombre='modificado'  
}  
  
cambiar  
printf '%s\n' "$nombre"
```

El valor exterior puede quedar modificado.

23. local

Dentro de una función puedes declarar una variable local:

```
saludar() {  
    local nombre='Ana'  
    printf '%s\n' "$nombre"  
}
```

Esa variable local existe en el ámbito de la función y de las funciones que esta llame, según las reglas de ámbito dinámico de Bash.

24. Evitar modificar accidentalmente una global

```
nombre='global'  
  
mostrar() {  
    local nombre='local'  
    printf 'Dentro: %s\n' "$nombre"  
}  
  
mostrar  
printf 'Fuera: %s\n' "$nombre"
```

Resultado conceptual:

```
Dentro: local
Fuera: global
```

local oculta temporalmente la variable exterior del mismo nombre.

25. Ámbito dinámico de Bash

Bash usa **ámbito dinámico** para variables locales de funciones.

Ejemplo:

```
segunda() {
    printf 'segunda ve: %s\n' "$dato"
}

primera() {
    local dato='local de primera'
    segunda
}

primera
```

segunda puede ver el dato local de primera porque fue llamada desde ese contexto.

Esto es diferente del ámbito léxico que probablemente encontrarás en otros lenguajes.

Para empezar, la regla práctica es: **mantén funciones pequeñas y pasa datos como argumentos en vez de depender demasiado de variables visibles implícitamente.**

26. local solo tiene sentido dentro de una función

Usaremos local únicamente dentro del cuerpo de funciones.

Ejemplo:

```
procesar() {
    local archivo=$1
    printf '%s\n' "$archivo"
}
```

27. Citar al asignar desde argumentos

En una asignación simple de Bash:

```
local archivo=$1
```

el contexto de asignación no realiza el mismo word splitting y globbing que una expansión ordinaria de comando.

Aun así, cuando una expansión pase posteriormente a una orden, mantenemos el patrón seguro:

```
printf '%s\n' "$archivo"
```

28. Una función, una responsabilidad clara

Evita funciones que hagan demasiadas cosas.

Mejor:

```
validar_ruta
mostrar_archivo
contar_líneas
```

que una única función gigantesca que valide, modifique, copie, borre y genere informes.

Las funciones pequeñas son más fáciles de probar y explicar.

29. Nombres de funciones

Usa nombres descriptivos:

```
mostrar_ayuda
validar_archivo
contar_argumentos
```

Evita nombres ambiguos como:

```
hacer
cosa
x
```

30. No sobrescribas nombres de órdenes sin necesidad

Bash permite que una función tenga el mismo nombre que una orden.

Eso puede cambiar qué se ejecuta cuando escribes ese nombre.

Para aprendizaje inicial, no defines funciones llamadas:

```
cd
ls
printf
test
```

Usa nombres propios y claros.

31. Función de validación básica

```
validar_archivo() {
    if (( $# != 1 )); then
        printf 'Uso: validar_archivo RUTA\n' >&2
        return 2
    fi

    if [[ ! -f $1 ]]; then
        printf 'No es un archivo regular: %s\n' "$1" >&2
        return 1
    fi

    return 0
}
```

Esta función separa dos casos:

- uso incorrecto;
- ruta que no es archivo regular.

Los significados 1 y 2 son convenciones internas de este ejemplo; no son universales.

32. Llamar a otra función

```
mostrar_archivo() {
    local ruta=$1
    printf 'Archivo: %s\n' "$ruta"
}

procesar_archivo() {
    local ruta=$1
    mostrar_archivo "$ruta"
}
```

Una función puede llamar a otra igual que llama una orden.

33. Pasar argumentos al llamar otra función

Correcto:

```
mostrar_archivo "$ruta"
```

Si necesitas reenviar todos los argumentos:

```
otra_funcion "$@"
```

Esto preserva sus límites.

34. Funciones y bucles

Puedes combinar lo aprendido:

```
mostrar_txt() {
    local archivo

    for archivo in ./*.txt; do
        [[ -e $archivo ]] || continue
        printf '<%s>\n' "$archivo"
    done
}
```

La variable de trabajo se declara local para no alterar accidentalmente otra variable archivo del script.

35. Funciones y sustitución de comandos: efecto de subshell

Cuando llamas una función dentro de:

```
resultado=$(mi_funcion)
```

la sustitución de comandos se ejecuta en un entorno de subshell.

Por eso no debes depender de que cambios de variables realizados dentro de esa llamada modifiquen después el shell exterior.

Usa la sustitución de comandos para capturar salida, no como mecanismo para cambiar estado global externo.

36. Función que calcula y produce un dato

```
sumar() {
    local a=$1
    local b=$2
    printf '%s\n' "$(( a + b ))"
}

resultado=$(sumar 4 7)
printf 'Resultado: %s\n' "$resultado"
```

Resultado:

```
Resultado: 11
```

Aquí return no se usa para transportar el número 11; el dato viaja por salida estándar.

37. Validar cantidad antes de usar \$1 o \$2

```
sumar() {
    if (( $# != 2 )); then
        printf 'Error: se requieren dos enteros\n' >&2
        return 2
    fi

    local a=$1
    local b=$2
    printf '%s\n' "$(( a + b ))"
}
```

La validación completa de que ambos argumentos sean enteros se ampliará después; aquí validamos primero la cantidad.

38. return sin número

Si escribes:

```
return
```

sin un argumento explícito, el estado retornado depende del estado de la última orden ejecutada antes de return.

Para ramas de validación importantes, preferimos estados explícitos como:

```
return 1
```

39. No uses `return` fuera de su contexto

`return` está diseñado para volver desde una función o desde un archivo ejecutado mediante `source/..`.

Para terminar un script ejecutado normalmente se usa `exit`.

40. Preparar el laboratorio

```
test -d ~/linux-lab && \  
cd ~/linux-lab && \  
mkdir -p modulo-28-funciones && \  
cd modulo-28-funciones && pwd
```

Etiqueta: **creación en laboratorio.**

Si alguna orden falla, la cadena se detiene: no continúes hasta revisar el mensaje. `test -d` comprueba primero que el laboratorio exista.

41. Práctica A — primera función

Crea `funcion_basica.sh`:

```
#!/usr/bin/env bash  
  
saludar() {  
    printf 'Hola desde Bash\n'  
}  
  
saludar
```

Valida:

```
bash -n funcion_basica.sh
```

Después ejecútalo.

42. Práctica B — argumento de función

Crea `funcion_argumento.sh`:

```
#!/usr/bin/env bash  
  
saludar() {  
    if (( $# != 1 )); then  
        printf 'Uso: saludar NOMBRE\n' >&2  
        return 2  
    fi  
  
    local nombre=$1  
    printf 'Hola, %s\n' "$nombre"  
}  
  
saludar 'Ana María'
```

Explica por qué Ana María sigue siendo un solo argumento.

43. Práctica C — variable local

Crea `funcion_local.sh`:

```
#!/usr/bin/env bash  
  
valor='global'  
  
demostrar_local() {  
    local valor='local'  
    printf 'Dentro: %s\n' "$valor"  
}  
  
demostrar_local  
printf 'Fuera: %s\n' "$valor"
```

Predice las dos líneas antes de ejecutar.

44. Práctica D — estado de retorno

Crea `funcion_estado.sh`:

```
#!/usr/bin/env bash

es_txt() {
    [[ $1 == *.txt ]]
}

if es_txt 'informe.txt'; then
    printf 'Sí termina en .txt\n'
else
    printf 'No termina en .txt\n'
fi
```

La función produce una decisión mediante su estado, no mediante el texto `true` o `false`.

45. Práctica E — capturar un dato

Crea `funcion_dato.sh`:

```
#!/usr/bin/env bash

sumar() {
    local a=$1
    local b=$2
    printf '%s\n' "$(( a + b ))"
}

resultado=$(sumar 8 5)
printf 'Suma: %s\n' "$resultado"
```

Explica por qué usamos `printf` para el dato y no `return` 13 como mecanismo general de resultado.

46. Práctica F — recorrer argumentos dentro de función

Crea `funcion_argumentos.sh`:

```
#!/usr/bin/env bash

mostrar_argumentos() {
    local argumento

    for argumento in "$@"; do
        printf '<%s>\n' "$argumento"
    done
}

mostrar_argumentos uno 'dos palabras' tres
```

Debe mostrar tres argumentos.

47. Errores frecuentes

Error	Problema	Corrección
definir una función y esperar que se ejecute sola	definición y llamada son operaciones distintas	llama la función después
llamar antes de que la definición haya sido procesada	la función aún no existe en ese punto	define antes de llamar
usar <code>\$@</code> sin comillas para reenviar	puede romper límites	usa <code>"\$@"</code>
usar <code>exit</code> queriendo salir solo de una función	termina el script/shell	usa <code>return</code>
usar <code>return</code> para devolver texto	<code>return</code> comunica un estado	usa <code>stdout</code> u otro diseño
asignar variables sin <code>local</code> dentro de funciones	puedes modificar estado exterior	usa <code>local</code> cuando la variable sea interna
depender demasiado de globales	crea acoplamiento difícil de probar	pasa argumentos explícitos
mezclar diagnóstico y dato en <code>stdout</code>	rompe capturas con <code>\$(...)</code>	usa <code>stderr</code> para diagnósticos
olvidar validar <code>\$#</code>	puedes usar argumentos ausentes	valida cantidad primero
asumir ámbito léxico	Bash usa ámbito dinámico	diseña funciones pequeñas y explícitas
esperar que una función en <code>\$(...)</code> cambie variables externas	la sustitución usa subshell	captura salida y evita ese efecto lateral

48. Detección de error 1

Analiza:

```
nombre='global'

cambiar() {
    nombre='local temporal'
}

cambiar
printf '%s\n' "$nombre"
```

Problema: la función modifica la variable exterior.

Corrección mínima:

```
cambiar() {
    local nombre='local temporal'
}
```

49. Detección de error 2

Analiza:

```
obtener_nombre() {
    return 'Ana'
}
```

Problema: return no transporta cadenas.

Corrección:

```
obtener_nombre() {
    printf '%s\n' 'Ana'
}
```

Si necesitas capturarlo:

```
nombre=$(obtener_nombre)
```

50. Detección de error 3

Analiza:

```
reenviar() {
    otra_funcion $@
}
```

Problema: se pueden perder los límites de los argumentos.

Corrección:

```
reenviar() {
    otra_funcion "$@"
}
```

51. Método para diseñar una función

Antes de escribirla:

1. define una sola responsabilidad;
2. elige un nombre descriptivo;
3. decide qué datos recibe como argumentos;
4. valida la cantidad mínima necesaria;
5. usa variables locales para trabajo interno;
6. preserva argumentos con quoting;
7. decide si el resultado será un estado o un dato;
8. usa return para estados;
9. usa stdout con disciplina si produces un dato capturable;
10. prueba la función de forma aislada antes de integrarla en un script mayor.

52. Práctica independiente

Crea `analizar_ruta.sh`.

Debe incluir al menos tres funciones:

```
validar_argumentos
clasificar_ruta
mostrar_resultado
```

Requisitos:

1. usar Bash;
2. recibir exactamente una ruta como argumento del script;
3. pasar esa ruta explícitamente a las funciones;
4. usar `local` para variables internas;
5. distinguir al menos archivo regular, directorio y otro/no existente;
6. usar estados de retorno para indicar éxito/fallo cuando corresponda;
7. no modificar la ruta;
8. no usar `sudo`;
9. pasar `bash -n`;
10. poder explicar qué datos entran y salen de cada función.

53. Mini evaluación

1. ¿definir una función también la ejecuta? A) Sí B) No
2. ¿cómo llamas una función llamada `saludar`? A) `saludar` B) `call saludar`
3. dentro de una función, ¿\$1 representa su primer argumento? A) Sí B) No
4. ¿\$# dentro de una función cuenta sus argumentos? A) Sí B) No
5. ¿"\$@" preserva límites de argumentos? A) Sí B) No
6. ¿`return 0` indica éxito? A) Sí B) No
7. ¿`return` y `exit` tienen el mismo alcance? A) Sí B) No
8. ¿`return` es el mecanismo general para devolver una cadena? A) Sí B) No
9. ¿`local` ayuda a evitar modificar una variable exterior del mismo nombre? A) Sí B) No
10. ¿Bash usa ámbito dinámico para variables locales de funciones? A) Sí B) No
11. ¿una función puede usarse directamente como condición de `if`? A) Sí B) No
12. ¿la salida estándar de una función puede capturarse con `${...}`? A) Sí B) No
13. ¿los diagnósticos deberían mezclarse sin cuidado con datos capturados por `stdout`? A) Sí B) No
14. ¿\${10} es la forma apropiada de referirse al décimo parámetro? A) Sí B) No
15. ¿una función llamada dentro de sustitución de comandos debe usarse para modificar variables del shell exterior? A) Sí B) No

54. Registro de aprendizaje

```
Una función es:
Definir una función significa:
Llamar una función significa:
Dentro de una función `$1` es:
Dentro de una función `$#` es:
`"$@"` sirve para:
`local` sirve para:
Ámbito dinámico significa, de forma básica:
`return 0` significa:
Diferencia entre `return` y `exit`:
¿cómo produzco un dato de texto desde una función?:
¿por qué separo stdout de stderr?:
¿por qué prefiero argumentos explícitos a muchas globales?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

55. Puerta de dominio

Para considerar este módulo **DOMINADO** deberás poder en más de una sesión:

1. definir y llamar una función sin copiar;
2. pasar un argumento con espacios sin romperlo;
3. explicar por qué `"$@"` es importante;
4. usar `local` y demostrar que la global no cambia;
5. distinguir un resultado de datos de un estado de retorno;
6. usar una función como condición de `if`;
7. detectar un uso incorrecto de `exit` dentro de una función;
8. dividir un script pequeño en dos o más funciones con responsabilidades claras.

56. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Bash Reference Manual — Shell Functions: https://www.gnu.org/software/bash/manual/html_node/Shell-Functions.html
- GNU Bash Reference Manual — Positional Parameters: https://www.gnu.org/software/bash/manual/html_node/Positional-Parameters.html
- GNU Bash Reference Manual — Bourne Shell Builtins (`return`): https://www.gnu.org/software/bash/manual/html_node/Bourne-Shell-Builtins.html
- GNU Bash Reference Manual — Bash Builtins (`local`, `declare`): https://www.gnu.org/software/bash/manual/html_node/Bash-Builtins.html
- GNU Bash Reference Manual — Command Substitution: https://www.gnu.org/software/bash/manual/html_node/Command-Substitution.html

Puntos verificados documentalmente:

- una función agrupa un comando compuesto bajo un nombre;
- cuando se ejecuta una función, sus argumentos pasan a ser temporalmente sus parámetros posicionales;
- al finalizar la función se restauran los parámetros posicionales anteriores;
- las variables son compartidas con el llamador salvo que se declaren locales;
- las variables locales de funciones siguen las reglas de ámbito dinámico de Bash;
- `return` finaliza una función y establece su estado;
- una función sin `return` explícito usa el estado de la última orden ejecutada como estado de la función;
- la sustitución de comandos ejecuta su contenido en un entorno de subshell y captura la salida estándar.

Se posponen:

- arrays como interfaz de funciones;
- `namerefs` (`local -n / declare -n`);
- `FUNCNAME`, `BASH_SOURCE` y pilas de llamadas;
- recursión;
- funciones exportadas;
- `source` como arquitectura de librerías;
- `getopts`;
- manejo avanzado de errores;
- `trap`, `mktemp`, `set -e`, `set -u` y `pipefail` como política.

Estado de la lección: redactada y revisada documentalmente contra GNU Bash. Las prácticas son locales, no destructivas y no requieren privilegios.

Módulo 29 — Manejo de errores, `trap`, `mktemp`, límites de `set -e`/`set -u` y `ShellCheck`

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Vigésimonovena entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué significa manejar errores en un script;
- diferenciar detectar un fallo, informarlo, recuperarse y terminar;
- validar explícitamente operaciones importantes;
- usar `trap` de forma básica con `EXIT`;
- crear archivos temporales de forma segura con `mktemp`;
- explicar por qué `mktemp -u` no debe usarse para crear después un archivo;
- comprender qué hace `set -e` y por qué no es un detector universal de errores;
- reconocer contextos donde `errexit` se ignora o tiene semántica especial;
- comprender qué hace `set -u` y cómo afecta variables no definidas;
- usar valores predeterminados cuando correspondan;
- comprender qué cambia `set -o pipefail`;
- separar limpieza, diagnóstico y lógica principal;
- usar `bash -n` para comprobar sintaxis;
- usar `ShellCheck` como análisis estático complementario;
- interpretar una advertencia de `ShellCheck` antes de aplicar cambios;
- evitar el patrón de “activar opciones estrictas y confiar ciegamente”.

Conocimientos previos:

- estados de salida;
- `if`, `&&`, `||`;
- tuberías;
- variables y expansiones;
- funciones;
- redirecciones;
- archivos y rutas.

Seguridad: las prácticas se limitan a `~/linux-lab/modulo-29-errores`. No se usan privilegios para los ejercicios. La limpieza automatizada solo eliminará archivos temporales creados expresamente por el propio ejercicio y comprobará primero la ruta guardada.

2. Qué significa manejar errores

Un script robusto no se limita a “seguir ejecutando”.

Debe decidir qué hacer cuando algo falla.

Modelo:

```
operación
↓
¿tuvo éxito?
├ sí → continuar
└ no → informar / recuperar / limpiar / terminar
```

3. Cuatro acciones distintas

Ante un fallo puedes necesitar:

1. **detectar** el estado;
2. **informar** qué ocurrió;
3. **recuperar o limpiar** recursos temporales;
4. **terminar** con un estado apropiado.

No todos los errores requieren las cuatro acciones.

4. Validación explícita

Una forma clara:

```
if ! cd ~/linux-lab; then
    printf 'Error: no pude entrar al laboratorio\n' >&2
    exit 1
fi
```

Ventaja: la intención queda visible.

5. Variante con `||`

También puedes escribir:

```
cd ~/linux-lab || {  
    printf 'Error: no pude entrar al laboratorio\n' >&2  
    exit 1  
}
```

Las llaves agrupan varias órdenes en la rama de fallo.

Para aprendizaje inicial, usa la forma que puedas explicar sin ambigüedad.

6. Mensajes de error por `stderr`

```
printf 'Error: falta un archivo\n' >&2
```

`>&2` envía el diagnóstico a la salida de error estándar.

Esto permite mantener `stdout` disponible para datos útiles.

7. Estados de error documentados

Puedes definir una convención interna:

```
0 → éxito  
1 → fallo general  
2 → uso incorrecto
```

Pero esos significados pertenecen a tu script; no son universales para todos los programas.

8. Función de error sencilla

```
error() {  
    printf 'Error: %s\n' "$1" >&2  
}
```

Uso:

```
error 'falta el archivo de entrada'
```

Una función así evita repetir formato de diagnóstico.

9. Qué es `trap`

`trap` permite indicar a Bash que ejecute una acción cuando ocurre un evento determinado, como:

- salida del shell mediante `EXIT`;
- determinadas señales;
- otros eventos soportados por Bash.

En este módulo nos concentraremos principalmente en `EXIT` para limpieza controlada.

10. Primer `trap` con `EXIT`

```
al_salir() {  
    printf 'El script está terminando\n'  
}  
  
trap al_salir EXIT
```

Cuando el shell termina, ejecuta la función registrada.

11. `trap` no significa “capturar todos los errores”

`trap` no convierte automáticamente cualquier fallo en una excepción.

Límite: la limpieza configurada con `trap ... EXIT` no está garantizada ante una terminación forzada o un apagado repentino. No dependas únicamente de esta limpieza para proteger información sensible.

Debes distinguir:

```
trap EXIT → acción al salir
trap ERR → evento relacionado con determinadas órdenes fallidas
```

ERR tiene excepciones y reglas relacionadas con las mismas construcciones que `set -e`; no lo usaremos como detector universal.

12. Consultar traps

Puedes inspeccionar traps instalados con:

```
trap -p
```

Esto es una operación de consulta.

13. Eliminar un trap

```
trap - EXIT
```

restaura la disposición original de EXIT para el shell actual.

No lo necesitas en la mayoría de scripts pequeños, pero debes reconocer la sintaxis.

14. Qué es un archivo temporal

Un archivo temporal guarda datos que solo necesitas durante una ejecución.

Un nombre fijo como:

```
/tmp/datos.txt
```

puede colisionar con otro proceso o usuario y no es una estrategia segura para creación temporal.

15. mktemp

GNU `mktemp` crea de forma segura un archivo o directorio temporal basado en una plantilla.

Ejemplo dentro de nuestro laboratorio:

```
temporal=$(mktemp "$HOME/linux-lab/modulo-29-errores/tmp.XXXXXXXXXX") || exit 1
```

`mktemp` crea realmente el archivo y escribe su nombre.

16. Plantilla con x

Una plantilla incluye una secuencia de X que `mktemp` sustituye.

Ejemplo:

```
tmp.XXXXXXXXXX
```

No inventes tú mismo un nombre “aleatorio” con `$RANDOM` como sustituto de `mktemp` para seguridad.

17. No usar `mktemp -u` para crear después

`mktemp -u` genera un nombre pero **no crea el objeto**.

Entre generar ese nombre y utilizarlo, otro proceso podría crear algo allí.

Por eso este manual no usa:

```
nombre=$(mktemp -u ...)
crear "$nombre" después
```

para archivos temporales.

18. Comprobar que `mktemp` funcionó

```
temporal=$(mktemp "$HOME/linux-lab/modulo-29-errores/tmp.XXXXXXXXXX") || {  
    printf 'Error: no se pudo crear el temporal\n' >&2  
    exit 1  
}
```

No uses una variable de ruta temporal si la creación falló.

19. Limpieza controlada con una función

```
temporal=''  
  
limpiar() {  
    if [[ -n $temporal && -f $temporal ]]; then  
        rm -f -- "$temporal"  
    fi  
}  
  
trap limpiar EXIT
```

Esta práctica sí usa `rm`, pero con límites explícitos:

- solo sobre una variable creada por este script;
- solo si no está vacía;
- solo si actualmente es un archivo regular;
- con `--` para finalizar opciones;
- sin `-r`;
- dentro de la carpeta de laboratorio.

20. Crear después del trap

Orden recomendado:

```
temporal=''  
  
limpiar() {  
    if [[ -n $temporal && -f $temporal ]]; then  
        rm -f -- "$temporal"  
    fi  
}  
  
trap limpiar EXIT  
  
temporal=$(mktemp "$HOME/linux-lab/modulo-29-errores/tmp.XXXXXXXXXX") || exit 1
```

Si algo falla después de la creación, el trap sigue teniendo una ruta conocida que limpiar.

21. No construyas limpieza peligrosa

Evita patrones genéricos como:

```
rm -rf "$variable"
```

en una rutina de aprendizaje.

Una variable vacía, equivocada o manipulada puede convertir una limpieza en una operación de gran impacto.

En este módulo usamos únicamente eliminación de un archivo temporal individual controlado.

22. Qué hace `set -e`

`set -e`, también llamado `errexit`, hace que Bash salga ante determinados estados de fallo.

```
set -e
```

Suena simple, pero su comportamiento depende del contexto sintáctico.

23. `set -e` no significa “cualquier comando no-cero mata el script”

Hay contextos donde Bash no sale por un estado no-cero, porque ese estado se está usando para tomar una decisión.

Entre ellos existen condiciones relacionadas con:

- pruebas de `if`;
- condiciones de `while` y `until`;
- listas `&&` y `||` salvo posiciones específicas;
- comandos de tuberías salvo las reglas aplicables al estado de la tubería;
- comandos negados con `!`.

Por eso no debes memorizar `set -e` como un `try/catch` universal.

24. Ejemplo: fallo usado por `if`

```
set -e

if false; then
    printf 'No aparece\n'
fi

printf 'El script sigue\n'
```

`false` forma parte de la condición de `if`, por lo que su fallo es información para la decisión.

25. Ejemplo: fallo usado por `||`

```
set -e

false || printf 'El fallo fue gestionado\n'
printf 'Continuación\n'
```

El estado de `false` participa en una lista OR.

26. Funciones y `set -e`: una interacción importante

Si una función o comando compuesto se ejecuta en un contexto donde `-e` está siendo ignorado, los comandos de su cuerpo también pueden quedar afectados por esa regla.

Esto hace que mover una función a un contexto distinto pueda cambiar cómo se comporta `errexit`.

Conclusión pedagógica:

■ usa comprobaciones explícitas para fallos que sean críticos para la lógica del script.

27. `set -e` dentro de sustitución de comandos

Las sustituciones de comandos:

```
resultado=$(comando)
```

se ejecutan en un entorno de subshell.

En Bash fuera de modo POSIX, `-e` normalmente se limpia en subshells creadas para sustitución de comandos, salvo configuraciones como `inherit_errexit`.

Esto es otra razón para no construir la robustez del script únicamente alrededor de `set -e`.

28. Qué hace `set -u`

`set -u`, también llamado `nounset`, trata como error determinadas expansiones de variables o parámetros no definidos.

```
set -u
printf '%s\n' "$variable_no_definida"
```

En un script no interactivo, Bash escribe un error y sale.

29. Variable vacía no es variable no definida

```
variable=''
```

está definida, aunque su contenido sea vacío.

Con `set -u`, debes distinguir:

```
unset → no definida
'' → definida pero vacía
```

30. Valor predeterminado con expansión de parámetros

Si una variable puede faltar y existe un valor razonable por defecto:

```
nombre=${1:-Invitado}
```

Esto evita depender de una expansión directa de un parámetro ausente.

No uses valores predeterminados para ocultar errores que deberían detectarse explícitamente.

31. Comprobar si una variable está definida

En Bash moderno puedes usar:

```
if [[ -v variable ]]; then
    printf 'Está definida\n'
fi
```

-v comprueba si la variable tiene un valor asignado.

32. set -u tampoco reemplaza validación

nounset puede detectar referencias accidentales a variables no definidas.

Pero no puede determinar por ti si:

- el dato tiene el formato correcto;
- una ruta es segura;
- un número está en un rango permitido;
- un argumento pertenece al usuario correcto;
- el contenido tiene sentido lógico.

33. Qué hace pipefail

Por defecto, el estado de una tubería es el estado de su último comando.

Con:

```
set -o pipefail
```

el estado de la tubería será el del comando situado más a la derecha que haya terminado con estado no-cero, o 0 si todos tuvieron éxito.

34. Ejemplo conceptual de pipefail

Sin pipefail:

```
comando_que_falla | comando_que_tiene_exito
```

la tubería puede terminar con éxito si el último comando devuelve 0.

Con pipefail, el fallo anterior puede reflejarse en el estado global de la tubería.

35. pipefail no vuelve segura una tubería

pipefail mejora la visibilidad del estado, pero no:

- valida datos;
- evita efectos laterales;
- evita pérdida de información;
- corrige quoting;
- garantiza que todos los programas produzcan estados adecuados.

Es una herramienta, no una garantía.

36. La combinación conocida set -euo pipefail

Es frecuente encontrar:

```
set -euo pipefail
```

Este manual **no** la presenta como una receta mágica.

Significa aproximadamente:

```
-e      → salir en determinados fallos
-u      → error en determinadas variables no definidas
pipefail → reflejar fallos anteriores de tuberías
```

Puede ser útil en scripts diseñados y probados para esas semánticas, pero debes conocer sus límites antes de adoptarla.

37. No activar opciones por copiar una plantilla

Antes de habilitar una opción global debes saber:

- qué código cambia;
- qué condiciones son deliberadamente no-cero;
- qué funciones dependen de ese comportamiento;
- qué tuberías existen;
- cómo manejarás errores esperados.

En ejercicios iniciales priorizamos lógica explícita y luego añadimos opciones deliberadamente.

38. Comprobar sintaxis con `bash -n`

Ya lo hemos usado:

```
bash -n script.sh
```

`-n` lee el script sin ejecutar las órdenes y puede detectar errores sintácticos.

Pero no detecta todos los errores lógicos.

Un script puede pasar `bash -n` y aun así fallar durante su ejecución.

39. Qué es ShellCheck

ShellCheck es una herramienta de análisis estático para scripts `sh`/`Bash` y otros shells compatibles.

Busca patrones problemáticos como:

- expansiones sin comillas;
- errores semánticos frecuentes;
- construcciones no portables;
- errores de lógica detectables estáticamente;
- usos sospechosos de comandos.

40. ShellCheck no ejecuta el script para entender todo

Análisis estático significa que estudia el código.

No puede saber siempre:

- qué archivos existirán;
- qué contenido real recibirás;
- qué resultado producirá un sistema remoto;
- qué intención humana tenías.

Una advertencia debe leerse y entenderse; no se corrige ciegamente.

41. Comprobar si ShellCheck está instalado

```
command -v shellcheck
```

Si devuelve una ruta, está disponible.

Esta es una operación de consulta.

42. Usar ShellCheck

```
shellcheck script.sh
```


Si no produce diagnósticos, eso no certifica que el script sea perfecto.

Solo significa que no encontró problemas dentro de las reglas aplicables.

43. Ejemplo típico: SC2086

ShellCheck suele advertir sobre expansiones sin comillas que pueden sufrir word splitting o globbing.

Ejemplo problemático:

```
printf '%s\n' $archivo
```

Corrección habitual:

```
printf '%s\n' "$archivo"
```

Antes de aplicar una corrección automática, confirma que coincide con la intención del script.

44. Instalación opcional de ShellCheck

Instalar un paquete modifica el sistema, así que primero identifica la distribución y revisa la operación.

Si no está instalado, consulta el gestor de paquetes correspondiente.

Ejemplos habituales:

```
Debian/Ubuntu → paquete shellcheck
Fedora/RHEL   → comprobar disponibilidad del paquete ShellCheck en repositorios habilitados
```

No es obligatorio instalarlo para continuar leyendo el manual. También existe el analizador en el sitio oficial de ShellCheck.

45. Orden de revisión recomendado

Para un script de práctica:

1. leer el código
2. `bash -n script.sh`
3. ShellCheck, si está disponible
4. corregir solo lo entendido
5. probar en el laboratorio
6. comprobar estados y resultados
7. guardar en Git

46. Preparar el laboratorio

```
test -d ~/linux-lab && \
cd ~/linux-lab && \
mkdir -p modulo-29-errores && \
cd modulo-29-errores && pwd
```

Etiqueta: **creación en laboratorio**.

Si alguna orden falla, la cadena se detiene: no continúes hasta revisar el mensaje. `test -d` comprueba primero que el laboratorio exista.

47. Práctica A — error explícito

Crea `validar_ruta.sh`:

```
#!/usr/bin/env bash

ruta=${1:-}

if [[ -z $ruta ]]; then
    printf 'Error: falta una ruta\n' >&2
    exit 2
fi

if [[ ! -e $ruta ]]; then
    printf 'Error: la ruta no existe: %s\n' "$ruta" >&2
    exit 1
fi

printf 'Ruta válida: %s\n' "$ruta"
exit 0
```

Prueba únicamente con rutas del laboratorio.

48. Práctica B — mktemp y limpieza con trap

Crea temporal.sh:

```
#!/usr/bin/env bash

temporal=''

limpiar() {
    if [[ -n $temporal && -f $temporal ]]; then
        rm -f -- "$temporal"
    fi
}

trap limpiar EXIT

temporal=$(mktemp "$HOME/linux-lab/modulo-29-errores/tmp.XXXXXXXXXX") || {
    printf 'Error: no se pudo crear el temporal\n' >&2
    exit 1
}

printf 'Temporal creado: %s\n' "$temporal"
printf 'dato de práctica\n' > "$temporal"
printf 'El trap lo eliminará al terminar\n'
```

Antes de ejecutar, confirma con `pwd` que estás en la carpeta de práctica.

49. Qué modifica la práctica B

El script:

1. crea un archivo temporal dentro de `~/linux-lab/modulo-29-errores`;
2. escribe una línea de prueba;
3. al salir, comprueba que la variable no esté vacía y que la ruta sea un archivo regular;
4. elimina únicamente ese archivo.

No utiliza borrado recursivo.

50. Práctica C — observar `set -u`

Crea nounset.sh:

```
#!/usr/bin/env bash

set -u

nombre=${1:-Invitado}
printf 'Hola, %s\n' "$nombre"
```

Prueba con y sin argumento.

Después explica por qué `${1:-Invitado}` evita depender de `$1` sin definir.

51. Práctica D — pipefail controlado

Crea pipefail.sh:

```
#!/usr/bin/env bash

set -o pipefail

if false | true; then
    printf 'La tubería informó éxito\n'
else
    printf 'La tubería informó fallo\n'
fi
```

Con pipefail, la tubería informa fallo porque uno de sus componentes devolvió un estado no-cero.

52. Práctica E — límite de set -e

Crea errexit.sh:

```
#!/usr/bin/env bash

set -e

if false; then
    printf 'No aparece\n'
fi

printf 'El script continúa después de la condición\n'
```

Explica por qué este ejemplo contradice la simplificación “cualquier fallo detiene el script”.

53. Práctica F — análisis estático

Si ShellCheck está disponible, crea ejemplo.sh:

```
#!/usr/bin/env bash

archivo='dos palabras.txt'
printf '%s\n' $archivo
```

Primero:

```
bash -n ejemplo.sh
```

Después:

```
shellcheck ejemplo.sh
```

Lee el diagnóstico.

Finalmente corrige:

```
printf '%s\n' "$archivo"
```

54. Errores frecuentes

Error	Problema	Corrección
creer que set -e captura cualquier fallo	tiene excepciones contextuales	valida explícitamente operaciones críticas
activar set -euo pipefail sin entenderlo	puede cambiar flujo esperado	estudia cada opción por separado
usar mktemp -u y crear después	ventana de carrera	deja que mktemp cree el objeto
usar nombre temporal fijo	colisiones/riesgo	usa mktemp
limpieza con rm -rf "\$variable"	demasiado impacto ante errores de variable	limita el objeto y valida antes
usar trap ERR como detector universal	comparte excepciones importantes	úsalo solo entendiendo su semántica
mezclar errores con stdout	contamina datos	envía diagnósticos a stderr
confiar en bash -n para lógica	solo analiza sintaxis	combina revisión y pruebas
obedecer ShellCheck sin entender	una regla puede no reflejar tu intención	lee código y documentación
asumir que ShellCheck certifica seguridad	es análisis estático limitado	prueba y revisa contexto

55. Detección de error 1

Analiza:

```
set -e

if grep -q 'texto' archivo.txt; then
    printf 'Encontrado\n'
fi
```

¿un resultado no-cero de grep necesariamente termina el script por set -e?

No en este contexto: grep está siendo usado como condición del if.

56. Detección de error 2

Analiza:

```
temporal=$(mktemp -u)
printf 'dato\n' > "$temporal"
```

Problema: se genera un nombre sin crear el archivo, dejando una ventana entre elección y creación.

Corrección conceptual:

```
temporal=$(mktemp "$HOME/linux-lab/modulo-29-errores/tmp.XXXXXXXXXX") || exit 1
```

57. Detección de error 3

Analiza:

```
limpiar() {
    rm -rf "$temporal"
}
```

Problema: la acción es demasiado amplia para una ruta variable y no verifica qué está eliminando.

En nuestro laboratorio solo necesitamos:

```
limpiar() {
    if [[ -n $temporal && -f $temporal ]]; then
        rm -f -- "$temporal"
    fi
}
```

58. Método de robustez del manual

Para scripts nuevos:

1. valida entradas;
2. cita expansiones;
3. comprueba operaciones críticas explícitamente;
4. separa datos y diagnósticos;
5. usa temporales creados de forma segura;
6. registra limpieza limitada con trap cuando exista un recurso temporal;
7. comprende una opción antes de habilitarla globalmente;
8. valida sintaxis con bash -n;
9. analiza con ShellCheck cuando esté disponible;
10. prueba fallos controlados además del camino exitoso.

59. Práctica independiente

Crea procesar_temporal.sh.

Debe:

1. usar Bash;
2. aceptar un argumento de texto no sensible;
3. validar que exista el argumento;
4. crear un temporal dentro de la carpeta del módulo usando mktemp;
5. registrar una función de limpieza mediante trap ... EXIT;
6. escribir únicamente el texto de práctica en el temporal;

7. leerlo y mostrarlo;
8. eliminar automáticamente solo ese archivo al terminar;
9. enviar errores a stderr;
10. pasar `bash -n`;
11. pasar ShellCheck si está disponible;
12. no usar `sudo`, `rm -r`, `rm -rf` ni rutas fuera del laboratorio.

60. Mini evaluación

1. ¿manejar errores significa solo usar `set -e`? A) Sí B) No
2. ¿stderr es apropiado para diagnósticos? A) Sí B) No
3. ¿trap ... EXIT puede usarse para limpieza al terminar? A) Sí B) No
4. ¿mktemp crea el objeto temporal? A) Sí B) No
5. ¿es seguro usar `mktemp -u` y crear después como patrón general? A) Sí B) No
6. ¿set -e sale ante absolutamente cualquier estado no-cero? A) Sí B) No
7. ¿una condición de `if` es uno de los contextos especiales de `set -e`? A) Sí B) No
8. ¿set -u detecta determinadas expansiones de variables no definidas? A) Sí B) No
9. ¿variable vacía y variable no definida son lo mismo? A) Sí B) No
10. ¿pipefail cambia el estado calculado de una tubería? A) Sí B) No
11. ¿pipefail valida automáticamente los datos? A) Sí B) No
12. ¿bash -n ejecuta todo el script? A) Sí B) No
13. ¿ShellCheck es análisis estático? A) Sí B) No
14. ¿una advertencia de ShellCheck debe entenderse antes de corregir? A) Sí B) No
15. ¿rm -rf "\$variable" es una buena práctica de limpieza para este módulo? A) Sí B) No

61. Registro de aprendizaje

```
Manejar un error significa:
`stderr` sirve para:
`trap` sirve para:
`EXIT` en trap significa:
`mktemp` sirve para:
¿por qué no usamos `mktemp -u` para crear después?:
`set -e` hace:
¿por qué `set -e` no es universal?:
`set -u` hace:
Diferencia entre unset y vacío:
`pipefail` cambia:
`bash -n` comprueba:
ShellCheck sirve para:
¿por qué no sigo una advertencia automáticamente?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

62. Puerta de dominio

Para considerar este módulo **DOMINADO** deberás poder en más de una práctica:

1. detectar explícitamente un fallo y devolver un estado;
2. explicar al menos dos excepciones conceptuales de `set -e`;
3. distinguir `set -u` de validación de contenido;
4. explicar qué resuelve `pipefail` y qué no;
5. crear un temporal con `mktemp` sin `-u`;
6. limpiar únicamente el temporal propio mediante `trap EXIT`;
7. interpretar una advertencia sencilla de ShellCheck;
8. explicar por qué un script que pasa `bash -n` aún puede contener errores.

63. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU Bash Reference Manual — The Set Builtin: https://www.gnu.org/software/bash/manual/html_node/The-Set-Builtin.html
- GNU Bash Reference Manual — Bourne Shell Builtins (trap): https://www.gnu.org/software/bash/manual/html_node/Bourne-Shell-Builtins.html
- GNU Bash Reference Manual — Pipelines: https://www.gnu.org/software/bash/manual/html_node/Pipelines.html
- GNU Bash Reference Manual — Command Execution Environment: https://www.gnu.org/software/bash/manual/html_node/Command-Execution-Environment.html
- GNU Coreutils — mktemp: https://www.gnu.org/software/coreutils/manual/html_node/mktemp-invocation.html
- ShellCheck: <https://www.shellcheck.net/>

- ShellCheck Wiki: <https://www.shellcheck.net/wiki/>

Puntos verificados documentalmente:

- `set -e` tiene excepciones asociadas a condiciones, listas y otros contextos; no equivale a “fallar ante cualquier no-cero”;
- si `-e` está siendo ignorado en el contexto de un comando compuesto o función, las órdenes del cuerpo pueden verse afectadas por esa misma regla;
- `set -u` trata determinadas expansiones de variables no definidas como error;
- `pipefail` devuelve el estado del comando no-cero situado más a la derecha en la tubería, o 0 si todos tienen éxito;
- por defecto, una tubería usa el estado de su último comando;
- `mktemp` crea de forma segura el archivo o directorio temporal;
- GNU Coreutils advierte que `mktemp -u` es inseguro para generar un nombre y crear el objeto posteriormente;
- ShellCheck es un analizador estático para scripts de shell y documenta sus diagnósticos mediante códigos como `SC2086`.

Se posponen:

- `ERR`, `DEBUG` y `RETURN` traps en profundidad;
- herencia de traps con `errtrace` y `functrace`;
- `inherit_errexist`;
- limpieza de árboles temporales complejos;
- archivos de bloqueo y concurrencia;
- señales y cleanup avanzado;
- `mktemp -d` con recursos múltiples;
- configuración avanzada de ShellCheck y directivas de exclusión;
- pruebas automatizadas de shell.

Estado de la lección: redactada y revisada documentalmente contra GNU Bash, GNU Coreutils y la documentación oficial de ShellCheck. Las prácticas son locales y limitan cualquier borrado al archivo temporal creado por el propio ejercicio.

Módulo 30 — Automatización con `cron` y temporizadores de `systemd`

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Trigésima entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué significa programar una tarea;
- distinguir ejecución manual de ejecución programada;
- aplicar la regla «probar primero, programar después»;
- reconocer `cron`, `crond` y `crontab`;
- leer los cinco campos temporales de una entrada de `crontab`;
- diferenciar el `crontab` de usuario de archivos de `cron` del sistema;
- reconocer diferencias de entorno entre tu terminal y `cron`;
- evitar comandos complejos directamente dentro del `crontab`;
- consultar tareas sin borrarlas accidentalmente;
- explicar qué ocurre con una ejecución perdida en `cron`;
- reconocer qué es una unidad `.timer` de `systemd`;
- entender la relación entre una unidad `.timer` y una `.service`;
- diferenciar `OnCalendar=` de temporizadores monotónicos como `OnActiveSec=`;
- usar `systemd-analyze calendar` para validar expresiones de calendario;
- explicar `Persistent=true`, `AccuracySec=` y `RandomizedDelaySec=` a nivel básico;
- consultar temporizadores con `systemctl list-timers`;
- crear una práctica de temporizador de usuario sin privilegios cuando el entorno lo soporte;
- documentar y retirar una automatización de prueba.

Conocimientos previos:

- scripts Bash;
- permisos de ejecución;
- rutas absolutas y relativas;
- variables de entorno;
- `systemd` y servicios;
- logs y `journalctl`;
- estados de salida y manejo de errores.

Seguridad: una tarea programada repite acciones sin que estés mirando. Por eso este módulo solo automatiza la escritura de una marca de tiempo dentro de `~/linux-lab/modulo-30-automatizacion`. No se automatizan borrados, actualizaciones del sistema, cambios de permisos, paquetes, usuarios, discos, red ni cortafuegos.

2. Qué es automatizar una tarea

Automatizar significa indicar que una acción se ejecute sin iniciarla manualmente cada vez.

Modelo:

```
script probado manualmente
↓
programador de tareas
↓
momento o intervalo
↓
ejecución automática
```

3. Regla principal del módulo

Primero prueba la tarea manualmente. Después prográmala.

Si un script falla cuando lo ejecutas tú, programarlo no corrige el problema; solo hace que falle automáticamente.

4. Qué es `cron`

`cron` es una familia tradicional de servicios para ejecutar órdenes de forma programada.

En sistemas que usan Cronie, el daemon suele llamarse:

```
crond
```

y las tablas de tareas se administran con:

```
crontab
```

No todas las distribuciones instalan o habilitan exactamente la misma implementación.

5. Comprobar qué existe

Consultas seguras:

```
command -v crontab
command -v systemctl
```

Si `crontab` no está disponible, no instales nada automáticamente solo para completar el ejercicio. Primero identifica la distribución y el paquete correspondiente.

Distribuciones y servicio cron: en Debian/Ubuntu normalmente se utiliza `cron.service` (paquete `cron`); en Fedora/RHEL suele emplearse `Cronie` con `crond.service`. La sintaxis básica de cinco campos es parecida, pero los nombres de unidad y registros dependen del sistema. Consulta la unidad instalada antes de diagnosticar fallos.

Temporizadores de usuario: el administrador `systemd --user` puede detenerse al cerrar la última sesión. La permanencia sin sesión requiere una configuración específica denominada *lingering*, administrada con `loginctl`. No actives *lingering* en este laboratorio: comprueba primero la política del equipo y prueba el temporizador durante una sesión abierta.

6. Consultar tu crontab

```
crontab -l
```

`-l` lista las tareas del usuario actual.

Si no existe una tabla todavía, la implementación puede informar que no hay `crontab`.

Esta es una operación de consulta.

7. Editar el crontab del usuario

```
crontab -e
```

abre la tabla del usuario actual en un editor.

Esto sí modifica configuración. Después de guardar, las nuevas entradas pueden comenzar a ejecutarse según su horario.

Por eso no añadiremos una línea hasta haber probado manualmente el script.

8. No usar `crontab -r` como práctica

`crontab -r` elimina la tabla del usuario.

Este manual no lo usa para limpiar un ejercicio porque podría borrar tareas ajenas a la práctica.

Para retirar una tarea, vuelve a:

```
crontab -e
```

y elimina únicamente la línea que tú añadiste.

9. Los cinco campos de tiempo

Una entrada de `crontab` de usuario tiene, de forma simplificada:

```
minuto hora día_del_mes mes día_de_la_semana comando
```

Ejemplo conceptual:

```
30 18 * * * comando
```

significa ejecutar el comando a las 18:30 cada día.

10. Rangos básicos

En `Cronie`:

minuto	0–59
hora	0–23
día del mes	1–31
mes	1–12
día semana	0–7

En día de semana, 0 y 7 representan domingo en esta implementación.

11. El asterisco *

* significa «cualquier valor permitido de este campo».

Ejemplo:

```
0 * * * *
```

coincide en el minuto 0 de cada hora.

12. Listas, rangos y pasos

Ejemplos de sintaxis común de Cronie:

```
1,15,30
1-5
*/10
```

Interpretación:

- 1,15,30 → valores concretos;
- 1-5 → rango;
- */10 → cada 10 unidades **dentro de ese campo**.

13. */N no siempre significa «cada N horas desde ahora»

Ejemplo:

```
*/23
```

en el campo hora selecciona valores dentro del rango diario; no crea automáticamente un intervalo continuo de 23 horas desde el momento actual.

Interpreta cada paso dentro del campo donde aparece.

14. Día del mes y día de la semana: detalle importante

En Cronie, si restringes tanto:

- día del mes;
- día de la semana;

la tarea se ejecuta cuando **cualquiera de esos dos campos coincide**, no únicamente cuando coinciden ambos.

Es una fuente frecuente de errores de calendario.

15. Ejemplo de horario

```
0 18 * * 1-5
```

se interpreta como:

```
minuto: 0
hora: 18
día del mes: cualquiera
mes: cualquiera
día de semana: lunes a viernes
```

16. Cron y el entorno

Una tarea ejecutada por cron no hereda necesariamente el mismo entorno interactivo de tu terminal.

En Cronie, entre otras cosas:

- SHELL suele inicializarse como `/bin/sh`;
- HOME y LOGNAME se establecen para el propietario de la tabla;
- PATH puede no coincidir con el que ves en una terminal interactiva.

Por eso un script que depende de alias, funciones interactivas o rutas implícitas puede fallar bajo cron.

17. Estrategia recomendada: programa un script, no una línea enorme

En lugar de poner lógica compleja dentro del crontab:

```
muchas órdenes && redirecciones && sustituciones ...
```

crea un script probado:

```
registro.sh
```

y programa únicamente su ruta.

Ventajas:

- puedes probarlo manualmente;
- puede pasar `bash -n` y ShellCheck;
- conserva su shebang;
- es más fácil de versionar;
- reduce problemas de quoting dentro del crontab.

18. El carácter % tiene significado especial en Cronie

En la parte de comando de una entrada de Cronie, un % sin escapar tiene tratamiento especial.

Esto es otra razón para mantener comandos complejos dentro de un script y dejar el crontab lo más simple posible.

19. Cambios de horario y DST

Los relojes civiles pueden saltar hacia adelante o repetirse por cambios de horario.

En Cronie, una hora que no existe puede no ejecutarse y una hora repetida puede provocar una coincidencia adicional según la situación.

Si una tarea es crítica, debes considerar zona horaria y cambios de reloj, no solo escribir cinco campos.

20. Cron no recupera automáticamente cualquier ejecución perdida

El daemon comprueba horarios mientras está funcionando.

Si el equipo está apagado en el instante programado, la ejecución tradicional de cron normalmente no ocurre después solo por haber encendido el equipo.

Existen herramientas complementarias como `anacron` para ciertos trabajos periódicos; no las desarrollaremos todavía.

21. Preparar el script seguro de laboratorio

```
test -d ~/linux-lab && \
cd ~/linux-lab && \
mkdir -p modulo-30-automatizacion && \
cd modulo-30-automatizacion && pwd
pwd
```

Crea `registro.sh`:

```
#!/usr/bin/env bash

archivo="$HOME/linux-lab/modulo-30-automatizacion/ejecuciones.log"
printf '%(%F %T)T\n' -1 >> "$archivo"
```

Este script solo añade una marca de tiempo a un archivo del laboratorio.

22. Validar antes de programar

```
bash -n registro.sh
bash registro.sh
cat ejecuciones.log
```

Si ShellCheck está disponible:

```
shellcheck registro.sh
```

No continúes con programación automática hasta comprobar que el archivo se actualiza correctamente.

23. Hacer ejecutable el script

Después de comprobarlo:

```
chmod u+x registro.sh
```

`chmod u+x` modifica solo el permiso de ejecución del archivo de práctica.

Prueba ejecución directa:

```
./registro.sh
```

24. Obtener la ruta exacta

```
realpath registro.sh
```

Guarda mentalmente o copia la ruta que devuelve.

Para tareas programadas, una ruta absoluta evita depender del directorio actual.

25. Diseñar una entrada cron de práctica

Ejemplo conceptual para ejecutar cada cinco minutos:

```
* /5 * * * * /ruta/absoluta/al/registro.sh
```

No copies literalmente `/ruta/absoluta/...` Usa la ruta real obtenida con `realpath`.

26. Instalar la práctica cron de forma controlada

Esta parte modifica tu `crontab` de usuario.

Antes:

```
crontab -l
```

Revisa si ya existen tareas.

Después:

```
crontab -e
```

añade **una sola línea** con tu script de laboratorio.

No borres ni alteres entradas existentes.

27. Verificar la instalación

```
crontab -l
```

Comprueba visualmente que:

- la línea aparece una sola vez;
- la ruta es la correcta;
- el horario es el que pretendías.

28. Comprobar el resultado

Después de una ejecución programada, consulta:

```
cat ~/linux-lab/modulo-30-automatizacion/ejecuciones.log
```

Debes ver una nueva marca de tiempo.

Si no aparece, investiga:

- servicio cron disponible;
- sintaxis;
- ruta;
- permiso de ejecución;
- entorno;
- logs de cron según la distribución.

29. Retirar únicamente la línea de práctica

Al terminar el ejercicio:

```
crontab -e
```

elimina solo la línea que apunta a registro.sh.

Después verifica:

```
crontab -l
```

No uses `crontab -r` para esta limpieza.

30. Qué es un temporizador systemd

Una unidad `.timer` puede programar la activación de otra unidad, normalmente una `.service` del mismo nombre.

Modelo:

```
lab-registro.timer
↓
momento/intervalo
↓
lab-registro.service
↓
script probado
```

31. Timer y service son responsabilidades diferentes

La unidad `.timer` responde:

```
¿cuándo?
```

La unidad `.service` responde:

```
¿qué se ejecuta?
```

Esta separación hace que la tarea pueda probarse manualmente iniciando primero el servicio.

32. Consultar timers del sistema

```
systemctl list-timers --all
```

muestra timers cargados, última y próxima activación cuando están disponibles.

Es una consulta.

33. Consultar timers del usuario

Si existe un administrador `systemd` de usuario:

```
systemctl --user list-timers --all
```

Esto no administra unidades globales del sistema.

Algunos contenedores, sistemas mínimos o entornos especiales pueden no tener disponible un user manager de `systemd`.

34. OnCalendar=

`OnCalendar=` programa según el reloj/calendario.

Ejemplos reconocidos por systemd:

```
hourly
daily
weekly
```

También admite expresiones de calendario más detalladas.

35. Validar una expresión sin instalar un timer

```
systemd-analyze calendar 'daily'
```

Esta orden analiza, normaliza y muestra la próxima coincidencia.

También puedes pedir varias:

```
systemd-analyze calendar --iterations=3 'Mon..Fri 18:00'
```

Esta es una excelente forma de comprobar la intención **antes** de crear el temporizador.

36. Temporizadores monotónicos

systemd también puede programar intervalos relativos, por ejemplo:

```
OnActiveSec=1min
OnUnitActiveSec=5min
```

OnActiveSec= cuenta desde que se activa el timer.

OnUnitActiveSec= cuenta desde la última activación de la unidad asociada.

Estos intervalos no expresan una hora civil como 18:30; expresan tiempo relativo.

37. Persistent=true

Para timers basados en OnCalendar=, Persistent=true guarda en disco cuándo fue activado por última vez el servicio asociado.

Cuando el timer vuelve a activarse, systemd comprueba si **habría vencido al menos una vez** durante el tiempo en que estuvo inactivo. Si es así, dispara la unidad inmediatamente, sujeto también a retrasos como RandomizedDelaySec=.

Esto sirve para recuperar una ejecución perdida, por ejemplo después de que el equipo haya estado apagado.

Pero hay una precisión importante:

```
Persistent=true ≠ cola de todas las ejecuciones perdidas
```

Si un trabajo diario habría vencido cinco veces mientras el timer estuvo inactivo, no debes asumir que systemd ejecutará cinco veces el servicio al volver. La documentación upstream establece que basta con que hubiera debido dispararse **al menos una vez** para generar la activación de recuperación.

Además, Persistent=true solo tiene efecto sobre timers configurados con OnCalendar=; no convierte automáticamente los timers monotónicos en persistentes.

Apagado/inactividad y suspensión no son exactamente lo mismo

Los timers de calendario usan el reloj de tiempo real. Si el sistema entra en suspensión o hibernación, ese reloj sigue avanzando. Cuando el sistema reanuda, los eventos de calendario que vencieron durante la suspensión pueden procesarse entonces.

La documentación de systemd también aclara que, si el mismo timer de calendario venció varias veces durante una suspensión continua, eso produce **una sola activación del servicio** al reanudarse.

Por tanto:

```
apagado/inactividad + Persistent=true → recuperación al activar el timer
suspensión + OnCalendar=                → recuperación al reanudar
múltiples vencimientos perdidos          → no asumir una ejecución por cada vencimiento
```

Esto ofrece una diferencia importante frente al cron tradicional para equipos que pueden permanecer apagados.

38. AccuracySec=

Un timer no necesariamente se dispara en el microsegundo exacto indicado.

AccuracySec= define una ventana de precisión y systemd puede agrupar despertares para ahorrar recursos.

El valor predeterminado documentado es de un minuto.

Para tareas normales, no reduzcas la precisión sin una razón real.

39. RandomizedDelaySec=

RandomizedDelaySec= añade un retraso aleatorio dentro de un intervalo.

Sirve para evitar que muchas máquinas ejecuten la misma tarea exactamente al mismo tiempo.

Ejemplo conceptual:

```
OnCalendar=daily
RandomizedDelaySec=15min
```

No necesitamos activarlo en la práctica local.

40. Unidad service de usuario para el laboratorio

Primero crea la carpeta de configuración de usuario si no existe:

```
mkdir -p ~/.config/systemd/user
```

Esto modifica únicamente configuración del usuario actual.

Crea:

```
~/.config/systemd/user/lab-registro.service
```

con:

```
[Unit]
Description=Práctica segura de registro automático

[Service]
Type=oneshot
ExecStart=%h/linux-lab/modulo-30-automatizacion/registro.sh
```

%h es un especificador de systemd que representa el directorio personal del usuario.

41. Probar el servicio antes del timer

Recarga las unidades de usuario:

```
systemctl --user daemon-reload
```

Después prueba **solo el servicio**:

```
systemctl --user start lab-registro.service
```

Comprueba:

```
cat ~/linux-lab/modulo-30-automatizacion/ejecuciones.log
```

Si esto falla, no continúes con el timer.

42. Consultar el estado del servicio

```
systemctl --user status lab-registro.service
```

Una unidad Type=oneshot puede aparecer como inactiva después de terminar correctamente; eso no significa necesariamente que haya fallado.

Revisa el estado/resultados y los logs.

43. Logs del servicio de usuario

```
journalctl --user -u lab-registro.service
```

Consulta eventos asociados a esa unidad cuando el entorno mantiene journal de usuario.

No publiques logs completos si contienen rutas o datos privados innecesarios.

44. Unidad timer de práctica

Crea:

```
~/config/systemd/user/lab-registro.timer
```

con:

```
[Unit]
Description=Temporizador de práctica para lab-registro

[Timer]
OnActiveSec=1min
OnUnitActiveSec=5min
Unit=lab-registro.service

[Install]
WantedBy=timers.target
```

Esta práctica usa tiempos relativos para poder observarla sin depender de una hora del día.

45. Recargar y revisar

```
systemctl --user daemon-reload
systemctl --user list-timers --all
```

Todavía no necesitas habilitar el timer para el arranque.

46. Iniciar el timer de práctica

```
systemctl --user start lab-registro.timer
```

start lo activa para la sesión/manager actual.

No usamos enable en la práctica principal porque no necesitamos dejarlo configurado para futuras sesiones.

47. Verificar el timer

```
systemctl --user status lab-registro.timer
systemctl --user list-timers --all
```

Busca:

- próxima ejecución;
- última ejecución;
- unidad que activa.

48. Detener el timer al terminar

```
systemctl --user stop lab-registro.timer
```

Después:

```
systemctl --user list-timers --all
```

Comprueba que ya no quede activo como práctica.

49. Retirar las dos unidades de práctica

Solo después de detener el timer y confirmar los nombres exactos:

```
rm -- "$HOME/.config/systemd/user/lab-registro.service" \
    "$HOME/.config/systemd/user/lab-registro.timer"
```

Este `rm` modifica configuración de usuario y elimina **únicamente los dos archivos exactos creados por el ejercicio**.

Después:

```
systemctl --user daemon-reload
```

No uses comodines ni borrado recursivo para esta limpieza.

50. start frente a enable

En systemd:

```
start → activa ahora
enable → configura activación futura según [Install]
```

enable --now hace ambas cosas.

En este laboratorio usamos start porque queremos una prueba temporal y reversible.

51. No pongas lógica de shell compleja directamente en ExecStart=

ExecStart= no es una línea de shell interactiva normal.

Operadores como:

```
>
&&
||
$HOME
```

no deben suponerse interpretados como lo haría Bash.

Por eso nuestra unidad ejecuta un **script Bash ya probado** y deja redirecciones y lógica dentro del script.

52. Cron frente a systemd timers

Característica	cron	systemd timer
modelo	tabla de horarios	unidades .timer + servicio
calendario	cinco campos en crontab	OnCalendar=
intervalos relativos	limitado/depende del diseño	directivas monotónicas
tarea perdida con equipo apagado	normalmente no se recupera sola	Persistent=true puede recuperar que hubo al menos una activación OnCalendar perdida; no reproduce necesariamente cada vencimiento
consulta	crontab -l	systemctl list-timers
logs	depende de implementación/configuración	journal de la unidad cuando aplica
entorno	cron suele ser reducido	entorno de unidad systemd
prueba separada de tarea	llamar script manualmente	iniciar .service manualmente

No existe una regla de que uno sea siempre mejor.

53. Cuándo preferir cron

Puede ser apropiado cuando:

- ya está instalado y gestionado en el entorno;
- la tarea es sencilla;
- la portabilidad entre sistemas Unix importa;
- el equipo tiene convenciones existentes basadas en crontab.

54. Cuándo preferir systemd timers

Puede ser apropiado en sistemas systemd cuando necesitas:

- integración con servicios;
- estado consultable;
- journal;
- temporizadores monotónicos;
- Persistent=true para calendarios;
- dependencias y propiedades de unidades.

55. Errores frecuentes

Error	Problema	Corrección
programar antes de probar	automatizas un fallo	prueba manualmente primero
usar rutas relativas	el directorio de ejecución puede ser distinto	usa rutas absolutas
confiar en el PATH interactivo	cron/systemd tienen otro entorno	usa script probado y rutas controladas
poner una línea enorme en cron	quoting y % complican el comportamiento	programa un script
usar crontab -r para retirar una línea	elimina toda la tabla	usa crontab -e
interpretar */N como intervalo universal	opera dentro del campo	analiza cada campo
olvidar la regla DOM/DOW de Cronie	puedes ejecutar más días de los esperados	verifica el calendario
asumir que cron recupera tareas perdidas	puede no ejecutarlas después	considera diseño apropiado/anacron/systemd
asumir que Persistent=true reproduce todas las ejecuciones omitidas	puede haber una sola activación de recuperación	interpretalo como detección de al menos un vencimiento perdido
confundir apagado con suspensión	systemd trata esos escenarios de forma distinta	distingue inactividad del timer y suspensión del sistema
crear timer sin probar service	dificulta diagnóstico	inicia el service manualmente
usar enable --now solo para probar	deja persistencia innecesaria	usa start en práctica temporal
escribir shell directamente en ExecStart=	systemd no es un shell	llama a un script
reducir AccuracySec sin motivo	aumenta precisión/costo de wakeups	conserva valores razonables

56. Detección de error 1

Analiza:

```
*/5 * * * * cd proyecto && ./tarea.sh > salida.log
```

Problemas:

- directorio relativo/implícito;
- varias operaciones dentro del crontab;
- resultado dependiente del entorno;
- más difícil probar y versionar.

Mejor diseño:

```
*/5 * * * * /ruta/absoluta/tarea_programada.sh
```

y dentro del script haces la validación, cambio de directorio y redirección de forma explícita.

57. Detección de error 2

Analiza:

```
0 4 1,15 * 5 comando
```

Si creías que significa «solo los días 1 o 15 que además sean viernes», tu interpretación es incorrecta para Cronie.

Al restringir ambos campos de día, Cronie ejecuta cuando coincide **día del mes o día de la semana**.

58. Detección de error 3

Analiza:

```
[Service]
ExecStart=echo hola >> $HOME/archivo.log
```

Problema: se escribió una línea como si ExecStart= fuera interpretado automáticamente por un shell.

Mejor:

```
[Service]
Type=oneshot
ExecStart=%h/linux-lab/modulo-30-automatizacion/registro.sh
```

59. Método seguro para automatizar

1. define exactamente la tarea;
2. limita su alcance;
3. ejecútala manualmente;
4. revisa su estado y salida;
5. usa rutas conocidas;
6. decide cron o systemd según el entorno;
7. valida el horario;
8. programa una tarea no destructiva primero;
9. verifica que realmente se ejecutó;
10. documenta cómo detenerla y retirarla.

60. Práctica independiente

Diseña una automatización llamada `estado_laboratorio`.

Debe:

1. usar un script Bash separado;
2. añadir fecha/hora a un archivo dentro de `~/linux-lab/modulo-30-automatizacion`;
3. pasar `bash -n`;
4. probarse manualmente antes de programarse;
5. tener una propuesta de horario cron;
6. tener una propuesta equivalente con timer systemd;
7. indicar cómo verificar que se ejecutó;
8. indicar cómo detenerla;
9. indicar cómo retirarla sin afectar otras tareas;
10. no usar `sudo`, borrados, paquetes, red, servicios del sistema ni datos sensibles.

61. Mini evaluación

1. ¿debes programar un script antes de probarlo manualmente? A) Sí B) No
2. ¿`crontab -l` consulta la tabla? A) Sí B) No
3. ¿usaremos `crontab -r` para retirar una sola práctica? A) Sí B) No
4. ¿una entrada cron de usuario tiene cinco campos de tiempo antes del comando? A) Sí B) No
5. ¿* significa cualquier valor permitido del campo? A) Sí B) No
6. ¿*/10 significa necesariamente cada diez horas desde ahora? A) Sí B) No
7. si DOM y DOW están restringidos en Cronie, ¿basta que coincida uno? A) Sí B) No
8. ¿cron garantiza el mismo PATH de tu terminal interactiva? A) Sí B) No
9. ¿es mejor programar un script probado que una línea cron enorme? A) Sí B) No
10. ¿cron tradicional recupera automáticamente toda ejecución perdida por apagado? A) Sí B) No
11. ¿un `.timer` suele activar una `.service`? A) Sí B) No
12. ¿`OnCalendar=` usa expresiones de calendario? A) Sí B) No
13. ¿`systemd-analyze calendar` puede validar una expresión antes de instalar el timer? A) Sí B) No
14. ¿`Persistent=true` aplica a recuperación de activaciones `OnCalendar` perdidas? A) Sí B) No
15. Si un timer habría vencido cinco veces mientras estuvo inactivo, ¿`Persistent=true` obliga a ejecutar cinco veces el servicio al volver? A) Sí B) No
16. Si un timer `OnCalendar=` vence varias veces durante una suspensión continua, ¿debes asumir varias activaciones al reanudar? A) Sí B) No
17. ¿`systemctl --user start` y `enable` significan exactamente lo mismo? A) Sí B) No
18. ¿`ExecStart=` debe tratarse como una línea de Bash interactiva? A) Sí B) No

62. Registro de aprendizaje

```
Automatizar significa:
Regla antes de programar:
`crontab -l` sirve para:
`crontab -e` sirve para:
Los cinco campos cron son:
`*` significa:
¿por qué una ruta absoluta ayuda?:
¿por qué no conviene una línea cron enorme?:
Una unidad `.timer` sirve para:
Una unidad `.service` sirve para:
`OnCalendar=` significa:
`OnActiveSec=` significa:
`Persistent=true` sirve para:
¿reproduce todas las ejecuciones perdidas?:
Diferencia entre apagado/inactividad y suspensión para timers de calendario:
`AccuracySec=` afecta:
`systemctl --user list-timers` muestra:
Diferencia entre `start` y `enable`:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

63. Puerta de dominio

Para considerar este módulo **DOMINADO** deberás poder en más de una ocasión:

1. leer correctamente una expresión cron sencilla;
2. detectar el error conceptual de */N fuera de su campo;
3. explicar el entorno reducido de cron;
4. probar un script antes de programarlo;
5. retirar solo tu propia entrada cron;
6. explicar la pareja timer/service;
7. validar un OnCalendar= con systemd-analyze calendar;
8. distinguir start, enable y Persistent=true;
9. diagnosticar una automatización que no se ejecutó sin realizar cambios destructivos.

64. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- Cronie upstream (código y manuales del proyecto): <https://github.com/cronie-crond/cronie>
- systemd upstream (código y manuales): <https://github.com/systemd/systemd/tree/main/man>
- Las páginas man7 siguientes son copias consultables de manuales; para contrastar cambios de versión, revisar upstream.
- Cronie crontab(5): <https://man7.org/linux/man-pages/man5/crontab.5.html>
- Cronie crond(8): <https://man7.org/linux/man-pages/man8/crond.8.html>
- systemd upstream — systemd.timer(5): <https://github.com/systemd/systemd/blob/main/man/systemd.timer.xml>
- systemd upstream — systemd.time(7): <https://github.com/systemd/systemd/blob/main/man/systemd.time.xml>
- systemd systemd-analyze(1): <https://man7.org/linux/man-pages/man1/systemd-analyze.1.html>
- systemd systemctl(1): <https://man7.org/linux/man-pages/man1/systemctl.1.html>
- Red Hat Enterprise Linux 10 — systemd unit files:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/using_systemd_unit_files_to_customize_and_optimize_your_sy

Puntos verificados documentalente:

- Cronie utiliza cinco campos de tiempo y revisa tareas por minuto;
- los comandos del crontab se ejecutan mediante /bin/sh por defecto salvo configuración de SHELL; HOME y LOGNAME se preparan para el usuario;
- % posee significado especial en la parte de comando de Cronie;
- cuando día del mes y día de semana están ambos restringidos, Cronie ejecuta si coincide cualquiera de ellos;
- systemd.timer admite OnCalendar= y temporizadores monotónicos como OnActiveSec= y OnUnitActiveSec=;
- Persistent=true solo afecta timers OnCalendar= y, al activarse el timer, dispara el servicio si habría vencido al menos una vez mientras estuvo inactivo; no debe interpretarse como una cola que reproduce todos los vencimientos perdidos;
- los timers OnCalendar= que vencen durante suspensión se procesan al reanudar; varios vencimientos durante una suspensión continua producen una sola activación del servicio;
- AccuracySec= tiene un valor predeterminado documentado de un minuto;
- systemd-analyze calendar analiza y normaliza la misma sintaxis utilizada por OnCalendar=;
- systemctl list-timers muestra temporizadores y sus próximas/últimas activaciones.

Se posponen:

- anacron en profundidad;

- `/etc/crontab` y `/etc/cron.d` como práctica administrativa;
- timers `systemd` globales con privilegios;
- `WakeSystem=`;
- calendarios avanzados y zonas horarias complejas;
- dependencias avanzadas de unidades;
- sandboxing de servicios `systemd`;
- `systemd-run` y timers transitorios;
- monitoreo centralizado de trabajos.

Estado de la lección: redactada y revisada documentalmente contra Cronie, `systemd` y documentación vigente de RHEL 10. La práctica principal usa scripts no destructivos; la parte de `systemd` se limita a unidades de usuario y se incluye un procedimiento de retirada explícito.

Módulo 31 — Archivos y compresión con `tar`, `gzip` y `xz`

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Trigésima primera entrega.

1. Qué aprenderás

Al terminar este módulo podrás:

- distinguir **archivar** de **comprimir**;
- explicar qué hace `tar` y qué hacen `gzip` y `xz`;
- crear un archivo `.tar` sin borrar los originales;
- listar el contenido de un `.tar` antes de extraerlo;
- crear y consultar archivos `.tar.gz` y `.tar.xz`;
- usar `gzip` y `xz` directamente sobre archivos individuales;
- conservar los archivos originales con `-k/--keep` en las primeras prácticas;
- comprobar archivos comprimidos con `gzip -t` y `xz -t`;
- consultar información con `gzip -l` y `xz -l` cuando corresponda;
- extraer en una carpeta nueva y vacía;
- evitar sobrescribir archivos existentes mediante `--keep-old-files`;
- reconocer el propósito de `--one-top-level`;
- explicar por qué no debemos usar `--absolute-names (-P)` sin comprender sus implicaciones;
- comparar tamaño original y comprimido sin asumir que “más compresión” siempre es mejor;
- reconocer `.tar`, `.tar.gz`, `.tgz` y `.tar.xz` como convenciones distintas.

Conocimientos previos:

- sistema de archivos y rutas;
- creación, copia y borrado seguro;
- permisos;
- pipes y redirecciones;
- estados de salida;
- Bash básico.

Seguridad: todas las prácticas se realizan dentro de `~/linux-lab/modulo-31-archivos`. No se extraen archivos descargados de Internet en directorios personales reales ni del sistema. No se usa `sudo`. Antes de extraer se inspecciona el contenido y se utiliza un destino vacío.

2. Archivar no es comprimir

Estos conceptos son distintos.

Archivar significa reunir varios archivos y directorios en un solo archivo contenedor.

Comprimir significa representar los datos con menos espacio cuando es posible.

Modelo:

```
varios archivos
  ↓
  tar
  ↓
archivo.tar
  ↓
gzip o xz
  ↓
archivo.tar.gz o archivo.tar.xz
```

3. Qué hace `tar`

`tar` crea y manipula archivos de archivo.

Operaciones fundamentales:

```
-c / --create    → crear
-t / --list      → listar
-x / --extract   → extraer
-f / --file      → indicar el archivo de archivo
```

En este manual escribiremos las opciones de forma explícita y consistente.

4. Qué hace `gzip`

`gzip` comprime datos y utiliza normalmente la extensión:

```
.gz
```

Puede comprimir un archivo individual, pero no reemplaza la función de `tar` para agrupar una jerarquía completa en un solo archivo.

5. Qué hace `xz`

`xz` es otra herramienta de compresión.

Su formato nativo usa:

```
.xz
```

También comprime archivos individuales. Para agrupar primero varios archivos o directorios, normalmente se combina con `tar`.

6. Extensiones comunes

Extensión	Significado habitual
.tar	archivo tar sin compresión externa
.tar.gz	archivo tar comprimido con gzip
.tgz	abreviatura convencional de .tar.gz
.tar.xz	archivo tar comprimido con xz
.gz	archivo comprimido con gzip
.xz	archivo comprimido con xz

Una extensión es una convención; no garantiza por sí sola el contenido real.

7. Preparar el laboratorio

```
test -d ~/linux-lab && \  
cd ~/linux-lab && \  
mkdir -p modulo-31-archivos/origen && \  
cd modulo-31-archivos && pwd
```

Etiqueta: **creación en laboratorio**.

Si alguna orden falla, la cadena se detiene: no continúes hasta revisar el mensaje. `test -d` comprueba primero que el laboratorio exista.

8. Crear datos de práctica

```
printf 'línea uno\n' > origen/uno.txt  
printf 'línea dos con espacios\n' > 'origen/dos palabras.txt'  
mkdir -p origen/subcarpeta  
printf 'dato interno\n' > origen/subcarpeta/tres.txt
```

Estas redirecciones sobrescriben únicamente los archivos de práctica con esos nombres dentro del laboratorio.

9. Revisar antes de archivar

```
find origen -maxdepth 3 -print
```

Comprueba que solo aparecen los objetos que esperas.

10. Crear un `.tar`

```
tar -cf practica.tar origen
```

Interpretación:

```
-c → crear archivo  
-f → el siguiente argumento es el nombre del archivo tar  
practica.tar → archivo de salida  
origen → objeto que se archiva
```

Los archivos originales permanecen en origen.

11. Listar antes de extraer

```
tar -tf practica.tar
```

-t lista miembros del archivo.

Esta operación **no extrae** el contenido.

Regla del manual:

Antes de extraer un archivo, inspecciona primero sus miembros.

12. Lista detallada

```
tar -tvf practica.tar
```

-v muestra información adicional como permisos, tamaños y nombres.

Esta vista ayuda a reconocer:

- archivos;
- directorios;
- enlaces;
- rutas inesperadas.

13. Crear un tar comprimido con gzip

GNU tar puede invocar gzip mediante -z:

```
tar -czf practica.tar.gz origen
```

Ahora tienes un archivo tar y compresión gzip en una sola operación.

14. Listar un .tar.gz

```
tar -tzf practica.tar.gz
```

De nuevo: primero listar, después decidir si extraer.

15. Crear un tar comprimido con xz

GNU tar puede usar xz mediante -J:

```
tar -cJf practica.tar.xz origen
```

16. Listar un .tar.xz

```
tar -tJf practica.tar.xz
```

Comprueba que los miembros sean los mismos que esperabas.

17. --auto-compress como alternativa

GNU tar también dispone de -a/--auto-compress para elegir compresión a partir del sufijo al crear ciertos archivos.

Ejemplo:

```
tar -caf practica-auto.tar.xz origen
```

Para aprender, seguiremos mostrando -z y -J porque dejan visible qué compresor interviene.

18. Comparar tamaños

```
ls -lh practica.tar practica.tar.gz practica.tar.xz
```

No esperes siempre grandes diferencias.

Archivos muy pequeños o ya comprimidos pueden reducirse poco o incluso crecer por la sobrecarga del formato.

19. gzip sobre un archivo individual

Primero crea una copia de práctica:

```
cp origen/uno.txt gzip-prueba.txt
```

Para conservar el original:

```
gzip -k gzip-prueba.txt
```

Después deberían existir:

```
gzip-prueba.txt  
gzip-prueba.txt.gz
```

20. Por qué usamos gzip -k

Sin -k/- -keep, gzip normalmente reemplaza el archivo de entrada por su versión comprimida cuando la operación tiene éxito.

En las primeras prácticas conservamos el original para poder comparar y evitar pérdidas accidentales.

21. Probar integridad gzip

```
gzip -t gzip-prueba.txt.gz  
estado=$?  
printf 'Estado gzip -t: %s\n' "$estado"
```

gzip -t comprueba la integridad del archivo comprimido.

Un estado 0 indica que la prueba terminó correctamente.

22. Consultar información gzip

```
gzip -l gzip-prueba.txt.gz
```

Muestra información de tamaño y relación de compresión cuando está disponible.

23. Descomprimir gzip conservando el .gz

```
mkdir -p restauracion-gzip && \  
cp gzip-prueba.txt.gz restauracion-gzip/ && \  
cd restauracion-gzip && \  
gzip -dk gzip-prueba.txt.gz
```

Las órdenes van encadenadas con &&: si una falla, las siguientes no se ejecutan y no trabajarás en una carpeta equivocada.

-d descomprime.

-k conserva el archivo comprimido.

Después vuelve al módulo:

```
cd ..
```

24. xz sobre un archivo individual

```
cp origen/uno.txt xz-prueba.txt  
xz -k xz-prueba.txt
```

Deberían quedar:

```
xz-prueba.txt  
xz-prueba.txt.xz
```

25. Por qué usamos xz -k

Por defecto, xz elimina el archivo de entrada después de una compresión exitosa en los casos normales.

-k/--keep conserva el original.

26. Probar integridad xz

```
xz -t xz-prueba.txt.xz
estado=$?
printf 'Estado xz -t: %s\n' "$estado"
```

27. Consultar información xz

```
xz -l xz-prueba.txt.xz
```

-l/--list muestra información del archivo .xz sin descomprimirlo a un archivo normal.

28. Descomprimir xz conservando el .xz

```
mkdir -p restauracion-xz && \
cp xz-prueba.txt.xz restauracion-xz/ && \
cd restauracion-xz && \
xz -dk xz-prueba.txt.xz && \
cd ..
```

29. Niveles de compresión

gzip y xz permiten seleccionar niveles/presets de compresión.

De forma general:

```
nivel menor → suele usar menos tiempo/recursos
nivel mayor → puede buscar mejor relación de compresión
```

Pero un nivel mayor no garantiza una mejora proporcional.

En xz, presets altos también pueden requerir bastante más memoria.

Para aprendizaje inicial usa los valores predeterminados.

30. Extraer no es una operación de solo lectura

Crear o listar un archivo es distinto de extraerlo.

Al extraer:

- se crean archivos y directorios;
- pueden existir conflictos con nombres ya presentes;
- se restauran metadatos según las reglas y opciones;
- un archivo no confiable puede contener una estructura inesperada.

Por eso la extracción requiere más precauciones.

31. Regla para archivos externos o no confiables

Antes de extraer:

1. identifica el archivo y su procedencia;
2. lista su contenido;
3. revisa rutas y tipos de miembros;
4. recuerda que **listar no certifica que el archivo sea seguro**;
5. crea un directorio nuevo y vacío;
6. asegúrate de que ese directorio y su directorio padre no puedan ser modificados por usuarios no confiables;
7. extrae un archivo no confiable **por separado**, sin mezclarlo con otros archivos no confiables;
8. revisa diagnósticos y estado de salida de tar;
9. inspecciona el resultado, incluidos enlaces y permisos, antes de mover o ejecutar nada.

GNU tar recomienda extraer archivos no confiables en un directorio vacío y controlado cuyo directorio y padre sean accesibles únicamente por usuarios de confianza.

Listar ayuda, pero no prueba seguridad

```
tar -tvf archivo.tar
```

permite observar nombres y tipos de miembros, pero no demuestra por sí solo que la extracción sea segura.

Un archivo puede contener, entre otras cosas:

- enlaces simbólicos;
- permisos inesperados;
- rutas diseñadas para causar conflictos;
- archivos ejecutables;
- estructuras que solo muestran su efecto real al extraerse.

Por eso:

```
listar → inspeccionar
extraer aislado → observar resultado real
ejecutar contenido → decisión separada
```

Un directorio distinto por cada archivo no confiable

GNU tar advierte que dos archivos no confiables no deben extraerse secuencialmente en el mismo árbol de trabajo. Un archivo extraído primero podría dejar enlaces u otra estructura que cambie el efecto de la extracción posterior.

Regla del manual:

un archivo no confiable → un directorio vacío y aislado propio

En nuestras prácticas normales usamos archivos creados por nosotros mismos. No necesitamos descargar contenido externo para aprender esta regla.

32. Crear destino vacío

Para nuestra práctica **con un archivo creado por nosotros**:

```
mkdir -p extraccion-segura
```

Comprueba:

```
ls -la extraccion-segura
```

Debe estar vacío salvo `.` y `...`

Si el archivo fuera realmente no confiable, además del directorio vacío tendrías que confirmar que usuarios no confiables no puedan modificar ese directorio ni su padre mientras ocurre la extracción. Esa verificación de permisos se estudia aquí como concepto; no necesitamos practicarla con contenido externo.

33. Extraer con `-c`

```
tar -xzf practica.tar.gz -C extraccion-segura
```

`-C` cambia el directorio de extracción para esa operación.

Después:

```
find extraccion-segura -maxdepth 4 -print
```

34. Extraer `.tar.xz`

En otra carpeta vacía:

```
mkdir -p extraccion-xz
tar -xJf practica.tar.xz -C extraccion-xz
```

35. Evitar reemplazar archivos existentes

GNU tar ofrece:

```
--keep-old-files
```

o su forma corta:

```
-k
```

durante extracción.

Si un miembro intentaría reemplazar un archivo existente, tar se niega y lo trata como error.

Ejemplo:

```
tar -xzf practica.tar.gz --keep-old-files -C extraccion-segura
```

Como extraccion-segura ya contiene la extracción de la sección 33, este ejemplo terminará con errores `File exists`: es justo lo que debe ocurrir. La Práctica F (sección 48) trabaja ese conflicto paso a paso.

36. --skip-old-files

También existe:

```
--skip-old-files
```

que omita archivos ya existentes sin tratarlos como error.

Para aprendizaje inicial preferimos `--keep-old-files` porque hace visible el conflicto.

37. --one-top-level

GNU tar dispone de:

```
--one-top-level
```

para crear una carpeta superior durante extracción y reducir el riesgo de un archivo tipo “tarbomb” que disperse muchos miembros en el directorio actual.

Ejemplo conceptual:

```
tar -xzf archivo.tar.gz --one-top-level
```

Seguimos prefiriendo además una carpeta de destino dedicada y vacía.

38. No usar --absolute-names (-P) por rutina

GNU tar normalmente aplica protecciones respecto a nombres absolutos.

`--absolute-names (-P)` cambia ese comportamiento y puede permitir que miembros del archivo se refieran a rutas fuera del directorio previsto.

Este manual **no lo usa** en prácticas.

39. Opciones de alto riesgo que no usamos como rutina

Evita copiar sin comprender opciones como:

```
--absolute-names  
--overwrite  
--recursive-unlink  
--remove-files  
--dereference
```

Pueden cambiar significativamente qué archivos se leen, eliminan o sobrescriben.

40. Archivos y enlaces simbólicos

Un archivo tar puede contener enlaces simbólicos.

Por eso:

```
tar -tvf archivo.tar
```

no sirve solo para ver nombres; también ayuda a reconocer tipos de miembros.

Con archivos externos, inspecciona enlaces antes de confiar en el resultado extraído.

Importante: revisar la lista no elimina todos los riesgos asociados a enlaces. GNU tar advierte que, durante extracción en un árbol que otros usuarios puedan modificar, un directorio podría cambiar por un enlace simbólico y alterar el destino efectivo de escritura.

La defensa principal no es “mirar mejor la lista”, sino combinar:

```
directorio vacío
+ padre controlado por usuarios de confianza
+ una extracción aislada por archivo no confiable
+ revisión posterior
```

No extraigas material no confiable como administrador si no existe una necesidad concreta y un entorno preparado para ello.

41. Seleccionar un miembro concreto

Después de listar el nombre exacto, puedes extraer un miembro específico.

Ejemplo dentro de una carpeta vacía:

```
mkdir -p extraccion-selectiva
tar -xzf practica.tar.gz -C extraccion-selectiva origen/uno.txt
```

Esto evita extraer miembros que no necesitas.

42. Extraer no significa «abrir»

Un archivo comprimido o archivado puede contener código, documentos, imágenes o cualquier otro dato.

Extraerlo solo restaura los miembros.

No ejecutes automáticamente archivos obtenidos de un archivo comprimido.

43. Práctica A — crear y listar .tar

```
tar -cf practica-a.tar origen
tar -tf practica-a.tar
```

Explica qué hace cada opción.

44. Práctica B — gzip con original conservado

```
cp origen/uno.txt ejercicio-gzip.txt
gzip -k ejercicio-gzip.txt
gzip -t ejercicio-gzip.txt.gz
ls -lh ejercicio-gzip.txt ejercicio-gzip.txt.gz
```

45. Práctica C — xz con original conservado

```
cp origen/uno.txt ejercicio-xz.txt
xz -k ejercicio-xz.txt
xz -t ejercicio-xz.txt.xz
ls -lh ejercicio-xz.txt ejercicio-xz.txt.xz
```

46. Práctica D — crear .tar.gz y .tar.xz

```
tar -czf copia.tar.gz origen
tar -cJf copia.tar.xz origen
```

Después:

```
tar -tzf copia.tar.gz
tar -tJf copia.tar.xz
```

47. Práctica E — extraer en destino vacío

```
mkdir -p restaurar-practica
tar -xzf copia.tar.gz --keep-old-files -C restaurar-practica
find restaurar-practica -maxdepth 4 -print
```

No extraigas encima de origen.

48. Práctica F — comprobar conflicto

Dentro de una carpeta de laboratorio separada, primero extrae una vez.

Después intenta repetir:

```
tar -xzf copia.tar.gz --keep-old-files -C restaurar-practica
```

Ahora tar debe advertir de miembros existentes en vez de reemplazarlos silenciosamente.

Esto es un fallo **controlado** de aprendizaje.

49. Errores frecuentes

Error	Problema	Corrección
creer que tar comprime por sí solo en todos los casos	archivar y comprimir son conceptos distintos	identifica tar + compresor
extraer antes de listar	no sabes qué se intentará escribir	usa tar -t... primero
creer que listar certifica seguridad	la lista no revela todas las consecuencias posibles	inspecciona, pero extrae aislado y revisa después
extraer en tu directorio personal	puede mezclar/sobrescribir archivos	usa carpeta vacía y controlada
reutilizar la misma carpeta para varios archivos no confiables	una extracción anterior puede afectar la siguiente	usa un directorio aislado distinto por archivo
usar gzip/xz sin saber que modifican entrada	puedes perder el original	usa -k durante aprendizaje
asumir extensión = formato real	nombres pueden engañar	inspecciona y prueba
usar -P por copiar un ejemplo	desactiva protecciones de nombres	no usar sin motivo y archivo confiable
usar --overwrite por defecto	permite reemplazos	usa destino vacío / --keep-old-files
comprimir un directorio directamente con gzip	gzip no crea archivo jerárquico	tar primero, luego gzip
creer que nivel 9 siempre es mejor	puede gastar tiempo/memoria por poco beneficio	usa predeterminado salvo necesidad
ejecutar contenido recién extraído	aumenta riesgo	inspecciona antes

50. Detección de error 1

Analiza:

```
tar -xzf descarga.tar.gz
```

si descarga.tar.gz es externa y estás en tu carpeta personal.

Problemas:

- no se inspeccionó;
- destino no está aislado;
- podrían existir conflictos.

Flujo recomendado:

```
tar -tzf descarga.tar.gz
recordar que listar no certifica seguridad
crear carpeta nueva/vacía y controlada
usar una carpeta distinta para cada archivo no confiable
extraer sin opciones de alto riesgo
revisar diagnósticos y estado de salida
inspeccionar enlaces, permisos y contenido
no ejecutar automáticamente nada extraído
```

51. Detección de error 2

Analiza:

```
gzip documento.txt
```

Si necesitabas conservar documento.txt, el comando no expresa esa intención.

Para práctica:

```
gzip -k documento.txt
```

52. Detección de error 3

Analiza:

```
tar -xPf archivo.tar
```

-P permite nombres absolutos y cambia protecciones importantes.

Corrección inicial:

```
tar -tf archivo.tar
```

y, si decides extraer, hacerlo en un destino controlado sin -P.

53. Método seguro para trabajar con archivos comprimidos

1. identifica el archivo y su procedencia;
2. conserva el original cuando estés aprendiendo;
3. lista el contenido de archivos tar;
4. recuerda que listar no certifica seguridad;
5. verifica integridad cuando la herramienta lo permita;
6. crea un destino nuevo, vacío y controlado;
7. usa un destino distinto por cada archivo no confiable;
8. evita sobrescrituras;
9. no uses opciones peligrosas por copiar recetas;
10. revisa diagnósticos y estado de salida;
11. inspecciona enlaces, permisos y contenido extraído;
12. no ejecute contenido automáticamente.

54. Práctica independiente

Crea un script empaquetar_practica.sh que:

1. use Bash;
2. reciba como argumento un directorio existente dentro del laboratorio;
3. valide que sea directorio;
4. cree un .tar.gz en la carpeta del módulo;
5. no borre ni modifique el directorio original;
6. liste el archivo creado con tar -tzf;
7. devuelva estado no-cero si falla la creación o el listado;
8. use quoting correcto;
9. pase bash -n;
10. pase ShellCheck si está disponible;
11. pueda explicarse línea por línea.

55. Mini evaluación

1. ¿archivar y comprimir son exactamente lo mismo? A) Sí B) No
2. ¿tar -c crea un archivo? A) Sí B) No
3. ¿tar -t lista miembros? A) Sí B) No
4. ¿tar -x extrae? A) Sí B) No
5. ¿debes listar un archivo externo antes de extraerlo? A) Sí B) No
6. ¿gzip agrupa por sí solo un árbol completo como tar? A) Sí B) No
7. ¿gzip -k conserva el original? A) Sí B) No
8. ¿xz -k conserva el original? A) Sí B) No
9. ¿gzip -t comprueba integridad? A) Sí B) No
10. ¿xz -t comprueba integridad? A) Sí B) No
11. ¿es mejor extraer un archivo no confiable en una carpeta vacía? A) Sí B) No
12. ¿listar con tar -t... demuestra por sí solo que el archivo sea seguro? A) Sí B) No
13. ¿conviene extraer dos archivos no confiables distintos en el mismo directorio? A) Sí B) No
14. ¿el directorio de extracción y su padre deben estar protegidos frente a modificaciones de usuarios no confiables? A) Sí B) No
15. ¿--keep-old-files evita reemplazar archivos existentes? A) Sí B) No
16. ¿--one-top-level ayuda frente a tarbombs? A) Sí B) No
17. ¿-P/--absolute-names debe usarse por defecto? A) Sí B) No
18. ¿una extensión garantiza completamente el formato real? A) Sí B) No

56. Registro de aprendizaje

```
Archivar significa:
Comprimir significa:
`tar -c` sirve para:
`tar -t` sirve para:
`tar -x` sirve para:
`-f` sirve para:
`-z` relaciona tar con:
`-J` relaciona tar con:
`gzip -k` sirve para:
`xz -k` sirve para:
`gzip -t` sirve para:
`xz -t` sirve para:
¿por qué extraigo en carpeta vacía?:
¿por qué listar no certifica seguridad?:
¿por qué uso una carpeta distinta por archivo no confiable?:
¿por qué también importa proteger el directorio padre?:
`--keep-old-files` hace:
`--one-top-level` ayuda a:
¿por qué evitamos `-P`?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

57. Puerta de dominio

Para considerar este módulo **DOMINADO** deberás poder en más de una ocasión:

1. explicar sin copiar la diferencia entre archivar y comprimir;
2. crear y listar un .tar;
3. crear un .tar.gz y un .tar.xz;
4. conservar originales al practicar con gzip y xz;
5. verificar un .gz y un .xz;
6. explicar por qué inspeccionar no equivale a certificar seguridad;
7. extraer a una carpeta vacía y controlada sin sobrescribir originales;
8. explicar por qué archivos no confiables distintos deben aislarse en destinos separados;
9. identificar por qué -P o --overwrite requieren especial precaución.

58. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- GNU tar Manual: <https://www.gnu.org/software/tar/manual/tar.html>
- GNU tar — opciones de extracción y archivos existentes: https://www.gnu.org/software/tar/manual/html_section/extract-options.html
- GNU tar — Reliability and Security: https://www.gnu.org/software/tar/manual/html_chapter/Reliability-and-security.html
- GNU Gzip Manual: <https://www.gnu.org/software/gzip/manual/gzip.html>
- XZ Utils — xz(1): <https://tukaani.org/xz/man/xz.1.html>

Puntos verificados documentalmente:

- GNU tar separa operaciones de creación (--create), listado (--list) y extracción (--extract);
- --keep-old-files impide reemplazar archivos existentes y trata el conflicto como error;
- --one-top-level crea un directorio superior durante extracción y puede proteger frente a tarbombs;
- GNU tar recomienda extraer archivos no confiables en un directorio vacío; ese directorio y su padre deben ser accesibles solo para usuarios de confianza;
- GNU tar recomienda extraer archivos no confiables distintos de forma independiente, en directorios vacíos diferentes;
- listar miembros ayuda a inspeccionar, pero no certifica que la extracción sea segura;
- GNU tar recomienda prestar atención a diagnósticos y estado de salida;
- GNU tar desaconseja opciones de riesgo como --absolute-names, --dereference, --overwrite, --recursive-unlink y --remove-files salvo comprensión explícita;
- el manual oficial de GNU gzip (versión 1.15, 3 de enero de 2026; consultado el 9 de octubre de 2026) documenta compresión y descompresión de archivos y dispone de --keep y --test;
- XZ Utils documenta -k/--keep, -d/--decompress, -l/--list y -t/--test;
- xz utiliza por defecto el formato .xz y recomienda xz -d/xz -dc en scripts en lugar de depender de alias como unxz o xzcat.

Se posponen:

- archivos incrementales de tar;
- ACL, xattrs y SELinux dentro de tar;
- formatos pax, ustar y GNU en profundidad;
- --transform y --strip-components;
- cifrado (tar/gzip/xz no proporcionan cifrado por sí solos);

- zstd, bzip2 y zip;
- recuperación forense de archivos dañados;
- procesamiento masivo paralelo.

Estado de la lección: redactada y revisada documentalmente contra GNU tar, GNU Gzip y XZ Utils. Las prácticas conservan originales y extraen únicamente en destinos controlados.

1. Qué aprenderás

Al terminar este módulo podrás:

- explicar qué hace `rsync` y qué no hace;
- distinguir copia, sincronización, espejo y respaldo con historial;
- comprobar si `rsync` está instalado y conocer su versión;
- realizar copias locales seguras;
- explicar la diferencia entre `origen` y `origen/`;
- usar `-a` comprendiendo qué conserva y qué no;
- usar `-v`, `-i`, `--stats` y `--dry-run` para revisar operaciones;
- excluir archivos mediante `--exclude`;
- entender cómo decide `rsync` si un archivo necesita actualizarse;
- reconocer cuándo `--checksum` puede ser útil y su coste;
- verificar una restauración en una carpeta separada;
- comprender el riesgo de `--delete` y simularlo antes de cualquier uso real;
- explicar por qué sincronización no sustituye automáticamente una política de backups;
- reconocer el uso remoto de `rsync` sobre SSH únicamente en equipos propios o autorizados;
- diseñar una copia reproducible sin sobrescribir datos de práctica innecesariamente.

Conocimientos previos:

- rutas absolutas y relativas;
- archivos y permisos;
- SSH en sistemas autorizados;
- compresión y restauración;
- estados de salida;
- Bash y quoting.

Seguridad: las prácticas se realizan únicamente dentro de `~/linux-lab/modulo-32-rsync`. No se sincronizan carpetas personales reales, unidades externas, servidores, configuraciones del sistema ni rutas remotas. `--delete` se estudia primero y se practica únicamente con `--dry-run` en este módulo.

2. Qué es `rsync`

`rsync` es una herramienta para transferir y sincronizar archivos de forma incremental.

Puede trabajar:

- entre dos rutas locales;
- entre una máquina local y otra remota;
- mediante un daemon `rsync`;
- mediante un shell remoto como SSH.

En este módulo la práctica principal será **local**.

3. Qué significa transferencia incremental

Después de una primera copia, `rsync` intenta evitar transferir de nuevo archivos que considera ya actualizados.

Modelo:

```
primera ejecución → copia inicial
segunda ejecución → compara
                    → transfiere solo lo necesario
```

Eso puede ahorrar tiempo y transferencia.

4. Copia no es lo mismo que respaldo histórico

Si sincronizas:

```
origen → destino
```

el destino puede terminar reflejando el estado actual del origen.

Pero un respaldo con historial suele necesitar además:

- versiones anteriores;
- snapshots;
- retención;
- protección frente a borrados accidentales;
- pruebas de restauración.

rsync puede formar parte de un sistema de backups, pero por sí solo no garantiza esas propiedades.

5. Sincronización y espejo

Una **sincronización** puede copiar cambios del origen al destino.

Un **espejo** intenta que el destino se parezca aún más al origen, incluyendo eventualmente eliminar elementos sobrantes mediante opciones como `--delete`.

Esa diferencia es crítica.

6. Comprobar si rsync está instalado

```
command -v rsync
```

Después:

```
rsync --version
```

En octubre de 2026, la versión oficial más reciente publicada por el proyecto rsync es 3.5.1. Eso **no significa** que tu distribución tenga que usar esa misma versión; primero revisa la versión instalada y el soporte de tu distribución.

7. Preparar el laboratorio

```
test -d ~/linux-lab && \
cd ~/linux-lab && \
mkdir -p modulo-32-rsync/origen && \
mkdir -p modulo-32-rsync/destino && \
mkdir -p modulo-32-rsync/restauracion && \
cd modulo-32-rsync && pwd
pwd
```

Etiqueta: **creación en laboratorio**.

Si alguna orden falla, la cadena se detiene: no continúes hasta revisar el mensaje. `test -d` comprueba primero que el laboratorio exista.

8. Crear datos de práctica

```
printf 'uno\n' > origen/uno.txt
printf 'dos palabras\n' > 'origen/dos palabras.txt'
mkdir -p origen/subcarpeta
printf 'tres\n' > origen/subcarpeta/tres.txt
```

Comprueba:

```
find origen -maxdepth 3 -print
```

9. Primera copia simple

```
rsync -av origen/ destino/
```

Explicación inicial:

```
-a → modo archivo
-v → salida más detallada
origen/ → contenido del directorio origen
destino/ → directorio receptor
```

10. La barra final cambia el significado

Este es uno de los puntos más importantes de rsync.

Con:

```
rsync -a origen/ destino/
```

se copian **los contenidos de origen** dentro de destino.

Con:

```
rsync -a origen destino/
```

se copia el directorio origen como un elemento dentro de destino.

Conceptualmente:

```
origen/ → copia contenido  
origen → copia el directorio contenedor
```

11. Verificar el resultado

```
find destino -maxdepth 3 -print
```

No confíes únicamente en que rsync terminó sin mensajes de error. Revisa la estructura.

12. Qué incluye -a

-a/--archive equivale a:

```
-r recursión  
-l enlaces simbólicos  
-p permisos  
-t tiempos de modificación  
-g grupo  
-o propietario  
-D archivos de dispositivo/especiales
```

De forma compacta:

```
-a = -rlptgoD
```

13. Qué NO incluye -a

-a no implica automáticamente, entre otros:

```
-A → ACL  
-X → atributos extendidos  
-H → conservación de relaciones de hardlinks
```

Tampoco implica conservar todos los metadatos posibles de todos los sistemas.

Por eso «usar -a» no significa «copia perfecta de cualquier filesystem».

14. Usuario normal y propietario/grupo

Opciones como conservación de propietario o grupo están sujetas a permisos y capacidades del sistema.

En nuestras prácticas todos los archivos pertenecen al usuario del laboratorio.

No usaremos sudo para forzar metadatos.

15. Simular antes de cambiar

--dry-run o -n realiza una prueba sin aplicar cambios de transferencia.

```
rsync -av --dry-run origen/ destino/
```

Es una de las herramientas más importantes antes de una sincronización sensible.

16. --dry-run no sustituye la revisión

Una simulación muestra qué **pretende** hacer rsync según las opciones actuales.

Aun debes revisar:

- origen;
- destino;
- barra final;
- exclusiones;
- opciones de borrado;
- sistema remoto, si existiera.

17. --itemize-changes (-i)

Para ver cambios con más detalle:

```
rsync -ai --dry-run origen/ destino/
```

-i muestra una representación compacta de lo que cambiaría.

Para aprendizaje, combínalo con --dry-run.

18. --stats

```
rsync -a --stats origen/ destino/
```

muestra estadísticas de la transferencia.

Puede ayudarte a distinguir cuántos archivos se examinaron y transfirieron.

19. Segunda ejecución sin cambios

Ejecuta de nuevo:

```
rsync -avi origen/ destino/
```

Como el destino ya debería estar actualizado, la transferencia será mínima o nula.

Esto ilustra el comportamiento incremental.

20. Modificar un archivo de práctica

```
printf 'línea nueva\n' >> origen/uno.txt
```

Después primero simula:

```
rsync -avi --dry-run origen/ destino/
```

Revisa qué archivo cambiaría.

21. Aplicar la actualización

Si la simulación coincide con tu intención:

```
rsync -avi origen/ destino/
```

Después:

```
cat destino/uno.txt
```

22. Cómo decide normalmente rsync

Sin --checksum, rsync normalmente utiliza una comprobación rápida basada principalmente en:

- tamaño;
- tiempo de modificación.

Si esos datos indican que el archivo está actualizado, puede omitirlo sin leer todo su contenido.

23. --checksum (-c)

Con:

```
rsync -ac origen/ destino/
```

rsync decide si necesita actualizar archivos comparando checksums de contenido en lugar de la comprobación rápida habitual.

Esto requiere leer los datos de ambos lados y puede ser mucho más costoso.

No lo uses automáticamente en todas las copias.

24. Checksum para decisión vs verificación de transferencia

La opción `--checksum` afecta principalmente la decisión de **si un archivo necesita actualizarse**.

Es distinta de las comprobaciones que rsync usa normalmente para verificar datos que realmente transfirió.

No interpretes `-c` como una garantía universal de backup correcto.

25. Excluir archivos

Ejemplo:

```
printf 'temporal\n' > origen/no-copiar.tmp
```

Simula:

```
rsync -avi --dry-run --exclude='*.tmp' origen/ destino/
```

El archivo `.tmp` debería quedar fuera de la transferencia.

26. Las exclusiones también importan con borrado

Las reglas de inclusión/exclusión pueden interactuar con opciones de eliminación.

No mezcles `--delete`, `--delete-excluded` y filtros complejos sin revisar exactamente qué quedará protegido o en riesgo.

En este módulo no usamos `--delete-excluded`.

27. Qué hace `--delete`

`--delete` elimina del destino ciertos archivos que ya no existen en el conjunto de origen.

Su propósito es acercar el destino a un espejo.

Eso significa que **puede borrar datos del destino**.

28. Regla permanente para `--delete`

Antes de cualquier uso real:

1. confirma origen;
2. confirma destino;
3. confirma la barra final;
4. revisa filtros;
5. usa `--dry-run`;
6. usa `-i`;
7. revisa cada borrado propuesto;
8. conserva otro respaldo si el contenido importa.

29. Práctica de `--delete` SOLO en simulación

Crea un archivo únicamente en el destino:

```
printf 'solo destino\n' > destino/sobrante.txt
```

Ahora:

```
rsync -avi --dry-run --delete origen/ destino/
```

Observa que rsync propondría eliminar `sobrante.txt`.

No retires `--dry-run` en esta práctica.

30. Variantes de eliminación

rsync ofrece opciones como:

```
--delete-before
--delete-during
--delete-delay
--delete-after
```

Cambian en qué momento se realizan eliminaciones.

No necesitas memorizarlas todavía.

La opción importante para este nivel es comprender que todas implican riesgo de borrado.

31. --remove-source-files no es una opción de backup

--remove-source-files elimina archivos del lado emisor después de una transferencia exitosa.

Este manual **no la usa** en las prácticas de copia.

Convertir una copia en un movimiento cambia completamente el riesgo.

32. Restaurar significa probar el camino inverso

Una copia no está completamente demostrada hasta que puedes recuperar datos.

En lugar de restaurar sobre origen, usaremos:

```
restauracion/
```

33. Restauración de prueba

Primero verifica que la carpeta esté vacía:

```
ls -la restauracion
```

Después simula:

```
rsync -avi --dry-run destino/ restauracion/
```

Si es correcto:

```
rsync -avi destino/ restauracion/
```

34. Verificar la restauración

```
find restauracion -maxdepth 3 -print
```

Después compara un archivo:

```
cmp origen/uno.txt restauracion/uno.txt
```

cmp termina sin salida cuando los archivos comparados son iguales.

35. Comparación con hashes

También puedes calcular:

```
sha256sum origen/uno.txt restauracion/uno.txt
```

Si ambos hashes son iguales, el contenido de esos archivos coincide para esta comprobación.

No necesitas usar hashes para cada copia normal, pero son útiles en ejercicios de verificación.

36. Verificación del árbol completo: concepto

Comparar únicamente un archivo no demuestra que todo el árbol esté correcto.

Una política real de backup debe definir:

- qué datos se incluyen;
- qué metadatos importan;
- cómo se verifica;
- con qué frecuencia se restaura de prueba;

- cuánto historial se conserva.

37. Copia local a otra carpeta no protege de todo

Si origen y destino están en el mismo disco:

- un fallo físico del disco puede afectar ambos;
- ransomware o errores del usuario pueden alcanzar ambos;
- un borrado con privilegios suficientes puede afectar ambos.

El laboratorio enseña la herramienta; no representa por sí mismo una estrategia completa de resiliencia.

38. Remoto sobre SSH: concepto

En sistemas propios o expresamente autorizados, rsync puede usar SSH.

Forma general:

```
rsync opciones origen/ usuario@host:ruta/
```

o en dirección inversa:

```
rsync opciones usuario@host:ruta/ destino/
```

No practicaremos con hosts externos en este módulo.

39. -e ssh

Rsync puede especificar el shell remoto:

```
rsync -a -e ssh origen/ usuario@host:ruta/
```

En muchos usos remotos modernos SSH ya es el transporte esperado.

No insertes contraseñas, tokens ni claves privadas en comandos o scripts.

40. -z en transferencias

-z/--compress comprime datos durante la transferencia.

Puede ayudar en redes lentas con datos compresibles.

Puede aportar poco o incluso consumir recursos innecesarios para:

- archivos ya comprimidos;
- transferencias locales;
- redes muy rápidas.

No confundas rsync -z con crear un archivo .gz permanente.

41. No versionar secretos

Si preparas scripts de rsync para GitHub, no incluyas:

- contraseñas;
- claves SSH privadas;
- tokens;
- cookies;
- archivos .env;
- nombres de host privados si revelan información sensible;
- datos personales innecesarios.

42. Enlaces simbólicos

-a incluye -l, por lo que conserva enlaces simbólicos como enlaces.

Otras opciones pueden cambiar cómo se tratan los enlaces.

No uses opciones que sigan enlaces fuera del árbol sin comprender su efecto.

43. --copy-links y riesgos de alcance

--copy-links (-L) transforma enlaces simbólicos encontrados en los objetos a los que apuntan.

Eso puede hacer que el conjunto copiado sea distinto del esperado.

Este manual no lo utiliza en las prácticas.

44. Hardlinks

-a no incluye -H.

Si necesitas preservar relaciones de hardlinks, existe:

```
-H / --hard-links
```

Puede aumentar el uso de memoria y tiempo.

No lo activamos sin una necesidad concreta.

45. ACL y atributos extendidos

Si el entorno requiere conservarlos, rsync dispone de:

```
-A → ACL  
-X → xattrs
```

No todas las copias necesitan esos metadatos, pero debes decidirlo explícitamente en backups reales.

46. Evitar escribir en el destino equivocado

Antes de ejecutar:

```
pwd  
find origen -maxdepth 2 -print  
find destino -maxdepth 2 -print
```

Después ejecuta primero:

```
rsync -avi --dry-run origen/ destino/
```

El nombre destino no es una protección: verifica su ruta real.

47. Práctica A — copia inicial

```
rsync -avi --dry-run origen/ destino/  
rsync -avi origen/ destino/  
find destino -maxdepth 3 -print
```

48. Práctica B — cambio incremental

```
printf 'cambio\n' >> origen/uno.txt  
rsync -avi --dry-run origen/ destino/
```

Predice qué archivo aparecerá como cambiado.

Después aplica la sincronización.

49. Práctica C — exclusión

```
printf 'temporal\n' > origen/ignorar.tmp  
rsync -avi --dry-run --exclude='*.tmp' origen/ destino/
```

Explica por qué ignorar.tmp no se transferiría.

50. Práctica D — restauración

```
rsync -avi --dry-run destino/ restauracion/  
rsync -avi destino/ restauracion/  
cmp origen/uno.txt restauracion/uno.txt
```


Si `cmp` no produce salida y devuelve 0, esos dos archivos coinciden.

51. Práctica E — observar `--delete` sin borrar

```
printf 'solo destino\n' > destino/sobran.te.txt
rsync -avi --dry-run --delete origen/ destino/
```

Identifica exactamente qué propondría eliminar.

No ejecute la misma orden sin `--dry-run` en este ejercicio.

52. Errores frecuentes

Error	Problema	Corrección
olvidar la barra final del origen	cambia la estructura copiada	predice antes origen vs origen/
usar <code>--delete</code> directamente	puede borrar datos del destino	primero <code>--dry-run -i</code>
creer que sincronización = backup histórico	puede propagar pérdidas	diseña retención/versiones aparte
asumir que <code>-a</code> conserva todo	no incluye ACL/xattrs/hardlinks	define metadatos requeridos
usar <code>-c</code> siempre	fuerza lectura de contenido y cuesta más	úsalo cuando haya razón
usar <code>-z</code> localmente por costumbre	puede añadir CPU sin beneficio	evalúa transporte/datos
restaurar encima del origen	puedes reemplazar datos	restaura primero en carpeta separada
usar rutas remotas sin autorización	acceso indebido	solo equipos propios/autorizados
incluir secretos en scripts	riesgo de exposición	usa gestión segura y no GitHub
usar <code>--remove-source-files</code> en un backup	elimina el origen	no usar en este módulo

53. Detección de error 1

Analiza:

```
rsync -a origen destino/
```

Si pretendías copiar **el contenido** de origen directamente dentro de destino, falta la barra final.

Corrección:

```
rsync -a origen/ destino/
```

54. Detección de error 2

Analiza:

```
rsync -a --delete ~/Documentos/ /mnt/copia/
```

sin haber verificado rutas ni hecho simulación.

Problema: `--delete` puede eliminar elementos del destino.

Primer paso correcto:

```
verificar origen y destino
realizar --dry-run -i
revisar cada eliminación propuesta
```

No uses esa ruta real como práctica.

55. Detección de error 3

Analiza:

```
rsync -a destino/ origen/
```

Si querías hacer backup pero invertiste origen y destino, puedes propagar una versión antigua hacia el lugar equivocado.

Regla:

lee siempre la orden como «desde ORIGEN hacia DESTINO» antes de ejecutarla.

56. Método seguro del manual

1. identifica origen y destino;
2. confirma si quieres el directorio o su contenido;
3. inspecciona ambos lados;
4. usa `--dry-run`;
5. usa `-i` para entender cambios;
6. evita opciones de borrado durante aprendizaje;
7. aplica la copia solo cuando la simulación sea correcta;
8. verifica el resultado;
9. realiza una restauración de prueba;
10. documenta qué metadatos y exclusiones forman parte del diseño.

57. Práctica independiente

Crea `copia_laboratorio.sh`.

Debe:

1. usar Bash;
2. aceptar dos argumentos: origen y destino;
3. comprobar que el origen sea un directorio;
4. comprobar que el destino esté dentro de `~/linux-lab/modulo-32-rsync`;
5. mostrar primero la orden en modo `--dry-run`;
6. requerir que el estudiante ejecute la transferencia real por separado, no automáticamente después de la simulación;
7. usar `-a`, `-i` y quoting correcto;
8. no incluir `--delete`;
9. no usar `sudo`;
10. pasar `bash -n`;
11. pasar ShellCheck si está disponible;
12. explicar la diferencia entre `origen` y `origen/`.

58. Mini evaluación

1. ¿rsync puede hacer transferencias incrementales? A) Sí B) No
2. ¿origen/ y origen significan exactamente lo mismo? A) Sí B) No
3. ¿--dry-run aplica cambios reales? A) Sí B) No
4. ¿-i ayuda a identificar cambios? A) Sí B) No
5. ¿-a incluye ACL automáticamente? A) Sí B) No
6. ¿-a incluye xattrs automáticamente? A) Sí B) No
7. ¿-a incluye preservación de hardlinks automáticamente? A) Sí B) No
8. ¿--checksum puede requerir leer más datos? A) Sí B) No
9. ¿--delete puede borrar archivos del destino? A) Sí B) No
10. ¿debes usar --delete sin simulación en este módulo? A) Sí B) No
11. ¿sincronización simple garantiza historial de versiones? A) Sí B) No
12. ¿una restauración de prueba ayuda a validar un backup? A) Sí B) No
13. ¿-z crea necesariamente un .gz permanente? A) Sí B) No
14. ¿rsync remoto debe limitarse a sistemas propios/autorizados? A) Sí B) No
15. ¿es importante comprobar la dirección origen → destino? A) Sí B) No

59. Registro de aprendizaje

```
`rsync` sirve para:
Transferencia incremental significa:
Diferencia entre copia y backup histórico:
`origen/` significa:
`origen` sin barra significa:
`-a` incluye:
`-a` NO incluye:
`--dry-run` sirve para:
`-i` sirve para:
`--checksum` cambia:
`--exclude` sirve para:
`--delete` hace:
¿por qué `--delete` es peligroso?:
¿por qué restauramos en otra carpeta?:
Dirección de una orden rsync:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

60. Puerta de dominio

Para considerar este módulo **DOMINADO** deberás poder en más de una ocasión:

1. predecir la diferencia entre origen y origen/;
2. realizar una simulación y explicar sus cambios;
3. ejecutar una sincronización local sin opciones de borrado;
4. explicar qué conserva -a y qué no;
5. usar una exclusión sencilla;
6. explicar por qué --delete requiere revisión especial;
7. restaurar hacia una carpeta separada;
8. verificar al menos un archivo restaurado;
9. explicar por qué rsync solo no constituye necesariamente un sistema completo de backups.

61. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- Sitio oficial de rsync: <https://rsync.samba.org/>
- Anuncio original de rsync 3.5.1, 21 de septiembre de 2026: <https://lists.samba.org/archive/rsync/2026-September/033395.html>

Fecha de verificación de la versión: 8 de octubre de 2026. La página oficial publica 3.5.1 (21-09-2026). No confundir versión upstream con la disponible en repositorios de Debian, Ubuntu, Fedora o RHEL; comprobar siempre rsync --version y los avisos de seguridad de la distribución. - Manual oficial rsync(1): <https://rsync.samba.org/ftp/rsync/rsync.1>

Puntos verificados documentalmente:

- rsync proporciona transferencia incremental de archivos;
- --dry-run realiza una simulación sin efectuar cambios;
- -a/--archive equivale a -rlptgoD y no incluye -A, -X ni -H;
- la barra final en una ruta de origen cambia si se transfiere el directorio contenedor o su contenido;
- --delete elimina archivos sobrantes del lado receptor según el conjunto y reglas aplicables;
- --checksum cambia la selección de archivos hacia comprobación por checksum en lugar de la comprobación rápida habitual;
- -z/--compress comprime datos durante la transferencia y no significa crear automáticamente un archivo .gz;
- la versión oficial más reciente publicada al 8 de octubre de 2026 es rsync 3.5.1, mientras que la versión instalada depende de la distribución.

Se posponen:

- daemon rsync y rsyncd.conf;
- --link-dest para snapshots basados en hardlinks;
- backups incrementales con retención;
- --backup y --backup-dir como estrategia;
- filtros avanzados y archivos de reglas;
- --files-from;
- límites de filesystem y dispositivos con -x;
- seguridad avanzada de symlinks;
- automatización remota;
- pruebas de recuperación ante desastre.

Estado de la lección: redactada y revisada documentalmente contra el manual oficial de rsync 3.5.1. Las prácticas son locales y --delete se limita a simulación con --dry-run.

Módulo 33 — Git y GitHub para scripts; puente al itinerario específico de Git

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Trigésima tercera entrega.

1. Propósito de este módulo

Este módulo no sustituye al **Manual Maestro de Git y GitHub — Edición 2026**.

Su función es darte autonomía suficiente para guardar de forma segura tus scripts de Linux/Bash en Git y GitHub sin duplicar todo el itinerario específico de Git.

Objetivo de evidencia:

Revisar diferencias y guardar commits sin secretos.

2. Qué aprenderás

Al terminar este módulo podrás:

- explicar la diferencia entre Git y GitHub;
- reconocer repositorio, working tree, staging area e historial;
- comprobar el estado con `git status`;
- revisar cambios con `git diff`;
- preparar cambios con `git add`;
- revisar exactamente lo que entrará al commit con `git diff --staged`;
- crear commits pequeños con mensajes claros;
- consultar historial con `git log`;
- entender qué hace `.gitignore` y qué no hace;
- detectar archivos sensibles antes de añadirlos;
- verificar un remoto con `git remote -v`;
- distinguir commit local de push a GitHub;
- realizar un push solo cuando el repositorio y la rama sean correctos;
- usar `git restore --staged` para sacar un archivo del staging sin borrar el archivo de trabajo;
- comprender por qué `git restore` puede descartar cambios y debe revisarse antes de usarlo;
- evitar comandos destructivos o de reescritura de historial mientras aprendes.

3. Git y GitHub no son lo mismo

Git es un sistema de control de versiones distribuido.

GitHub es una plataforma que puede alojar repositorios Git y añadir colaboración, revisión, issues, pull requests, Actions y otras funciones.

Puedes usar Git sin GitHub.

También puedes tener un repositorio Git local que todavía no se haya publicado en ningún servidor.

4. Modelo mental mínimo

```
working tree
├─ git add
staging area / index
├─ git commit
historial local
├─ git push
repositorio remoto en GitHub
```

Cada flecha es una acción distinta.

5. Qué es el working tree

Es la copia de archivos que estás editando en tu carpeta de proyecto.

Ejemplo:

```
script.sh
README.md
.gitignore
```

Los cambios que haces primero existen en tu working tree.

6. Qué es el staging area

También se llama **index**.

Sirve para preparar el contenido del próximo commit.

Git no obliga a incluir todos tus cambios juntos.

Puedes preparar solo algunos archivos o incluso partes de archivos.

7. Qué es un commit

Un commit registra el contenido preparado en el index junto con metadatos y un mensaje.

Un commit debe representar una unidad lógica de cambio.

Ejemplo de este proyecto:

```
linux: añadir práctica de funciones Bash
```

Mejor que:

```
cambios
```

8. GitHub entra después

Un commit existe localmente aunque no hayas hecho push.

git push publica commits hacia un remoto configurado.

Por eso:

```
commit ≠ push
```

9. Comprobar Git

```
git --version
```

Después:

```
command -v git
```

Son operaciones de consulta.

10. Preparar el laboratorio

```
test -d ~/linux-lab && \  
cd ~/linux-lab && \  
mkdir -p modulo-33-git && \  
cd modulo-33-git && pwd
```

Etiqueta: **creación en laboratorio**.

Si alguna orden falla, la cadena se detiene: no continúes hasta revisar el mensaje. test -d comprueba primero que el laboratorio exista.

11. Inicializar un repositorio de práctica

```
git init -b main
```

Esto crea metadatos Git dentro de:

```
.git/
```

No sube nada a Internet.

12. Consultar estado

```
git status
```

git status muestra, entre otras cosas:

- cambios preparados para commit;
- cambios modificados pero no preparados;
- archivos no rastreados;
- información de rama.

Es uno de los comandos que debes ejecutar con más frecuencia.

Antes del ejemplo: qué es un here-document. La sintaxis `<<'EOF'` entrega al comando varias líneas de entrada hasta encontrar una línea que contenga únicamente EOF. Las comillas alrededor de EOF impiden que Bash expanda variables y sustituciones de comandos dentro del bloque. En `cat > archivo`, el signo `>` crea o sobrescribe el archivo: comprueba antes que el nombre está libre y que estás en el directorio del laboratorio. Si ya existe, conserva la versión anterior y usa Vim para editarla sin perder el trabajo.

13. Crear un archivo de práctica

```
cat > hola.sh <<'EOF'
#!/usr/bin/env bash
printf 'Hola desde Git y Bash\n'
EOF
```

Después:

```
git status
```

`hola.sh` debe aparecer como archivo no rastreado.

14. Revisar antes de añadir

Para un archivo no rastreado pequeño puedes inspeccionarlo con:

```
cat hola.sh
```

También:

```
bash -n hola.sh
```

Si ShellCheck está disponible:

```
shellcheck hola.sh
```

Primero revisa contenido; después prepara el commit.

15. git add

```
git add hola.sh
```

`git add` añade el contenido actual del archivo al index.

Importante:

Si modificas el archivo después de `git add`, debes volver a ejecutar `git add` para preparar esa nueva versión.

16. Confirmar estado después de git add

```
git status
```

Ahora `hola.sh` debe aparecer preparado para commit.

17. Revisar lo preparado

```
git diff --staged
```

Este paso es obligatorio en nuestro flujo de aprendizaje.

No basta con saber qué archivos están staged; debes revisar **qué contenido** entrará.

18. Flujo mínimo antes de commit

```
git status
git diff
git add archivo
git diff --staged
git status
git commit
```

Esta secuencia reduce commits accidentales.

19. Crear el primer commit

```
git commit -m 'linux: versionar primer script Bash'
```

El commit registra únicamente el contenido preparado.

20. Consultar historial

```
git log --oneline --decorate
```

Esto muestra una vista compacta del historial.

21. Modificar el script

```
printf "printf 'Segunda línea de práctica\\n\\n' >> hola.sh
```

Después:

```
git status
git diff
```

git diff muestra cambios del working tree que todavía no están staged.

22. git diff y git diff --staged

Modelo:

```
git diff
working tree ↔ index

git diff --staged
index ↔ HEAD
```

HEAD suele referirse al commit actual de la rama.

23. Preparar segundo cambio

```
git add hola.sh
git diff --staged
```

Solo si la diferencia es correcta:

```
git commit -m 'linux: ampliar práctica de salida con printf'
```

24. Commits pequeños

Evita guardar en un mismo commit:

- un script Bash;
- una configuración de red;
- documentación no relacionada;
- archivos binarios grandes;
- credenciales;
- cambios experimentales de otro tema.

Mejor:

```
un cambio lógico → un commit claro
```

25. Qué hace .gitignore

Un archivo .gitignore especifica patrones de archivos **no rastreados** que Git debe ignorar.

Ejemplo:

```
*.log
tmp/
.env
```

26. Qué .gitignore NO hace

Si un archivo ya está rastreado, añadirlo después a .gitignore no hace que Git deje de rastrearlo automáticamente.

Esto es importante con secretos.

.gitignore es prevención, no una herramienta mágica para borrar historial.

27. Regla de secretos

Nunca prepares ni publiques:

- contraseñas;
- tokens;
- API keys;
- cookies;
- claves SSH privadas;
- archivos .env con secretos;
- credenciales de nube;
- datos personales innecesarios.

GitHub advierte explícitamente que no debes añadir, confirmar ni subir información sensible a un repositorio remoto.

Si el secreto YA fue confirmado o publicado

Añadir el archivo a .gitignore o borrar el secreto en un commit nuevo **no invalida una credencial que ya fue expuesta**.

GitHub recomienda tratar una credencial filtrada como comprometida. El orden defensivo es:

1. detener nuevos pushes o cambios innecesarios
2. identificar qué credencial fue expuesta y qué servicio la emitió
3. revocar o rotar la credencial con su proveedor
4. actualizar de forma segura los sistemas legítimos que dependían de ella
5. retirar el secreto del código y del working tree
6. decidir si hace falta sanear el historial del repositorio
7. coordinar la limpieza de clones, forks o ramas afectadas si corresponde
8. revisar por qué ocurrió y añadir prevención

La prioridad es **invalidar la credencial**, no “hacer desaparecer el texto” primero.

GitHub señala que reescribir historial puede tener efectos secundarios importantes:

- cambia hashes de commits;
- puede afectar ramas y pull requests;
- otros clones pueden volver a introducir el secreto;
- requiere coordinación con colaboradores;
- un secreto puede seguir existiendo en forks, clones o referencias históricas.

Por eso este módulo **no enseña una receta automática de force-push o reescritura de historial**. Si realmente ocurre una exposición, primero se revoca/rota la credencial y después se sigue la documentación oficial de GitHub para la limpieza apropiada.

Push protection y secret scanning

GitHub puede detectar ciertos secretos y bloquear un push mediante **push protection**.

Si un push es bloqueado por un secreto real:

```
no lo fuerces ni lo ignores por comodidad
→ retira el secreto
→ revoca/rota si ya pudo quedar expuesto
→ vuelve a revisar el commit
```

Push protection es una capa preventiva; no reemplaza tu propia revisión.

Antes de ejecutar: el siguiente bloque vuelve a usar un *here-document* (explicado antes de la sección 13). Recuerda que `>` crea o sobrescribe `.gitignore`: comprueba antes que no existe.

28. Crear `.gitignore` de práctica

```
cat > .gitignore <<'EOF'
*.log
tmp/
.env
EOF
```

Después:

```
git status
```

29. Probar que `.gitignore` funciona

```
printf 'registro\n' > prueba.log
mkdir -p tmp
printf 'temporal\n' > tmp/dato.txt
```

Después:

```
git status
```

`prueba.log` y `tmp/dato.txt` no deberían aparecer como candidatos normales para seguimiento por esos patrones.

30. Ver por qué algo está ignorado

Puedes consultar:

```
git check-ignore -v prueba.log
```

Esto ayuda a identificar qué patrón de ignore coincidió.

31. Revisar antes de `git add .`

`git add .` puede preparar muchos cambios bajo el directorio actual.

Antes de usarlo:

```
git status
git diff
```

En este manual preferimos al principio:

```
git add archivo_especifico
```

porque obliga a decidir qué entra.

32. Si añadiste algo al staging por error

Para sacar un archivo del staging sin borrar el archivo de trabajo:

```
git restore --staged archivo
```

Después verifica:

```
git status
```

33. Cuidado con `git restore sin --staged`

```
git restore archivo
```

puede reemplazar cambios del working tree con contenido de la fuente de restauración.

Eso puede descartar trabajo no guardado.

Regla:

Antes de usar `git restore` sobre el working tree, revisa `git status` y `git diff`.

34. Comandos destructivos fuera del núcleo

No usamos como rutina en este módulo:

```
git reset --hard
git clean -fd
git checkout -- archivo
git push --force
git rebase --onto ...
```

Algunos son útiles en contextos concretos, pero pueden descartar trabajo o reescribir historial.

Se estudiarán en el itinerario específico de Git cuando corresponda.

35. Configurar identidad de commits

Git necesita una identidad para crear commits.

Consulta primero:

```
git config --get user.name
git config --get user.email
```

Si falta, configura solo lo necesario siguiendo tu política de privacidad.

No publiques información personal innecesaria.

36. Configuración local vs global

```
--local → solo repositorio actual
--global → configuración del usuario
```

Para un laboratorio, una configuración local puede ser suficiente y evita cambiar otros repositorios.

Ejemplo conceptual:

```
git config --local user.name 'Nombre de práctica'
git config --local user.email 'correo autorizado'
```

No inventes ni publiques una dirección personal si no quieres asociarla a commits.

37. Qué es un remoto

Un remoto es una referencia a otro repositorio.

Consulta:

```
git remote -v
```

Si no aparece nada, el repositorio de práctica no tiene remoto configurado.

38. origin no es GitHub por definición

`origin` es solo un nombre convencional de remoto.

Puede apuntar a:

- GitHub;
- otro servidor;
- otra ubicación accesible.

No asumas el destino: verifica `git remote -v`.

39. Añadir remoto

Forma general:

```
git remote add origin URL_DEL_REPOSITORIO
```

Esto modifica la configuración local del repositorio.

Antes de ejecutarlo en una práctica real, verifica:

- propietario;
- nombre del repositorio;
- protocolo;
- URL completa.

40. git push

Forma típica:

```
git push -u origin main
```

push envía commits locales hacia el remoto.

Antes:

```
git status
git log --oneline --decorate -n 5
git remote -v
git branch --show-current
```

Debes saber exactamente qué rama y qué repositorio estás publicando.

41. Autenticación a GitHub

GitHub no acepta la contraseña normal de tu cuenta como contraseña Git para operaciones HTTPS.

Según el método configurado, puedes usar:

- Git Credential Manager u otro gestor compatible;
- GitHub CLI;
- token de acceso personal para HTTPS;
- claves SSH.

Nunca escribas tokens o claves privadas dentro del repositorio.

42. Este proyecto ya usa integración GitHub

Durante la construcción de este manual, el tutor guarda cada módulo en el repositorio:

```
budoninja010-hub/aprendizaje-programacion-ciberseguridad
```

Este Módulo 33 formaliza cómo podrás entender y ejecutar tú mismo ese flujo.

43. Flujo seguro para tus ejercicios

Para cada práctica con valor de aprendizaje:

```
1. crear/corregir ejercicio
2. revisar código
3. comprobar que no contiene secretos
4. git status
5. git diff
6. git add archivo concreto
7. git diff --staged
8. git status
9. git commit -m 'mensaje claro'
10. verificar remoto/rama
11. git push cuando corresponda
```

44. Ejemplos de mensajes de commit

```
python: agregar ejercicio de variables y print
java: practicar clases y objetos
linux: añadir práctica de permisos
linux: agregar ejercicio de funciones Bash
security: documentar laboratorio defensivo de análisis de logs
```

Formato de este repositorio: área: verbo en infinitivo + qué cambia. Las áreas y ejemplos están en [GUÍA_DE_TRABAJO.md](#).

El mensaje debe explicar qué aprendiste o qué cambió.

45. No sobrescribir aprendizaje anterior

Si corriges un ejercicio:

```
commit 1 → versión inicial / WIP
commit 2 → corrección
```

Así conservas el historial.

No reemplaces silenciosamente el aprendizaje anterior si el historial tiene valor.

46. Archivo en progreso

Un ejercicio incompleto puede guardarse si tiene valor pedagógico.

Mensaje posible:

```
linux: guardar práctica de bucles en progreso (WIP)
```

Después:

```
linux: corregir práctica de bucles y quoting
```

47. Revisar un commit después de crearlo

```
git show --stat --oneline HEAD
```

Para revisar contenido:

```
git show HEAD
```

Confirma que no se haya colado un archivo inesperado.

48. Revisar archivos rastreados

```
git ls-files
```

Esto ayuda a responder:

```
¿qué archivos están bajo seguimiento?
```

49. Diferencia entre no rastreado e ignorado

```
untracked → Git lo ve pero todavía no está en historial
ignored    → una regla indica que normalmente no debe añadirse
tracked    → Git ya lo sigue
```

Son estados diferentes.

50. Git no es una copia de seguridad universal

Git es excelente para archivos de texto y proyectos versionables.

No reemplaza automáticamente:

- backup del sistema;
- copias de discos;
- snapshots;
- almacenamiento de grandes binarios;
- protección frente a secretos ya filtrados.

51. GitHub tampoco debe ser un almacén de secretos

Un repositorio privado reduce exposición pública, pero no convierte las credenciales en contenido apropiado para versionar.

Los secretos deben gestionarse con mecanismos específicos.

Si un secreto real ya llegó al repositorio:

```
repositorio privado ≠ credencial segura
borrar en el commit siguiente ≠ credencial revocada
.gitignore ≠ limpieza del historial
```

La primera acción de seguridad es invalidar o rotar la credencial con el proveedor correspondiente.

52. Práctica A — estado y primer commit

Si seguiste las secciones 13 a 23, ese commit ya existe y `git commit` responderá que no hay nada que confirmar. Repite el flujo desde cero en un repositorio nuevo del laboratorio:

```
test -d ~/linux-lab && \
cd ~/linux-lab && \
mkdir modulo-33-git-practica && \
cd modulo-33-git-practica && \
git init -b main
```

Crea `hola.sh` como en la sección 13 y `.gitignore` como en la sección 28. Después:

```
git status
git add hola.sh
git diff --staged
git commit -m 'linux: versionar primer script Bash'
git log --oneline --decorate
```

Explica qué cambió en cada etapa.

53. Práctica B — segundo commit pequeño

Modifica `hola.sh`.

Después:

```
git status
git diff
git add hola.sh
git diff --staged
git commit -m 'linux: ampliar práctica de printf'
```

54. Práctica C — .gitignore

```
git add .gitignore
git diff --staged
git commit -m 'chore: agregar exclusiones de laboratorio'
```

Después comprueba:

```
git check-ignore -v prueba.log
```

55. Práctica D — sacar algo del staging

Modifica dos archivos de práctica.

Prepara ambos:

```
git add hola.sh .gitignore
```

Después saca solo uno:

```
git restore --staged .gitignore
git status
```

Verifica que `.gitignore` siga existiendo en tu working tree.

56. Práctica E — auditoría antes de publicar

Ejecuta:

```
git status
git diff
git diff --staged
git ls-files
git remote -v
git branch --show-current
```

Responde antes de cualquier push:

```
¿qué cambios publicaré?
¿hay secretos?
¿qué rama uso?
¿qué remoto es?
¿el repositorio es el correcto?
```

57. Errores frecuentes

Error	Problema	Corrección
git add . sin revisar	puedes preparar archivos inesperados	status/diff y archivos concretos
commit sin diff --staged	no sabes exactamente qué entra	revisar staged antes
subir .env	posible exposición de secretos	ignorar y gestionar secretos fuera de Git
creer que borrar el secreto en un commit nuevo lo invalida	la credencial puede seguir activa y existir en historial	revocar/rotar primero; después limpiar código/historial
creer que .gitignore elimina un archivo ya rastreado	no afecta automáticamente a tracked ni borra historial	corregir seguimiento y evaluar saneamiento histórico
confundir commit con push	son etapas distintas	commit local, push remoto
asumir que origin es correcto	puede apuntar a otro destino	git remote -v
usar git restore archivo sin revisar	puede descartar cambios	status + diff primero
usar reset --hard para «arreglar todo»	puede destruir trabajo	no usar en este nivel
push --force por rutina	puede reescribir historial remoto	posponer hasta dominar consecuencias
commits gigantes	dificultan revisión/aprendizaje	commits pequeños y temáticos

58. Detección de error 1

Analiza:

```
git add .
git commit -m 'cosas'
```

Problemas:

- no se revisó estado;
- no se revisó diferencia;
- no se comprobó contenido staged;
- mensaje poco informativo.

Mejor flujo:

```
git status
git diff
git add archivo_concreto
git diff --staged
git commit -m 'linux: describe el cambio real'
```

59. Detección de error 2

Analiza un repositorio que contiene:

```
.env
token.txt
id_rsa
```

Antes de cualquier git add, detén el flujo.

Esos nombres pueden indicar secretos o credenciales.

No deben subirse.

Si una credencial real ya hubiera sido confirmada o publicada, el procedimiento cambia:

```
detener → revocar/rotar → retirar del código → evaluar limpieza de historial
```

No intentes “solucionarlo” únicamente agregando el archivo a `.gitignore`.

60. Detección de error 3

Analiza:

```
git push origin main
```

sin revisar el remoto.

Primer paso correcto:

```
git remote -v
git branch --show-current
```

Después confirma el repositorio y la rama.

61. Práctica independiente

Crea `estado_sistema.sh` dentro de un repositorio de laboratorio.

Debe:

1. usar Bash;
2. mostrar solo información no sensible como `uname -s` y `pwd` del laboratorio;
3. pasar `bash -n`;
4. pasar ShellCheck si está disponible;
5. tener un `.gitignore` razonable;
6. ejecutar `git status` antes de preparar cambios;
7. revisar `git diff`;
8. añadir solo los archivos necesarios;
9. revisar `git diff --staged`;
10. crear un commit con mensaje claro;
11. no contener secretos ni datos personales;
12. explicar qué cambiaría antes de un eventual push.

62. Mini evaluación

1. ¿Git y GitHub son exactamente lo mismo? A) Sí B) No
2. ¿`git status` muestra estado del working tree/index? A) Sí B) No
3. ¿`git add` prepara contenido para el próximo commit? A) Sí B) No
4. ¿un commit publica automáticamente en GitHub? A) Sí B) No
5. ¿`git diff` y `git diff --staged` comparan lo mismo? A) Sí B) No
6. ¿debes revisar `git diff --staged` antes del commit? A) Sí B) No
7. ¿`.gitignore` afecta automáticamente archivos ya rastreados? A) Sí B) No
8. ¿un token debe subirse si el repositorio es privado? A) Sí B) No
9. Si una credencial real ya fue publicada, ¿la primera prioridad es revocarla o rotarla? A) Sí B) No
10. ¿añadir un secreto ya publicado a `.gitignore` invalida esa credencial? A) Sí B) No
11. ¿reescribir historial puede afectar commits, ramas, PRs y clones? A) Sí B) No
12. ¿`git restore --staged` puede sacar un archivo del staging? A) Sí B) No
13. ¿`git restore` archivo puede descartar cambios del working tree? A) Sí B) No
14. ¿origin garantiza que el remoto sea GitHub correcto? A) Sí B) No
15. ¿debes revisar `git remote -v` antes de un push importante? A) Sí B) No
16. ¿los commits pequeños facilitan revisión? A) Sí B) No
17. ¿Git reemplaza todos los tipos de backup? A) Sí B) No
18. ¿este módulo sustituye el itinerario completo de Git/GitHub? A) Sí B) No

63. Registro de aprendizaje

```
Git es:
GitHub es:
Working tree significa:
Staging area significa:
`git status` sirve para:
`git diff` muestra:
`git diff --staged` muestra:
`git add` hace:
`git commit` hace:
`git push` hace:
`.gitignore` sirve para:
¿qué no hace `.gitignore`?:
`git restore --staged` sirve para:
¿por qué reviso el remoto antes de push?:
Regla de secretos:
Si un secreto ya fue publicado, primero debo:
¿Por qué `.gitignore` no resuelve una exposición pasada?:
¿Por qué reescribir historial requiere coordinación?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

64. Puerta de dominio

Para considerar este módulo **DOMINADO** deberás poder en más de una práctica:

1. explicar Git vs GitHub;
2. distinguir working tree, index y commit;
3. revisar cambios antes de prepararlos;
4. preparar solo archivos elegidos;
5. revisar staged antes de commit;
6. crear un commit claro sin copiar el mensaje;
7. detectar un archivo que no debe subirse;
8. explicar qué hacer si una credencial real ya fue expuesta;
9. distinguir revocar/rotar una credencial de limpiar historial Git;
10. verificar rama y remoto antes de push;
11. explicar por qué un repositorio privado tampoco debe contener secretos.

65. Enlace al itinerario específico de Git

Este módulo cubre únicamente la autonomía mínima necesaria para tus scripts Linux.

El itinerario específico de Git/GitHub profundiza en:

- modelo de objetos de Git;
- ramas;
- merge;
- conflictos;
- rebase;
- tags;
- remotos avanzados;
- pull requests;
- estrategias de colaboración;
- reflog;
- reset/restore/revert con más detalle;
- GitHub Actions;
- GitOps y su Apéndice B específico.

No adelantes esos temas aquí si todavía no corresponden en el itinerario de Git.

66. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- Git — git status: <https://git-scm.com/docs/git-status>
- Git — git add: <https://git-scm.com/docs/git-add>
- Git — git commit: <https://git-scm.com/docs/git-commit>
- Git — git restore: <https://git-scm.com/docs/git-restore>
- Git — .gitignore: <https://git-scm.com/docs/gitignore>

- GitHub Docs — Adding locally hosted code to GitHub: <https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github>
- GitHub Docs — Personal access tokens: <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>
- GitHub Docs — Removing sensitive data from a repository: <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository>
- GitHub Docs — Remediating a leaked secret: <https://docs.github.com/en/code-security/tutorials/remediate-leaked-secrets/remediating-a-leaked-secret>
- GitHub Docs — Push protection: <https://docs.github.com/en/code-security/concepts/secret-security/push-protection>

Puntos verificados documentalmente:

- `git status` distingue cambios entre HEAD, index y working tree y muestra archivos no rastreados;
- `git add` añade al index el contenido que existe en el momento de ejecutar el comando;
- `git commit` crea un commit con el contenido actual del index;
- `git restore --staged` puede restaurar contenido del index desde HEAD;
- `git restore` sobre el working tree puede reemplazar contenido local;
- `.gitignore` especifica archivos intencionalmente no rastreados y no afecta automáticamente archivos ya rastreados;
- GitHub advierte que nunca deben añadirse, confirmarse o subirse contraseñas, API keys u otra información sensible;
- si una credencial real ya fue expuesta, GitHub recomienda revocarla o rotarla como primera medida; retirar texto del repositorio no invalida por sí solo la credencial;
- reescribir historial puede cambiar hashes, afectar colaboradores y permitir recontaminación desde clones o ramas antiguas, por lo que requiere coordinación;
- push protection puede bloquear pushes que contienen secretos compatibles con sus detectores, pero no sustituye la revisión humana;
- un repositorio local puede conectarse a GitHub mediante un remoto y posteriormente publicarse con `git push` tras autenticación.

Se posponen al itinerario específico:

- `git reset` en profundidad;
- `git clean`;
- `rebase`;
- `cherry-pick`;
- `reflog`;
- `force push`;
- resolución avanzada de conflictos;
- hooks;
- submodules/subtrees;
- firma de commits;
- Git LFS;
- GitHub Actions;
- estrategias de ramas y GitOps en profundidad.

Estado de la lección: redactada y revisada documentalmente contra Git y GitHub Docs. El flujo pedagógico exige revisar diferencias, excluir secretos y confirmar remoto/rama antes de publicar.

Módulo 34 — Almacenamiento: `lsblk`, `df`, `du`, montaje y `fstab`

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Trigésima cuarta entrega.

1. Propósito del módulo

Este módulo introduce almacenamiento desde una regla central:

primero inspeccionar; después comprender; solo entonces modificar.

El objetivo de evidencia definido por la arquitectura es:

Separar inspección de cambios de alto impacto.

Durante esta lección **no montaremos discos reales, no desmontaremos sistemas de archivos reales y no editaremos `/etc/fstab`.**

2. Qué aprenderás

Al terminar este módulo podrás:

- distinguir dispositivo de bloque, partición, sistema de archivos y punto de montaje;
- usar `lsblk` para inspeccionar dispositivos de bloque;
- usar `lsblk --fs` para consultar sistemas de archivos, UUID y puntos de montaje;
- usar `findmnt` para consultar montajes actuales;
- usar `df` para consultar espacio del sistema de archivos;
- usar `du` para estimar espacio utilizado por archivos/directorios;
- explicar por qué `df` y `du` pueden mostrar cifras diferentes;
- explicar qué significa montar un sistema de archivos;
- explicar qué ocurre con el contenido previo de un directorio usado como punto de montaje;
- reconocer opciones como `ro`, `rw`, `noauto`, `nofail`, `user` y `defaults`;
- leer los seis campos de una línea de `/etc/fstab`;
- explicar por qué se prefieren identificadores persistentes como `UUID=`;
- consultar `/etc/fstab` sin modificarlo;
- crear una **copia de práctica** con sintaxis tipo `fstab`;
- validar esa copia con `findmnt --verify --tab-file` sin montarla;
- reconocer por qué `mount -a`, edición real de `fstab`, particionado y formateo son operaciones de alto impacto.

3. Modelo mental del almacenamiento

Un modelo inicial:

```
dispositivo de bloque
  ↓
partición o volumen
  ↓
sistema de archivos
  ↓
punto de montaje
  ↓
archivos y directorios visibles
```

No todos los sistemas usan exactamente una relación uno-a-uno entre estas capas.

4. Qué es un dispositivo de bloque

Es un dispositivo que Linux maneja mediante acceso por bloques.

Ejemplos frecuentes:

```
discos
SSD
particiones
volúmenes lógicos
dispositivos de almacenamiento virtuales
```

Los nombres concretos dependen del sistema.

5. Qué es una partición

Una partición es una división lógica de un dispositivo de almacenamiento.

Una partición **puede** contener un sistema de archivos, pero también puede tener otros usos.

No confundas:

```
partición ≠ sistema de archivos
```

6. Qué es un sistema de archivos

Es la estructura utilizada para organizar datos y metadatos.

Ejemplos habituales en Linux:

```
ext4
xfs
btrfs
vfat
tmpfs
```

Cada tipo tiene características y reglas propias.

7. Qué es un punto de montaje

Es el directorio del árbol Linux donde un sistema de archivos se hace accesible.

Ejemplo conceptual:

```
dispositivo/sistema de archivos → /datos
```

Al montarse, sus archivos aparecen bajo /datos.

8. Qué ocurre si el directorio ya tenía contenido

Cuando otro sistema de archivos se monta sobre un directorio, el contenido previo de ese directorio deja de ser accesible por esa ruta mientras el montaje permanece activo.

No significa necesariamente que se haya borrado.

Esta es una razón para comprobar el punto de montaje antes de realizar cambios.

9. `lsblk`: inspeccionar dispositivos

```
lsblk
```

`lsblk` lista información sobre dispositivos de bloque.

Es una operación de **consulta**.

10. Salida en árbol

La salida predeterminada suele representar relaciones entre dispositivos:

```
disco
├─ partición
└─ partición
```

Pero configuraciones complejas pueden producir relaciones más elaboradas.

11. `lsblk --fs`

```
lsblk --fs
```

muestra información orientada a sistemas de archivos.

En `util-linux` equivale a solicitar columnas como:

```
NAME
FSTYPE
FSVER
LABEL
UUID
FSAVAIL
FSUSE%
MOUNTPOINTS
```

12. UUID

Un UUID identifica de forma persistente un sistema de archivos.

Ejemplo conceptual:

```
UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

Para configuración persistente suele ser preferible a depender exclusivamente de un nombre como `/dev/sdb1`, porque ciertos nombres de dispositivos pueden cambiar entre arranques o según detección.

13. `lsblk` para scripts

Cuando una salida vaya a ser procesada por un script, no dependas de las columnas predeterminadas.

Selecciona columnas explícitas:

```
lsblk -o NAME,SIZE,FSTYPE,UUID,MOUNTPOINTS
```

La documentación de `util-linux` advierte que la salida predeterminada puede cambiar.

14. No adivines el disco por su nombre

No asumas:

```
/dev/sda siempre es X
/dev/sdb siempre es USB
```

Antes de cualquier operación administrativa real debes identificar el dispositivo mediante varias evidencias:

- tamaño;
- modelo;
- tipo;
- sistema de archivos;
- UUID;
- punto de montaje;
- contexto del equipo.

15. `findmnt`: inspeccionar sistemas montados

```
findmnt
```

`findmnt` muestra sistemas de archivos montados y sus relaciones.

Es preferible a parsear la salida histórica de `mount` en scripts.

16. Buscar el sistema de archivos de una ruta

```
findmnt --target "$HOME"
```

`--target` acepta un archivo o directorio y muestra el sistema de archivos que contiene esa ruta.

17. Consultar un punto de montaje concreto

Ejemplo:

```
findmnt --mountpoint /
```

consulta si `/` es un punto de montaje y muestra su información.

18. Elegir columnas en `findmnt`

```
findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS
```

Para automatización, declarar columnas evita depender de la salida predeterminada.

19. `df`: espacio del sistema de archivos

```
df
```

`df` informa sobre espacio utilizado y disponible en sistemas de archivos.

Versión más legible para humanos:

```
df -h
```

20. Consultar el sistema de archivos de una ruta con `df`

```
df -h "$HOME"
```

Esto responde principalmente:

```
¿cuánto espacio tiene y cuánto usa el sistema de archivos que contiene esta ruta?
```

21. `df -T`

En GNU/Linux:

```
df -hT
```

incluye el tipo del sistema de archivos.

No confundas el tipo con el nombre del dispositivo.

22. `du`: uso atribuido a archivos y directorios

```
du
```

`du` estima espacio utilizado por un conjunto de archivos.

Para una carpeta concreta:

```
du -sh ~/linux-lab
```

23. `du -s` y `du -h`

```
-s / --summarize      → un total por argumento  
-h / --human-readable → unidades legibles
```

Por eso:

```
du -sh directorio
```

es una consulta muy común.

24. `df` y `du` responden preguntas distintas

```
df → estado del sistema de archivos  
du → espacio asociado a archivos/directorios recorridos
```

No esperes que sus cifras sean siempre idénticas.

25. Por qué pueden diferir

Entre otras causas:

- archivos eliminados que siguen abiertos por procesos;

- bloques reservados o metadatos del sistema de archivos;
- archivos sparse;
- permisos que impiden a `du` recorrer parte del árbol;
- montajes dentro del árbol;
- diferencias entre espacio lógico y bloques realmente asignados.

Una diferencia no significa automáticamente que una herramienta esté equivocada.

26. Consultar primero el laboratorio

```
cd ~/linux-lab
pwd
du -sh .
df -h .
findmnt --target .
```

Estas órdenes son de **consulta**.

27. Qué significa montar

Montar significa asociar un sistema de archivos a un punto del árbol de directorios.

Forma conceptual:

```
mount FUENTE DESTINO
```

Esto es una operación de **modificación del estado del sistema**.

En este módulo no la ejecutaremos sobre dispositivos reales.

28. Antes de montar: qué debe verificarse

En una administración real, como mínimo:

1. fuente correcta;
2. sistema de archivos correcto;
3. destino correcto;
4. que el destino no tenga un montaje inesperado;
5. contenido previo del destino;
6. opciones de montaje;
7. permisos necesarios;
8. estrategia de recuperación.

29. Consultar si una ruta está montada

```
findmnt --mountpoint /ruta
```

o para saber qué sistema contiene una ruta:

```
findmnt --target /ruta
```

Estas dos preguntas no son exactamente iguales.

30. Opciones de montaje básicas

Algunas opciones comunes:

```
ro      → solo lectura
rw      → lectura y escritura
noauto  → no incluir en montaje automático por `mount -a`
nofail  → no tratar ausencia del dispositivo como error crítico en ciertos flujos
user    → permite determinados montajes por usuario no root
defaults → conjunto de comportamiento por defecto dependiente del sistema/filesystem
```

No memorices `defaults` como una lista universal rígida; consulta la documentación del entorno.

31. `ro` no vuelve mágicamente seguro todo

Un montaje de solo lectura reduce la posibilidad de escribir a través de ese montaje.

Pero no:

- valida el contenido;
- vuelve confiables los archivos;
- impide ejecutar todos los tipos de contenido;
- corrige una fuente equivocada.

32. Desmontar también modifica el sistema

La orden:

```
umount PUNTO
```

separa un sistema de archivos del árbol.

Puede fallar si está ocupado y puede afectar aplicaciones que lo estén usando.

No la practicamos sobre sistemas reales en este módulo.

33. Qué es /etc/fstab

/etc/fstab contiene información estática sobre sistemas de archivos que el sistema puede montar.

Es leído por varias herramientas y servicios.

Un error puede afectar montajes durante el arranque.

Por eso **no editaremos el archivo real en esta lección**.

34. Consultar /etc/fstab

```
cat /etc/fstab
```

o:

```
findmnt --fstab
```

Son operaciones de consulta.

Antes de compartir su salida públicamente, revisa si contiene UUID, rutas de red u otra información que no quieras divulgar.

35. Los seis campos de fstab

Una entrada se describe mediante seis campos principales. Los campos quinto y sexto pueden omitirse y, en ese caso, toman el valor 0:

1. fuente
2. punto de montaje
3. tipo de sistema de archivos
4. opciones
5. dump
6. orden de fsck

Ejemplo conceptual:

```
UUID=... /datos xfs defaults 0 0
```

36. Campo 1: fuente

Puede identificar la fuente mediante:

- UUID=;
- LABEL=;
- PARTUUID=;
- PARTLABEL=;
- ruta de dispositivo;
- otras fuentes según el tipo de montaje.

Para almacenamiento persistente local suele preferirse un identificador persistente.

37. Campo 2: punto de montaje

Es la ruta donde aparecerá el sistema de archivos.

Ejemplo:

```
/datos
```

Si contiene espacios o tabulaciones, `fstab` usa escapes especiales; no debes simplemente escribir espacios literales y esperar que se interpreten como parte del campo.

38. Campo 3: tipo

Ejemplos:

```
xfs
ext4
vfat
tmpfs
nfs
cifs
```

El tipo debe corresponder al recurso.

39. Campo 4: opciones

Es una lista separada por comas.

Ejemplo:

```
defaults,noauto
```

Las opciones válidas dependen también del tipo de sistema de archivos.

40. Campo 5: `fs_freq`

Tradicionalmente lo utiliza `dump` para decidir qué sistemas deben respaldarse.

Si no se especifica, su valor predeterminado es 0.

En muchas configuraciones modernas aparece como 0.

41. Campo 6: `fs_passno`

Se relaciona con el orden de comprobación mediante `fsck` durante el arranque.

Si se omite `fs_passno`, el valor predeterminado es 0. El significado apropiado depende del sistema de archivos y de la política del sistema.

No cambies este valor por ensayo y error.

42. `systemd` y `fstab`

En sistemas `systemd`, `systemd-fstab-generator` transforma dinámicamente entradas de `fstab` en unidades de montaje.

Esto explica por qué modificar `fstab` puede tener consecuencias más amplias que una simple línea de texto.

43. `mount -a`: operación de alto impacto

`mount -a` intenta montar las entradas aplicables de `fstab`, normalmente excepto las marcadas con `noauto`.

Por eso:

```
NO se usa como validador inocuo.
```

Para revisar sintaxis y consistencia preferimos primero herramientas de validación.

44. `findmnt --verify`

```
findmnt --verify
```

puede verificar la parseabilidad y usabilidad de la tabla de montaje.

Pero aplicado sin opciones trabaja sobre la configuración real.

Para el laboratorio utilizaremos:

```
--tab-file ARCHIVO
```


con una copia creada por nosotros.

45. Preparar laboratorio de `fstab` sin tocar `/etc/fstab`

```
test -d ~/linux-lab && \  
cd ~/linux-lab && \  
mkdir -p modulo-34-almacenamiento/origen-bind && \  
mkdir -p modulo-34-almacenamiento/punto-bind && \  
cd modulo-34-almacenamiento && pwd  
pwd
```

Solo creamos directorios ordinarios.

46. Crear una tabla de práctica

```
printf '%s %s none bind 0 0\n' \  
"$HOME/linux-lab/modulo-34-almacenamiento/origen-bind" \  
"$HOME/linux-lab/modulo-34-almacenamiento/punto-bind" \  
> fstab-practica
```

Este archivo **no es** `/etc/fstab`.

No cambia montajes.

47. Leer la tabla de práctica

```
cat fstab-practica
```

Identifica sus seis campos.

48. Validar la tabla de práctica

```
findmnt --verify --verbose --tab-file fstab-practica
```

Esta práctica verifica la tabla indicada sin instalarla como `/etc/fstab`.

Si informa un error, detente y entiende el diagnóstico antes de cambiar nada.

49. Crear un error deliberado y seguro

Haz una segunda copia:

```
cp fstab-practica fstab-error
```

Edita **solo** `fstab-error` para introducir una línea mal formada; por ejemplo, deja una línea con únicamente el nombre de la fuente, sin punto de montaje ni tipo. Omitir únicamente el quinto o sexto campo no constituye por sí solo un error: sus valores predeterminados son 0.

Después:

```
findmnt --verify --verbose --tab-file fstab-error
```

El objetivo es aprender a detectar una configuración incorrecta sin afectar el sistema.

50. No copies la práctica a `/etc/fstab`

El archivo de laboratorio usa rutas del usuario y existe solo con fines pedagógicos.

No debes hacer:

```
sudo cp fstab-practica /etc/fstab
```

Esa acción reemplazaría configuración crítica del sistema.

51. Por qué una copia previa importa

Antes de una modificación real de `fstab`, una administración responsable necesita:

- copia verificable del archivo anterior;
- identificar exactamente el sistema de archivos;
- confirmar UUID;

- comprobar punto de montaje;
- validar sintaxis;
- plan de recuperación si el arranque falla.

Esta lección se detiene antes de esa fase administrativa.

52. No formatear para aprender este módulo

Órdenes como:

```
mkfs
wipefs
fdisk
parted
```

pueden alterar o destruir estructuras de almacenamiento.

No son necesarias para alcanzar los objetivos del Módulo 34.

Se mantienen fuera de las prácticas.

53. No usar dd para preparar dispositivos aquí

dd puede sobrescribir grandes cantidades de datos con una orden equivocada.

No necesitamos usarlo para aprender lsblk, df, du, montajes y fstab.

54. Práctica A — mapa del almacenamiento

Ejecuta únicamente consultas:

```
lsblk
lsblk --fs
findmnt
df -h
```

Después explica:

```
qué es dispositivo
qué es filesystem
qué está montado
qué espacio está disponible
```

55. Práctica B — una ruta concreta

```
findmnt --target "$HOME/linux-lab"
df -h "$HOME/linux-lab"
du -sh "$HOME/linux-lab"
```

Explica por qué las tres órdenes responden preguntas distintas.

56. Práctica C — columnas explícitas

```
lsblk -o NAME,SIZE,FSTYPE,UUID,MOUNTPOINTS
findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS
```

Identifica qué columnas pertenecen al dispositivo y cuáles al montaje.

57. Práctica D — leer fstab real sin modificar

```
findmnt --fstab
```

Después:

```
cat /etc/fstab
```

Compara cómo presenta la información cada herramienta.

No edites el archivo.

58. Práctica E — validar archivo de laboratorio

```
findmnt --verify --verbose --tab-file fstab-practica
```

Explica:

- qué archivo se validó;
- por qué no fue `/etc/fstab`;
- por qué esto es más seguro que probar cambios reales.

59. Errores frecuentes

Error	Problema	Corrección
confundir disco con filesystem	son capas distintas	construir modelo dispositivo → filesystem → mountpoint
creer que <code>df</code> y <code>du</code> miden exactamente lo mismo	responden preguntas distintas	usar cada herramienta según objetivo
asumir <code>/dev/sdb</code> por memoria	nombres pueden cambiar	identificar con <code>lsblk</code> , UUID y contexto
editar <code>/etc/fstab</code> para «probar»	puede afectar arranque	usar copia de laboratorio
usar <code>mount -a</code> como simple prueba	puede montar muchas entradas	validar primero con <code>findmnt --verify</code>
montar sobre directorio con datos	oculta temporalmente contenido previo	revisar destino antes
copiar UUID de otro equipo	identifica otro filesystem	obtenerlo del sistema correcto
usar <code>sudo</code> innecesariamente	aumenta impacto	consultas normales primero
probar <code>mkfs/wipefs</code> en un disco	riesgo de pérdida de datos	no usar en este módulo
parsear salida predeterminada de <code>lsblk</code> en script	columnas pueden cambiar	definir <code>-o</code> explícitamente

60. Detección de error 1

Analiza:

```
UUID=abc /datos ext4 defaults 0
```

La ausencia del sexto campo no demuestra por sí sola un error: si se omite, `fs_passno` toma el valor 0. El ejemplo tampoco demuestra que `UUID=abc` identifique un sistema de archivos existente. Hay que distinguir una entrada parseable de una fuente y un destino utilizables.

No lo pruebes instalándolo.

Colócalo en una copia de laboratorio y valida con:

```
findmnt --verify --verbose --tab-file archivo-prueba
```

61. Detección de error 2

Analiza:

```
quiero montar un USB y supongo que siempre es /dev/sdb1
```

Problema: el nombre no constituye identificación suficiente.

Primero:

```
lsblk --fs
```

y verifica tamaño, filesystem, UUID, montaje y contexto.

62. Detección de error 3

Analiza:

```
sudo mount -a
```

después de editar varias líneas de `fstab` sin validarlas.

Problema: intenta aplicar múltiples montajes y puede afectar el sistema.

El flujo correcto empieza con:

```
copia
revisión
identificación
validación
plan de recuperación
```

63. Método seguro del manual

1. consulta con `lsblk`, `findmnt`, `df` y `du`;
2. identifica la capa que estás observando;
3. no adivines dispositivos;
4. prefiere identificadores persistentes para configuraciones permanentes;
5. revisa el punto de montaje;
6. no edites `fstab` durante exploración;
7. prueba sintaxis en una copia;
8. valida antes de aplicar;
9. documenta procedimiento de recuperación;
10. aumenta privilegios solo cuando una tarea real lo requiera y comprendas el efecto.

64. Práctica independiente

Crea `informe_almacenamiento.sh`.

Debe:

1. usar Bash;
2. ejecutar solo operaciones de consulta;
3. mostrar `lsblk` con columnas explícitas;
4. mostrar `df -h` para `~/linux-lab`;
5. mostrar `du -sh` para `~/linux-lab`;
6. usar `findmnt --target` para esa ruta;
7. no usar `sudo`;
8. no montar ni desmontar;
9. no leer dispositivos con herramientas destructivas;
10. pasar `bash -n`;
11. pasar ShellCheck si está disponible;
12. poder explicar qué pregunta responde cada comando.

65. Mini evaluación

1. ¿dispositivo de bloque y sistema de archivos son exactamente lo mismo? A) Sí B) No
2. ¿`lsblk` sirve para listar dispositivos de bloque? A) Sí B) No
3. ¿`lsblk --fs` muestra información de filesystem/UUID? A) Sí B) No
4. ¿`findmnt` sirve para consultar montajes? A) Sí B) No
5. ¿`df` y `du` responden exactamente la misma pregunta? A) Sí B) No
6. ¿`du -sh` directorio resume uso de ese árbol? A) Sí B) No
7. ¿montar puede ocultar temporalmente contenido previo del punto de montaje? A) Sí B) No
8. ¿`/etc/fstab` tiene seis campos principales por entrada? A) Sí B) No
9. ¿UUID suele ser más persistente que un nombre `/dev/sdX`? A) Sí B) No
10. ¿`mount -a` es una operación puramente de consulta? A) Sí B) No
11. ¿`findmnt --verify --tab-file` permite validar otra tabla? A) Sí B) No
12. ¿editaremos `/etc/fstab` real en esta práctica? A) Sí B) No
13. ¿`mkfs` es necesario para aprender este módulo? A) Sí B) No
14. ¿debes confirmar el punto de montaje antes de montar? A) Sí B) No
15. ¿salida predeterminada de `lsblk` debe asumirse estable en scripts? A) Sí B) No

66. Registro de aprendizaje

```
Dispositivo de bloque significa:
Partición significa:
Sistema de archivos significa:
Punto de montaje significa:
`lsblk` responde:
`lsblk --fs` añade:
`findmnt` responde:
`df` responde:
`du` responde:
¿por qué pueden diferir df y du?:
UUID sirve para:
Los seis campos de fstab son:
`noauto` significa:
`findmnt --verify` sirve para:
¿por qué no usamos mount -a para probar?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

67. Puerta de dominio

Para considerar este módulo **DOMINADO** deberás poder en más de una práctica:

1. distinguir dispositivo, filesystem y punto de montaje;
2. leer una salida básica de `lsblk --fs`;
3. explicar `df` frente a `du`;
4. localizar el filesystem de una ruta con `findmnt`;
5. enumerar los seis campos de `fstab`;
6. explicar por qué UUID es útil;
7. validar una tabla de práctica sin editar `/etc/fstab`;
8. identificar por qué `mount -a`, `mkfs` y edición de `fstab` son cambios de mayor impacto.

68. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- util-linux upstream (código fuente y manuales de `lsblk`, `findmnt`, `mount`, `fstab`): <https://github.com/util-linux/util-linux>
- util-linux upstream (documentación del proyecto): <https://www.kernel.org/pub/linux/utils/util-linux/>
- Las páginas man7 siguientes se mantienen como copias HTML prácticas, no como fuente upstream.
- util-linux — `lsblk(8)`: <https://man7.org/linux/man-pages/man8/lsblk.8.html>
- util-linux — `findmnt(8)`: <https://man7.org/linux/man-pages/man8/findmnt.8.html>
- util-linux — `mount(8)`: <https://man7.org/linux/man-pages/man8/mount.8.html>
- util-linux — `fstab(5)`: <https://man7.org/linux/man-pages/man5/fstab.5.html>
- GNU Coreutils — `df` y `du`: <https://www.gnu.org/software/coreutils/manual/coreutils.html>
- Red Hat Enterprise Linux 10 — Mounting file systems:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/managing_file_systems/mounting-file-systems
- Red Hat Enterprise Linux 10 — Persistently mounting file systems:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/managing_file_systems/persistently-mounting-file-systems

Puntos verificados documentalente:

- `lsblk` lista dispositivos de bloque y `--fs` solicita información de sistemas de archivos;
- util-linux recomienda columnas explícitas cuando la salida se procesa en scripts;
- `findmnt` consulta sistemas de archivos montados y puede buscar por `target/mountpoint`;
- `findmnt --verify` puede comprobar una tabla y `--tab-file` permite indicar un archivo distinto de `/etc/fstab`;
- `df` informa espacio de sistemas de archivos y `du` estima espacio utilizado por archivos;
- montar un filesystem sobre un directorio hace inaccesible por esa ruta el contenido anterior mientras el montaje está activo;
- `fstab` describe fuente, punto de montaje, tipo, opciones, campo `dump` y orden de `fsck`;
- RHEL 10 documenta el uso de UUID para montajes persistentes y la generación de unidades `systemd` a partir de `fstab`;
- `mount -a` intenta aplicar las entradas correspondientes de `fstab` y por eso no es una operación de validación inocua.

Se posponen:

- particionado real con `fdisk/parted`;
- creación de filesystems con `mkfs`;
- borrado de firmas con `wipefs`;
- LVM;
- RAID;
- LUKS;
- cuotas;

- montaje NFS/CIFS real;
- recuperación de filesystems;
- edición administrativa real de `/etc/fstab`;
- montaje de imágenes mediante loop devices;
- tuning de opciones específicas de ext4/xfs/btrfs.

Estado de la lección: redactada y revisada documentalmente contra util-linux, GNU Coreutils y RHEL 10. Todas las prácticas son de consulta o utilizan archivos/directorios ordinarios dentro de `~/linux-lab`; no se modifican discos ni la tabla real `/etc/fstab`.

Módulo 35 — Defensa, actualizaciones, mínimo privilegio, auditoría y AIDE

Manual Maestro de Linux y Shell Scripting — Edición 2026 · v3 · Trigésima quinta entrega.

1. Propósito del módulo

Este módulo cierra el núcleo inicial del manual.

Su objetivo no es convertir Linux en un sistema “invulnerable”, sino enseñarte una rutina defensiva verificable:

```
conocer el sistema
↓
reducir privilegios
↓
mantener software actualizado
↓
revisar logs y estado
↓
crear una línea base
↓
detectar cambios
↓
documentar evidencia y recuperación
```

Objetivo de evidencia de la arquitectura:

Documentar un laboratorio propio con evidencia y recuperación.

2. Qué aprenderás

Al terminar este módulo podrás:

- explicar el principio de mínimo privilegio;
- consultar tu identidad, grupos y privilegios permitidos;
- distinguir operación de consulta de operación administrativa;
- revisar actualizaciones disponibles sin instalarlas automáticamente;
- diferenciar actualización general de actualización de seguridad;
- reconocer que actualizar puede requerir reinicio de procesos o kernel;
- revisar servicios fallidos y logs locales;
- identificar procesos que escuchan puertos en tu propio equipo;
- crear una línea base de integridad dentro de `~/linux-lab`;
- detectar cambios mediante hashes;
- explicar qué es AIDE y cómo funciona;
- distinguir inicialización, comprobación y actualización de una base AIDE;
- reconocer falsos positivos después de cambios legítimos;
- documentar evidencia antes y después de una modificación;
- definir un procedimiento de recuperación;
- evitar automatizaciones defensivas que cambien el sistema sin revisión.

3. Alcance de seguridad

Todas las prácticas son sobre:

- tu propio equipo;
- una máquina virtual propia;
- un laboratorio;
- o un sistema expresamente autorizado.

No se incluyen:

- exploración de equipos ajenos;
- persistencia;
- evasión;
- explotación;
- robo de credenciales;
- movimiento lateral;
- técnicas para ocultar actividad.

4. Defensa no significa ejecutar todo como root

Una mala práctica defensiva es usar privilegios máximos por costumbre.

El principio correcto es:

```
usar el menor privilegio necesario
durante el menor tiempo necesario
para una tarea concreta
```

5. Consultar identidad

```
id
```

muestra información como UID, GID y grupos.

También:

```
whoami
groups
```

Son operaciones de consulta.

6. Usuario normal frente a root

root tiene privilegios administrativos muy amplios.

Un usuario normal tiene un alcance más limitado.

Para aprendizaje diario:

trabaja como usuario normal y eleva privilegios solo cuando la tarea lo requiera explícitamente.

7. Qué es `sudo`

`sudo` permite ejecutar órdenes autorizadas con otra identidad, normalmente root, según reglas definidas por administración.

No significa:

```
"hacer cualquier cosa como administrador"
```

Las reglas pueden limitar usuarios, hosts y comandos.

8. Consultar tus permisos `sudo`

```
sudo -l
```

muestra qué órdenes tienes autorizadas según la configuración aplicable.

Puede solicitar autenticación.

No modifica configuración.

9. No editar `sudoers` directamente como práctica

Los archivos principales son:

```
/etc/sudoers
/etc/sudoers.d/
```

Una regla incorrecta puede romper acceso administrativo o ampliar privilegios indebidamente.

Este módulo se limita a **consulta y concepto**.

10. Actualizaciones como medida defensiva

Las vulnerabilidades conocidas suelen corregirse mediante actualizaciones.

Pero actualizar también es un cambio del sistema.

Flujo responsable:


```
consultar
revisar
entender impacto
planificar ventana
aplicar
verificar
recuperar si fuera necesario
```

11. Consultar actualizaciones en RHEL 10

```
dnf check-update
```

consulta paquetes instalados que tienen actualizaciones disponibles.

Puede contactar repositorios configurados.

En esta práctica **no instalamos nada**.

Recuerda la semántica específica documentada por DNF:

```
0   → no hay actualizaciones disponibles
100 → hay actualizaciones disponibles
1   → ocurrió un error
```

Esto importa especialmente en un script defensivo. No debes escribir una regla genérica del tipo:

```
si estado != 0 → error
```

para `dnf check-update`, porque interpretarías incorrectamente el estado 100.

Este punto conecta con el **Módulo 25 — Códigos de salida** y con el **Módulo 15 — Paquetes, repositorios y actualizaciones**: el significado de un código debe consultarse en la documentación de la herramienta concreta.

Equivalentes de consulta en otras distribuciones: en Debian y Ubuntu, `apt list --upgradable` muestra paquetes con actualizaciones disponibles según el índice local; para actualizar ese índice normalmente se utiliza `apt update`, operación que consulta repositorios y puede requerir privilegios. No instales ni actualices paquetes en esta práctica. En Fedora reciente con DNF5, la comprobación se documenta como `dnf5 check-upgrade`; revisa la distinción entre DNF4 y DNF5 del Módulo 15. Para AIDE, consulta los procedimientos y rutas de configuración propios de Debian/Ubuntu o Fedora antes de administrar una base real.

12. Consultar avisos de seguridad

En RHEL 10 puede usarse:

```
dnf updateinfo list updates security
```

para listar actualizaciones de seguridad disponibles cuando el sistema y sus repositorios lo soportan.

13. Consultar una advisory concreta

Forma general:

```
dnf updateinfo info ID_DE_ADVISORY
```

Sirve para revisar severidad, paquetes y CVE asociados antes de aplicar cambios.

14. Aplicar actualizaciones es otra fase

Órdenes como:

```
dnf upgrade
dnf upgrade --security
```

modifican el sistema.

No las ejecutaremos en este laboratorio.

Antes de hacerlo en un equipo real debes considerar:

- compatibilidad;
- dependencias;
- espacio disponible;
- servicios afectados;

- kernel;
- ventana de mantenimiento;
- copia/rollback apropiado.

15. Kernel y reinicios

Una actualización puede instalar un kernel nuevo o actualizar bibliotecas utilizadas por procesos en ejecución.

Por eso:

```
actualización instalada ≠ cambio plenamente activo en todos los procesos
```

Después de actualizaciones reales puede ser necesario revisar reinicios.

16. No automatizar actualizaciones sin política

RHEL puede automatizar actualizaciones mediante herramientas como `dnf-automatic`.

Eso puede ser apropiado en ciertos entornos, pero requiere una política:

- qué se actualiza;
- cuándo;
- cómo se prueba;
- cómo se notifica;
- qué ocurre si falla;
- cómo se recupera.

No activaremos timers de actualización en este módulo.

17. Estado general de systemd

Consulta:

```
systemctl --failed
```

muestra unidades que están en estado fallido.

Es una buena comprobación defensiva inicial.

18. Consultar una unidad concreta

```
systemctl status NOMBRE.service
```

Primero identifica una unidad real.

No reinicies ni habilites servicios solo porque aparezcan mensajes que no comprendas.

19. Logs con journalctl

Consulta general del arranque actual:

```
journalctl -b
```

Puede contener mucha información.

No publiques logs completos sin revisar datos privados.

20. Prioridades de log

Puedes filtrar por prioridad:

```
journalctl -p warning..alert -b
```

Esto ayuda a concentrarse en eventos más importantes del arranque actual.

Una advertencia no prueba por sí sola un incidente de seguridad.

21. Logs de una unidad

```
journalctl -u NOMBRE.service -b
```

Permite correlacionar eventos con una unidad concreta.

22. Buscar antes de concluir

Un log puede representar:

- configuración normal;
- fallo de hardware;
- error de software;
- permiso insuficiente;
- cambio administrativo;
- o actividad maliciosa.

No saltes directamente a la última explicación.

23. Puertos locales en escucha

En tu propio equipo:

```
ss -lntup
```

puede mostrar sockets locales en escucha y procesos cuando los permisos lo permiten.

Esta práctica es de **inventario local**, no de escaneo de otras máquinas.

24. Preguntas defensivas sobre un listener

Si aparece un puerto:

1. ¿qué proceso lo abrió?;
2. ¿qué servicio corresponde?;
3. ¿lo esperabas?;
4. ¿escucha solo en localhost o en interfaces externas?;
5. ¿es necesario?;
6. ¿está actualizado?;
7. ¿qué logs produce?

No detengas procesos por intuición.

25. Procesos

Consulta:

```
ps -ef
```

o una vista acotada:

```
ps -eo pid,user,comm,args --sort=user
```

La presencia de un proceso desconocido no demuestra que sea malicioso.

26. Permisos dentro del laboratorio

Busca archivos escribibles por cualquiera únicamente dentro del laboratorio:

```
find ~/linux-lab -type f -perm -0002 -print
```

Esto no modifica nada.

Si aparece un archivo, revisa por qué tiene ese permiso antes de cambiarlo.

27. umask

Consulta:

```
umask
```

umask influye en permisos iniciales de nuevos archivos/directorios.

No reemplaza una revisión explícita de permisos.

28. Auditoría: qué significa

Auditar no significa únicamente “buscar ataques”.

Significa reunir evidencia sobre:

- quién;
- qué;
- cuándo;
- dónde;
- estado anterior;
- cambio;
- resultado.

29. auditd como concepto

RHEL incluye un subsistema de auditoría que puede registrar eventos definidos por políticas.

Configurar reglas de auditoría es una tarea administrativa.

En este módulo solo comprobaremos si el servicio existe:

```
systemctl status auditd
```

No añadiremos reglas al sistema.

30. Evidencia mínima de laboratorio

Para una práctica técnica, registra:

```
fecha
objetivo
equipo/laboratorio autorizado
estado inicial
comandos ejecutados
resultado esperado
resultado observado
archivos modificados
cómo volver al estado anterior
```

31. Preparar el laboratorio final

```
test -d ~/linux-lab && \
cd ~/linux-lab && \
mkdir -p modulo-35-defensa/evidencia && \
mkdir -p modulo-35-defensa/integridad && \
cd modulo-35-defensa && pwd
pwd
```

Antes de ejecutar: `cat > archivo <<'EOF'` utiliza un *here-document*: envía a `cat` las líneas siguientes hasta el delimitador `EOF`. Las comillas impiden expansiones de variables y comandos dentro del bloque. La redirección `>` **crea o sobrescribe** el destino; verifica que estés en el laboratorio y que el archivo no existe antes de continuar. Si ya existe, revísalo y editalo con Vim o conserva una copia.

32. Crear archivo de evidencia inicial

```
cat > evidencia/registro.md <<'EOF'
# Laboratorio defensivo – Módulo 35

## Objetivo
Crear una línea base y detectar cambios controlados.

## Alcance
Solo ~/linux-lab/modulo-35-defensa

## Recuperación
Restaurar desde copia de laboratorio creada antes del cambio.
EOF
```

33. Crear datos de integridad

```
printf 'configuracion=original\n' > integridad/config.txt
printf 'dato estable\n' > integridad/datos.txt
```

34. Crear copia de recuperación

```
mkdir -p recuperacion
cp -a integridad recuperacion/integridad-inicial
```

Esta copia existe solo dentro del laboratorio.

35. Qué es una línea base

Una línea base representa un estado conocido contra el cual se comparan estados posteriores.

Ejemplo:

```
archivo
tamaño
permisos
propietario
hash
fecha
```

36. Línea base con SHA-256

```
find integridad -type f -print0 \
| sort -z \
| xargs -0 sha256sum \
> evidencia/baseline.sha256
```

Esta práctica calcula hashes solo de nuestros archivos de laboratorio.

37. Revisar la línea base

```
cat evidencia/baseline.sha256
```

Guarda también el estado:

```
stat integridad/config.txt integridad/datos.txt > evidencia/stat-inicial.txt
```

38. Verificar sin modificar

```
sha256sum -c evidencia/baseline.sha256
```

Si no hubo cambios, los archivos deben validar correctamente.

39. Crear un cambio controlado

```
printf 'configuracion=modificada\n' > integridad/config.txt
```

Este cambio afecta únicamente un archivo creado por la práctica.

40. Detectar el cambio

```
sha256sum -c evidencia/baseline.sha256
```

Ahora la comprobación debe informar que config.txt cambió.

Eso demuestra detección de integridad.

41. Hash distinto no explica la causa

Un hash diferente indica:

```
el contenido ya no coincide
```

No indica automáticamente:

- quién cambió el archivo;
- por qué;
- si fue autorizado;
- si fue un ataque.

Para atribución necesitas evidencia adicional.

42. Recuperación del archivo de práctica

Primero revisa la copia:

```
cat recuperacion/integridad-inicial/config.txt
```

Después restaura únicamente el archivo de laboratorio:

```
cp recuperacion/integridad-inicial/config.txt integridad/config.txt
```

43. Verificar recuperación

```
sha256sum -c evidencia/baseline.sha256
```

El archivo debe volver a coincidir con la línea base.

44. Qué es AIDE

AIDE significa **Advanced Intrusion Detection Environment**.

Su modelo es:

```
configuración
↓
base de datos inicial
↓
estado posterior del filesystem
↓
comparación
↓
informe de archivos añadidos/cambiados/eliminados
```

45. AIDE no bloquea cambios

AIDE es principalmente una herramienta de **detección de integridad**.

No impide por sí sola que un archivo cambie.

Su valor está en detectar diferencias respecto de una línea base.

46. Comprobar si AIDE existe

```
command -v aide
```

Si existe:

```
aide --version
```

Estas son consultas.

Si no está instalado, no lo instalaremos automáticamente en esta práctica.

47. Flujo administrativo AIDE en RHEL 10

En un sistema RHEL administrado, el flujo documentado incluye:

```
instalar aide
configurar /etc/aide.conf
inicializar base
activar base inicial
ejecutar comprobaciones
actualizar la base después de cambios legítimos
```

Este flujo normalmente requiere privilegios administrativos.

48. Inicialización del sistema

La documentación de RHEL muestra:

```
aide --init
```

para generar una base inicial conforme a `/etc/aide.conf`.

No ejecute esa inicialización sobre todo tu sistema como parte de este laboratorio.

Puede recorrer una gran cantidad de archivos y escribir la base del sistema.

49. Comprobación AIDE

Una vez correctamente inicializado por un administrador:

```
aide --check
```

compara el estado actual con la base configurada.

Los resultados pueden incluir:

- añadidos;
- eliminados;
- modificados.

50. Actualizar la base después de cambios legítimos

Instalar paquetes, aplicar actualizaciones o cambiar configuración de forma autorizada puede producir diferencias.

Después de verificar que esos cambios son legítimos, la base de AIDE debe actualizarse según el procedimiento de la distribución.

No actualices una línea base solo para hacer desaparecer una alerta.

51. Proteger la línea base

Si un atacante pudiera modificar al mismo tiempo:

- archivos del sistema;
- configuración de AIDE;
- base de referencia;
- binario de AIDE;

la confianza de la comprobación disminuiría.

Por eso Red Hat recomienda proteger base, configuración y herramienta en una ubicación segura cuando el nivel de riesgo lo justifique.

52. Falsos positivos y cambios legítimos

Una detección de AIDE puede corresponder a:

- actualización autorizada;
- edición administrativa;
- cambio normal de una aplicación;
- instalación;
- o modificación no autorizada.

La herramienta detecta diferencia; el analista interpreta contexto.

53. AIDE no reemplaza backups

AIDE puede decirte que algo cambió.

No necesariamente conserva la versión anterior.

Necesitas una estrategia de respaldo y recuperación aparte.

54. AIDE no reemplaza logs

Integridad responde principalmente:

```
¿cambió?
```

Los logs pueden ayudar con:

```
¿cuándo?  
¿qué proceso?  
¿qué usuario?  
¿qué ocurrió alrededor?
```

Las fuentes se complementan.

55. Rutina defensiva básica semanal

Para un laboratorio propio:

1. revisar actualizaciones disponibles;
2. revisar `systemctl --failed`;
3. revisar eventos importantes del journal;
4. revisar listeners locales esperados;
5. verificar cambios de integridad;
6. documentar hallazgos;
7. corregir solo después de comprender;
8. verificar recuperación.

56. No reaccionar destruyendo evidencia

Ante algo inesperado no hagas inmediatamente:

```
borrar archivos  
limpiar logs  
reiniciar múltiples servicios  
reinstalar sin documentar
```

Primero registra el estado y preserva evidencia relevante.

57. No compartir evidencia sensible

Antes de subir a GitHub o enviar un informe, revisa que no contenga:

- tokens;
- contraseñas;
- direcciones privadas innecesarias;
- nombres de usuarios reales;
- rutas personales sensibles;
- claves SSH;
- cookies;
- logs con información privada.

58. Guardar evidencia en GitHub

Solo se versiona evidencia **sanitizada**.

Ejemplo de archivos apropiados:

```
README.md  
registro.md sin datos sensibles  
script de consulta  
ejemplo de baseline del laboratorio
```

No subas dumps completos del sistema.

59. Script defensivo de consulta

Crea `revision_basica.sh`:


```
#!/usr/bin/env bash

printf '%s\n' '=== Identidad ==='
id

printf '%s\n' '=== Unidades fallidas ==='
systemctl --failed --no-pager

printf '%s\n' '=== Espacio del laboratorio ==='
df -h "$HOME/linux-lab"

printf '%s\n' '=== Uso del laboratorio ==='
du -sh "$HOME/linux-lab"
```

No usa sudo ni modifica el sistema.

60. Validar el script

```
bash -n revision_basica.sh
```

Si ShellCheck está disponible:

```
shellcheck revision_basica.sh
```

Después:

```
bash revision_basica.sh
```

61. Errores frecuentes

Error	Problema	Corrección
usar root para todo	aumenta impacto	usuario normal primero
actualizar sin revisar	puede afectar servicios/kernel	consultar y planificar
interpretar una alerta como ataque confirmado	falta contexto	correlacionar evidencia
borrar algo sospechoso inmediatamente	destruye evidencia	documentar primero
ejecutar AIDE --init sin entender configuración	puede escanear/escribir base del sistema	revisar alcance y configuración
actualizar baseline ante cualquier alerta	normaliza cambios no investigados	validar legitimidad primero
creer que AIDE es backup	no conserva versiones por sí solo	backup separado
publicar logs completos	puede filtrar información	sanitizar evidencia
detener servicio desconocido por intuición	puede romper sistema	identificar y documentar
automatizar remediación sin pruebas	amplifica errores	detectar primero, cambiar después

62. Detección de error 1

Analiza:

```
veo un proceso que no conozco → lo mato
```

Problema: desconocido no significa malicioso.

Primeros pasos:

```
identificar proceso
ver propietario
ver servicio asociado
revisar logs
consultar documentación
```

63. Detección de error 2

Analiza:

```
AIDE reporta 200 cambios después de una actualización → intrusión confirmada
```

Problema: una actualización legítima también modifica archivos.

Debes correlacionar con:

- transacción de paquetes;
- fecha/hora;
- advisories;
- logs;
- cambios autorizados.

64. Detección de error 3

Analiza:

```
hay actualizaciones → dnf upgrade -y inmediatamente
```

Problema: aplicar sin revisión elimina una fase importante de gestión del cambio.

En este nivel:

```
consultar → revisar → planificar → aplicar → verificar
```

65. Proyecto final del núcleo

Crea una carpeta:

```
~/linux-lab/modulo-35-defensa/proyecto-final/
```

Debe contener:

- README.md;
- revision_basica.sh;
- evidencia/baseline.sha256 de archivos de práctica;
- explicación de un cambio controlado;
- detección del cambio;
- procedimiento de recuperación;
- resultado después de restaurar.

66. README del proyecto final

Debe responder:

```
¿qué sistema/laboratorio se usó?
¿cuál era el objetivo?
¿qué se consultó?
¿qué se modificó?
¿qué evidencia se guardó?
¿cómo se detectó el cambio?
¿cómo se recuperó?
¿qué comandos NO se ejecutaron por riesgo?
```

67. Criterio de aprobación

El proyecto se considera correcto si:

1. todo ocurre en laboratorio autorizado;
2. no requiere sudo para la práctica principal;
3. no borra logs;
4. no toca /etc;
5. no cambia servicios;
6. crea una línea base;
7. detecta un cambio controlado;
8. restaura el archivo;
9. vuelve a verificar;
10. no contiene secretos;
11. puede guardarse en GitHub con un commit pequeño.

68. Commit recomendado

Cuando el ejercicio esté revisado:

```
linux: añadir laboratorio final de integridad y defensa
```

Antes de guardarlo:

```
git status
git diff
git diff --staged
```

y confirma que no hay información sensible.

69. Mini evaluación

1. ¿mínimo privilegio significa usar root siempre? A) Sí B) No
2. ¿sudo -l consulta tus autorizaciones? A) Sí B) No
3. ¿dnf check-update instala automáticamente paquetes? A) Sí B) No
4. ¿un estado 100 de dnf check-update significa que hay actualizaciones disponibles? A) Sí B) No
5. ¿un estado 1 de dnf check-update representa un error? A) Sí B) No
6. ¿aplicar actualizaciones puede afectar kernel o servicios? A) Sí B) No
7. ¿systemctl --failed sirve para revisar unidades fallidas? A) Sí B) No
8. ¿una advertencia del journal prueba por sí sola un ataque? A) Sí B) No
9. ¿ss -lntup puede ayudar a inventariar listeners locales? A) Sí B) No
10. ¿un hash distinto demuestra quién cambió el archivo? A) Sí B) No
11. ¿AIDE usa una base de referencia? A) Sí B) No
12. ¿AIDE impide todos los cambios? A) Sí B) No
13. ¿cambios legítimos pueden generar diferencias en AIDE? A) Sí B) No
14. ¿debes actualizar la baseline sin investigar una alerta? A) Sí B) No
15. ¿AIDE reemplaza un sistema de backups? A) Sí B) No
16. ¿la evidencia debe sanitizarse antes de GitHub? A) Sí B) No
17. ¿una defensa responsable incluye recuperación? A) Sí B) No

70. Registro de aprendizaje

```
Mínimo privilegio significa:
`sudo -l` sirve para:
`dnf check-update` sirve para:
Estado 0 de `dnf check-update` significa:
Estado 100 de `dnf check-update` significa:
Estado 1 de `dnf check-update` significa:
¿Por qué no debo tratar todo estado no-cero como error?:
`systemctl --failed` sirve para:
`journalctl` sirve para:
`ss -lntup` sirve para:
Una línea base es:
`sha256sum -c` sirve para:
AIDE es:
`aide --init` conceptualmente hace:
`aide --check` conceptualmente hace:
¿por qué cambia AIDE después de updates?:
¿por qué AIDE no es un backup?:
¿qué evidencia debo guardar?:
¿cómo recupero un cambio?:
Algo que todavía confundo:
Estado: EN APRENDIZAJE / PRACTICADO
```

71. Puerta de dominio

Para considerar este módulo **DOMINADO** deberás poder en más de una ocasión:

1. explicar mínimo privilegio con tus palabras;
2. separar consulta de modificación administrativa;
3. revisar actualizaciones sin aplicarlas automáticamente;
4. interpretar un listener local sin asumir que sea malicioso;
5. crear una línea base de integridad;
6. detectar un cambio controlado;
7. restaurar y volver a verificar;
8. explicar el modelo de AIDE;
9. distinguir detección, evidencia y recuperación;
10. producir un informe sanitizado y versionarlo sin secretos.

72. Fuentes y límites

Clasificación y estado de los enlaces de esta sección: [catálogo de fuentes](#) (jerarquía E04, comprobación del 9 de octubre de 2026).

Fuentes principales:

- iproute2 upstream (origen de ss): <https://git.kernel.org/pub/scm/network/iproute2/iproute2.git/>

- AIDE (proyecto original): <https://aide.github.io/>
- systemd upstream (manuales originales de systemctl y journalctl): <https://github.com/systemd/systemd/tree/main/man>
- Las guías oficiales RHEL y DNF siguientes respaldan los procedimientos específicos de distribución.
- Red Hat Enterprise Linux 10 — Security hardening:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/security_hardening/
- RHEL 10 — Checking integrity with AIDE:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/security_hardening/checking-integrity-with-aide
- RHEL 10 — Managing sudo access:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/security_hardening/managing-sudo-access
- DNF Project — Command Reference, check-update: https://dnf.readthedocs.io/en/latest/command_ref.html#check-update-command
- RHEL 10 — Managing software with DNF:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/managing_software_with_the_dnf_tool/updating-rhel-content
- RHEL 10 — Managing and monitoring security updates:
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/risk_reduction_and_recovery_operations/managing-and-monitoring-security-updates
- systemd — systemctl, journalctl: <https://www.freedesktop.org/software/systemd/man/latest/>
- iproute2 — ss(8): <https://man7.org/linux/man-pages/man8/ss.8.html>
- GNU Coreutils — sha256sum: <https://www.gnu.org/software/coreutils/manual/coreutils.html>

Puntos verificados documentalmente:

- RHEL 10 recomienda gestionar sudo para permitir tareas administrativas específicas sin iniciar sesión directamente como root;
- dnf check-update lista actualizaciones disponibles; DNF documenta estado 0 si no hay actualizaciones, 100 si hay actualizaciones y 1 si ocurre un error; RHEL permite consultar advisories de seguridad con dnf updateinfo;
- dnf upgrade --security instala actualizaciones de seguridad y por tanto es una operación modificadora, no una consulta;
- AIDE crea una base de datos de archivos y luego compara el estado actual para detectar diferencias;
- RHEL documenta aide --init para inicializar la base y aide --check para realizar comprobaciones;
- después de cambios legítimos, como actualizaciones de software, la base de AIDE debe actualizarse tras validar esos cambios;
- Red Hat recomienda proteger la base/configuración de AIDE cuando el nivel de seguridad lo requiera;
- una detección de integridad no sustituye copias de respaldo ni análisis de logs.

Se posponen para itinerarios posteriores de administración/ciberseguridad defensiva:

- reglas avanzadas de Audit;
- SELinux en profundidad;
- OpenSCAP;
- SCAP Security Guide;
- hardening CIS/STIG;
- gestión centralizada de logs;
- SIEM;
- respuesta a incidentes forense;
- gestión de vulnerabilidades a escala;
- automatización de remediación;
- políticas AIDE complejas;
- administración empresarial de sudo;
- EDR y monitoreo de endpoints.

Estado de la lección: redactada y revisada documentalmente contra RHEL 10 y documentación oficial de las herramientas utilizadas. La práctica es local, defensiva, reversible y no requiere privilegios administrativos.

73. Cierre del núcleo v3

Con este módulo quedan redactados los **35 módulos del núcleo** del Manual Maestro de Linux y Shell Scripting — Edición 2026 v3.

Esto significa:

contenido redactado ≠ contenido dominado

Las prácticas del estudiante todavía deben ejecutarse, explicarse y revisarse durante las clases.

Siguiente fase editorial:

auditoría transversal completa de los Módulos 1-35, corrección de inconsistencias, referencias cruzadas, seguridad, progresión pedagógica y preparación de edición consolidada.

Actualización: esta auditoría se realizó el 8 de octubre de 2026. Ver [03 — Auditoría transversal completa v3](#).

Fin del núcleo v3. [← Módulo 34 — Almacenamiento: lsb1k, df, du, montaje y fstab](#) · [Volver al índice](#)